

MobilityDB 1.3 User's Manual

Esteban Zimányi

COLLABORATORS

	<i>TITLE :</i> MobilityDB 1.3 User's Manual		
<i>ACTION</i>	<i>NAME</i>	<i>DATE</i>	<i>SIGNATURE</i>
WRITTEN BY	Esteban Zimányi	July 30, 2026	

REVISION HISTORY

NUMBER	DATE	DESCRIPTION	NAME

Contents

1	Introduction	1
1.1	Project Steering Committee	1
1.2	Other Code Contributors	2
1.3	Sponsors	2
1.4	Licenses	2
1.5	Installation from Sources	2
1.5.1	Short Version	2
1.5.2	Get the Sources	3
1.5.3	Enabling the Database	4
1.5.4	Dependencies	4
1.5.5	Configuring	5
1.5.6	Build and Install	5
1.5.7	Testing	5
1.5.8	Documentation	6
1.6	Installation from Binaries	6
1.6.1	Debian-based Linux Distributions	6
1.6.2	Windows	6
1.7	Support	6
1.7.1	Reporting Problems	7
1.7.2	Mailing Lists	7
1.8	Migrating from Version 1.2 to Version 1.3	7
2	Set and Span Types	8
2.1	Input and Output	9
2.2	Constructors	11
2.3	Conversions	12
2.4	Accessors	14
2.5	Transformations	17
2.6	Spatial Reference System	20
2.7	Set Operations	21

2.8	Bounding Box Operations	21
2.8.1	Topological Operations	21
2.8.2	Position Operations	22
2.8.3	Splitting Operations	23
2.9	Distance Operations	24
2.10	Comparisons	25
2.11	Aggregations	26
2.12	Indexing	27
3	Bounding Box Types	28
3.1	Input and Output	28
3.2	Constructors	30
3.3	Conversions	31
3.4	Accessors	32
3.5	Transformations	34
3.6	Spatial Reference System	35
3.7	Splitting Operations	36
3.8	Set Operations	37
3.9	Bounding Box Operations	37
3.9.1	Topological Operations	37
3.9.2	Position Operations	39
3.10	Comparisons	40
3.11	Aggregations	40
3.12	Indexing	41
4	Temporal Types (Part 1)	42
4.1	Introduction	42
4.2	Examples of Temporal Types	44
4.3	Validity of Temporal Types	46
4.4	Temporalizing Operations	46
4.5	Notation	47
4.6	Input and Output	48
4.7	Constructors	52
4.8	Conversions	54
4.9	Accessors	54
4.10	Transformations	58

5	Temporal Types (Part 2)	61
5.1	Modifications	61
5.2	Restrictions	66
5.3	Bounding Box Operators	68
5.4	Comparisons	68
5.4.1	Traditional Comparisons	68
5.4.2	Ever and Always Comparisons	69
5.4.3	Temporal Comparisons	70
5.5	Miscellaneous	71
6	Temporal Alphanumeric Types	72
6.1	Notation	72
6.2	Conversions	72
6.3	Accessors	73
6.4	Transformations	74
6.5	Restrictions	74
6.6	Boolean Operations	76
6.7	Mathematical Operations	76
6.8	Text Operations	79
7	Temporal Geometry Types (Part 1)	80
7.1	Spatiotemporal Types	80
7.2	Notation	80
7.3	Input and Output	82
7.4	Conversions	84
7.5	Accessors	85
7.6	Transformations	88
8	Temporal Geometry Types (Part 2)	92
8.1	Restrictions	92
8.2	Spatial Reference System	93
8.3	Bounding Box Operations	94
8.4	Distance Operations	94
8.5	Spatial Relationships	97
8.5.1	Ever and Always Relationships	99
8.5.2	Spatiotemporal Relationships	100

9	Temporal Types: Analytics Operations	102
9.1	Simplification	102
9.2	Reduction	103
9.3	Similarity	105
9.4	Extended Kalman Filter	108
9.5	Splitting Operations	108
9.6	Multidimensional Tiling	112
9.6.1	Bin Operations	113
9.6.2	Tile Operations	113
9.6.3	Bounding Box Operations	116
9.6.4	Splitting Operations	117
10	Temporal Types: Aggregation and Indexing	121
10.1	Aggregation	121
10.2	Indexing	123
10.3	Statistics and Selectivity	124
10.3.1	Statistics Collection	124
10.3.2	Selectivity Estimation	126
11	Temporal Poses	127
11.1	Static Poses	127
11.1.1	Input and Output	128
11.1.2	Constructors	129
11.1.3	Conversions	129
11.1.4	Accessors	130
11.1.5	Transformations	130
11.1.6	Spatial Reference System	130
11.1.7	Comparisons	131
11.1.8	OGC GeoPose v1.0 support	131
11.1.8.1	JSON input and output	132
11.1.8.2	Quaternion renormalization	132
11.1.8.3	Euler-angle accessors	132
11.1.8.4	Frame metadata registry	133
11.1.8.5	Body↔world rigid transform	133
11.1.8.6	Cross-SRID frame transforms	134
11.1.8.7	Temporal JSON I/O	134
11.1.8.8	Speed and angular speed	135
11.2	Temporal Poses	135
11.3	Validity of Temporal Poses	136

11.4	Input and Output	136
11.5	Constructors	138
11.6	Conversions	139
11.7	Accessors	139
11.8	Transformations	140
11.9	Modifications	141
11.10	Restrictions	141
11.11	Spatial Reference System	142
11.12	Bounding Box Operations	142
11.13	Distance Operations	143
11.14	Spatial Relationships	144
11.15	Comparisons	145
11.16	Aggregations	145
11.17	Indexing	146
12	Temporal Network Points	147
12.1	Static Network Types	147
12.1.1	Constructors	149
12.1.2	Conversions	149
12.1.3	Accessors	149
12.1.4	Transformations	150
12.1.5	Spatial Operations	150
12.1.6	Comparisons	151
12.2	Temporal Network Points	151
12.3	Validity of Temporal Network Points	152
12.4	Constructors	152
12.5	Conversions	153
12.6	MF-JSON Input and Output	153
12.7	Accessors	154
12.8	Transformations	154
12.9	Modifications	155
12.10	Restrictions	156
12.11	Distance Operations	156
12.12	Spatial Operations	157
12.13	Bounding Box Operations	157
12.14	Spatial Relationships	158
12.15	Comparisons	158
12.16	Aggregations	159
12.17	Indexing	160

12.18 Aggregation	160
12.19 Production guidance	160
12.19.1 When to use tnpoint vs tgeompoint	161
12.19.2 SRID inheritance from the network registry	161
12.19.3 Numerical hazards	161
12.19.4 Durability and storage	162
13 Temporal Circular Buffers	163
13.1 Static Circular Buffers	164
13.1.1 Constructors	164
13.1.2 Conversions	164
13.1.3 Accessors	165
13.1.4 Transformations	165
13.1.5 Spatial Operations	165
13.1.6 Comparisons	166
13.2 Temporal Circular Buffers	166
13.3 Validity of Temporal Circular Buffers	167
13.4 Input and Output	167
13.5 Constructors	169
13.6 Conversions	170
13.7 Accessors	170
13.8 Transformations	171
13.9 Modifications	171
13.10 Spatial Operations	172
13.11 Distance Operations	173
13.12 Restrictions	174
13.13 Comparisons	175
13.14 Bounding Box Operations	176
13.15 Spatial Relationships	177
13.16 Aggregations	178
13.17 Simplification	178
13.18 Indexing	179
14 Temporal Rigid Geometries	180
14.1 Input and Output	180
14.2 Constructors	181
14.3 Conversions	182
14.4 Accessors	182
14.5 Traversed Area	183

14.6 Spatial Functions	184
14.7 Motion Metrics	184
14.8 Modifications	185
14.9 Spatial Restrictions	185
14.10 Distance Operations	186
14.11 Similarity	187
14.12 Comparisons	187
14.13 Bounding-Box Operators	188
14.14 Tiles	188
14.15 Bounding Boxes	189
14.16 Aggregations	189
14.17 Indexing	189
14.18 Production guidance	190
14.18.1 When to use trgeometry vs tpose vs tgeometry	190
14.18.2 Frame conventions	190
14.18.3 Numerical hazards	191
14.18.4 Durability and storage	191
15 JSON Types in MobilityDB	192
15.1 JSONB Set Operations	192
15.2 Temporal JSONB values	198
15.3 Validity of temporal JSONB values	199
15.4 Input and Output	199
15.5 Constructors	200
15.6 Conversions	201
15.7 Accessors	202
15.8 Transformations	202
15.9 Temporal JSON Operations	203
15.10 Restrictions	209
15.11 Comparisons	210
15.12 Bounding Box Operations	210
15.13 Aggregations	211
15.14 Indexing	212
16 Temporal H3 Cell Indices	213
16.1 Input and Output	213
16.2 Conversions	214
16.3 Static Geometry Coverage	214
16.4 Accessors	215

16.5	Index Inspection	215
16.6	Hierarchy	216
16.7	Lat/Lng Conversions	217
16.8	Directed Edges	217
16.9	Vertices	218
16.10	Grid Traversal	218
16.11	Metrics	218
16.12	Comparisons	218
16.12.1	Ever and Always Comparisons	218
16.12.2	Temporal Comparisons	219
16.13	Bounding-Box Operators	219
16.14	Set-Returning Helpers	220
16.15	Spatial Relationships	221
16.16	Indexing	221
16.17	Operational notes	222
16.17.1	When to use th3index	222
16.17.2	H3-specific hazards	222
16.17.3	Durability and storage	223
17	Temporal QUADBIN Cell Indices	224
17.1	Input and Output	224
17.2	Conversions	225
17.3	Static Cells	225
17.4	Accessors	225
17.5	Index Inspection	226
17.6	Hierarchy	226
17.7	Geometry Conversions	227
17.8	Tiles and Quadkeys	228
17.9	Metrics	228
17.10	Comparisons	228
17.11	Indexing	229
17.12	Operational notes	229
17.12.1	When to use tquadbin	229
17.12.2	QUADBIN-specific hazards	229
17.12.3	Durability and storage	230

18 Temporal Point Clouds	231
18.1 Quickstart	231
18.1.1 Register a schema	231
18.1.2 Build base and temporal values	232
18.1.3 Query with the bbox surface	232
18.1.4 Index for bbox-driven scans	233
18.1.5 Aggregate	233
18.1.6 Per-point access	233
18.1.7 What's next	233
18.2 Static Types <code>pcpoint</code> and <code>pcpatch</code>	233
18.2.1 Ergonomic constructors	234
18.2.2 Accessors	234
18.3 Set Types <code>pcpointset</code> and <code>pcpatchset</code>	235
18.3.1 Constructors	235
18.4 Bounding Box <code>tpcbox</code>	235
18.4.1 Constructors	235
18.4.2 Conversions	235
18.4.3 Accessors	236
18.4.4 Transformations	237
18.4.5 Set Operations	237
18.4.6 Topological Operators	237
18.4.7 Position Operators	238
18.4.8 Comparisons	238
18.5 Temporal Type <code>tpcpoint</code>	239
18.5.1 Constructors	239
18.5.2 Accessors	239
18.5.3 Per-Dimension Projections	240
18.5.4 Conversions	240
18.5.5 Comparisons	240
18.5.6 Restrictions	240
18.5.7 Bounding-box operators	241
18.5.8 B-tree ordering — what <code>ORDER BY tpcpoint</code> means	241
18.5.9 Spatial relationships	242
18.6 Temporal Type <code>tpcpatch</code>	242
18.6.1 Constructors	242
18.6.2 Accessors	242
18.6.3 Restrictions	243
18.6.4 Spatial relationships	244
18.6.5 Bounding-box operators	244

18.6.6 B-tree ordering — what ORDER BY tpcpatch means	244
18.6.7 Per-point operations: what is and is not available	244
18.7 Indexes	245
18.8 Aggregations	245
18.9 Binary representation and portability	246
18.10 MF-JSON output	246
18.11 PDAL integration: readers.tpcpatch and writers.tpcpatch	247
18.11.1 readers.tpcpatch	247
18.11.2 writers.tpcpatch	247
18.11.3 Supported compression schemes	248
18.11.4 Limitations	248
18.12 PCL bridge: SQL-callable filters and registration	248
18.13 Caveats	249
19 Raster Sampling	250
19.1 Sampling Operator	250
19.2 Raquet / QUADBIN Sampling	251
19.3 Raquet Accessors and Comparison	253
19.4 Restriction and Predicate Operators	255
19.5 Build and Install	256
19.6 Production Roadmap	256
20 Portable Named-Function Dialect	257
A MobilityDB Reference	260
A.1 MobilityDB Types	260
A.2 Set and Span Types	260
A.2.1 Input and Output	260
A.2.2 Constructors	261
A.2.3 Conversions	261
A.2.4 Accessors	261
A.2.5 Transformations	261
A.2.6 Spatial Reference System	262
A.2.7 Set Operations	262
A.2.8 Bounding Box Operations	262
A.2.8.1 Topological Operations	262
A.2.8.2 Position Operations	262
A.2.8.3 Splitting Operations	262
A.2.9 Distance Operations	262
A.2.10 Comparisons	262

A.2.11 Aggregations	262
A.3 Box Types	263
A.3.1 Input and Output	263
A.3.2 Constructors	263
A.3.3 Conversions	263
A.3.4 Accessors	263
A.3.5 Transformations	263
A.3.6 Spatial Reference System	264
A.3.7 Splitting Operations	264
A.3.8 Set Operations	264
A.3.9 Bounding Box Operations	264
A.3.9.1 Topological Operations	264
A.3.9.2 Position Operations	264
A.3.10 Comparisons	264
A.3.11 Aggregations	264
A.4 Temporal Types	265
A.4.1 Input and Output	265
A.4.2 Constructors	265
A.4.3 Conversions	265
A.4.4 Accessors	265
A.4.5 Transformations	266
A.4.6 Modifications	266
A.4.7 Restrictions	266
A.4.8 Comparisons	266
A.4.9 Miscellaneous	266
A.5 Temporal Alphanumeric Types	266
A.5.1 Conversions	266
A.5.2 Accessors	267
A.5.3 Transformations	267
A.5.4 Restrictions	267
A.5.5 Boolean Operations	267
A.5.6 Mathematical Operations	267
A.5.7 Text Operations	267
A.6 Temporal Geometry Types	268
A.6.1 Input and Output	268
A.6.2 Conversions	268
A.6.3 Spatial Reference System	268
A.6.4 Accessors	268
A.6.5 Transformations	269

A.6.6	Restrictions	269
A.6.7	Distance Operations	269
A.6.8	Spatial Relationships	269
A.6.8.1	Ever or Always Spatial Relationships	269
A.6.8.2	Spatiotemporal Relationships	269
A.7	Operations for Temporal Types: Analytics Operations	270
A.7.1	Simplification	270
A.7.2	Reduction	270
A.7.3	Similarity	270
A.7.4	Splitting Operations	270
A.7.5	Multidimensional Tiling	270
A.7.5.1	Bin Operations	270
A.7.5.2	Tile Operations	271
A.7.5.3	Bounding Box Operations	271
A.7.5.4	Splitting Operations	271
A.7.6	Aggregations	271
A.8	Temporal Poses	271
A.8.1	Static Poses	271
A.8.1.1	Input and Output	271
A.8.1.2	Constructors	272
A.8.1.3	Conversions	272
A.8.1.4	Accessors	272
A.8.1.5	Transformations	272
A.8.1.6	Spatial Reference System	272
A.8.1.7	Comparisons	272
A.8.2	Temporal Poses	272
A.8.2.1	Input and Output	272
A.8.2.2	Constructors	273
A.8.2.3	Conversions	273
A.8.2.4	Accessors	273
A.8.2.5	Transformations	273
A.8.2.6	Modifications	273
A.8.2.7	Restrictions	273
A.8.2.8	Spatial Reference System	273
A.8.2.9	Bounding Box Operations	273
A.8.2.10	Distance Operations	274
A.8.2.11	Comparisons	274
A.8.2.12	Aggregations	274
A.9	Temporal Network Points	274

A.9.1	Static Network Types	274
A.9.1.1	Constructors	274
A.9.1.2	Conversions	274
A.9.1.3	Accessors	274
A.9.1.4	Transformations	274
A.9.1.5	Spatial Operations	274
A.9.1.6	Comparisons	275
A.9.2	Temporal Network Points	275
A.9.2.1	Constructors	275
A.9.2.2	Conversions	275
A.9.2.3	Accessors	275
A.9.2.4	Transformations	275
A.9.2.5	Modifications	275
A.9.2.6	Restrictions	275
A.9.2.7	Bounding Box Operations	275
A.9.2.8	Distance Operations	276
A.9.2.9	Spatial Operations	276
A.9.2.10	Comparisons	276
A.9.2.11	Aggregations	276
A.10	Temporal Circular Buffers	276
A.10.1	Static Circular Buffers	276
A.10.1.1	Constructors	276
A.10.1.2	Conversions	276
A.10.1.3	Accessors	276
A.10.1.4	Transformations	276
A.10.1.5	Spatial Operations	277
A.10.1.6	Comparisons	277
A.10.2	Temporal Circular Buffers	277
A.10.2.1	Input and Output	277
A.10.2.2	Constructors	277
A.10.2.3	Conversions	277
A.10.2.4	Accessors	277
A.10.2.5	Transformations	277
A.10.2.6	Modifications	278
A.10.2.7	Restrictions	278
A.10.2.8	Distance Operations	278
A.10.2.9	Spatial Operations	278
A.10.2.10	Bounding Box Operations	278
A.10.2.11	Spatial Relationships	278

A.10.2.12 Comparisons	278
A.10.2.13 Aggregations	278
A.11 Temporal Rigid Geometries	279
A.11.1 Input and Output	279
A.11.2 Constructors	279
A.11.3 Conversions	279
A.11.4 Accessors	279
A.11.5 Traversed Area	279
A.11.6 Spatial Functions	279
A.11.7 Motion Metrics	280
A.11.8 Modifications	280
A.11.9 Restrictions	280
A.11.10 Distance Operations	280
A.11.11 Similarity	280
A.11.12 Comparisons	280
A.11.13 Bounding Box Operations	280
A.11.14 Tiles	280
A.11.15 Bounding Boxes	281
A.11.16 Aggregations	281
A.12 Temporal JSON	281
A.12.1 Input and Output	281
A.12.2 Constructors	281
A.12.3 Conversions	281
A.12.4 Accessors	281
A.12.5 Transformations	281
A.12.6 Temporal JSON operations	282
A.12.7 Restrictions	282
A.12.8 Bounding Box Operations	282
A.12.9 Comparisons	282
A.12.10 Aggregations	282
A.13 Temporal H3 Cell Indices	283
A.13.1 Conversions	283
A.13.2 Static Geometry Coverage	283
A.13.3 Accessors	283
A.13.4 Index Inspection	283
A.13.5 Hierarchy	283
A.13.6 Lat/Lng Conversions	283
A.13.7 Directed Edges	284
A.13.8 Vertices	284

A.13.9 Grid Traversal	284
A.13.10 Metrics	284
A.13.11 Ever and Always Comparisons	284
A.13.12 Temporal Comparisons	284
A.13.13 Bounding Box Operations	284
A.13.14 Set-Returning Helpers	285
A.13.15 Spatial Relationships	285
A.13.16 Indexing	285
A.14 Temporal Quadbin Cell Indices	285
A.14.1 Conversions	285
A.14.2 Static Cells	285
A.14.3 Accessors	286
A.14.4 Index Inspection	286
A.14.5 Hierarchy	286
A.14.6 Geometry Conversions	286
A.14.7 Tiles and Quadkeys	286
A.14.8 Metrics	286
A.14.9 Comparisons	286
A.14.10 Indexing	286
A.15 Temporal Point Clouds	287
A.15.1 Static Types <code>pcpoint</code> and <code>pcpatch</code>	287
A.15.1.1 Constructors	287
A.15.1.2 Accessors	287
A.15.2 Set Types <code>pcpointset</code> and <code>pcpatchset</code>	287
A.15.2.1 Constructors	287
A.15.3 Bounding Box <code>tpcbox</code>	287
A.15.3.1 Constructors	287
A.15.3.2 Conversions	287
A.15.3.3 Accessors	287
A.15.3.4 Transformations	287
A.15.3.5 Set Operations	288
A.15.3.6 Topological Operators	288
A.15.3.7 Position Operators	288
A.15.3.8 Comparisons	288
A.15.4 Temporal Type <code>tpcpoint</code>	288
A.15.4.1 Constructors	288
A.15.4.2 Accessors	288
A.15.4.3 Conversions	288
A.15.4.4 Comparisons	289

A.15.4.5 Restrictions	289
A.15.4.6 Bounding-box Operators	289
A.15.4.7 Spatial Relationships	289
A.15.5 Temporal Type <code>tpcpatch</code>	289
A.15.5.1 Constructors	289
A.15.5.2 Accessors	289
A.15.5.3 Restrictions	289
A.15.5.4 Spatial Relationships	290
A.15.5.5 Bounding-box Operators	290
A.15.6 Aggregations	290
A.15.7 Indexes	290
A.15.8 Binary Representation	290
A.15.9 MF-JSON Output	290
A.15.10 PDAL Integration	290
A.15.11 PCL Bridge	290
A.16 Raster Sampling	291
A.16.1 Sampling Operator	291
A.16.2 Raquet / QUADBIN Sampling	291
A.16.3 Raquet Accessors and Comparison	291
A.16.4 Restriction and Predicate Operators	291
B Synthetic Data Generator	292
B.1 Generator for PostgreSQL Types	292
B.2 Generator for PostGIS Types	293
B.3 Generator for MobilityDB Span, Time, and Box Types	294
B.4 Generator for MobilityDB Temporal Types	294
B.5 Generation of Tables with Random Values	295
B.6 Generator for Temporal Network Point Types	300

List of Figures

3.1	Multiresolution grid on Brussels data obtained using the BerlinMOD generator. Each cell contains at most 10,000 (left) and 1,000 (right) instants across the entire simulation period (four days in this case). On the left, we can see the high density of the traffic in the ring around Brussels, while on the right we can see other main axes in the city.	37
5.1	Insert operation for temporal values.	61
5.2	Update and delete operation for temporal values.	62
5.3	Modification operations for temporal tables in SQL	62
7.1	Hierarchy of spatiotemporal types in MobilityDB.	80
7.2	Illustration of the use of the <code>tgeompoint</code> (top) and the <code>tgeometry</code> (bottom) types for modelling the trajectory and the wind swath of tropical storms in 2024.	81
7.3	Visualizing the speed of a moving object using a color ramp in QGIS.	90
9.1	Difference between the spatial and the synchronous distance.	103
9.2	Sampling of temporal floats with discrete, step, and linear interpolation.	104
9.3	Changing the precision of temporal floats with discrete, step, and linear interpolation.	106
9.4	Multidimensional tiling for temporal floats.	112
13.1	Illustration of the use of the <code>tcbuffer</code> type for modelling the circular buffer around the wind swath of tropical storms in 2024.	163

List of Tables

1.1	Variables for the user's and the developer's documentation	6
A.1	MobilityDB current instantiations of template types	260

Abstract

MobilityDB is an extension to the [PostgreSQL](#) database system and its spatial extension [PostGIS](#). It allows temporal and spatio-temporal objects to be stored in the database, that is, objects whose attribute values and/or location evolves in time. MobilityDB includes functions for analysis and processing of temporal and spatio-temporal objects and provides support for GiST and SP-GiST indexes. MobilityDB is open source and its code is available on [Github](#). A binding for the Python programming language is also available on [Github](#).



MobilityDB is developed by the Computer & Decision Engineering Department of the Université Libre de Bruxelles (ULB) under the direction of Prof. Esteban Zimányi. ULB is an OGC Associate Member and member of the OGC Moving Feature Standard Working Group ([MF-SWG](#)).



The MobilityDB Manual is licensed under a [Creative Commons Attribution-Share Alike 3.0 License](#). Feel free to use this material any way you like, but we ask that you attribute credit to the MobilityDB Project and wherever possible, a link back to [MobilityDB](#).



Chapter 1

Introduction

MobilityDB is an extension of [PostgreSQL](#) and [PostGIS](#) that provides *temporal types*. Such data types represent the evolution on time of values of some element type, called the *base type* of the temporal type. For instance, temporal integers may be used to represent the evolution on time of the gear used by a moving car. In this case, the data type is *temporal integer* and the base type is *integer*. Similarly, a temporal float may be used to represent the evolution on time of the speed of a car. As another example, a temporal point may be used to represent the evolution on time of the location of a car, as reported by GPS devices. Temporal types are useful because representing values that evolve in time is essential in many applications, for example in mobility applications. Furthermore, the operators on the base types (such as arithmetic operators and aggregation for integers and floats, topological and distance relationships for geometries) can be intuitively generalized when the values evolve in time.

MobilityDB provides the temporal types `tbool`, `tint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint`, and `tgeogpoint`. These temporal types are based, respectively, on the `bool`, `integer`, `float`, and `text` base types provided by PostgreSQL, and on the `geometry` and `geography` base types provided by PostGIS, where `tgeometry` and `tgeography` accept arbitrary geometries/geographies, while `tgeompoint` and `tgeogpoint` only accept 2D or 3D points.¹ Furthermore, MobilityDB provides *set*, *span*, and *span set* template types for representing, respectively, sets of values, ranges of values, and sets of ranges of values of base types or time types. Examples of values of set types are `intset`, `floatset`, and `tstzset`, where the latter represents set of `timestampz` values. Examples of values of span types are `intspan`, `floatspan`, and `tstzspan`. Examples of values of span set types are `intspanset`, `floatspanset`, and `tstzspanset`.

1.1 Project Steering Committee

The MobilityDB Project Steering Committee (PSC) coordinates the general direction, release cycles, documentation, and outreach efforts for the MobilityDB project. In addition, the PSC provides general user support, accepts and approves patches from the general MobilityDB community and votes on miscellaneous issues involving MobilityDB such as developer commit access, new PSC members or significant API changes.

The current members in alphabetical order and their main responsibilities are given next:

- Mohamed Bakli: [MobilityDB-docker](#), cloud and distributed versions, integration with [Citrus](#)
- Krishna Chaitanya Bommakanti: [MEOS \(Mobility Engine Open Source\)](#), [pyMEOS](#)
- Anita Graser: integration with [Moving Pandas](#) and the Python ecosystem, integration with [QGIS](#)
- Darafei Praliaskouski: integration with [PostGIS](#)
- Mahmoud Sakr: co-founder of the MobilityDB project, [MobilityDB workshop](#), co-chair of the OGC Moving Feature Standard Working Group ([MF-SWG](#))
- Vicky Vergara: integration with [pgRouting](#), liason with [OSGeo](#)

¹ Although 4D temporal points can be represented, the M dimension is currently not taken into account.

- Esteban Zimányi (chair): co-founder of the MobilityDB project, overall project coordination, main contributor of the backend code, [BerlinMOD generator](#)

1.2 Other Code Contributors

- Arthur Lesuisse
- Xinyiang Li
- Maxime Schoemans

1.3 Sponsors

These are research funding organizations (in alphabetical order) that have contributed with monetary funding to the MobilityDB project.

- [European Commission](#)
- [Fonds de la Recherche Scientifique \(FNRS\), Belgium](#)
- [Innoviris, Belgium](#)

These are corporate entities (in alphabetical order) that have contributed developer time or monetary funding to the MobilityDB project.

- [Adonmo, India](#)
- [Georepublic, Germany](#)
- [Université libre de Bruxelles, Belgium](#)

1.4 Licenses

The following licenses can be found in MobilityDB:

Resource	Licence
MobilityDB code	PostgreSQL Licence
MobilityDB documentation	Creative Commons Attribution-Share Alike 3.0 License

1.5 Installation from Sources

1.5.1 Short Version

To compile assuming you have all the dependencies in your search path

```
git clone https://github.com/MobilityDB/MobilityDB
mkdir MobilityDB/build
cd MobilityDB/build
cmake ..
make
sudo make install
```

The above commands install the `master` branch. If you want to install another branch, for example, `develop`, you can replace the first command above as follows

```
git clone --branch develop https://github.com/MobilityDB/MobilityDB
```

You should also set the following in the `postgresql.conf` file depending on the version of PostGIS you have installed (below we use PostGIS 3):

```
shared_preload_libraries = 'postgis-3'
max_locks_per_transaction = 128
```

If you do not preload the PostGIS library with the above configuration you will not be able to load the MobilityDB library and will get an error message such as the following one:

```
ERROR: could not load library "/usr/local/pgsql/lib/libMobilityDB-1.1.so":
        undefined symbol: ST_Distance
```

You can find the location of the `postgresql.conf` file as given next.

```
$ which postgres
/usr/local/pgsql/bin/postgres
$ ls /usr/local/pgsql/data/postgresql.conf
/usr/local/pgsql/data/postgresql.conf
```

As can be seen, the PostgreSQL binaries are in the `bin` subdirectory while the `postgresql.conf` file is in the `data` subdirectory.

Once MobilityDB is installed, it needs to be enabled in each database you want to use it in. In the example below we use a database named `mobility`.

```
createdb mobility
psql mobility -c "CREATE EXTENSION PostGIS"
psql mobility -c "CREATE EXTENSION MobilityDB"
```

The two extensions PostGIS and MobilityDB can also be created using a single command.

```
psql mobility -c "CREATE EXTENSION MobilityDB cascade"
```

1.5.2 Get the Sources

The MobilityDB latest release can be found in <https://github.com/MobilityDB/MobilityDB/releases/latest>
wget

To download this release:

```
wget -O mobilitydb-1.3.tar.gz https://github.com/MobilityDB/MobilityDB/archive/v1.3.tar.gz
```

Go to Section 1.5.1 to the extract and compile instructions.

git

To download the repository

```
git clone https://github.com/MobilityDB/MobilityDB.git
cd MobilityDB
git checkout v1.3
```

Go to Section 1.5.1 to the compile instructions (there is no tar ball).

1.5.3 Enabling the Database

MobilityDB is an extension that depends on PostGIS. Enabling PostGIS before enabling MobilityDB in the database can be done as follows

```
CREATE EXTENSION postgis;  
CREATE EXTENSION mobilitydb;
```

Alternatively, this can be done in a single command by using `CASCADE`, which installs the required PostGIS extension before installing the MobilityDB extension

```
CREATE EXTENSION mobilitydb CASCADE;
```

1.5.4 Dependencies

Compilation Dependencies

To be able to compile MobilityDB, make sure that the following dependencies are met:

- CMake cross-platform build system.
- C compiler `gcc` or `clang`. Other ANSI C compilers can be used but may cause problems compiling some dependencies.
- GNU Make (`gmake` or `make`) version 3.1 or higher. For many systems, GNU make is the default version of make. Check the version by invoking `make -v`.
- PostgreSQL version 12 or higher. PostgreSQL is available from <http://www.postgresql.org>.
- PostGIS version 3 or higher. PostGIS is available from <https://postgis.net/>.
- GNU Scientific Library (GSL). GSL is available from <https://www.gnu.org/software/gsl/>. GSL is used for the random number generators.

Notice that PostGIS has its own dependencies such as Proj, GEOS, LibXML2, or JSON-C, and these libraries are also used in MobilityDB. Refer to <http://trac.osgeo.org/postgis/wiki/UsersWikiPostgreSQLPostGIS> for a support matrix of PostGIS with PostgreSQL, GEOS, and Proj.

Optional Dependencies

For the user's documentation

- The DocBook DTD and XSL files are required for building the documentation. For Ubuntu, they are provided by the packages `docbook` and `docbook-xsl`.
- The XML validator `xmllint` is required for validating the XML files of the documentation. For Ubuntu, it is provided by the package `libxml2`.
- The XSLT processor `xsltproc` is required for building the documentation in HTML format. For Ubuntu, it is provided by the package `libxslt`.
- The program `dblatex` is required for building the documentation in PDF format. For Ubuntu, it is provided by the package `dblatex`.
- The program `dbtoepub` is required for building the documentation in EPUB format. For Ubuntu, it is provided by the package `dbtoepub`.

For the developers's documentation

- The program `doxygen` is required for building the documentation. For Ubuntu, it is provided by the package `doxygen`.

Example: Installing dependencies on Linux

Database dependencies

```
sudo apt-get install postgresql-16 postgresql-server-dev-16 postgresql-16-postgis
```

Build dependencies

```
sudo apt-get install cmake gcc libgsl-dev
```

1.5.5 Configuring

MobilityDB uses the `cmake` system to do the configuration. The build directory must be different from the source directory.

To create the build directory

```
mkdir build
```

To see the variables that can be configured

```
cd build
cmake -L ..
```

1.5.6 Build and Install

Notice that the current version of MobilityDB has been tested on Linux, MacOS, and Windows systems. It may work on other Unix-like systems, but remain untested. We are looking for volunteers to help us to test MobilityDB on multiple platforms.

The following instructions start from `path/to/MobilityDB` on a Linux system

```
mkdir build
cd build
cmake ..
make
sudo make install
```

When the configuration changes

```
rm -rf build
```

and start the build process as mentioned above.

1.5.7 Testing

MobilityDB uses `ctest`, the CMake test driver program, for testing. This program will run the tests and report results.

To run all the tests

```
ctest
```

To run a given test file

```
ctest -R '021_tbox'
```

To run a set of given test files you can use wildcards

```
ctest -R '022_*'
```

1.5.8 Documentation

MobilityDB user's documentation can be generated in HTML, PDF, and EPUB format. Furthermore, the documentation is available in English and in other languages (currently, only in Spanish). The user's documentation can be generated in all formats and in all languages, or specific formats and/or languages can be specified. MobilityDB developer's documentation can only be generated in HTML format and in English.

The variables used for generating user's and the developer's documentation are as follows:

Variable	Default value	Comment
DOC_ALL	BOOL=OFF	The user's documentation is generated in HTML, PDF, and EPUB formats.
DOC_HTML	BOOL=OFF	The user's documentation is generated in HTML format.
DOC_PDF	BOOL=OFF	The user's documentation is generated in PDF format.
DOC_EPUB	BOOL=OFF	The user's documentation is generated in EPUB format.
LANG_ALL	BOOL=OFF	The user's documentation is generated in English and in all available translations.
ES	BOOL=OFF	The user's documentation is generated in English and in Spanish.
DOC_DEV	BOOL=OFF	The English developer's documentation is generated in HTML format.

Table 1.1: Variables for the user's and the developer's documentation

Generate the user's and the developer's documentation in all formats and in all languages.

```
cmake -D DOC_ALL=ON -D LANG_ALL=ON -D DOC_DEV=ON ..
make doc
make doc_dev
```

Generate the user's documentation in HTML format and in all languages.

```
cmake -D DOC_HTML=ON -D LANG_ALL=ON ..
make doc
```

Generate the English user's documentation in all formats.

```
cmake -D DOC_ALL=ON ..
make doc
```

Generate the user's documentation in PDF format in English and in Spanish.

```
cmake -D DOC_PDF=ON -D ES=ON ..
make doc
```

1.6 Installation from Binaries

1.6.1 Debian-based Linux Distributions

Support for Debian-based Linux systems, such as Ubuntu and Arch Linux, is being developed.

1.6.2 Windows

Since PostGIS version 3.3.3, MobilityDB is distributed in the PostGIS Bundle for Windows, which is available on application stackbuilder and OSGeo website. For more information, refer to the [PostGIS documentation](#).

1.7 Support

MobilityDB community support is available through the MobilityDB github page, documentation, tutorials, mailing lists and others.

1.7.1 Reporting Problems

Bugs are reported and managed in an [issue tracker](#). Please follow these steps:

1. Search the tickets to see if your problem has already been reported. If so, add any extra context you might have found, or at least indicate that you too are having the problem. This will help us prioritize common issues.
2. If your problem is unreported, create a [new issue](#) for it.
3. In your report include explicit instructions to replicate your issue. The best tickets include the exact SQL necessary to replicate a problem. Please also, note the operating system and versions of MobilityDB, PostGIS, and PostgreSQL.
4. It is recommended to use the following wrapper on the problem to pin point the step that is causing the problem.

```
SET client_min_messages TO debug;  
<your code>  
SET client_min_messages TO notice;
```

1.7.2 Mailing Lists

There are two mailing lists for MobilityDB hosted on OSGeo mailing list server:

- User mailing list: <http://lists.osgeo.org/mailman/listinfo/mobilitydb-users>
- Developer mailing list: <http://lists.osgeo.org/mailman/listinfo/mobilitydb-dev>

For general questions and topics about how to use MobilityDB, please write to the user mailing list.

1.8 Migrating from Version 1.2 to Version 1.3

MobilityDB 1.3 is a major revision with respect to version 1.2. The most important change in version 1.3 was to enable new spatiotemporal types. While version 1.2 defined the spatiotemporal types `tgeompoint` (temporal geometry point), `tgeogpoint` (temporal geography point), and `tnpoint` (temporal network point), version 1.3 added the new types `tgeometry` (temporal geometry), `tgeography` (temporal geography), `tcbuffer` (temporal circular buffer), `tpose` (temporal pose), and `trgeometry` (temporal rigid geometry). At a conceptual level, all the spatiotemporal types are organized in the hierarchy depicted in Figure 7.1.

In order to enable the above, a refactoring of the code base was necessary. All the types provided by version 1.2 in addition to the new types `tgeometry` and `tgeography` are defined as core types and thus, are always included in the building process. Each one of the other spatiotemporal types can be included in the building process by setting its corresponding compilation flag. For example, when adding `-DCBUFFER=1` the types built upon the `cbuffer` base type, that is `cbuffer`, `cbufferset`, and `tcbuffer`, will be included in the building process. The possibility to select the types to be included in the executable program is important in particular for stream processing, due to the limited capability of the edge devices. Please notice that the new types `tcbuffer`, `tpose`, and `trgeometry` are still in development and will be finalized in a forthcoming release.

In addition, the API of MobilityDB and MEOS was extended with new functionality and streamlined to improve usability. Finally, version 1.3 also enables the latest versions PostgreSQL 18 and PostGIS 3.6.0. For all these reasons, the binary format of the temporal types have changed from version 1.2 to 1.3 and thus a backup and restore is needed for migrating to the newer version 1.3 of MobilityDB.

Chapter 2

Set and Span Types

MobilityDB provides *set*, *span*, and *span set* types for representing set of values another type, which is called the *base type*. Set types are akin to *array types* in PostgreSQL restricted to one dimension, but enforce the constraint that sets do not have duplicates. Span and span set types in MobilityDB correspond to the *range and multirange types* in PostgreSQL but have additional constraints. In particular, span types in MobilityDB are of fixed length and do not allow empty spans and infinite bounds. While span types provide similar functionality to range types, they enable increasing performance. In particular, the overhead of processing variable-length types is removed and, in addition, pointer arithmetics and binary search can be used.

The base types used for constructing set, span, and span set types are the types `integer`, `bigint`, `float`, `text`, `date`, and `timestamptz` (timestamp with time zone) provided by PostgreSQL, the types `geometry` and `geography` provided by PostGIS, and the types `cbuffer` (circular buffer, see Chapter 13) and `npoint` (network point, see Chapter 12) provided by MobilityDB. MobilityDB provides the following set and span types:

- `set`: `intset`, `bigintset`, `floatset`, `textset`, `dateset`, `tstzset`, `geomset`, `geogset`, `cbufferset`, `npointset`.
- `span`: `intspan`, `bigintspan`, `floatspan`, `datespan`, `tstzspan`.
- `spanset`: `intspanset`, `bigintspanset`, `floatspanset`, `datespanset`, `tstzspanset`.

We present next the functions and operators for set and span types. These functions and operators are polymorphic, that is, their arguments may be of several types, and the result type may depend on the type of the arguments. To express this in the signature of the functions and operators, we use the following notation:

- `set` represents any set type, such as `intset` or `tstzset`.
- `span` represents any span type, such as `intspan` or `tstzspanset`.
- `spanset` represents any span set type, such as `intspanset` or `tstzspanset`.
- `spans` represents any span or span set type, such as `intspan` or `tstzspanset`.
- `base` represents any base type of a set or span type, such as `integer` or `timestamptz`
- `number` represents any base type of a number span type, such as `integer` or `float`,
- `numset` represents any number set type, such as `intset` or `floatset`.
- `numspans` represents any number span type, such as `intspan` or `floatspanset`.
- `numbers` represents any number set or range type, such as `integer`, `intset`, `intspan`, or `intspanset`,
- `dates` represents any time type with date granularity, that is, `date`, `dateset`, `datespan`, or `datespanset`,
- `times` represents any time type with `timestamptz` granularity, that is, `timestamptz`, `tstzset`, `tstzspan`, or `tstzspanset`,

- A set of types such as `{set, spans}` represents any of the types listed,
- A set of operators such as `{=, <>}` represents any of the operators listed,
- `type[]` represents an array of `type`.

As an example, the signature of the contains operator (`@>`) is as follows:

```
{set, spans} @> {set, spans, base} → boolean
```

Notice that the signature above is an abridged version of the more precise signature below

```
set @> {set, base} → boolean
spans @> {spans, base} → boolean
```

since sets and spans cannot be mixed in operations and thus, for instance, we cannot ask whether a span contains a set. In the following, for conciseness, we use the abridged style of signatures above. Furthermore, the time part of the timestamps is omitted in most examples. Recall that in that case PostgreSQL assumes the time `00:00:00`.

In what follows, since span and span set types have similar functions and operators, when we speak about span types we mean both span and span set types, unless we explicitly refer to *unit* span types and span *set* types to distinguish them.

2.1 Input and Output

MobilityDB generalizes Open Geospatial Consortium's (OGC) Well-Known Text (WKT) and Well-Known Binary (WKB) input and output format for all its types. In this way, applications can exchange data between them using a standardized exchange format. The WKT format is human-readable while the WKB format is more compact and more efficient than the WKT format. The WKB format can be output either as a binary string or as a character string encoded in hexadecimal ASCII.

The set types represent an *ordered* set of *distinct* values. A set must contain at least one element. Examples of set values are as follows:

```
SELECT tstzset '{2001-01-01 08:00:00, 2001-01-03 09:30:00}';
-- Singleton set
SELECT textset '{"highway"}';
-- Erroneous set: unordered elements
SELECT floatset '{3.5, 1.2}';
-- Erroneous set: duplicate elements
SELECT geomset '{"Point(1 1)", "Point(1 1)}';
-- Circular-buffer set: each element is a cbuffer WKT
SELECT cbufferset '{"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)}';
```

Notice that the elements of the sets `textset`, `geomset`, `geogset`, `cbufferset`, and `npointset` must be enclosed between double quotes. Notice also that geometries and geographies follow the order defined in PostGIS.

A value of a unit span type has two bounds, the *lower bound* and the *upper bound*, which are values of the underlying *base type*. For example, a value of the `tstzspan` type has two bounds, which are `timestamp_tz` values. The bounds can be inclusive or exclusive. An inclusive bound means that the boundary instant is included in the span, while an exclusive bound means that the boundary instant is not included in the span. In the text form of a span value, inclusive and exclusive lower bounds are represented, respectively, by “[” and “(”. Likewise, inclusive and exclusive upper bounds are represented, respectively, by “]” and “)”. In a span value, the lower bound must be less than or equal to the upper bound. A span value with equal and inclusive bounds is called an *instantaneous span* and corresponds to a base type value. Examples of span values are as follows:

```
SELECT intspan '[1, 3)';
SELECT floatspan '[1.5, 3.5)';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-03 09:30:00)';
-- Instant spans
SELECT intspan '[1, 1]';
SELECT floatspan '[1.5, 1.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00]';
```

```
-- Erroneous span: invalid bounds
SELECT tstzspan '[2001-01-01 08:10:00, 2001-01-01 08:00:00]';
-- Erroneous span: empty span
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00]';
```

Values of `intspan`, `bigintspan`, and `datespan` are converted into *normal form* so that equivalent values have identical representations. In the canonical representation of these types, the lower bound is inclusive and the upper bound is exclusive as shown in the following examples:

```
SELECT intspan '[1, 1]';
-- [1, 2)
SELECT bigintspan '(1, 3]';
-- [2, 4)
SELECT datespan '[2001-01-01, 2001-01-03]';
-- [2001-01-01, 2001-01-04)
```

A value of a span set type represents an *ordered* set of *disjoint* span values. A span set value must contain at least one element, in which case it corresponds to a single span value. Examples of span set values are as follows:

```
SELECT floatspanset '{{[8.1, 8.5],[9.2, 9.4]}}';
-- Singleton spanset
SELECT tstzspanset '{{[2001-01-01 08:00:00, 2001-01-01 08:10:00]}}';
-- Erroneous spanset: unordered elements
SELECT intspanset '{{[3,4],[1,2]}}';
-- Erroneous spanset: overlapping elements
SELECT tstzspanset '{{[2001-01-01 08:00:00, 2001-01-01 08:10:00],
[2001-01-01 08:05:00, 2001-01-01 08:15:00]}}';
```

Values of the span set types are converted into *normal form* so that equivalent values have identical representations. For this, consecutive adjacent span values are merged when possible. Examples of transformation into normal form are as follows:

```
SELECT intspanset '{{[1,2],[3,4]}}';
-- {[1, 5)}
SELECT floatspanset '{{[1.5,2.5],[2.5,4.5]}}';
-- {[1.5, 4.5]}
SELECT tstzspanset '{{[2001-01-01 08:00:00, 2001-01-01 08:10:00],
[2001-01-01 08:10:00, 2001-01-01 08:10:00], (2001-01-01 08:10:00, 2001-01-01 08:20:00]}}';
-- {[2001-01-01 08:00:00+00,2001-01-01 08:20:00+00]}
```

We give next the functions for input and output of set and span types in Well-Known Text and Well-Known Binary format. The default output format of all set and span types is the Well-Known Text format. The function `asText` given next enables to determine the output of floating point values.

- Return the Well-Known Text (WKT) representation

```
asText({floatset, floatspans}, maxdecdigits=15) → text
```

The `maxdecdigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

```
SELECT asText(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT asText(floatspanset '{{[1.55,2.55],[4,5]}}', 0);
-- {[2, 3], [4, 5]}
```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

```
asBinary({set, spans}, endian text='') → bytea
```

```
asHexWKB({set, spans}, endian text='') → text
```

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```

SELECT asBinary(datasets '{2001-01-01, 2001-01-03}');
-- \x010500010200000006e01000070010000
SELECT asBinary(intspan '[1, 3]');
-- \x011300010100000003000000
SELECT asBinary(floatspanset '{{[1, 2], [4, 5]}}', 'XDR');
-- \x00000e00000002033ff000000000000040000000000000034010000000000004014000000000000
SELECT asHexWKB(datasets '{2001-01-01, 2001-01-03}');
-- 010500010200000006E01000070010000
SELECT asHexWKB(intspan '[1, 3]');
-- 011300010100000003000000
SELECT asHexWKB(floatspanset '{{[1, 2], [4, 5]}}', 'XDR');
-- 00000E00000002033FF0000000000000400000000000003401000000000004014000000000000

```

- Input from the Well-Known Binary (WKB) representation

settypeFromBinary(bytea) → set

spantypeFromBinary(bytea) → span

spansettypeFromBinary(bytea) → spanset

There is one function per set or span (set) type, the name of the function has as prefix the name of the type

```

SELECT datasetsFromBinary('\x010500010200000006e01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromBinary('\x011300010100000003000000');
-- [1, 3]
SELECT floatspansetFromBinary(
  '\x00000e00000002033ff0000000000000400000000000003401000000000004014000000000000');
-- {[1, 2], [4, 5]}

```

- Input from the Hexadecimal Well-Known Binary (HexWKB) representation

settypeFromHexWKB(text) → set

spantypeFromHexWKB(text) → span

spansettypeFromHexWKB(text) → spanset

There is one function per set or span (set) type, the name of the function has as prefix the name of the type.

```

SELECT datasetsFromHexWKB('010500010200000006E01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromHexWKB('011300010100000003000000');
-- [1, 3]
SELECT floatspanFromHexWKB('01060001000000000000F83F000000000000440');
-- [1.5, 2.5]
SELECT floatspansetFromHexWKB(
  '00000E00000002033FF0000000000000400000000000003401000000000004014000000000000');
-- {[1, 2], [4, 5]}

```

2.2 Constructors

The constructor function for the set types has a single argument that is an array of values of the corresponding base type. The values must be ordered and cannot have nulls or duplicates.

- Constructor for set types

set(base[]) → set

```

SELECT set(ARRAY['highway', 'primary', 'secondary']);
-- {"highway", "primary", "secondary"}
SELECT set(ARRAY[timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00']);
-- {2001-01-01 08:00:00+00, 2001-01-03 09:30:00+00}

```


The unit span types have a constructor function that accepts four arguments. The first two arguments specify, respectively, the lower and upper bound, and the last two arguments are Boolean values stating, respectively, whether the lower and upper bounds are inclusive or not. The last two arguments are assumed to be, respectively, true and false if not specified. Notice that integer spans are transformed into *normal form*, that is, with inclusive lower bound and exclusive upper bound.

- Constructor for span types

`span(lower base, upper base, leftInc bool=true, rightInc bool=false) → span`

```
SELECT span(20.5, 25);
-- [20.5, 25)
SELECT span(20, 25, false, true);
-- [21, 26)
SELECT span(timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00', false, true);
-- (2001-01-01 08:00:00, 2001-01-03 09:30:00]
```

The constructor function for span set types have a single argument that is an array of span values of the same subtype.

- Constructor for span set types

`spanset(span[]) → spanset`

```
SELECT spanset(ARRAY[intspan '[10,12]', '[13,15]']);
-- {[10, 16)}
SELECT spanset(ARRAY[floatspan '[10.5,12.5]', '[13.5,15.5]']);
-- {[10.5, 12.5], [13.5, 15.5]}
SELECT spanset(ARRAY[tstzspan '[2001-01-01 08:00, 2001-01-01 08:10]',
                              '[2001-01-01 08:20, 2001-01-01 08:40]']);
-- {[2001-01-01 08:00, 2001-01-01 08:10], [2001-01-01 08:20, 2001-01-01 08:40]}
```

2.3 Conversions

Values of set and span types can be converted to one another or converted to and from PostgreSQL range types using the function `CAST` or using the `::` notation.

- Convert a base value to a set, span, or span set value

`base:: {set, span, spanset}`

`set(base) → set`

`span(base) → span`

`spanset(base) → spanset`

```
SELECT CAST(timestamptz '2001-01-01 08:00:00' AS tstzset);
-- {2001-01-01 08:00:00}
SELECT timestamptz '2001-01-01 08:00:00'::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:00:00]
SELECT spanset(timestamptz '2001-01-01 08:00:00');
-- {[2001-01-01 08:00:00, 2001-01-01 08:00:00]}
```

- Convert a set value to a span set value

`set::spanset`

`spanset(set) → spanset`

```
SELECT spanset(tstzset '{2001-01-01 08:00:00, 2001-01-01 08:15:00,
                        2001-01-01 08:25:00}');
/* {[2001-01-01 08:00:00, 2001-01-01 08:00:00],
    [2001-01-01 08:15:00, 2001-01-01 08:15:00],
    [2001-01-01 08:25:00, 2001-01-01 08:25:00]} */
```

- Convert a span value to a span set value

`span::spanset`

`spanset(span) → spanset`

```
SELECT floatspan '[1.5,2.5]':floatspanset;
-- {[1.5, 2.5]}
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00]':tstzspanset;
-- {[2001-01-01 08:00:00, 2001-01-01 08:30:00]}
```

- Convert a set or a span set into a span, ignoring the potential time gaps

`{set, spanset}::span`

`span({set, spanset}) → span`

```
SELECT span(dateset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- [2001-01-01, 2001-01-06]
SELECT span(tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}');
-- [2001-01-01, 2001-01-04]
```

- Convert a span value to and from a PostgreSQL range value

`span::range`

`range::span`

`range(span) → range`

`span(range) → span`

Notice that PostgreSQL range values accept empty ranges and ranges with infinite values, which are not allowed as span values in MobilityDB

```
SELECT intspan '[10, 20]':int4range;
-- [10,20)
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00]':tstzrange;
-- ["2001-01-01 08:00:00","2001-01-01 08:30:00")
SELECT int4range '[10, 20)':intspan;
-- [10,20)
SELECT int4range 'empty':intspan;
-- ERROR: Range cannot be empty
SELECT int4range '[10,)':intspan;
-- ERROR: Range bounds cannot be infinite
SELECT tstzrange '[2001-01-01 08:00:00, 2001-01-01 08:30:00]':tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:30:00]
```

- Convert a span set value to and from a PostgreSQL multirange value

`spanset::multirange`

`multirange::spanset`

`multirange(spanset) → multirange`

`spanset(multirange) → spanset`

```
SELECT intspanset '{{[1,2],[4,5]]':int4multirange;
-- {[1,3), [4,6)}
SELECT tstzspanset '{{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]]':tstzmultirange;
-- {[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}
SELECT int4multirange '{{[1,2],[4,5]]':intspanset;
-- {[1, 3), [4, 6)}
SELECT tstzmultirange '{{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]]':tstzspanset;
-- {[2001-01-01, 2001-01-02], [2001-01-04, 2001-01-05]}
```

2.4 Accessors

- Return the memory size in bytes

`memSize({set, spanset}) → integer`

```
SELECT memSize(tstzset '{2001-01-01, 2001-01-02, 2001-01-03}');
-- 48
SELECT memSize(tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04],
[2001-01-05, 2001-01-06]}');
-- 112
```

- Return the lower or upper bound

`lower(spans) → base`

`upper(spans) → base`

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
-- 1
```

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-01
SELECT upper(intspanset '{[1,2],[4,5]}');
-- 6
SELECT lower(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
-- 1
```

```
SELECT upper(floatspan '[20.5, 25.3]');
-- 25.3
SELECT upper(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-05
```

- Is the lower or upper bound inclusive?

`lowerInc(spans) → boolean`

`upperInc(spans) → boolean`

```
SELECT lowerInc(datespan '[2001-01-01, 2001-01-05)');
-- true
SELECT lowerInc(intspanset '{[1,2],[4,5]}');
-- true
```

```
SELECT upper(floatspan '[20.5, 25.3]');
-- true
SELECT upperInc(tstzspan '[2001-01-01, 2001-01-05)');
-- false
```

- Return the width of the span as a float

`width(numspan) → float`

`width(numspanset, boundspan=false) → float`

An additional parameter can be set to true to compute the width of the bounding span, thus ignoring the potential value gaps

```
SELECT width(floatspan '[1, 3)');
-- 2
SELECT width(intspanset '{{[1,3), [5,7)}}');
-- 4
SELECT width(intspanset '{{[1,3), [5,7)}}', true);
-- 6
```

- Return the duration

`duration({datespan,tstzspan}) → interval`

`duration({datespanset,tstzspanset},boundspan bool=false) → interval`

An additional parameter can be set to true to compute the duration of the bounding time span, thus ignoring the potential time gaps

```
SELECT duration(datespan '[2001-01-01, 2001-01-03)');
-- 2 days
SELECT duration(tstzspanset '{{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}}');
-- 3 days
SELECT duration(tstzspanset '{{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}}', true);
-- 4 days
```

- Return the (number of) values

`numValues(set) → integer`

`getValues(set) → base[]`

```
SELECT numValues(intset '{1,3,5,7}');
-- 4
SELECT getValues(tstzset '{{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}}');
-- {"2001-01-01","2001-01-03","2001-01-05","2001-01-07"}
```

- Return the start, end, or n-th value

`startValue(set) → base`

`endValue(set) → base`

`valueN(set,integer) → base`

```
SELECT startValue(intset '{1,3,5,7}');
-- 1
SELECT endValue(dateset '{{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}}');
-- 2001-01-07
SELECT valueN(floatset '{1,3,5,7}',2);
-- 3
```

- Return the (number of) spans

`numSpans(spanset) → integer`

`spans(spanset) → span[]`

```
SELECT numSpans(intspanset '{{[1,3), [4,5), [6,7)}}');
-- 3
SELECT numSpans(datespanset '{{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05),
[2001-01-06, 2001-01-07)}}');
-- 3
```

```
SELECT spans(floatspanset '{{[1,3), [4,4), [6,7)}}');
-- {"[1,3)","[4,4)","[6,7)"}
SELECT spans(tstzspanset '{{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
[2001-01-05, 2001-01-06)}}');
-- {"[2001-01-01,2001-01-03)","[2001-01-04,2001-01-04)","[2001-01-05,2001-01-06)"}

```

- Return the start, end, or n-th span

`startSpan(spanset) → span`

`endSpan(spanset) → span`

`spanN(spanset, integer) → span`

```
SELECT startSpan(intspanset '{[1,3],[4,5],[6,7]}');
-- [1,3)
SELECT startSpan(datespanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05),
[2001-01-06, 2001-01-07)}');
-- [2001-01-01,2001-01-03)
```

```
SELECT endSpan(floatspanset '{[1,3],[4,4],[6,7]}');
-- [6,7)
SELECT endSpan(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
[2001-01-05, 2001-01-06)}');
-- [2001-01-05,2001-01-06)
```

```
SELECT spanN(floatspanset '{[1,3],[4,4],[6,7]}',2);
-- [4,4]
SELECT spanN(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
[2001-01-05, 2001-01-06)}', 2);
-- [2001-01-04,2001-01-04]
```

- Return the (number of) different dates

`numDates(datespanset) → integer`

`dates(datespanset) → dateset`

```
SELECT numDates(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 4
SELECT dates(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04}
```

- Return the start, end, or n-th date

`startDate(datespanset) → date`

`endDate(datespanset) → date`

`dateN(datespanset, integer) → date`

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startDate(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 2001-01-01
SELECT endDate(datespanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 2001-01-05
SELECT dateN(datespanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

- Return the (number of) different timestamps

`numTimestamps(tstzspanset) → integer`

`timestamps(tstzspanset) → tstzset`

```
SELECT numTimestamps(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 3
SELECT timestamps(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- {"2001-01-01 00:00:00", "2001-01-03 00:00:00", "2001-01-05 00:00:00"}
```

- Return the start, end, or n-th timestamp

`startTimestamp(tstzspanset) → timestamptz`

`endTimestamp(tstzspanset) → timestamptz`

`timestampN(tstzspanset, integer) → timestamptz`

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startTimestamp(tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04]}');
-- 2001-01-01
SELECT endTimestamp(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05]}');
-- 2001-01-05
SELECT timestampN(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

2.5 Transformations

- Expand or shrink the bounds by a value or an interval

`expand(numspan, base) → numspan`

`expand(tstzspan, interval) → tstzspan`

The function returns NULL if the value or interval given as second argument is negative and the span resulting from shifting the bounds with the argument is empty.

```
SELECT expand(floatspan '[1, 3]', 1);
-- [0, 4]
SELECT expand(floatspan '[1, 3]', -1);
-- [2, 2]
SELECT expand(floatspan '[1, 3]', -1);
-- NULL
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
-- [2000-12-31, 2001-01-04]
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-1 day');
-- [2001-01-02, 2001-01-02]
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-2 day');
-- NULL
```

- Shift by a value or interval

`shift(numbers, base) → numbers`

`shift(dates, integer) → dates`

`shift(times, interval) → times`

```
SELECT shift(dateset '{2001-01-01, 2001-01-03, 2001-01-05}', 1);
-- {2001-01-02, 2001-01-04, 2001-01-06}
SELECT shift(intspan '[1, 4]', -1);
-- [0, 3]
SELECT shift(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
-- [2001-01-02, 2001-01-04]
SELECT shift(floatspanset '{[1, 2], [3, 4]}', -1);
-- {[0, 1], [2, 3]}
SELECT shift(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}',
interval '1 day');
-- {[2001-01-02, 2001-01-04], [2001-01-05, 2001-01-06]}
```

- Scale by a value or interval

`scale(numbers, base) → numbers`

`scale(dates, integer) → dates`

`scale(times,interval) → times`

If the width or time span of the input value is zero (for example, for a singleton timestamp set), the result is the input value. The given value or interval must be strictly greater than zero.

```
SELECT scale(tstzset '{2001-01-01}', '1 day');
-- {2001-01-01}
SELECT scale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '2 days');
-- {2001-01-01, 2001-01-02, 2001-01-03}
SELECT scale(intspan '[1, 4)', 4);
-- [1, 6)
SELECT scale(datespan '[2001-01-01, 2001-01-04)', 4);
-- [2001-01-01, 2001-01-06)
SELECT scale(tstzspan '[2001-01-01, 2001-01-03]', '1 day');
-- [2001-01-01, 2001-01-02]
SELECT scale(floatspanset '{{[1, 2], [3, 4]}}', 6);
-- {[1, 3], [5, 7]}
SELECT scale(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}', '1 day');
/* {[2001-01-01 00:00:00, 2001-01-01 12:00:00],
   [2001-01-01 18:00:00, 2001-01-02 00:00:00]} */
SELECT scale(tstzset '{2001-01-01}', '-1 day');
-- ERROR: The duration must be a positive interval: -1 days
```

- Shift and scale by the values or intervals

`shiftScale(numbers,base,base) → numbers`

`shiftScale(dates,integer,integer) → dates`

`shiftScale(times,interval,interval) → times`

This function combines the functions **shift** and **scale**.

```
SELECT shiftScale(tstzset '{2001-01-01}', '1 day', '1 day');
-- {2001-01-02}
SELECT shiftScale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '1 day','2 days');
-- {2001-01-02, 2001-01-03, 2001-01-04}
SELECT shiftScale(intspan '[1, 4)', -1, 4);
-- [0, 5)
SELECT shiftScale(datespan '[2001-01-01, 2001-01-04)', -1, 4);
-- [2001-12-31, 2001-01-05)
SELECT shiftScale(tstzspan '[2001-01-01, 2001-01-03]', '1 day', '1 day');
-- [2001-01-02, 2001-01-03]
SELECT shiftScale(floatspanset '{{[1, 2], [3, 4]}}', -1, 6);
-- {[0, 2], [4, 6]}
SELECT shiftScale(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}',
  '1 day', '1 day');
/* {[2001-01-02 00:00:00, 2001-01-02 12:00:00],
   [2001-01-02 18:00:00, 2001-01-03 00:00:00]} */
```

- Round down or up to the nearest integer

`floor({floatset,floatspans}) → {floatset,floatspans}`

`ceil({floatset,floatspans}) → {floatset,floatspans}`

```
SELECT floor(floatset '{1.5,2.5}');
-- {1, 2}
SELECT ceil(floatspan '[1.5,2.5)');
-- [2, 3)
SELECT floor(floatspan '(1.5, 1.6)');
-- [1, 1]
SELECT ceil(floatspanset '{{[1.5, 2.5],[3.5,4.5]}}');
-- {[2, 3], [4, 5]}
```

- Round to a number of decimal places

`round({floatset,floatspans},integer=0) → {floatset,floatspans}`

```
SELECT round(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT round(floatspan '[1.123456789,2.123456789]', 3);
-- [1.123,2.123]
SELECT round(floatspan '[1.123456789, inf]', 3);
-- [1.123,Infinity)
SELECT round(floatspanset '{[1.123456789, 2.123456789],[3.123456789,4.123456789]}', 3);
-- {[1.123, 2.123], [3.123, 4.123]}
```

- Convert to degrees or radians

`degrees({floatset,floatspans}, normalize=false) → {floatset,floatspans}`

`radians({floatset,floatspans}) → {floatset,floatspans}`

The additional parameter in the degrees function can be used to normalize the values between 0 and 360 degrees.

```
SELECT round(degrees(floatset '{0, 0.5, 0.7, 1.0}', true), 3);
-- {0, 28.648, 40.107, 57.296}
SELECT round(radians(floatspanset ' {[0, 45], [90, 135]}'), 3);
-- {[0, 0.785], [1.571, 2.356]}
```

- Transform to lowercase, uppercase, or initcap

`lower(textset) → textset`

`upper(textset) → textset`

`initcap(textset) → textset`

```
SELECT lower(textset '{"AAA", "BBB", "CCC"}');
-- {"aaa", "bbb", "ccc"}
SELECT upper(textset '{"aaa", "bbb", "ccc"}');
-- {"AAA", "BBB", "CCC"}
SELECT initcap(textset '{"aaa", "bbb", "ccc"}');
-- {"Aaa", "Bbb", "Ccc"}
```

- Text concatenation

`{text,textset} || {text,textset} → textset`

```
SELECT textset '{aaa, bbb}' || text 'XX';
-- {"aaaXX", "bbbXX"}
SELECT text 'XX' || textset '{aaa, bbb}';
-- {"XXaaa", "XXbbb"}
```

- Set the temporal precision of the time value to the interval with respect to the origin

`tprecision(times,interval,origin timestampz='2000-01-03') → times`

If the origin is not specified, it is set by default to Monday, January 3, 2000

```
SELECT tprecision(timestampz '2001-12-03', '30 days');
-- 2001-11-23
SELECT tprecision(timestampz '2001-12-03', '30 days', '2001-12-01');
-- 2001-12-01
SELECT tprecision(tstzset '{2001-01-01 08:00, 2001-01-01 08:10, 2001-01-01 09:00,
  2001-01-01 09:10}', '1 hour');
-- {"2001-01-01 08:00:00+01", "2001-01-01 09:00:00+01"}
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-01 09:00]', '1 day');
-- [2001-12-01, 2001-12-02)
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-15 09:00]', '1 day');
-- [2001-12-01, 2001-12-16)
```



```
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
[2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02)}
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
[2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02)}
```

2.6 Spatial Reference System

- Return or set the spatial reference identifier

SRID(spatialset) → integer

setSRID(spatialset) → spatialset

```
SELECT SRID(geomset '{"Point(1 1)", "Point(2 2)}"');
-- 0
SELECT SRID(geogset '{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 4326
SELECT SRID(geomset 'SRID=5676;{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 5676
SELECT asEWKT(setSRID(geomset '{Point(1 1), Point(2 2)}', 5676));
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(setSRID(poseset '{"Pose(Point(1 1),1)", "Pose(Point(2 2),3)}"', 5676));
-- SRID=5676;{"Pose(POINT(2 2),3)", "Pose(POINT(1 1),1)"}
SELECT SRID(cbuffer(geomset 'SRID=4326;{"Cbuffer(Point(1 1),0.5)",
"Cbuffer(Point(2 2),1.0)}"');
-- 4326
SELECT asEWKT(setSRID(cbuffer(geomset '{"Cbuffer(Point(1 1),0.5)",
"Cbuffer(Point(2 2),1.0)}"', 3812));
-- SRID=3812;{"Cbuffer(POINT(1 1),0.5)", "Cbuffer(POINT(2 2),1)"}

```

- Transform to a spatial reference identifier

transform(spatialset, to_srid integer) → spatialset

transformPipeline(spatialset, pipeline text, to_srid integer, is_forward bool=true) → spatialset

The transform function specifies the transformation with a target SRID. An error is raised when the input set has an unknown SRID (represented by 0). The transformPipeline function specifies the transformation with a coordinate transformation pipeline in the following format:

urn:ogc:def:coordinateOperation:AUTHORITY::CODE

The SRID of the input set is ignored, and the SRID of the output set will be set to zero unless a value is provided via the optional to_srid parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(geomset 'SRID=4326;{"Point(2.340088 49.400250),
Point(6.575317 51.553167)}', 3812), 6);
-- SRID=3812;{"POINT(502773.429981 511805.120402)", "POINT(803028.908265 751590.742629)"}
WITH test(geomset, pipeline) AS (
  SELECT geogset 'SRID=4326;{"Point(4.3525 50.846667 100.0)",
    "Point(-0.1275 51.507222 100.0)}"',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(geomset, pipeline, 4326), pipeline,
4326, false), 6)
FROM test;
-- SRID=4326;{"POINT Z (4.3525 50.846667 100)", "POINT Z (-0.1275 51.507222 100)"}

```

2.7 Set Operations

The set and span types have associated set operators, namely union, difference, and intersection, which are represented, respectively by +, -, and *. The set operators for the set and span types are given next.

- Union, difference, or intersection of sets or spans

set {+, -, *} set → set

spans {+, -, *} spans → spans

```
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' +
  dateset '{2001-01-03, 2001-01-06}';
-- {2001-01-01, 2001-01-03, 2001-01-05, 2001-01-06}
SELECT intspan '[1, 3]' + intspan '[3, 5]';
-- [1, 5]
SELECT floatspan '[1, 3]' + floatspan '[4, 5]';
-- {[1, 3), [4, 5)}
SELECT tstzspanset ' {[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}' +
  tstzspan '[2001-01-03, 2001-01-04]';
-- {[2001-01-01, 2001-01-05)}
```

```
SELECT intset '{1, 3, 5}' - intset '{3, 6}';
-- {1, 5}
SELECT datespan '[2001-01-01, 2001-01-05)' - datespan '[2001-01-03, 2001-01-07)';
-- {[2001-01-01, 2001-01-03)}
SELECT floatspan '[1, 5]' - floatspan '[3, 4]';
-- {[1, 3), (4, 5]}
SELECT tstzspanset ' {[2001-01-01, 2001-01-06], [2001-01-07, 2001-01-10]} ' -
  tstzspanset ' {[2001-01-02, 2001-01-03], [2001-01-04, 2001-01-05],
    [2001-01-08, 2001-01-09]} ' ;
/* {[2001-01-01,2001-01-02), (2001-01-03,2001-01-04), (2001-01-05,2001-01-06],
    [2001-01-07,2001-01-08), (2001-01-09,2001-01-10)} */
```

```
SELECT tstzset '{2001-01-01, 2001-01-03}' * tstzset '{2001-01-03, 2001-01-05}';
-- {2001-01-03}
SELECT intspan '[1, 5]' * intspan '[3, 6]';
-- [3, 5]
SELECT floatspanset ' {[1, 5), [6, 8)} ' * floatspan '[1, 6]';
-- {[1, 5)}
SELECT tstzspan '[2001-01-01, 2001-01-05)' * tstzspan '[2001-01-03, 2001-01-07)';
-- [2001-01-03, 2001-01-05)
```

2.8 Bounding Box Operations

2.8.1 Topological Operations

The topological operations available for the set and span types are given next.

- Do the values overlap (have values in common)?

{set, spans} && {set, spans} → boolean

```
SELECT intset '{1, 3}' && intset '{2, 3, 4}';
-- true
SELECT floatspan '[1, 3]' && floatspan '[3, 4]';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-05)' && tstzspan '[2001-01-02, 2001-01-07)';
-- true
```

```
SELECT floatspanset '{{[1, 5],[6, 8]}}' && floatspan '[1, 6]';
-- true
```

- Does the first value contain the second one?

{set, spans} @> {base, set, spans} → boolean

```
SELECT floatset '{1.5, 2.5}' @> 2.5;
-- true
SELECT tstzspan '[2001-01-01, 2001-05-01]' @> timestampz '2001-02-01';
-- true
SELECT floatspanset '{{[1, 2],[2, 3]}}' @> 2.0;
-- false
```

- Is the first value contained by the second one?

{base, set, spans} <@ {set, spans} → boolean

```
SELECT timestampz '2001-01-10' <@ tstzspan '[2001-01-01, 2001-05-01]';
-- true
SELECT floatspan '[2, 5]' <@ floatspan '[1, 5]';
-- false
SELECT tstzspan '[2001-02-01, 2001-03-01]' <@ tstzspan '[2001-01-01, 2001-05-01]';
-- true
SELECT floatspanset '{{[1,2],[3,4]}}' <@ floatspan '[1, 6]';
-- true
```

- Is the first value adjacent to the second one?

spans -|- spans → boolean

```
SELECT intspan '[2, 6]' -|- intspan '[6, 7]';
-- true
SELECT floatspan '[2, 5]' -|- floatspan '(5, 6)';
-- false
SELECT floatspanset '{{[2, 3],[4, 5]}}' -|- floatspan '(5, 6)';
-- true
SELECT tstzspanset '{{[2001-01-01, 2001-01-02]}}' -|- tstzspan '[2001-01-02, 2001-01-03]';
-- false
```

2.8.2 Position Operations

The position operations available for set and span types are given next. Notice that the operators for time types have an additional # to distinguish them from the operators for number types.

- Is the first value strictly left of the second one?

numbers << numbers → boolean

times <<# times → boolean

```
SELECT intspan '[15, 20]' << 20;
-- true
SELECT intspanset '{{[15, 17],[18, 20]}}' << 20;
-- true
SELECT floatspan '[15, 20]' << floatspan '(15, 20)';
-- false
SELECT dateset '{2001-01-01, 2001-01-02}' <<# dateset '{2001-01-03, 2001-01-05}';
-- true
```

- Is the first value strictly to the right of the second one?

numbers >> numbers → boolean

times #>> times → boolean

```
SELECT intspan '[15, 20)' >> 10;
-- true
SELECT floatspan '[15, 20)' >> floatspan '[5, 10]';
-- true
SELECT floatspanset '{{[15, 17], [18, 20}}' >> floatspan '[5, 10]';
-- true
SELECT tstzspan '[2001-01-04, 2001-01-05)' #>>
  tstzspanset '{{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06}}';
-- true
```

- Is the first value not to the right of the second one?

numbers &< numbers → boolean

times &<# times → boolean

```
SELECT intspan '[15, 20)' &< 18;
-- false
SELECT intspanset '{{[15, 16], [17, 18}}' &< 18;
-- true
SELECT floatspan '[15, 20)' &< floatspan '[10, 20]';
-- true
SELECT dateset '{2001-01-02, 2001-01-05}' &<# dateset '{2001-01-01, 2001-01-04}';
-- false
```

- Is the first value not to the left of the second one?

numbers &> numbers → boolean

times #&> times → boolean

```
SELECT intspan '[15, 20)' &> 30;
-- true
SELECT floatspan '[1, 6]' &> floatspan '(1, 3)';
-- false
SELECT floatspanset '{{[1, 2], [3, 4}}' &> floatspan '(1, 3)';
-- false
SELECT timestamp '2001-01-01' #&> tstzspan '[2001-01-01, 2001-01-05)';
-- true
```

2.8.3 Splitting Operations

When creating indexes for set or span set types, what is stored in the index is not the actual value but instead, a bounding box that *represents* the value. In this case, the index will provide a list of candidate values that *may* satisfy the query predicate, and a second step is needed to filter out candidate values by computing the query predicate on the actual values.

However, when the bounding boxes have a large empty space not covered by the actual values, the index will generate many candidate values that do not satisfy the query predicate, which reduces the efficiency of the index. In these situations, it may be better to represent a value not with a *single* bounding box, but instead with *multiple* bounding boxes. This increases considerably the efficiency of the index, provided that the index is able to manage multiple bounding boxes per value. The following functions are used for generating multiple spans from a single set or span set value.

- Return an array of N spans obtained by merging the elements of a set or the spans of a spanset

splitNSpans(set, integer) → span[]

splitNSpans(spanset, integer) → span[]

The last argument specifies the number of output spans. If the number of input elements or spans is less than the given number, the resulting array will have one span per input element or span. Otherwise, the given number of output spans will be obtained by merging several consecutive input elements or spans.

```
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
/* {"[1, 11)"} */
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 3);
-- {"[1, 5)", "[5, 8)", "[8, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 3)", "[3, 5)", "[5, 7)", "[7, 9)", "[9, 10)", "[10, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
/* {"[1, 2)", "[2, 3)", "[3, 4)", "[4, 5)", "[5, 6)", "[6, 7)", "[7, 8)", "[8, 9)",
    "[9, 10)", "[10, 11)"} */
```

```
SELECT splitNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}');
-- {"[1, 2)", "[3, 4)", "[5, 6)", "[7, 8)", "[9, 10)"}
SELECT splitNSpans(floatspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 4)", "[5, 8)", "[9, 10)"}
SELECT splitNSpans(datespanset '{[2000-01-01, 2000-01-04), [2000-01-05, 2000-01-10)}', 3);
-- {"[2000-01-01, 2000-01-04)", "[2000-01-05, 2000-01-10)"}

```

- Return an array of spans obtained by merging N consecutive elements of a set or N consecutive spans of a spanset

`splitEachNSpans(set, integer) → span[]`

`splitEachNSpans(spanset, integer) → span[]`

The last argument specifies the number of input elements that are merged to produce an output span. If the number of input elements is less than the given number, the resulting array will have one output span per element. Otherwise, the given number of consecutive input elements will be merged into a single output span in the answer. Notice that, contrary to the `splitNSpans` function, the number of spans in the result depends on the number of input elements or spans.

```
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
/* {"[1, 2)", "[2, 3)", "[3, 4)", "[4, 5)", "[5, 6)", "[6, 7)", "[7, 8)", "[8, 9)",
    "[9, 10)", "[10, 11)"} */
SELECT splitEachNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 6)", "[7, 10)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 7)", "[7, 11)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
-- {"[1, 11)"}

```

2.9 Distance Operations

The distance operator `<->` for set and span types consider the bounding span and returns a the smallest distance between the two values. In the case of time values, the operator returns the number of days or the number of seconds between the two time values. The distance operator can also be used for nearest neighbor searches using a GiST or an SP-GiST index (see Section 2.12).

- Return the smallest distance ever

`numbers <-> numbers → base`

`dates <-> dates → integer`

`times <-> times → float`

```
SELECT 3 <-> intspan '[6, 8)';
-- 3
SELECT floatspan '[1, 3]' <-> floatspan '(5.5, 7]';
-- 2.5
SELECT floatspan '[1, 3]' <-> floatspanset '{(5.5, 7], [8, 9]}';

```

```
-- 2.5
SELECT tstzspan '[2001-01-02, 2001-01-06)' <-> timestamptz '2001-01-07';
-- 86400
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' <->
    dateset '{2001-01-02, 2001-01-04}';
-- 0
```

2.10 Comparisons

The comparison operators (`=`, `<`, and so on) require that the left and right arguments be of the same type. Excepted equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on set and span types. For span values, the operators compare first the lower bound, then the upper bound. For set and span set values, the operators compare first the bounding spans, and if those are equal, they compare the first *N* values or spans, where *N* is the minimum of the number of composing values or spans of both values.

The comparison operators available for the set and span types are given next. Recall that integer spans are always represented by their canonical form.

- Traditional comparisons

```
set {=, <>, <, >, <=, >=} set → boolean
spans {=, <>, <, >, <=, >=} spans → boolean
cmp(set, set) → int
cmp(span, span) → int
cmp(spanset, spanset) → int
```

Functions `cmp`, `cmp`, and `cmp` return -1, 0, or 1 depending on whether the first argument is, respectively, less than, equal to, or greater than the second one.

```
SELECT intspan '[1,3]' = intspan '[1,4)';
-- true
SELECT floatspanset '{{[1, 2), [2, 3])}' = floatspanset '{{[1, 3])}';
-- true
SELECT tstzset '{2001-01-01, 2001-01-04}' <> tstzset '{2001-01-01, 2001-01-05}';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-04)' <> tstzspan '[2001-01-03, 2001-01-05)';
-- true
SELECT floatspan '[3, 4]' < floatspan '(3, 4]';
-- true
SELECT intspanset '{{[1, 2), [3, 4])}' < intspanset '{{[3, 4])}';
-- true
SELECT floatspan '[3, 4]' > floatspan '[3, 4)';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-04)' > tstzspan '[2001-01-02, 2001-01-05)';
-- true
SELECT floatspanset '{{[1, 4])}' <= floatspanset '{{[1, 5), [6, 7])}';
-- true
SELECT tstzspanset '{{[2001-01-01, 2001-01-04])}' <=
    tstzspanset '{{[2001-01-01, 2001-01-05), [2001-01-06, 2001-01-07])}';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-05)' >= tstzspan '[2001-01-03, 2001-01-04)';
-- true
SELECT intspanset '{{[1, 4])}' >= intspanset '{{[1, 5), [6, 7])}';
-- false
SELECT cmp(intspanset '{{[1, 4])}', intspanset '{{[1, 5), [6, 7])}');
-- -1
```

2.11 Aggregations

There are several aggregate functions defined for set and span types. They are described next.

- Function `extent` returns a bounding span that encloses a set of set or span values.
- Union is a very useful operation for set and span types. As we have seen in Section 2.7, we can compute the union of two set or span values using the `+` operator. However, it is also very useful to have an aggregate version of the union operator for combining an arbitrary number of values. Functions `setUnion`, `spanUnion`, and `spansetUnion` can be used for this purpose.
- Function `tCount` generalizes the traditional aggregate function `count`. The temporal count can be used to compute at each point in time the number of available objects (for example, number of spans). Function `tCount` returns a temporal integer (see Chapter 4). The function has two optional parameters that specify the granularity (an `interval`) and the origin of time (a `timestamp_tz`). When these parameters are given, the temporal count is computed at time bins of the given granularity (see Section 9.6).

- Bounding span

`extent({set, spans}) → span`

```
WITH spans(r) AS (
  SELECT floatspan '[1, 4]' UNION SELECT floatspan '(5, 8)' UNION
  SELECT floatspan '(7, 9)' )
SELECT extent(r) FROM spans;
-- [1,9)
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT extent(ts) FROM times;
-- [2001-01-01, 2001-01-06]
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}' UNION
  SELECT tstzspanset '{[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06]}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-06]}' )
SELECT extent(ps) FROM periods;
-- [2001-01-01, 2001-01-06]
```

- Aggregate union

`setUnion({value, set}) → set`

`spanUnion(spans) → spanset`

`spansetUnion(spansets) → spanset`

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT setUnion(ts) FROM times;
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04, 2001-01-05, 2001-01-06}
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-03], [2001-01-05, 2001-01-06]}' UNION
  SELECT tstzspanset '{[2001-01-07, 2001-01-08]}' )
SELECT spansetUnion(ps) FROM periods;
-- {[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06], [2001-01-07, 2001-01-08]}
```

- Temporal count

`tCount(times) → {tintSeq, tintSeqSet}`

```

WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT tCount(ts) FROM times;
-- {2@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-04, 1@2001-01-05, 1@2001-01-06}

WITH periods(ps) AS (
  SELECT tstzspanset ' {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}' UNION
  SELECT tstzspanset ' {[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06)}' UNION
  SELECT tstzspanset ' {[2001-01-02, 2001-01-06)}' )
SELECT tCount(ps) FROM periods;
-- {[2@2001-01-01, 3@2001-01-03, 1@2001-01-04, 2@2001-01-05, 2@2001-01-06)}

```

2.12 Indexing

GiST and SP-GiST indexes can be created for table columns of the set and span types. The GiST index implements an R-tree while the SP-GiST index implements a quad-tree. An example of creation of a GiST index in a column `During` of type `tstzspan` in a table `Reservation` is as follows:

```

CREATE TABLE Reservation (ReservationID integer PRIMARY KEY, RoomID integer,
  During tstzspan);
CREATE INDEX Reservation_During_Idx ON Reservation USING GIST(During);

```

A GiST or an SP-GiST index can accelerate queries involving the following operators: `=`, `&&`, `<@`, `@>`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<#`, `#>>`, `&<#`, `#&>`, and `<->`.

In addition, B-tree indexes can be created for table columns of a set or span types. For these index types, basically the only useful operation is equality. There is a B-tree sort ordering defined for values of span time types with corresponding `<`, `<=`, `>`, and `>=` operators, but the ordering is rather arbitrary and not usually useful in the real world. The B-tree support is primarily meant to allow sorting internally in queries, rather than creation of actual indexes.

Finally, hash indexes can be created for table columns of a set or span types. For these types of indexes, the only operation defined is equality.

Chapter 3

Bounding Box Types

We present next the functions and operators for bounding box types. These functions and operators are polymorphic, that is, their arguments can be of various types and their result type may depend on the type of the arguments. To express this in the signature of the operators, we use the following notation:

- `box` represents any bounding box type, that is, `tbox` or `stbox`.

3.1 Input and Output

MobilityDB generalizes Open Geospatial Consortium's Well-Known Text (WKT) and Well-Known Binary (WKB) input and output format for all temporal types. We present next the functions for input and output box types.

A `tbox` is composed of a numeric and/or time dimensions. For each dimension, a span given, that is, one of an `intspan`, `bigintspan`, or a `floatspan` for the value dimension and a `tstzspan` for the time dimension. Examples of input of `tbox` values are as follows:

```
-- Both value and time dimensions
SELECT tbox 'TBOXINT XT([1,3],[2001-01-01,2001-01-02]);'
SELECT tbox 'TBOXBIGINT XT([1,3],[2001-01-01,2001-01-02]);'
SELECT tbox 'TBOXFLOAT XT([1.5,2.5],[2001-01-01,2001-01-02]);'
-- Only value dimension
SELECT tbox 'TBOXINT X([1,3]);'
SELECT tbox 'TBOXBIGINT X([1,3]);'
SELECT tbox 'TBOXFLOAT X([1.5,2.5]);'
-- Only time dimension
SELECT tbox 'TBOX T([2001-01-01,2001-01-02]);'
```

An `stbox` is composed of a spatial and/or time dimensions, where the coordinates of the spatial dimension may be 2D or 3D. For the time dimension a `tstzspan` is given and for the spatial dimension minimum and maximum coordinate values are given, where the latter may be Cartesian (planar) or geodetic (spherical). The SRID of the coordinates may be specified; if it is not the case, a value of 0 (unknown) and 4326 (corresponding to WGS84) is assumed, respectively, for planar and geodetic boxes. Geodetic boxes always have a Z dimension to account for the curvature of the underlying sphere or spheroid. Examples of input of `stbox` values are as follows:

```
-- Only value dimension with X and Y coordinates
SELECT stbox 'STBOX X((1.0,2.0),(1.0,2.0));'
-- Only value dimension with X, Y, and Z coordinates
SELECT stbox 'STBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0));'
-- Both value (with X and Y coordinates) and time dimensions
SELECT stbox 'STBOX XT(((1.0,2.0),(1.0,2.0)), [2001-01-03,2001-01-03]);'
-- Both value (with X, Y, and Z coordinates) and time dimensions
SELECT stbox 'STBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)), [2001-01-01,2001-01-03]);'
```

```
-- Only time dimension
SELECT stbox 'STBOX T([2001-01-03,2001-01-03])';
-- Only value dimension with X, Y, and Z geodetic coordinates
SELECT stbox 'GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Both value (with X, Y and Z geodetic coordinates) and time dimension
SELECT stbox 'GEODSTBOX ZT((1.0,2.0,3.0),(1.0,2.0,3.0)), [2001-01-04,2001-01-04]';
-- Only time dimension for geodetic box
SELECT stbox 'GEODSTBOX T([2001-01-03,2001-01-03])';
-- SRID is given
SELECT stbox 'SRID=5676;STBOX XT(((1.0,2.0),(1.0,2.0)), [2001-01-04,2001-01-04])';
SELECT stbox 'SRID=4326;GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
```

We give next the functions for input and output of box types in Well-Known Text and Well-Known Binary format.

- Return the Well-Known Text (WKT) representation

`asText(box,maxdecdigits=15) → text`

The `maxdecdigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

```
SELECT asText(tbox 'TBOXFLOAT XT([1.123456789,2.123456789],[2001-01-01,2001-01-02])', 3);
-- TBOXFLOAT XT([1.123, 2.123],[2001-01-01 00:00:00+01, 2001-01-02 00:00:00+01])
SELECT asText(stbox 'STBOX Z((1.55,1.55,1.55),(2.55,2.55,2.55))', 0);
-- STBOX Z((2,2,2),(3,3,3))
```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

`asBinary(box,endian text='') → bytea`

`asHexWKB(box,endian text='') → text`

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])');
-- \x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])', 'XDR');
-- \x000300270100001cc1d3579c0000001cd5f12efc00000d013ff00000000000004000000000000000
SELECT asBinary(stbox 'STBOX X((1,1),(2,2))');
-- \x0101000000000000f03f0000000000000040000000000000f03f0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])');
-- 0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])', 'XDR');
-- 000300270100001cc1d3579c0000001cd5f12efc00000d013ff00000000000004000000000000000
SELECT asHexWKB(stbox 'STBOX X((1,1),(2,2))');
-- 0101000000000000f03f0000000000000040000000000000f03f0000000000000040
```

- Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

`boxFromBinary(bytea) → box`

`boxFromHexWKB(text) → box`

In the signatures above, `box` replaces any box type, that is, `tbox` or `stbox`.

```
SELECT tboxFromBinary(
  '\x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040');
-- TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])
SELECT stboxFromBinary(
  '\x0101000000000000f03f0000000000000040000000000000f03f0000000000000040');
-- STBOX X((1,1),(2,2))
SELECT tboxFromHexWKB(
  '0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040');
-- TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])
```

```
SELECT stboxFromHexWKB(
  '0101000000000000F03F00000000000004000000000000F03F000000000000040');
-- STBOX X((1,1),(2,2))
```

3.2 Constructors

Type `tbox` has several constructor functions depending on whether the value and/or the time extent are given. The value extent can be specified by a number or a span, while the time extent can be specified by a time type.

- Constructor for `tbox`

```
tbox({number,numspan}) → tbox
tbox({timestampz,tstzspan}) → tbox
tbox({number,numspan},{timestampz,tstzspan}) → tbox
```

```
-- Both value and time dimensions
SELECT tbox(1.0, timestampz '2001-01-01');
SELECT tbox(floatspan '[1.0, 2.0]', tstzspan '[2001-01-01,2001-01-02]');
-- Only value dimension
SELECT tbox(floatspan '[1.0,2.0]');
-- Only time dimension
SELECT tbox(tstzspan '[2001-01-01,2001-01-02]');
```

Type `stbox` has several constructor functions depending on whether the space and/or the time extent are given. The coordinates for the spatial extent can be 2D or 3D and can be either Cartesian or geodetic. The spatial extent can be specified by the minimum and maximum coordinate values. The SRID can be specified in an optional last argument. If not given, a value 0 (respectively 4326) is assumed by default for planar (respectively geodetic) boxes. The spatial extent can also be specified by a geometry or a geography. The temporal extent can be specified by a time type.

- Constructor for `stbox`

```
stboxX(float,float,float,float,srid=0) → stbox
stboxZ(float,float,float,float,float,float,srid=0) → stbox
stboxT({timestampz,tstzspan}) → stbox
stboxXT(float,float,float,float,{timestampz,tstzspan},srid=0) → stbox
stboxZT(float,float,float,float,float,float,{timestampz,tstzspan},srid=0) → stbox
geodstboxZ(float,float,float,float,float,float,srid=4326) → stbox
geodstboxT({timestampz,tstzspan}) → stbox
geodstboxZT(float,float,float,float,float,float,{timestampz,tstzspan},srid=4326)
→ stbox
stbox(geo) → stbox
stbox(geo,{timestampz,tstzspan}) → stbox
```

```
-- Only value dimension with X and Y coordinates
SELECT stboxX(1.0,2.0,1.0,2.0);
-- Only value dimension with X, Y, and Z coordinates
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Only value dimension with X, Y, and Z coordinates and SRID
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0,5676);
-- Only time dimension
SELECT stboxT(tstzspan '[2001-01-03,2001-01-03]');
-- Both value (with X and Y coordinates) and time dimensions
SELECT stboxXT(1.0,2.0, 1.0,2.0, tstzspan '[2001-01-03,2001-01-03]');
```

```
-- Both value (with X, Y, and Z coordinates) and time dimensions
SELECT stboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-03]');
-- Only value dimension with X, Y, and Z geodetic coordinates
SELECT geodstboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Only time dimension for geodetic box
SELECT geodstboxT(tstzspan '[2001-01-03,2001-01-03]');
-- Both value (with X, Y, and Z geodetic coordinates) and time dimensions
SELECT geodstboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-04]');
-- Geometry and time dimension
SELECT stbox(geometry 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03,2001-01-05]');
-- Geography and time dimension
SELECT stbox(geography 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03,2001-01-05]');
```

3.3 Conversions

- Convert a tbox to another type

tbox::{intspan,floatspan,tstzspan}

intspan(tbox) → tbox

floatspan(tbox) → tbox

tstzspan(tbox) → tbox

```
SELECT tbox 'TBOXINT XT([1,4],[2001-01-01,2001-01-02])'::intspan;
-- [1,4)
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])'::floatspan;
-- (1, 2)
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])'::tstzspan;
-- [2001-01-01, 2001-01-02)
```

- Convert another type to a tbox

{numbers,times,tnumber}::tbox

tbox({numbers,times,tnumber}) → tbox

```
SELECT intset '{1,2}'::tbox;
-- TBOXINT X([1, 3))
SELECT intspan '[1,3]'::tbox;
-- TBOXINT X([1, 3))
SELECT floatspan '(1.0,2.0)'::tbox;
-- TBOXFLOAT X((1, 2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::tbox;
-- TBOX T((2001-01-01,2001-01-04))
```

- Convert an stbox to a another type

stbox::{box2d,box3d,geo,tstzspan}

box2d(stbox) → box2d

box3d(stbox) → box2d

geometry(stbox) → geometry

geography(stbox) → geography

tstzspan(stbox) → tstzspan

```
SELECT stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)), [2001-01-01,2001-01-03])'::box2d;
-- BOX(1 2,3 4)
SELECT ST_AsEWKT(stbox 'SRID=4326;STBOX XT(((1,1),(5,5)), [2001-01-01,2001-01-05])'::
  geometry);
-- SRID=4326;POLYGON((1 1,1 5,5 5,5 1,1 1))
```

```

SELECT ST_AsEWKT(stbox 'STBOX XT((1,1),(1,5)), [2001-01-01,2001-01-05]]'::geometry);
-- LINESTRING(1 1,1 5)
SELECT ST_AsEWKT(stbox 'GEODSTBOX XT((1,1),(1,1)), [2001-01-01,2001-01-05]]'::geography);
-- SRID=4326;POINT(1 1)
SELECT ST_AsEWKT(stbox 'STBOX ZT((1,1,1),(5,5,5)), [2001-01-01,2001-01-05]]'::
  geometry);
/* POLYHEDRALSURFACE(((1 1 1,1 5 1,5 5 1,5 1 1,1 1 1)),
  ((1 1 5,5 1 5,5 5 5,1 5 5,1 1 5)), ((1 1 1,1 1 5,1 5 5,1 5 1,1 1 1)),
  ((5 1 1,5 5 1,5 5 5,5 1 5,5 1 1)), ((1 1 1,5 1 1,5 1 5,1 1 5,1 1 1)),
  ((1 5 1,1 5 5,5 5 5,5 5 1,1 5 1))) */
SELECT stbox 'STBOX XT((1.0,2.0),(3.0,4.0)), [2001-01-01,2001-01-03]]'::tstzspan;
-- [2001-01-01, 2001-01-03]

```

- Convert another type to an stbox

```

{box2d,box3d,geo,time,tgeo}::stbox
stbox({box2d,box3d,geo,time,tgeo}) → stbox

```

```

SELECT geometry 'Linestring(1 1 1,2 2 2)'::box3d::stbox;
-- STBOX Z((1,1,1),(2,2,2))
SELECT geography 'Linestring(1 1,2 2)'::stbox;
-- SRID=4326;GEODSTBOX X((1,1),(2,2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::stbox;
-- STBOX T((2001-01-01,2001-01-04))

```

3.4 Accessors

- Has X/Z/T dimension?

```
hasX(box) → boolean
```

```
hasZ(stbox) → boolean
```

```
hasT(box) → boolean
```

```

SELECT hasX(tbox 'TBOX T([2001-01-01,2001-01-03])');
-- false
SELECT hasX(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- true
SELECT hasZ(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false
SELECT hasT(tbox 'TBOXFLOAT XT((1.0,3.0), [2001-01-01,2001-01-03])');
-- true
SELECT hasT(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false

```

- Is geodetic?

```
isGeodetic(stbox) → boolean
```

```

SELECT isGeodetic(stbox 'GEODSTBOX Z((1.0,1.0,0.0),(3.0,3.0,1.0))');
-- true
SELECT isGeodetic(stbox 'STBOX XT((1.0,2.0),(3.0,4.0)), [2001-01-01,2001-01-02])');
-- false

```

- Return the minimum X/Y/Z/T value

```
xMin(box) → float
```

```
yMin(stbox) → float
```

```
zMin(stbox) → float
```

```
tMin(box) → timestamptz
```

```

SELECT xMin(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- 1
SELECT yMin(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 2
SELECT zMin(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 3
SELECT tMin(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-01

```

Notice that for `tbox` the result value for `xMin` and `xMin` is converted to a float for the temporal boxes with an integer span.

- Return the maximum X/Y/Z/T value

`xMax(box) → float`

`yMax(stbox) → float`

`zMax(stbox) → float`

`tMax(box) → timestampz`

```

SELECT xMax(tbox 'TBOXINT X([1,4))');
-- 3
SELECT yMax(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 4
SELECT zMax(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 6
SELECT tMax(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-03

```

Notice that for `tbox` the result value for `xMin` and `xMin` is converted to a float for the temporal boxes with an integer span.

- Is the minimum X/T value inclusive?

`xMinInc(tbox) → bool`

`tMinInc(box) → bool`

```

SELECT xMinInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMinInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true

```

- Is the maximum X/T value inclusive?

`xMaxInc(tbox) → bool`

`tMaxInc(box) → bool`

```

SELECT xMaxInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMaxInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true

```

- Area, volume, perimeter

`area(stbox, spheroid bool=true) → float`

`volume(stbox) → float`

`perimeter(stbox, spheroid bool=true) → float`

For geodetic boxes, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used. The `volume` function do not accept geodetic boxes.

```

SELECT area(stbox 'STBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03]))';
-- 4
SELECT volume(stbox 'STBOX ZT(((1,1,1),(3,3,3)), [2001-01-01,2001-01-03]))';
-- 8
SELECT perimeter(stbox 'STBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03]))';
-- 8
SELECT area(stbox 'GEODSTBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03]))';
-- 49209676328.36632
SELECT perimeter(stbox 'GEODSTBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03])', false);
-- 889221.9544681838

```

3.5 Transformations

- Shift and/or scale the span of a bounding box by one or two values

shiftValue(box, {integer, float}) → box

scaleValue(box, {integer, float}) → box

shiftScaleValue(tbox, {integer, float}, {integer, float}) → box

```

SELECT shiftValue(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])', 1.0);
-- TBOXFLOAT XT([2.5, 3.5], [2001-01-01, 2001-01-02])
SELECT scaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])', 2.0);
-- TBOXFLOAT XT([1.5, 3.5], [2001-01-01, 2001-01-02])
SELECT shiftScaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])', 2.0, 3.0);
-- TBOXFLOAT XT([3.5, 6.5], [2001-01-01, 2001-01-02])

```

- Shift and/or scale the span or the period of the bounding box to a value or interval

shiftTime(box, interval) → box

scaleTime(box, interval) → box

shiftScaleTime(box, interval, interval) → box

For scaling, if the width or the time span of the period is zero (that is, the lower and upper bound are equal), the result is the box. The given value or interval must be strictly greater than zero.

```

SELECT shiftTime(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])',
  interval '1 day');
-- TBOXFLOAT XT([1.5, 2.5], [2001-01-02, 2001-01-03])
SELECT shiftTime(stbox 'STBOX T([2001-01-01,2001-01-02])', interval '-1 day');
-- STBOX T([2001-12-31, 2001-01-01])
SELECT shiftTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '1 day');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-02, 2001-01-03])
SELECT scaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])',
  interval '2 days');
-- TBOXFLOAT XT([1.5, 2.5], [2001-01-01, 2001-01-03])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '1 hour');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01 00:00:00, 2001-01-01 01:00:00])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '-1 day');
-- ERROR: The interval must be positive: -1 days
SELECT shiftScaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])',
  interval '1 day', interval '3 days');
-- TBOXFLOAT XT([1.5, 2.5], [2001-01-02, 2001-01-05])
SELECT shiftScaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '1 hour', interval '3 hours');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01 01:00:00, 2001-01-01 04:00:00])

```

- Return the spatial dimension of the bounding box, removing the temporal dimension if any

`getSpace(stbox) → stbox`

```
SELECT getSpace(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-03])');
-- STBOX Z((1,1,1),(2,2,2))
```

The functions given next expand the bounding boxes on the value and the time dimension or set the precision of the value dimension. These functions raise an error if the corresponding dimension is not present.

- Expand the numeric, spatial, or temporal dimension of a bounding box by a value or an interval

`expandValue(tbox,{integer,float}) → tbox`

`expandSpace(stbox,float) → stbox`

`expandTime(box,interval) → box`

The function returns NULL if the value or interval given as second argument is negative and the span resulting from shifting the bounds with the argument is empty.

```
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-03])', 1.0);
-- TBOXFLOAT XT((0,3), [2001-01-01,2001-01-03])
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-03])', -1.0);
-- NULL
SELECT expandValue(tbox 'TBOX T([2001-01-01,2001-01-03])', 1);
-- The box must have value dimension
```

```
SELECT expandSpace(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-03])', 1);
-- STBOX ZT(((0,0,0),(3,3,3)), [2001-01-01,2001-01-03])
SELECT expandSpace(stbox 'STBOX T([2001-01-01,2001-01-03])', 1);
-- The box must have space dimension
```

```
SELECT expandTime(tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-03])', interval '1 day');
-- TBOXFLOAT XT((1,2), [2000-12-31,2001-01-04])
SELECT expandTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-03])',
  interval '-1 day');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-02,2001-01-02])
SELECT expandTime(tbox 'TBOX XT((1,2), [2001-01-01,2001-01-03])', interval '-2 days');
-- NULL
```

- Round the value or the coordinates of the bounding box to a number of decimal places

`round(box,integer=0) → box`

```
SELECT round(tbox 'TBOXFLOAT XT((1.12345,2.12345), [2001-01-01,2001-01-02])', 2);
-- TBOXFLOAT XT((1.12,2.12), [2001-01-01, 2001-01-02])
SELECT round(stbox 'STBOX XT(((1.12345, 1.12345), (2.12345, 2.12345)),
  [2001-01-01,2001-01-02])', 2);
-- STBOX XT(((1.12,1.12),(2.12,2.12)), [2001-01-01, 2001-01-02])
SELECT round(tstzspan '[2000-01-01, 2001-01-02]'::tbox);
-- The tbox must have X dimension
```

3.6 Spatial Reference System

- Return or set the spatial reference identifier

`SRID(stbox) → integer`

`setSRID(stbox) → stbox`


```

SELECT SRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)), [2001-01-01,2001-01-02]))';
-- 0
SELECT SRID(stbox 'SRID=5676;STBOX XT(((1.0,2.0),(4.0,5.0)), [2001-01-01,2001-01-02]))';
-- 5676
SELECT SRID(stbox 'GEODSTBOX T([2001-01-01,2001-01-02]))';
-- ERROR: The box must have space dimension

SELECT setSRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)),
[2001-01-01,2001-01-02]), 5676);
-- SRID=5676;STBOX ZT(((1,2,3),(4,5,6)), [2001-01-01,2001-01-02])

```

- Transform to a spatial reference identifier

`transform(stbox, to_srid integer) → stbox`

`transformPipeline(stbox, pipeline text, to_srid integer, is_forward bool=true) → stbox`

The `transform` function specifies the transformation with a target SRID. An error is raised when the input box has an unknown SRID (represented by 0). The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input box is ignored, and the SRID of the output box will be set to zero unless a value is provided via the optional `to_srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```

SELECT round(transform(stbox 'SRID=4326;STBOX XT(((2.340088, 49.400250),
(6.575317, 51.553167)), [2001-01-01,2001-01-02]), 3812), 6);
/* SRID=3812;STBOX XT(((502773.429981,511805.120402),(803028.908265,751590.742629)),
[2001-01-01, 2001-01-02]) */
WITH test(box, pipeline) AS (
  SELECT stbox 'SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))',
  text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(box, pipeline, 4326),
pipeline, 4326, false), 6)
FROM test;
-- SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))

```

3.7 Splitting Operations

- Split the bounding box in quadrants or octants { }

`quadSplit(stbox) → {stbox}`

As indicated by the { } symbol, this function is a *set-returning function* (also known as a *table function*) since it typically return more than one value.

```

SELECT quadSplit(stbox 'STBOX XT(((0,0),(4,4)), [2001-01-01,2001-01-05]))';
/* {"STBOX XT(((0,0),(2,2)), [2001-01-01, 2001-01-05])",
"STBOX XT(((2,0),(4,2)), [2001-01-01, 2001-01-05])",
"STBOX XT(((0,2),(2,4)), [2001-01-01, 2001-01-05])",
"STBOX XT(((2,2),(4,4)), [2001-01-01, 2001-01-05])"} */
SELECT quadSplit(stbox 'STBOX Z((0,0,0),(4,4,4))');
/* {"STBOX Z((0,0,0),(2,2,2))", "STBOX Z((2,0,0),(4,2,2))", "STBOX Z((0,2,0),(2,4,2))",
"STBOX Z((2,2,0),(4,4,2))", "STBOX Z((0,0,2),(2,2,4))", "STBOX Z((2,0,2),(4,2,4))",
"STBOX Z((0,2,2),(2,4,4))", "STBOX Z((2,2,2),(4,4,4))"} */

```

This function is typically used for multiresolution grids, where the space is split in cells such that the cells have a maximum number of elements. Figure 3.1 shows an example of the result of using this function using synthetic trajectories in Brussels.

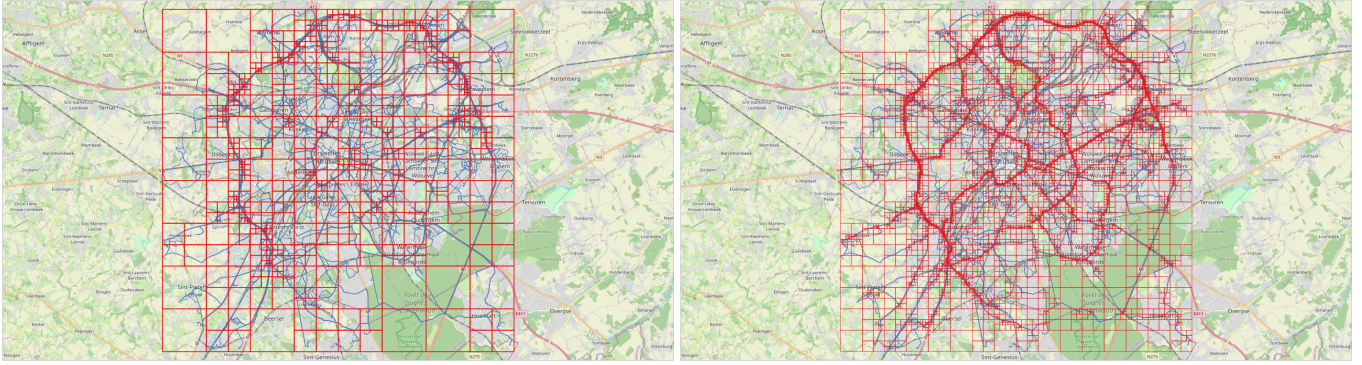


Figure 3.1: Multiresolution grid on Brussels data obtained using the **BerlinMOD** generator. Each cell contains at most 10,000 (left) and 1,000 (right) instants across the entire simulation period (four days in this case). On the left, we can see the high density of the traffic in the ring around Brussels, while on the right we can see other main axes in the city.

3.8 Set Operations

The set operators for box types are union (+) and intersection (*). In the case of union, the operands must have exactly the same dimensions, otherwise an error is raised. Furthermore, if the operands do not overlap on all the dimensions and error is raised, since in this would result in a box with disjoint values, which cannot be represented. The operator computes the union on all dimensions that are present in both arguments. In the case of intersection, the operands must have at least one common dimension, otherwise an error is raised. The operator computes the intersection on all dimensions that are present in both arguments.

- Union and intersection of two bounding boxes

`box {+, *} box → box`

```
SELECT tbox 'TBOXINT XT([1,3],[2001-01-01,2001-01-03])' +
  tbox 'TBOXINT XT([2,4],[2001-01-02,2001-01-04])';
-- TBOXINT XT([1,4],[2001-01-01,2001-01-04])
SELECT stbox 'STBOX ZT((1,1,1),(2,2,2),[2001-01-01,2001-01-02])' +
  stbox 'STBOX XT((2,2),(3,3)),[2001-01-01,2001-01-03]';
-- ERROR: The arguments must be of the same dimensionality
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-02])' +
  tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- ERROR: Result of box union would not be contiguous
```

```
SELECT tbox 'TBOXINT XT([1,3],[2001-01-01,2001-01-03])' *
  tbox 'TBOX T([2001-01-02,2001-01-04])';
-- TBOX T([2001-01-02,2001-01-03])
SELECT stbox 'STBOX ZT((1,1,1),(3,3,3),[2001-01-01,2001-01-02])' *
  stbox 'STBOX X((2,2),(4,4))';
-- STBOX X((2,2),(3,3))
```

3.9 Bounding Box Operations

3.9.1 Topological Operations

There are five topological operators: overlaps (&&), contains (@>), contained (<@), same (~=), and adjacent (-|-). The operators verify the topological relationship between the bounding boxes taking into account the value and/or the time dimension for as many dimensions that are present on both arguments.

- Do the bounding boxes overlap?

`box && box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' &&
       tbox 'TBOXFLOAT XT((2,4),[2001-01-02,2001-01-04])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' &&
       stbox 'STBOX T([2001-01-02,2001-01-02])';
-- true
```

- Does the first bounding box contain the second one?

`box @> box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' @>
       tbox 'TBOXFLOAT XT((2,3),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' @>
       stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- Is the first bounding box contained in the second one?

`box <@ box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <@
       tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' <@
       stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])';
-- true
```

- Are the bounding boxes equal in their common dimensions?

`box ~= box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' ~=
       tbox 'TBOXFLOAT T([2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03])' ~=
       stbox 'STBOX Z((1,1,1),(3,3,3))';
-- true
```

- Are the bounding boxes adjacent?

`box -|- box → boolean`

Two boxes are adjacent if they share n dimensions and their intersection is at most of $n-1$ dimensions.

```
SELECT tbox 'TBOXINT XT([1,2],[2001-01-01,2001-01-02])' -|-
       tbox 'TBOXINT XT([2,3],[2001-01-02,2001-01-03])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' -|-
       tbox 'TBOX T([2001-01-02,2001-01-03])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03])' -|-
       stbox 'STBOX XT(((2,2),(4,4)),[2001-01-03,2001-01-04])';
-- true
```

3.9.2 Position Operations

The position operators consider the relative position of the bounding boxes. The operators `<<`, `>>`, `&<`, and `&>` consider the X value for the `tbox` type and the X coordinates for the `stbox` type, the operators `<<|`, `|>>`, `&<|`, and `|>>` consider the Y coordinates for the `stbox` type, the operators `<</`, `/>>`, `&</`, and `/>>` consider the Z coordinates for the `stbox` type, and the operators `<<#`, `#>>`, `#&<`, and `#&>` consider the time dimension for the `tbox` and `stbox` types. The operators raise an error if both boxes do not have the required dimension.

The operators for the numeric dimension of the `tbox` type are given next.

- Are the X/Y/Z/T values of the first bounding box strictly less than those of the second one?

`tbox {<<, <<#} tbox → boolean`

`stbox {<<, <<|, <</, <<#} stbox → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<
       tbox 'TBOXFLOAT T([2001-01-03,2001-01-04])';
-- ERROR: The box must have value dimension
SELECT stbox 'STBOX Z((1,1,1),(2,2,2))' <<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(2,2,2))' <</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<#
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- Are the X/Y/Z/T values of the first bounding box strictly greater than those of the second one?

`tbox {>>, #>>} tbox → boolean`

`stbox {>>, |>>, />>, #>>} stbox → boolean`

```
SELECT tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])' >>
       tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |>> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' />> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT(((3,3),(4,4)),[2001-01-03,2001-01-04])' #>>
       stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- Are the X/Y/Z/T values of the first bounding box not greater than those of the second one?

`tbox {&<, &<#} {tbox,stbox} → boolean`

`stbox {&<, &<|, &</, &<#} stbox → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' &<
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' &<#
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- Are the X/Y/Z/T values of the first bounding box not less than those of the second one?

tbox {&>, #&>} tbox → boolean

stbox {&>, |&>, /&>, #&>} stbox → boolean

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' &>
       tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- false
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' /&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT((1,1),(2,2),[2001-01-01,2001-01-02])' #&>
       stbox 'STBOX XT((1,1),(4,4),[2001-01-01,2001-01-04])';
-- true
```

3.10 Comparisons

The traditional comparison operators (=, <, and so on) can be applied to box types. Excepted equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on box types. These operators compare first the timestamps and if those are equal, compare the values.

- Traditional comparisons

box {=, <>, <, <=, >=} box → boolean

cmp(tbox,tbox) → int

cmp(stbox,stbox) → int

Functions cmp and cmp returns -1, 0, or 1 depending on whether the first argument is, respectively, less than, equal to, or greater than the second one.

```
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' =
       tbox 'TBOXINT XT([2,2],[2001-01-03,2001-01-05])';
-- false
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' <>
       tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <
       tbox 'TBOXINT XT([1,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-03,2001-01-04])' >
       tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <=
       tbox 'TBOXINT XT([2,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' >=
       tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])';
-- false
SELECT cmp(tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])',
          tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])');
-- -1
```

3.11 Aggregations

- Bounding box extent

extent(box) → box

```

WITH boxes(b) AS (
  SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' UNION
  SELECT tbox 'TBOXFLOAT XT((5,7),[2001-01-05,2001-01-07])' UNION
  SELECT tbox 'TBOXFLOAT XT((6,8),[2001-01-06,2001-01-08])' )
SELECT extent(b) FROM boxes;
-- TBOXFLOAT XT((1,8),[2001-01-01,2001-01-08])
WITH boxes(b) AS (
  SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' UNION
  SELECT stbox 'STBOX Z((5,5,5),(7,7,7))' UNION
  SELECT stbox 'STBOX Z((6,6,6),(8,8,8))' )
SELECT extent(b) FROM boxes;
-- STBOX Z((1,1,1),(8,8,8))

```

3.12 Indexing

GiST and SP-GiST indexes can be created for table columns of the `tbox` and `stbox` types. The GiST index implements an R-tree and the SP-GiST index implements an n-dimensional quad-tree. An example of creation of a GiST index in a column `Box` of type `stbox` in a table `Trips` is as follows:

```

CREATE TABLE Trips(TripID integer PRIMARY KEY, Trip tgeompoint, Box stbox);
CREATE INDEX Trips_Box_Idx ON Trips USING GIST(bbox);

```

A GiST or SP-GiST index can accelerate queries involving the following operators: `&&`, `<@`, `@>`, `~=`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<|`, `|>>`, `&<|`, `|&>`, `<</`, `/>>`, `&</`, `/&>`, `<<#`, `#>>`, `&<#`, and `#&>`.

In addition, B-tree indexes can be created for table columns of a bounding box type. For these index types, basically the only useful operation is equality. There is a B-tree sort ordering defined for values of bounding box types, with corresponding `<` and `>` operators, but the ordering is rather arbitrary and not usually useful in the real world. The B-tree support is primarily meant to allow sorting internally in queries, rather than creation of actual indexes.

Chapter 4

Temporal Types (Part 1)

4.1 Introduction

MobilityDB provides several temporal types based on PostgreSQL or PostGIS types, namely, `tbool`, `tint`, `tbigint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint`, and `tgeogpoint`, which are, respectively, based on the base types `bool`, `integer`, `bigint`, `float`, `text`, `geometry`, and `geography`, where `tgeometry` and `tgeography` accept arbitrary geometries/geographies, while `tgeompoint` and `tgeogpoint` only accept 2D or 3D points with Z dimension. In addition, MobilityDB provides other specific spatial types, namely, `cbuffer` (circular buffer), `npoint` (network point), and `pose`, and its corresponding spatiotemporal types, namely, `tcbuffer`, `tnpoint`, `tpose` and `trgeometry` (temporal rigid geometry). In this chapter and the next one we present the functions and operators available for all temporal types, while the specific functions for the spatiotemporal types will be given in subsequent chapters.

The *interpolation* of a temporal value states how the value evolves between successive instants. The interpolation is *discrete* when the value is unknown between two successive instants. They can represent, for example, checkins/checkouts when using an RFID card reader to enter or exit a building. The interpolation is *step* when the value remains constant between two successive instants. For example, the gear used by a moving car may be represented with a temporal integer, which indicates that its value is constant between two time instants. On the other hand, the interpolation is *linear* when the value evolves linearly between two successive instants. For example, the speed of a car may be represented with a temporal float, which indicates that the values are known at the time instants but continuously evolve between them. Similarly, the location of a vehicle may be represented by a temporal point where the location between two consecutive GPS readings is obtained by linear interpolation. Temporal types based on discrete base types, that is the `tbool`, `tint`, or `ttext` evolve necessarily in a step manner. On the other hand, temporal types based on continuous base types, that is `tfloat`, `tgeompoint`, or `tgeogpoint` may evolve in a step or linear manner. Note that the types `tgeometry` and `tgeography` only support discrete or step interpolation, since it is not possible to linearly interpolate two arbitrary geometries/geographies.

The *subtype* of a temporal value states the temporal extent at which the evolution of values is recorded. Temporal values come in three subtypes, explained next.

A temporal value of *instant* subtype (briefly, *an instant value*) represents the value at a time instant, for example

```
SELECT tfloat '17@2018-01-01 08:00:00';
```

A temporal value of *sequence* subtype (briefly, *a sequence value*) represents the evolution of the value during a sequence of time instants, where the values between these instants are interpolated using a discrete, step, or a linear function (see above). An example is as follows:

```
-- Discrete interpolation
SELECT tfloat '{17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00, 18@2018-01-01 08:10:00}';
-- Step interpolation
SELECT tfloat 'Interp=Step;(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00,
  15@2018-01-01 08:10:00)';
-- Linear interpolation
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00)';
```

As can be seen, a sequence value has a lower and an upper bound that can be inclusive (represented by '[' and ']') or exclusive (represented by '(' and ')'). By definition, both bounds must be inclusive when the interpolation is discrete or when the sequence has a single instant (called an *instantaneous sequence*), as the next example

```
SELECT tint '[10@2018-01-01 08:00:00]';
```

Sequence values must be *uniform*, that is, they must be composed of instant values of the same base type. Sequence values with step or linear interpolation are referred to as *continuous sequences*.

The value of a temporal sequence is interpreted by assuming that the period of time defined by every pair of consecutive values $v_1@t_1$ and $v_2@t_2$ is lower inclusive and upper exclusive, unless they are the first or the last instants of the sequence and in that case the bounds of the whole sequence apply. Furthermore, the value taken by the temporal sequence between two consecutive instants depends on whether the interpolation is step or linear. For example, the temporal sequence above represents that the value is 10 during (2018-01-01 08:00:00, 2018-01-01 08:05:00), 20 during [2018-01-01 08:05:00, 2018-01-01 08:10:00), and 15 at the end instant 2018-01-01 08:10:00. On the other hand, the following temporal sequence

```
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00)';
```

represents that the value evolves linearly from 10 to 20 during (2018-01-01 08:00:00, 2018-01-01 08:05:00) and evolves from 20 to 15 during [2018-01-01 08:05:00, 2018-01-01 08:10:00).

Finally, a temporal value of *sequence set* subtype (briefly, a *sequence set value*) represents the evolution of the value at a set of sequences, where the values between these sequences are unknown. An example is as follows:

```
SELECT tfloat '{[17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00],
[18@2018-01-01 08:10:00, 18@2018-01-01 08:15:00]}';
```

As shown in the above examples, sequence set values can only be of step or linear interpolation. Furthermore, all composing sequences of a sequence set value must be of the same base type and the same interpolation.

Continuous sequence values are converted into *normal form* so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. For step interpolation, three consecutive instant values can be merged into two if they have the same value. For linear interpolation, three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into normal form are as follows.

```
SELECT tint '[1@2001-01-01, 2@2001-01-03, 2@2001-01-04, 2@2001-01-05]';
-- [1@2001-01-01 00:00:00+00, 2@2001-01-03 00:00:00+00, 2@2001-01-05 00:00:00+00)
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
Point(1 1)@2001-01-01 08:10:00)');
-- [Point(1 1)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:10:00)
SELECT tfloat '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]';
-- [1@2001-01-01 00:00:00+00, 3@2001-01-05 00:00:00+00)
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(2 2)@2001-01-01 08:05:00,
Point(3 3)@2001-01-01 08:10:00]');
-- [Point(1 1)@2001-01-01 08:00:00, Point(3 3)@2001-01-01 08:10:00]
```

Similary, temporal sequence set values are converted into normal form. For this, consecutive sequence values are merged when possible. Examples of transformation into a normal form are as follows.

```
SELECT tint '{[1@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)]}';
-- '{[1@2001-01-01, 2@2001-01-03, 2@2001-01-05)]}'
SELECT tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-03, 3@2001-01-05)]}';
-- '{[1@2001-01-01, 3@2001-01-05)]}'
SELECT tfloat '{[1@2001-01-01, 3@2001-01-05), [3@2001-01-05)]}';
-- '{[1@2001-01-01, 3@2001-01-05)]}'
SELECT asText(tgeompoint '{[Point(0 0)@2001-01-01 08:00:00,
Point(1 1)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00),
[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)]}');
/* {[[Point(0 0)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
Point(1 1)@2001-01-01 08:15:00)] */
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,Point(2 2)@2001-01-01 08:05:00),
```



```
[Point(2 2)@2001-01-01 08:05:00, Point(3 3)@2001-01-01 08:10:00}');
-- {[Point(1 1)@2001-01-01 08:00:00, Point(3 3)@2001-01-01 08:10:00]}
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,Point(3 3)@2001-01-01 08:10:00),
[Point(3 3)@2001-01-01 08:10:00]}');
-- {[Point(1 1)@2001-01-01 08:00:00, Point(3 3)@2001-01-01 08:10:00]}
```

Temporal types support *type modifiers* (or *typmod* in PostgreSQL terminology), which specify additional information for a column definition. For example, in the following table definition:

```
CREATE TABLE Department (DeptNo integer, DeptName varchar(25), NoEmps tint (Sequence));
```

the type modifier for the type `varchar` is the value 25, which indicates the maximum length of the values of the column, while the type modifier for the type `tint` is the string `Sequence`, which restricts the subtype of the values of the column to be sequences. In the case of temporal alphanumeric types (that is, `tbool`, `tint`, `tbigint`, `tfloat`, and `ttext`), the possible values for the type modifier are `Instant`, `Sequence`, and `SequenceSet`. If no type modifier is specified for a column, values of any subtype are allowed.

On the other hand, in the case of temporal geometry types (that is, `tgeompoint`, `tgeogpoint`, `tgeometry`, or `tgeography`) the type modifier may be used to specify the subtype, the geometry type, and/or the spatial reference identifier (SRID). For example, in the following table definition:

```
CREATE TABLE Flight (FlightNo integer, Route tgeogpoint (Sequence, PointZ, 4326));
CREATE TABLE Storm (StormId integer, Route tgeography (Sequence, Linestring, 4326));
```

the type modifiers for the `tgeogpoint` and `tgeography` types are composed of three values, the first one indicating the subtype as above, the second one the spatial type of the geographies composing the temporal point and geographies, and the last one the SRID of the composing geographies. For temporal points, the possible values for the first argument of the type modifier are as above, those for the second argument are either `Point` or `PointZ`, and those for the third argument are valid SRIDs. All the three arguments are optional and if any of them is not specified for a column, values of any subtype, dimensionality, and/or SRID are allowed.

Each temporal type is associated to another type, referred to as its *bounding box*, which represent its extent in the value and/or the time dimension. The bounding box of the various temporal types are as follows:

- The `tstzspan` type for the `tbool` and `ttext` types, where only the temporal extent is considered.
- The `tbox` (temporal box) type for the `tint`, `tbigint`, and `tfloat` types, where the value and the time extents are defined, respectively, by a number span and a time span.
- The `stbox` (spatiotemporal box) type for the `tgeompoint` and `tgeogpoint` types, where the spatial extent is defined in the X, Y, and Z dimensions, and the time extent by a time span.

A rich set of functions and operators is available to perform various operations on temporal types. They are explained in this chapter and the following ones. Some of these operations, in particular those related to indexes, manipulate bounding boxes for efficiency reasons.

4.2 Examples of Temporal Types

Examples of usage of temporal alphanumeric types are given next.

```
CREATE TABLE Department (DeptNo integer, DeptName varchar(25), NoEmps tint);
INSERT INTO Department VALUES
(10, 'Research', tint '[10@2001-01-01, 12@2001-04-01, 12@2001-08-01)'),
(20, 'Human Resources', tint '[4@2001-02-01, 6@2001-06-01, 6@2001-10-01)');
CREATE TABLE Temperature (RoomNo integer, Temp tfloat);
INSERT INTO Temperature VALUES
(1001, tfloat '{18.5@2001-01-01 08:00:00, 20.0@2001-01-01 08:10:00}'),
(2001, tfloat '{19.0@2001-01-01 08:00:00, 22.5@2001-01-01 08:10:00}');
```

```

-- Value at a timestamp
SELECT RoomNo, valueAtTimestamp(Temp, '2001-01-01 08:10:00')
FROM temperature;
-- 1001 | 20
-- 2001 | 22.5

-- Restriction to a value
SELECT DeptNo, atValue(NoEmps, 10)
FROM Department;
-- 10 | [10@2001-01-01, 10@2001-04-01]
-- 20 |

-- Restriction to a period
SELECT DeptNo, atTime(NoEmps, tstzspan '[2001-01-01, 2001-04-01]')
FROM Department;
-- 10 | [10@2001-01-01, 12@2001-04-01]
-- 20 | [4@2001-02-01, 4@2001-04-01]

-- Temporal comparison
SELECT DeptNo, NoEmps #<= 10
FROM Department;
-- 10 | [t@2001-01-01, f@2001-04-01, f@2001-08-01]
-- 20 | [t@2001-02-01, t@2001-10-01]

-- Temporal aggregation
SELECT tsum(NoEmps)
FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01,
    18@2001-06-01, 6@2001-08-01, 6@2001-10-01)} */

```

Examples of usage of temporal point types are given next.

```

CREATE TABLE Trips(CarId integer, TripId integer, Trip tgeompoint);
INSERT INTO Trips VALUES
  (10, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:00:00, Point(2 0)@2001-01-01 08:10:00,
Point(2 1)@2001-01-01 08:15:00)}'),
  (20, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00,
Point(3 3)@2001-01-01 08:20:00)}');

-- Value at a given timestamp
SELECT CarId, ST_AsText(valueAtTimestamp(Trip, timestamptz '2001-01-01 08:10:00'))
FROM Trips;
-- 10 | POINT(2 0)
-- 20 | POINT(1 1)

-- Restriction to a value
SELECT CarId, asText(atValue(Trip, geometry 'Point(2 0)'))
FROM Trips;
-- 10 | {"[POINT(2 0)@2001-01-01 08:10:00+00]"}
-- 20 |

-- Restriction to a period
SELECT CarId, asText(atTime(Trip, tstzspan '[2001-01-01 08:05:00,2001-01-01 08:10:00]'))
FROM Trips;
-- 10 | {[POINT(1 0)@2001-01-01 08:05:00+00, POINT(2 0)@2001-01-01 08:10:00+00]}
-- 20 | {[POINT(0 0)@2001-01-01 08:05:00+00, POINT(1 1)@2001-01-01 08:10:00+00]}

-- Temporal distance
SELECT T1.CarId, T2.CarId, T1.Trip <-> T2.Trip
FROM Trips T1, Trips T2
WHERE T1.CarId < T2.CarId;

```

```
/* 10 | 20 | {[1@2001-01-01 08:05:00+00, 1.4142135623731@2001-01-01 08:10:00+00,
1@2001-01-01 08:15:00+00)} */
```

4.3 Validity of Temporal Types

Values of temporal types must satisfy several constraints so that they are well defined. These constraints are given next.

- The constraints on the base type and the `timestamp_tz` type must be satisfied.
- A sequence value must be composed of at least one instant value.
- An instantaneous sequence value or a sequence value with discrete interpolation must have inclusive lower and upper bounds.
- In a sequence value, the timestamps of the composing instants must be different and ordered.
- In a sequence value with step interpolation, the last two values must be equal if upper bound is exclusive.
- A sequence set value must be composed of at least one sequence value.
- In a sequence set value, the composing sequence values must be non overlapping and ordered.

An error is raised whenever one of these constraints are not satisfied. Examples of incorrect temporal values are as follows.

```
-- Incorrect value for base type
SELECT tbool '1.5@2001-01-01 08:00:00';
-- Base type value is not a point
SELECT tgeompoint 'Linestring(0 0,1 1)@2001-01-01 08:05:00';
-- Incorrect timestamp
SELECT tint '2@2001-02-31 08:00:00';
-- Empty sequence
SELECT tint '';
-- Incorrect bounds for instantaneous sequence
SELECT tint '[1@2001-01-01 09:00:00)';
-- Duplicate timestamps
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00]';
-- Unordered timestamps
SELECT tint '[1@2001-01-01 08:10:00, 2@2001-01-01 08:00:00]';
-- Incorrect end value for step interpolation
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:10:00)';
-- Empty sequence set
SELECT tint '{}';
-- Duplicate timestamps
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00}';
-- Overlapping periods
SELECT tint '{[1@2001-01-01 08:00:00, 1@2001-01-01 10:00:00),
[2@2001-01-01 09:00:00, 2@2001-01-01 11:00:00)}';
```

4.4 Temporalizing Operations

A common way to generalize the traditional operations to the temporal types is to apply the operation *at each instant*, which yields a temporal value as result. In that case, the operation is only defined on the intersection of the temporal extents of the operands; if the temporal extents are disjoint, then the result is null. For example, the temporal comparison operators, such as `#<`, test whether the values taken by their operands at each instant satisfy the condition and return a temporal Boolean. Examples of the various generalizations of the operators are given next.

```

-- Temporal comparison
SELECT tfloat '[2@2001-01-01, 2@2001-01-03]' #< tfloat '[1@2001-01-01, 3@2001-01-03]';
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
SELECT tfloat '[1@2001-01-01, 3@2001-01-03]' #< tfloat '[3@2001-01-03, 1@2001-01-05]';
-- NULL

-- Temporal addition
SELECT tint '[1@2001-01-01, 1@2001-01-03]' + tint '[2@2001-01-02, 2@2001-01-05]';
-- [3@2001-01-02, 3@2001-01-03]

-- Temporal intersects
SELECT tIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04]',
  geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}

-- Temporal distance
SELECT tgeompoint '[Point(0 0)@2001-01-01 08:00:00, Point(0 1)@2001-01-03 08:10:00]' <->
  tgeompoint '[Point(0 0)@2001-01-02 08:05:00, Point(1 1)@2001-01-05 08:15:00]';
-- [0.5@2001-01-02 08:05:00+00, 0.745184033794557@2001-01-03 08:10:00+00]

```

Another common requirement is to determine whether the operands *ever* or *always* satisfy a condition with respect to an operation. These can be obtained by applying the *ever* or *always* comparison operators. These operators are denoted by prefixing the traditional comparison operators with, respectively, `?` (*ever*) and `%` (*always*). Examples of *ever* and *always* comparison operators are given next.

```

-- Does the operands ever intersect?
SELECT eIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04]',
  geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))') ?= true;
-- true

-- Does the operands always intersect?
SELECT aIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04]',
  geometry 'Polygon((0 0,0 2,4 2,4 0,0 0))') %= true;
-- true

-- Is the left operand ever less than the right one ?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03]' #<
  tfloat '[3@2001-01-01, 1@2001-01-03]') ?= true;
-- true

-- Is the left operand always less than the right one ?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03]' #<
  tfloat '[2@2001-01-01, 4@2001-01-03]') %= true;
-- true

```

For example, the `eIntersects` function determines whether there is an instant at which the two arguments spatially intersect. We describe next the functions and operators for temporal types. For conciseness, in the examples we mostly use sequences composed of two instants.

4.5 Notation

We present next the functions and operators for temporal types. These functions and operators are polymorphic, that is, their arguments may be of several types, and the result type may depend on the type of the arguments. To express this, we use the following notation:

- `time` represents any time type, that is, `timestamp_tz`, `tstzspan`, `tstzset`, or `tstzspanset`,
- `tttype` represents any temporal type,

- `ttypeInst`, `ttypeSeq`, and `ttypeSeqSet` represent any temporal type with, respectively, instant, sequence, and sequence set subtype,
- `tdisc` represents any temporal type with a discrete base type, that is, `tbool`, `tint`, `tbigint`, or `ttext`,
- `tcont` represents any temporal type with a continuous base type, that is, `tfloat`, `tgeompoint`, or `tgeogpoint`,
- `ttypeDiscSeq` and `ttypeContSeq` represent any temporal type with sequence subtype and, respectively, discrete and continuous interpolation,
- `base` represents any base type of a temporal type, that is, `boolean`, `integer`, `float`, `text`, `geometry`, or `geography`,
- `values` represents any set of values of a base type of a temporal type, for example, `integer`, `intset`, `intspan`, and `intspanset` for the base type `integer`
- `type[]` represents an array of `type`.
- `<type>` in the name of a function represents the functions obtained by replacing `<type>` by a specific type. For example, `tintSeq` or `tfloatSeq` are represented by `ttypeSeq`.

4.6 Input and Output

MobilityDB generalizes Open Geospatial Consortium's Well-Known Text (WKT), Well-Known Binary (WKB), and Moving Features JSON (MF-JSON) input and output format for all temporal types. We start by describing the WKT format.

An *instant value* is a couple of the form `v@t`, where `v` is a value of the base type and `t` is a `timestampz` value. The temporal extent of an instant value is a single timestamp. Examples of instant values are as follows:

```
SELECT tbool 'true@2001-01-01 08:00:00';
SELECT tint '1@2001-01-01 08:00:00';
SELECT tbigint '5@2001-01-01 08:00:00';
SELECT tfloat '1.5@2001-01-01 08:00:00';
SELECT ttext 'AAA@2001-01-01 08:00:00';
SELECT tgeompoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeogpoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeometry 'LineString(0 0,1 1)@2017-01-01 08:00:05';
SELECT tgeography 'Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:00:05';
```

A *sequence value* is a set of values `v1@t1, ..., vn@tn` delimited by lower and upper bounds, which can be inclusive (represented by '[' and ']') or exclusive (represented by '(' and ')'). A sequence value composed of a single couple `v@t` is called an *instantaneous sequence*. Sequence values have an associated *interpolation function* which may be discrete, linear, or step. By definition, the lower and upper bounds of an instantaneous sequence or of a sequence value with discrete interpolation are inclusive. The temporal extent of a sequence value with discrete interpolation is a timestamp set. Examples of sequence values with discrete interpolation are as follows.

```
SELECT tbool '{true@2001-01-01 08:00:00, false@2001-01-03 08:00:00}';
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00}';
SELECT tbigint '{1@2001-01-01 08:00:00}'; -- Instantaneous sequence
SELECT tfloat '{1.0@2001-01-01 08:00:00, 2.0@2001-01-03 08:00:00}';
SELECT ttext '{AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00}';
SELECT tgeompoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeogpoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeometry '{Point(0 0)@2017-01-01 08:00:00,
  LineString(0 0,0 1)@2017-01-02 08:05:00}';
SELECT tgeography '{Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00}';
```

The temporal extent of a sequence value with linear or step interpolation is a period defined by the first and last instants as well as the lower and upper bounds. Examples of sequence values with linear interpolation are as follows:

```

SELECT tbool '[true@2001-01-01 08:00:00, true@2001-01-03 08:00:00]';
SELECT tint '[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00]';
SELECT tfloat '[2.5@2001-01-01 08:00:00, 3@2001-01-03 08:00:00, 1@2001-01-04 08:00:00]';
SELECT tfloat '[1.5@2001-01-01 08:00:00]'; -- Instantaneous sequence
SELECT ttext '[BBB@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00]';
SELECT tgeompoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 0)@2017-01-01 08:05:00]';
SELECT tgeogpoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00,
    Point(0 0)@2017-01-01 08:10:00]';
SELECT tgeometry '[Point(0 0)@2017-01-01 08:00:00,
    Linestring(0 0,0 1)@2017-01-02 08:05:00]';
SELECT tgeography '[Point(0 0)@2017-01-01 08:00:00,
    Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00]';

```

Sequence values whose base type is continuous may specify that the interpolation is step with the prefix `Interp=Step`. If this is not specified, it is supposed that the interpolation is linear by default. Example of sequence values with step interpolation are given next:

```

SELECT tfloat 'Interp=Step;[2.5@2001-01-01 08:00:00, 3@2001-01-01 08:10:00]';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
    Point(1 1)@2017-01-01 08:05:00, Point(1 1)@2017-01-01 08:10:00]';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
    Point(1 1)@2017-01-01 08:05:00, Point(0 0)@2017-01-01 08:10:00]';
ERROR: Invalid end value for temporal sequence with step interpolation
SELECT tgeogpoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
    Point(1 1)@2017-01-01 08:10:00]';

```

The last two instants of a sequence value with discrete interpolation and exclusive upper bound must have the same base value, as shown in the second and third examples above.

A *sequence set value* is a set $\{v_1, \dots, v_n\}$ where every v_i is a sequence value. The interpolation of sequence set values can only be linear or step, not discrete. All sequences composing a sequence set value must have the same interpolation. The temporal extent of a sequence set value is a set of periods. Examples of sequence set values with linear interpolation are as follows:

```

SELECT tbool '{{[false@2001-01-01 08:00:00, false@2001-01-03 08:00:00),
    [true@2001-01-03 08:00:00], (false@2001-01-04 08:00:00, false@2001-01-06 08:00:00)}}';
SELECT tint '{{[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00),
    [2@2001-01-04 08:00:00, 3@2001-01-05 08:00:00, 3@2001-01-06 08:00:00)}}';
SELECT tfloat '{{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00, 2@2001-01-04 08:00:00,
    3@2001-01-06 08:00:00)}}';
SELECT ttext '{{[AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00, BBB@2001-01-04 08:00:00),
    [CCC@2001-01-05 08:00:00, CCC@2001-01-06 08:00:00)}}';
SELECT tgeompoint '{{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}}';
SELECT tgeogpoint '{{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}}';
SELECT tgeometry
    '{{[Point(0 0)@2017-01-01 08:00:00, Linestring(0 0,1 1)@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}}';
SELECT tgeography
    '{{[Point(0 0)@2017-01-01 08:00:00, Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Linestring(0 0,1 1)@2017-01-01 08:15:00)}}';

```

Sequence set values whose base type is continuous may specify that the interpolation is step with the prefix `Interp=Step`. If this is not specified, it is supposed that the interpolation is linear by default. Example of sequence set values with step interpolation are given next:

```

SELECT tfloat 'Interp=Step;{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00,
    2@2001-01-04 08:00:00, 3@2001-01-06 08:00:00]}';
SELECT tgeompoint 'Interp=Step;{[Point(0 0)@2017-01-01 08:00:00,
    Point(0 1)@2017-01-01 08:05:00], [Point(0 1)@2017-01-01 08:10:00,

```

```
Point(0 1)@2017-01-01 08:15:00}}';
SELECT tgeogpoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
Point(0 1)@2017-01-01 08:05:00], [Point(0 1)@2017-01-01 08:10:00,
Point(0 1)@2017-01-01 08:15:00}}';
```

For temporal geometries, it is possible to specify the spatial reference identifier (SRID) using the Extended Well-Known text (EWKT) representation as follows:

```
SELECT tgeompoint 'SRID=5435;[Point(0 0)@2001-01-01,Point(0 1)@2001-01-02] '
SELECT tgeography 'SRID=7844;[Point(0 0)@2001-01-01,Linestring(1 0,0 1)@2001-01-02] '
```

All component geometries of a temporal geometry will then be of the given SRID. Furthermore, each component geometry can specify its SRID with the EWKT format as in the following example

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=5435;Point(0 1)@2001-01-02] '
```

An error is raised if the component geometries are not all in the same SRID or if the SRID of a component geometry is different from the one of the temporal point, as shown below.

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=4326;Point(0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (5435)
SELECT tgeography 'SRID=7844;[SRID=4326;Point(0 0)@2001-01-01,
SRID=4326;Linestring(1 0,0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (7844)
```

If not specified, the default SRID for temporal geometric points is 0 (unknown) and for temporal geographies is 4326 (WGS 84). Temporal geometries with step interpolation may also specify the SRID, as shown next.

```
SELECT tgeompoint 'SRID=5435,Interp=Step;[Point(0 0)@2001-01-01,
Point(0 1)@2001-01-02]';
SELECT tgeogpoint 'Interp=Step;[SRID=4326;Point(0 0)@2001-01-01,
SRID=4326;Point(0 1)@2001-01-02]';
```

We give below the input and output functions in Well-Known Text (WKT), Well-Known Binary (WKB), and Moving Features JSON (MF-JSON) format for temporal alphanumeric types. The corresponding functions for temporal geometries are detailed in Section 7.3. The default output format of all temporal alphanumeric types is the Well-Known Text format. The function `asText` given next enables to determine the output of temporal float values.

- Return the Well-Known Text (WKT) representation

`asText(ttype,maxdecdigits=15) → text`

The `maxdecdigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

```
SELECT asText(tfloat '[10.55@2001-01-01, 25.55@2001-01-02]', 0);
-- [11@2001-01-01, 26@2001-01-02]
SELECT asText(tgeometry
'[Point(1.55 1.55)@2001-01-01,Linestring(1.55 1.55,3.55 3.55)@2001-01-02]', 0);
-- [Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-02]
```

- Return the Moving Features JSON (MF-JSON) representation

`asMFJSON(ttype,options=0,flags=0,maxdecdigits=15) → bytea`

The `options` argument can be used to add a bounding box in the MFJSON output:

- 0: means no option (default value)
- 1: MFJSON BBOX

The `flags` argument can be used to customize the JSON output, for example, to produce an easy-to-read (for human readers) JSON output. Refer to the documentation of the `json-c` library for the possible values. Typical values are as follows:

- 0: means no option (default value)
- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

The `maxdecdigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

```
SELECT asMFJSON(tbool 't@2001-01-01 18:00:00', 1);
/* {"type":"MovingBoolean","period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:00:00+01","lowerInc":true,"upperInc":true},
    "values":[true],"datetimes":["2001-01-01T18:00:00+01"],"interpolation":"None"} */
SELECT asMFJSON(tint '10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00', 1);
/* {"type":"MovingInteger","bbox":[10,25],"period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:10:00+01"},"values":[10,25],"datetimes":["2001-01-01T18:00:00+01",
    "2001-01-01T18:10:00+01"],"lowerInc":true,"upperInc":true,
    "interpolation":"Discrete"} */
SELECT asMFJSON(tfloat '10.5@2001-01-01 18:00:00+02, 25.5@2001-01-01 18:10:00+02');
/* {"type":"MovingFloat","values":[10.5,25.5],"datetimes":["2001-01-01T17:00:00+01",
    "2001-01-01T17:10:00+01"],"lowerInc":true,"upperInc":true,"interpolation":"Linear"} */
SELECT asMFJSON(ttext '1@2001-01-01 18:00:00+02,
    driving@2001-01-01 18:10:00+02');
/* {"type":"MovingText","sequences":[{"values":["walking","driving"],
    "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"],
    "lowerInc":true,"upperInc":true}], "interpolation":"Step"} */
SELECT asMFJSON(tgeometry '1@2001-01-01 18:00:00+02,
    Linestring(1 1,2 2)@2001-01-01 18:10:00+02');
/* {"type":"MovingGeometry","sequences":[{"values":[{"type":"Point",
    "coordinates":[1,1]},{ "type":"LineString","coordinates":[[1,1],[2,2]]}],
    "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"],
    "lower_inc":true,"upper_inc":true}], "interpolation":"Step"} */
```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (WKB) representation

`asBinary(ttype, endian text=)` → `bytea`

`asHexWKB(ttype, endian text=)` → `text`

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(tbool 'true@2001-01-01');
-- \x011a000101009c57d3c11c0000
SELECT asBinary(tfloat '1.5@2001-01-01');
-- \x01210001000000000000f83f009c57d3c11c0000
SELECT asHexWKB(tint '1@2001-01-01', 'XDR');
-- 000023010000000100001CC1D3579C00
SELECT asHexWKB(ttext 'AAA@2001-01-01');
-- 012900010400000000000000041414100009C57D3C11C0000
```

- Input from the Moving Features JSON (MF-JSON) representation

`ttypeFromMFJSON(bytea)` → `ttype`

There is one function per temporal type, the name of the function has as prefix the name of the type, which is `tbool` or `tint` in the examples below.

```
SELECT tboolFromMFJSON(text
    '{"type":"MovingBoolean","period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:00:00+01","lowerInc":true,"upperInc":true},
    "values":[true],"datetimes":["2001-01-01T18:00:00+01"],"interpolation":"None"}');
-- t@2001-01-01 18:00:00
SELECT tintFromMFJSON(text
    '{"type":"MovingInteger","bbox":[10,25],"period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:10:00+01"},"values":[10,25],"datetimes":["2001-01-01T18:00:00+01",
```



```

    "2001-01-01T18:10:00+01"], "lowerInc": true, "upperInc": true,
    "interpolation": "Discrete"}');
-- {10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}
SELECT tfloatFromMFJSON(text
  '{"type": "MovingFloat", "values": [10.5, 25.5], "datetimes": ["2001-01-01T17:00:00+01",
    "2001-01-01T17:10:00+01"], "lowerInc": true, "upperInc": true,
    "interpolation": "Linear"}');
-- [10.5@2001-01-01 18:00:00, 25.5@2001-01-01 18:10:00]
SELECT ttextFromMFJSON(text
  '{"type": "MovingText", "sequences": [{"values": ["walking", "driving"],
    "datetimes": ["2001-01-01T17:00:00+01", "2001-01-01T17:10:00+01"],
    "lowerInc": true, "upperInc": true}], "interpolation": "Step"}');
-- [{"walking"@2001-01-01 18:00:00, "driving"@2001-01-01 18:10:00}]);
SELECT asText(tgeometryFromMFJSON(text
  '{"type": "MovingGeometry", "sequences": [{"values": [{"type": "Point",
    "coordinates": [1, 1]}, {"type": "LineString", "coordinates": [[1, 1], [2, 2]]}],
    "datetimes": ["2001-01-01T17:00:00+01", "2001-01-01T17:10:00+01"],
    "lower_inc": true, "upper_inc": true}], "interpolation": "Step"}'));
-- {[POINT(1 1)@2001-01-01 17:00:00, LINESTRING(1 1, 2 2)@2001-01-01 17:10:00]}

```

- Input from the Well-Known Binary (WKB) or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

ttypeFromBinary(bytea) → ttype

ttypeFromHexWKB(text) → ttype

There is one function per temporal type, the name of the function has as prefix the name of the type, which is tbool or tint in the examples below.

```

SELECT tboolFromBinary('\x011a000101009c57d3c11c0000');
-- t@2001-01-01
SELECT tintFromBinary('\x000023010000000100001cc1d3579c00');
-- 1@2001-01-01
SELECT tfloatFromBinary('\x012100010000000000000f83f009c57d3c11c0000');
-- 1.5@2001-01-01
SELECT ttextFromBinary('\x0129000104000000000000041414100009c57d3c11c0000');
-- "AAA"@2001-01-01
SELECT tboolFromHexWKB('011A000101009C57D3C11C0000');
-- t@2001-01-01
SELECT tintFromHexWKB('000023010000000100001CC1D3579C00');
-- 1@2001-01-01
SELECT tfloatFromHexWKB('012100010000000000000F83F009C57D3C11C0000');
-- 1.5@2001-01-01
SELECT ttextFromHexWKB('0129000104000000000000041414100009C57D3C11C0000');
-- "AAA"@2001-01-01

```

4.7 Constructors

We give next the constructor functions for the various subtypes. Using the constructor function is frequently more convenient than writing a literal constant.

- Constructors for temporal types having a constant value

These constructors have two arguments, a base type and a time value, where the latter is a timestamp_{tz}, a tstzset, a tstzspan, or a tstzspanset value for constructing, respectively, an instant, a sequence with discrete interpolation, a sequence with linear or step interpolation, or a sequence set value. The functions for sequence or sequence set values with continuous base type have an optional third argument stating whether the resulting temporal value has linear or step interpolation. Linear interpolation is assumed by default if the argument is not specified.

ttype(base, timestamp_{tz}) → ttypeInst

```
ttype(base,tstzset) → ttypeDiscSeq
ttype(base,tstzspan,interp='linear') → ttypeContSeq
ttype(base,tstzspanset,interp='linear') → ttypeSeqSet
```

```
SELECT tbool(true, timestampz '2001-01-01');
SELECT tint(1, timestampz '2001-01-01');
SELECT tfloat(1.5, tstzset '{2001-01-01, 2001-01-02}');
SELECT ttext('AAA', tstzset '{2001-01-01, 2001-01-02}');
SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]');
SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]', 'step');
SELECT tgeompoint('Point(0 0)', tstzspan '[2001-01-01, 2001-01-02]');
SELECT tgeography('SRID=7844;Point(0 0)', tstzspanset '{[2001-01-01, 2001-01-02],
[2001-01-03, 2001-01-04]}', 'step');
```

- Constructors for temporal types of sequence subtype

These constructors have a first mandatory argument, which is an array of values of the corresponding instant values and additional optional arguments. The second argument states the interpolation. If the argument is not given, it is by default step for discrete base types such as integer, and linear for continuous base types such as float. An error is raised when linear interpolation is stated for temporal values with discrete base types. For continuous sequences, the third and fourth arguments are Boolean values stating, respectively, whether the left and right bounds are inclusive or exclusive. These arguments are assumed to be true by default if they are not specified.

```
ttypeSeq(ttypeInst[],interp={'step','linear'},leftInc bool=true,
rightInc bool=true) → ttypeSeq
```

```
SELECT tboolSeq(ARRAY[tbool 'true@2001-01-01 08:00:00','false@2001-01-01 08:05:00'],
'discrete');
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00']);
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00'], 'linear');
-- ERROR: The temporal type cannot have linear interpolation
SELECT tfloatSeq(ARRAY[tfloat '1.0@2001-01-01 08:00:00', '2.0@2001-01-01 08:05:00'],
'step', false, true);
SELECT ttextSeq(ARRAY[ttext 'AAA@2001-01-01 08:00:00', 'BBB@2001-01-01 08:05:00']);
SELECT tgeompointSeq(ARRAY[tgeompoint 'Point(0 0)@2001-01-01 08:00:00',
'Point(0 1)@2001-01-01 08:05:00', 'Point(1 1)@2001-01-01 08:10:00'], 'discrete');
SELECT tgeographySeq(ARRAY[tgeography 'Point(1 1)@2001-01-01 08:00:00',
'Point(2 2)@2001-01-01 08:05:00']);
```

- Constructors for temporal types of sequence set subtype

```
ttypeSeqSet(ttypeContSeq[]) → ttypeSeqSet
```

```
ttypeSeqSetGaps(ttypeInst[],maxt=NULL,maxdist=NULL,interp='linear') → ttypeSeqSet
```

A first set of constructors have a single argument, which is an array of values of the corresponding *sequence* values. The interpolation of the resulting temporal value depends on the interpolation of the composing sequences. An error is raised if the sequences composing the array have different interpolation.

```
SELECT tboolSeqSet(ARRAY[tbool '[false@2001-01-01 08:00:00, false@2001-01-01 08:05:00]',
'[true@2001-01-01 08:05:00]', '(false@2001-01-01 08:05:00, false@2001-01-01 08:10:00)']);
SELECT tintSeqSet(ARRAY[tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:05:00,
2@2001-01-01 08:10:00, 2@2001-01-01 08:15:00]']);
SELECT tfloatSeqSet(ARRAY[tfloat '[1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00,
2.0@2001-01-01 08:10:00]', '[2.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00]']);
SELECT tfloatSeqSet(ARRAY[tfloat 'Interp=Step;1.0@2001-01-01 08:00:00,
2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
'Interp=Step;3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00]');
SELECT ttextSeqSet(ARRAY[ttext '[AAA@2001-01-01 08:00:00, AAA@2001-01-01 08:05:00]',
'[BBB@2001-01-01 08:10:00, BBB@2001-01-01 08:15:00]']);
SELECT tgeompointSeqSet(ARRAY[tgeompoint '[Point(0 0)@2001-01-01 08:00:00,
Point(0 1)@2001-01-01 08:05:00, Point(0 1)@2001-01-01 08:10:00]',
'[Point(0 1)@2001-01-01 08:15:00, Point(0 0)@2001-01-01 08:20:00]']);
```

```

SELECT tgeographySeqSet (ARRAY[tgeography
  '[Point(0 0)@2001-01-01 08:00:00, Point(0 0)@2001-01-01 08:05:00]',
  '[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00]']);
SELECT tfloatSeqSet (ARRAY[tfloat 'Interp=Step;[1.0@2001-01-01 08:00:00,
  2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
  '[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00]']);
-- ERROR: The temporal values must have the same interpolation

```

Another set of constructors for sequence set values have as first argument an array of the corresponding *instant* values, and two optional arguments stating a maximum time interval and a maximum distance such that a gap is introduced between sequences of the result whenever two consecutive input instants have a time gap or a distance greater than these values. For temporal geometries, the distance is specified in units of the coordinate system. When none of the gap arguments are given, the resulting value will be a singleton sequence set. In addition, when the base type is continuous, an additional last argument states whether the interpolation is step or linear. If this argument is not specified it is assumed to be linear by default.

The parameters of the function depend on the temporal type. For example, the interpolation parameter is not allowed for temporal types with discrete subtype such as `tint`. Similarly, the parameter `maxdist` is not allowed for scalar types such as `ttext` that do not have a standard distance function.

```

SELECT tintSeqSetGaps (ARRAY[tint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
  '5@2001-01-05']);
-- {[1@2001-01-01, 3@2001-01-02, 4@2001-01-03, 5@2001-01-05]}
SELECT tbigintSeqSetGaps (ARRAY[tbigint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
  '5@2001-01-05'], '1 day', 1);
-- {[1@2001-01-01], [3@2001-01-02, 4@2001-01-03], [5@2001-01-05]}
SELECT ttextSeqSetGaps (ARRAY[ttext 'AA@2001-01-01', 'BB@2001-01-02', 'AA@2001-01-03',
  'CC@2001-01-05'], '1 day');
-- [{"AA"@2001-01-01, "BB"@2001-01-02, "AA"@2001-01-03}, [{"CC"@2001-01-05}]}
SELECT asText (tgeometrySeqSetGaps (ARRAY[tgeometry 'Point(1 1)@2001-01-01',
  'Linestring(2 2,3 3)@2001-01-02', 'Point(4 2)@2001-01-03',
  'Polygon((5 0,6 1,7 0,5 0))@2001-01-05'], '1 day', 1));
/* {[POINT(1 1)@2001-01-01], [LINESTRING(2 2,3 3)@2001-01-02],
  [POINT(4 2)@2001-01-03], [POLYGON((5 0,6 1,7 0,5 0))@2001-01-05]} */
SELECT asText (tgeompointSeqSetGaps (ARRAY[tgeompoint 'Point(1 1)@2001-01-01',
  'Point(2 2)@2001-01-02', 'Point(3 2)@2001-01-03', 'Point(3 2)@2001-01-05'],
  '1 day', 1, 'step'));
/* Interp=Step;{[POINT(1 1)@2001-01-01], [POINT(2 2)@2001-01-02, POINT(3 2)@2001-01-03],
  [POINT(3 2)@2001-01-05]} */

```

4.8 Conversions

A temporal value can be converted into a compatible type using the notation `CAST(ttype1 AS ttype2)` or `ttype1::ttype2`.

- Convert a temporal value to a timestampz span

`ttype::tstzspan`

```

SELECT tint '[1@2001-01-01, 2@2001-01-03]':::tstzspan;
-- [2001-01-01, 2001-01-03]
SELECT ttext '(A@2001-01-01, B@2001-01-03, C@2001-01-05)':::tstzspan;
-- (2001-01-01, 2001-01-05]
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]':::tstzspan;
-- [2001-01-01, 2001-01-03]

```

4.9 Accessors

- Return the memory size in bytes

`memSize(ttype) → integer`

```
SELECT memSize(tint '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- 176
```

- Return the temporal type

tempSubtype(ttype) → {'Instant', 'Sequence', 'SequenceSet'}

```
SELECT tempSubtype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Sequence
```

- Return the base type

tempBasetype(ttype) → text

```
SELECT tempBasetype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- int4
SELECT tempBasetype(tgeompoint 'Point(1 1)@2001-01-01');
-- geometry
```

- Return the interpolation

interp(ttype) → {'None', 'Discrete', 'Step', 'Linear'}

```
SELECT interp(tbool 'true@2001-01-01');
-- None
SELECT interp(tfloat '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- Discrete
SELECT interp(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Linear
SELECT interp(tfloat 'Interp=Step;[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tgeompoint 'Interp=Step;[Point(1 1)@2001-01-01,
    Point(2 2)@2001-01-02, Point(3 3)@2001-01-03]');
-- Step
SELECT interp(tgeometry '[Point(1 1)@2001-01-01,
    Linestring(1 1,2 2)@2001-01-02, Polygon((3 3,4 4,5 3,3 3))@2001-01-03]');
-- Step
```

- Return the value or the timestamp of an instant

getValue(ttypeInst) → base

getTimestamp(ttypeInst) → timestamptz

```
SELECT getValue(tint '1@2001-01-01');
-- 1
SELECT getTimestamp(tfloat '1@2001-01-01');
-- 2001-01-01
```

- Return the values or the time on which a temporal value is defined

getValues(ttype) → baseset

getTime(ttype) → tstzspanset

```
SELECT getValues(tbool '[false@2001-01-01, true@2001-01-02, false@2001-01-03]');
-- {f,t}
SELECT getValues(tint '[1@2001-01-01, 3@2001-01-02, 1@2001-01-03]');
-- {[1, 2), [3, 4)}
SELECT getValues(tint '[{1@2001-01-01, 2@2001-01-02, 1@2001-01-03},
    [4@2001-01-04, 4@2001-01-05]}');
-- {[1, 3), [4, 5)}
```

```

SELECT getValues(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[1, 1], [2, 2]}
SELECT getValues(tfloat 'Interp=Step;{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 1], [2, 2], [3, 3]}
SELECT getValues(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- {[1, 2]}
SELECT getValues(tfloat '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 2], [3, 3]}

SELECT getTime(ttext 'walking@2001-01-01');
-- {[2001-01-01, 2001-01-01]}
SELECT getTime(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[2001-01-01, 2001-01-01], [2001-01-02, 2001-01-02], [2001-01-03, 2001-01-03]}
SELECT getTime(tint '[1@2001-01-01, 1@2001-01-15]');
-- {[2001-01-01, 2001-01-15]}
SELECT getTime(tfloat '{[1@2001-01-01, 1@2001-01-10], [12@2001-01-12, 12@2001-01-15]}');
-- {[2001-01-01, 2001-01-10], [2001-01-12, 2001-01-15]}

```

- Return the bounding period on which a temporal value is defined

timeSpan(ttype) → tstzspan

```

SELECT timeSpan(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- [2001-01-01, 2001-01-03]
SELECT timeSpan(tint '[1@2001-01-01, 1@2001-01-15]');
-- [2001-01-01, 2001-01-15]

```

- Return the start, end, or n-th value

startValue(ttype) → base

endValue(ttype) → base

valueN(ttype,int) → base

The functions do not take into account whether the bounds are inclusive or not.

```

SELECT startValue(tfloat '(1@2001-01-01, 2@2001-01-03)');
-- 1
SELECT endValue(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}');
-- 5
SELECT valueN(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}', 3);
-- 3

```

- Return the value at a timestamp

valueAtTimestamp(ttype,timestamptz) → base

```

SELECT valueAtTimestamp(tfloat '[1@2001-01-01, 4@2001-01-04]', '2001-01-02');
-- 2

```

- Return the time interval

duration(ttype,boundspace=false) → interval

An additional parameter can be set to true to compute the duration of the bounding time span, thus ignoring the potential time gaps

```

SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}');
-- 00:00:00
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}', true);
-- 4 days
SELECT duration(tfloat '[1@2001-01-01, 2@2001-01-03, 2@2001-01-05]');

```

```
-- 4 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05)}');
-- 3 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05)}',
    true);
-- 4 days
```

- Is the start/end instant inclusive?

lowerInc(ttype) → bool
upperInc(ttype) → bool

```
SELECT lowerInc(tint '[1@2001-01-01, 2@2001-01-02)');
-- true
SELECT upperInc(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- false
```

- Return the (number of) different instants

numInstants(ttype) → integer
instants(ttype) → ttypeInst[]

```
SELECT numInstants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- 3
SELECT instants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- {"1@2001-01-01", "2@2001-01-02", "3@2001-01-03"}
```

- Return the start, end, or n-th instant

startInstant(ttype) → ttypeInst
endInstant(ttype) → ttypeInst
instantN(ttype, integer) → ttypeInst

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startInstant(tfloat '{[1@2001-01-01, 2@2001-01-02),
    (2@2001-01-02, 3@2001-01-03)}');
-- 1@2001-01-01
SELECT endInstant(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- 3@2001-01-03
SELECT instantN(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}', 3);
-- 3@2001-01-03
```

- Return the (number of) different timestamps

numTimestamps(ttype) → integer
timestamps(ttype) → timestamptz[]

```
SELECT numTimestamps(tfloat '{[1@2001-01-01, 2@2001-01-03),
    [3@2001-01-03, 5@2001-01-05)}');
-- 3
SELECT timestamps(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"2001-01-01", "2001-01-03", "2001-01-05"}
```

- Return the start, end, or n-th timestamp

startTimestamp(ttype) → timestamptz
endTimestamp(ttype) → timestamptz
timestampN(ttype, integer) → timestamptz

The functions do not take into account whether the bounds are inclusive or not.

```

SELECT startTimestamp(tfloat '[1@2001-01-01, 2@2001-01-03]');
-- 2001-01-01
SELECT endTimestamp(tfloat '{[1@2001-01-01, 2@2001-01-03],
[3@2001-01-03, 5@2001-01-05]}');
-- 2001-01-05
SELECT timestampN(tfloat '{[1@2001-01-01, 2@2001-01-03],
[3@2001-01-03, 5@2001-01-05]}', 3);
-- 2001-01-05

```

- Return the (number of) sequences

`numSequences({ttypeContSeq,ttypeSeqSet})` → integer
`sequences({ttypeContSeq,ttypeSeqSet})` → ttypeContSeq[]

```

SELECT numSequences(tfloat '{[1@2001-01-01, 2@2001-01-03],
[3@2001-01-03, 5@2001-01-05]}');
-- 2
SELECT sequences(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}');
-- {"[1@2001-01-01, 2@2001-01-03]", "[3@2001-01-03, 5@2001-01-05]"}

```

- Return the start, end, or n-th sequence

`startSequence({ttypeContSeq,ttypeSeqSet})` → ttypeContSeq
`endSequence({ttypeContSeq,ttypeSeqSet})` → ttypeContSeq
`sequenceN({ttypeContSeq,ttypeSeqSet},integer)` → ttypeContSeq

```

SELECT startSequence(tfloat '{[1@2001-01-01, 2@2001-01-03],
[3@2001-01-03, 5@2001-01-05]}');
-- [1@2001-01-01, 2@2001-01-03]
SELECT endSequence(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}');
-- [3@2001-01-03, 5@2001-01-05]
SELECT sequenceN(tfloat '{[1@2001-01-01, 2@2001-01-03],
[3@2001-01-03, 5@2001-01-05]}', 2);
-- [3@2001-01-03, 5@2001-01-05]

```

- Return the segments

`segments({ttypeContSeq,ttypeSeqSet})` → ttypeContSeq[]

```

SELECT segments(tint '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
(3@2001-01-03, 5@2001-01-05]}');
/* {"[1@2001-01-01, 1@2001-01-02]", "[3@2001-01-02, 3@2001-01-03]", "[2@2001-01-03]",
"(3@2001-01-03, 3@2001-01-05)", "[5@2001-01-05]"} */
SELECT segments(tfloat '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
(3@2001-01-03, 5@2001-01-05]}');
/* {"[1@2001-01-01, 3@2001-01-02]", "[3@2001-01-02, 2@2001-01-03]",
"(3@2001-01-03, 5@2001-01-05)"} */

```

4.10 Transformations

A temporal value can be transformed to another subtype. An error is raised if the subtypes are incompatible.

- Transform a temporal value to another subtype

`ttypeInst(ttype)` → ttypeInst
`ttypeSeq(ttype)` → ttypeSeq
`ttypeSeqSet(ttype)` → ttypeSeqSet

```

SELECT tboolInst(tbool '{{true@2001-01-01}}');
-- t@2001-01-01
SELECT tboolInst(tbool '{{true@2001-01-01, true@2001-01-02}}');
-- ERROR: Cannot transform input value to a temporal instant
SELECT tintSeq(tint '1@2001-01-01');
-- [1@2001-01-01]
SELECT tfloatSeqSet(tfloat '2.5@2001-01-01');
-- {[2.5@2001-01-01]}
SELECT tfloatSeqSet(tfloat '{2.5@2001-01-01, 1.5@2001-01-02, 3.5@2001-01-03}');
-- {[2.5@2001-01-01], [1.5@2001-01-02], [3.5@2001-01-03]}

```

- Transform a temporal value to another interpolation

setInterp(ttype, interp) → ttype

```

SELECT setInterp(tbool 'true@2001-01-01', 'discrete');
-- {t@2001-01-01}
SELECT setInterp(tfloat '{{[1@2001-01-01], [2@2001-01-02], [1@2001-01-03]]}', 'discrete');
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]}
SELECT setInterp(tfloat 'Interp=Step;[1@2001-01-01, 2@2001-01-02,
1@2001-01-03, 2@2001-01-04]', 'linear');
/* {[1@2001-01-01, 1@2001-01-02), [2@2001-01-02, 2@2001-01-03),
[1@2001-01-03, 1@2001-01-04), [2@2001-01-04]} */
SELECT asText(setInterp(tgeompoint 'Interp=Step;{[Point(1 1)@2001-01-01,
Point(2 2)@2001-01-02], [Point(3 3)@2001-01-05, Point(4 4)@2001-01-06]}', 'linear'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02), [POINT(2 2)@2001-01-02,
[POINT(3 3)@2001-01-05, POINT(3 3)@2001-01-06), [POINT(4 4)@2001-01-06]} */
SELECT setInterp(tgeometry '[Point(1 1)@2001-01-01,
Linestring(1 1,2 2)@2001-01-02]', 'linear');
-- ERROR: The temporal type cannot have linear interpolation

```

- Return a temporal boolean stating whether each segment of a continuous temporal sequence (set) has, respectively, at least or at most a given duration

segmentMinDuration(temp, interval, bool strict=true) → tbool

segmentMaxDuration(temp, interval, bool strict=true) → tbool

If `strict` is not specified, the value `true` is assumed by default, and therefore the function checks whether the segment duration is strictly less than or greater than the interval. If the argument is set to `false`, the function checks whether the segment duration is less/greater than or *equal* to the interval.

```

SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
-1@2001-01-06]', '1 day');
-- {[t@2001-01-01, f@2001-01-03, t@2001-01-04, t@2001-01-06]}
SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
-1@2001-01-06]', '1 day', false);
-- {[t@2001-01-01, t@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
-1@2001-01-06]', '1 day');
-- {[f@2001-01-01 00:00:00+01, f@2001-01-06 00:00:00+01]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
-1@2001-01-06]', '1 day', false);
-- {[f@2001-01-01, t@2001-01-03, f@2001-01-04, f@2001-01-06]}

```

- Shift and/or scale the time span of a temporal value by one or two intervals

The given intervals must be strictly greater than zero. If the time span of the temporal value is zero (for example, for a temporal instant), the result of a scale operation is the temporal value.

shiftTime(ttype, interval) → ttype

scaleTime(ttype, interval) → ttype

shiftScaleTime(ttype, interval, interval) → ttype


```

SELECT shiftTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-02, 2@2001-01-04, 1@2001-01-06}
SELECT shiftTime(tfpoint '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-02, 2@2001-01-04]
SELECT asText(shiftTime(tgeompoint '[[Point(1 1)@2001-01-01, Point(2 2)@2001-01-03],
  [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]]', '1 day'));
/* {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-04],
  [POINT(2 2)@2001-01-05, POINT(1 1)@2001-01-06]} */
SELECT scaleTime(tint '1@2001-01-01', '1 day');
-- 1@2001-01-01
SELECT scaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-01 00:00:00+01, 2@2001-01-01 12:00:00+01, 1@2001-01-02 00:00:00+01}
SELECT scaleTime(tfpoint '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-01, 2@2001-01-02]
SELECT asText(scaleTime(tgeompoint '[[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03], [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]]', '1 day'));
/* {[POINT(1 1)@2001-01-01 00:00:00+01, POINT(2 2)@2001-01-01 06:00:00+01,
  POINT(1 1)@2001-01-01 12:00:00+01], [POINT(2 2)@2001-01-01 18:00:00+01,
  POINT(1 1)@2001-01-02 00:00:00+01]} */
SELECT scaleTime(tint '1@2001-01-01', '-1 day');
-- ERROR: The interval must be positive: -1 days
SELECT shiftScaleTime(tint '1@2001-01-01', '1 day', '1 day');
-- 1@2001-01-02
SELECT shiftScaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day', '1 day');
-- {1@2001-01-02 00:00:00+01, 2@2001-01-02 12:00:00+01, 1@2001-01-03 00:00:00+01}
SELECT shiftScaleTime(tfpoint '[1@2001-01-01, 2@2001-01-03]', '1 day', '1 day');
-- [1@2001-01-02, 2@2001-01-03]
SELECT asText(shiftScaleTime(tgeometry '[[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03], [Point(2 2)@2001-01-04,
  Polygon((1 1,2 2,3 1,1 1))@2001-01-05]]', '1 day', '1 day'));
/* {[POINT(1 1)@2001-01-02 00:00:00, LINESTRING(1 1,2 2)@2001-01-02 06:00:00,
  POINT(1 1)@2001-01-02 12:00:00], [POINT(2 2)@2001-01-02 18:00:00,
  POLYGON((1 1,2 2,3 1,1 1))@2001-01-03 00:00:00]} */

```

- Transform a nonlinear temporal value into a set of rows, each one is a pair composed of a base value and a period set during which the temporal value has the base value {}

`unnest(ttype) → {(value,time)}`

```

SELECT (un).value, (un).time
FROM (SELECT unnest(tfpoint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}') AS un) t;
-- 1 | {[2001-01-01, 2001-01-01], [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-02]}
SELECT (un).value, (un).time
FROM (SELECT unnest(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- 1 | {[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-03]}
SELECT (un).value, (un).time
FROM (SELECT unnest(tfpoint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- ERROR: The temporal value cannot have linear interpolation
SELECT ST_AsText((un).value), (un).time
FROM (SELECT unnest(tgeompoint 'Interp=Step;[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
-- POINT(1 1) | {[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-03]}
-- POINT(2 2) | {[2001-01-02, 2001-01-03]}
SELECT ST_AsText((un).value), (un).time
FROM (SELECT unnest(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
-- POINT(1 1) | {[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-03]}
-- LINESTRING(1 1,2 2) | {[2001-01-02, 2001-01-03]}

```

Chapter 5

Temporal Types (Part 2)

5.1 Modifications

We explain next the semantics of the modification operations (that is, `insert`, `update`, and `delete`) for temporal types. These operations have similar semantics as the corresponding operations for application-time temporal tables introduced in the [SQL:2011](#) standard. The main difference is that SQL uses *tuple timestamping* (where timestamps are attached to tuples), while temporal values in MobilityDB use *attribute timestamping* (where timestamps are attached to attribute values). Please refer to this [article](#) for a detailed discussion about tuple versus attribute timestamping in temporal databases.

The `insert` operation adds to a temporal value the instants of another one without modifying the existing instants, as illustrated in Figure 5.1.

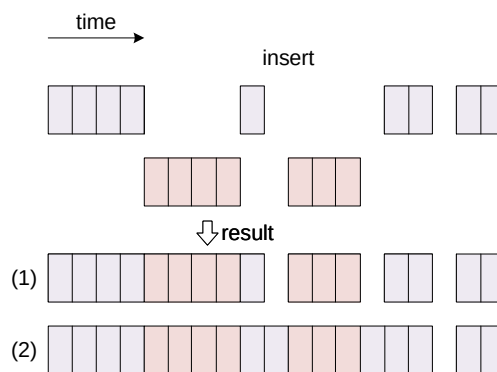


Figure 5.1: Insert operation for temporal values.

As shown in the figure, the temporal values may only intersect at their boundary, and in that case, they must have the same base value at their common timestamps, otherwise an error is raised. The result of the operation is the union of the instants for both temporal values, as shown in the first result of the figure. This is equivalent to a `merge` operation explained below. Alternatively, as shown in the second result of the figure, the inserted fragments that are disjoint with the original value are connected to the last instant before and the first instant after the fragment. A Boolean parameter `connect` is used to choose between the two results, and the parameter is set to true by default. Notice that this only applies to continuous temporal values.

The `update` operation replaces the instants in the first temporal value with those of the second one as illustrated in Figure 5.2.

As in the case of an `insert` operation, an additional Boolean parameter determines whether the replaced disconnected fragments are connected in the resulting value, as shown in the two possible results in the figure. When the two temporal values are either disjoint or only overlap at their boundary, this corresponds to an `insert` operation as explained above. In this case, the `update` operation behaves as an `upsert` operation in SQL.

The `deleteTime` operation removes the instants of a temporal value that intersect a time value. This operation can be used in two different situations, illustrated in Figure 5.2.

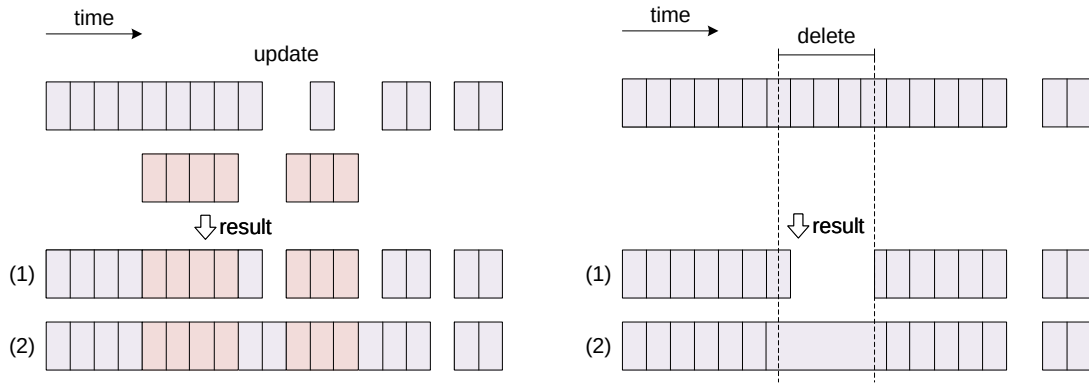


Figure 5.2: Update and delete operation for temporal values.

1. In the first case, shown as the top result in the figure, the meaning of operation is to introduce time gaps after removing the instants of the temporal value intersecting the time value. This is equivalent to the restriction operations (Section 5.2), which restrict a temporal value to the complement of the time value.
2. The second case, shown as the bottom result in the figure, is used for removing erroneous values (e.g., detected as outliers) without introducing a time gap, or for removing time gaps. In this case, the instants of the temporal value are deleted and the last instant before and the first instant after a removed fragment are connected. This behaviour is specified by setting an additional Boolean parameter of the operation. Notice that this only applies to continuous temporal values.

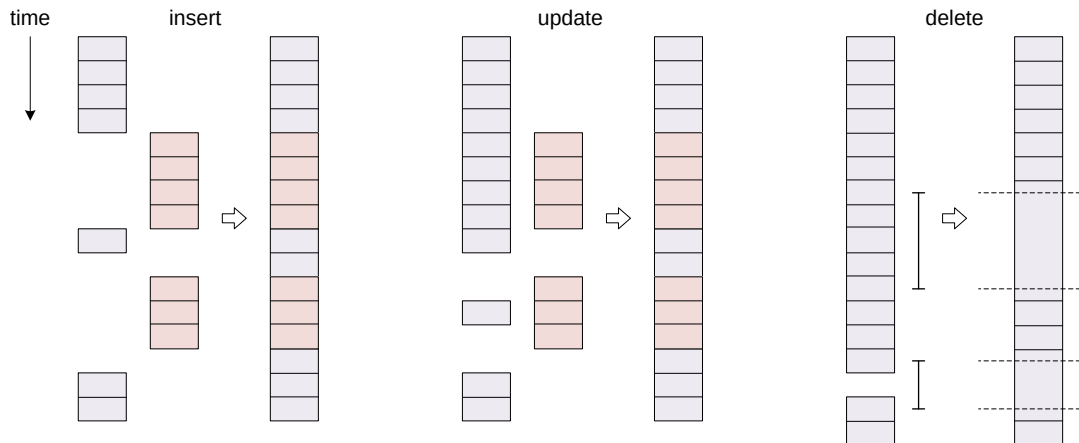


Figure 5.3: Modification operations for temporal tables in SQL

Figure 5.3 shows the equivalent modification operations for temporal tables in the SQL standard. Intuitively, these figures are obtained by rotating 90 degrees clockwise the corresponding figures for temporal values (Figure 5.1 and Figure 5.2). This follows from the fact that in SQL consecutive tuples ordered by time are typically connected through the `LEAD` and `LAG` window functions.

- Insert a temporal value into another one

```
insert (ttype,ttype,connect=true) → ttype
```

```
SELECT insert(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{3@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 3@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT insert(tbigint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tbigint '{5@2001-01-03, 7@2001-01-07}');
```

```
-- ERROR: The temporal values have different value at their overlapping instant 2001-01-03
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-05]
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]', false);
-- {[1@2001-01-01, 2@2001-01-02], [1@2001-01-03, 1@2001-01-05]}
SELECT asText(insert(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04], [Point(1 1 1)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-03'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (1 1 1)@2001-01-03,
  POINT Z (3 3 3)@2001-01-04], [POINT Z (1 1 1)@2001-01-05]} */
```

- Update a temporal value with another one

update(ttype, ttype, connect=true) → ttype

```
SELECT update(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{5@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 5@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT update(tfloat '[1@2001-01-01, 1@2001-01-05]',
  tfloat '[1@2001-01-02, 3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 1@2001-01-02, 3@2001-01-03, 1@2001-01-04, 1@2001-01-05]}
SELECT asText(update(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(1 1 1)@2001-01-05], [Point(1 1 1)@2001-01-07]}',
  tgeompoint 'Point(2 2 2)@2001-01-02, Point(2 2 2)@2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (2 2 2)@2001-01-04,
  POINT Z (1 1 1)@2001-01-05], [POINT Z (1 1 1)@2001-01-07]} */
SELECT asText(update(
  tgeometry 'Point(1 1)@2001-01-01, Point(3 3)@2001-01-03, Point(1 1)@2001-01-05',
  tgeometry 'Linestring(2 2,3 3)@2001-01-02, Linestring(3 3,2 2)@2001-01-04]'));
/* [POINT(1 1)@2001-01-01, LINESTRING(2 2,3 3)@2001-01-02,
  LINESTRING(3 3,2 2)@2001-01-04], (POINT(3 3)@2001-01-04, POINT(1 1)@2001-01-05) */
```

- Delete the instants of a temporal value that intersect a time value

deleteTime(ttype, time, connect=true) → ttype

```
SELECT deleteTime(tint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-02', false);
-- {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03]}
SELECT deleteTime(tbigint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-04');
-- [1@2001-01-01, 1@2001-01-03]
SELECT deleteTime(tfloat '[1@2001-01-01, 4@2001-01-02, 2@2001-01-04, 5@2001-01-05]',
  tstzspan '[2001-01-02, 2001-01-04]');
-- [1@2001-01-01, 5@2001-01-05]
SELECT asText(deleteTime(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tstzspan '[2001-01-02, 2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-04,
  POINT Z (3 3 3)@2001-01-05]} */
```

- Append a temporal instant to a temporal value

appendInstant(ttype, ttypeInst, interp=NULL) → ttype

appendInstant(ttypeInst, interp=NULL, maxdist=NULL, maxt=NULL) → ttypeSeq

The first version of the function returns the result of appending the second argument to the first one. If either input is NULL, then NULL is returned.

The second version of the function above is an *aggregate* function that returns the result of successively appending a set of rows of temporal values. This means that it operates in the same way the SUM() and AVG() functions do and like most aggregates, it also ignores NULL values. Two optional arguments state a maximum distance and a maximum time interval such that a gap is introduced whenever two consecutive instants have a distance or a time gap greater than these values. For temporal points

the distance is specified in units of the coordinate system. If one of the arguments are not given, it is not taken into account for determining the gaps.

```
SELECT appendInstant(tint '1@2001-01-01', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT appendInstant(tbigint '1@2001-01-01', tbigint '1@2001-01-02', 'discrete');
-- {1@2001-01-01, 1@2001-01-02}
SELECT appendInstant(tint '[1@2001-01-01]', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT asText(appendInstant(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-06'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
  [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
  POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendInstant(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(2 2,3 3)@2001-01-04, Point(3 3)@2001-01-05]}',
  tgeometry 'Linestring(1 1,2 2)@2001-01-06'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(2 2,3 3)@2001-01-04,
  POINT(3 3)@2001-01-05, LINESTRING(1 1,2 2)@2001-01-06]} */
```

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst ORDER BY inst) FROM temp;
-- [1@2001-01-01, 5@2001-01-05]
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst, 'discrete' ORDER BY inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
WITH temp(inst) AS (
  SELECT tgeogpoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeogpoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeogpoint 'Point(3 3)@2001-01-03' UNION
  SELECT tgeogpoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeogpoint 'Point(5 5)@2001-01-05' )
SELECT asText(appendInstant(inst ORDER BY inst)) FROM temp;
/* [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03,
  POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05] */
```

Notice that in the first query above with `tfloat`, the intermediate observations were removed by the normalization process since they were redundant due to linear interpolation. This is not the case for the second query with discrete interpolation or the third query with `tgeogpoint`, where geodetic coordinates are used.

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '4@2001-01-04' UNION SELECT tfloat '5@2001-01-05' UNION
  SELECT tfloat '7@2001-01-07' )
SELECT appendInstant(inst, NULL, 0.0, '1 day' ORDER BY inst) FROM temp;
-- {[1@2001-01-01, 2@2001-01-02], [4@2001-01-04, 5@2001-01-05], [7@2001-01-07]}
WITH temp(inst) AS (
  SELECT tgeompoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeompoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeompoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeompoint 'Point(5 5)@2001-01-05' UNION
  SELECT tgeompoint 'Point(7 7)@2001-01-07' )
SELECT asText(appendInstant(inst, NULL, sqrt(2), '1 day' ORDER BY inst)) FROM temp;
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02],
  [POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05], [POINT(7 7)@2001-01-07]} */
```

- Append a temporal sequence to a temporal value

```
appendSequence(ttype, ttypeSeq) → {ttypeSeq, ttypeSeqSet}
```

```
appendSequence(ttypeSeq) → {ttypeSeq, ttypeSeqSet}
```

The first version of the function returns the result of appending the second argument to the first one. If either input is NULL, then NULL is returned.

The second version of the function above is an *aggregate* function that returns the result of successively appending a set of rows of temporal values. This means that it operates in the same way the SUM() and AVG() functions do and like most aggregates, it also ignores NULL values.

```
SELECT appendSequence(tint '1@2001-01-01', tint '{2@2001-01-02, 3@2001-01-03}');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT appendSequence(tbigint '[1@2001-01-01, 2@2001-01-02]',
  tbigint '[2@2001-01-02, 3@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 3@2001-01-03]
SELECT asText(appendSequence(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint '[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]}'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
  [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
  POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendSequence(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(3 3,2 2)@2001-01-04, Point(3 3)@2001-01-05]}',
  tgeometry '[Point(3 3)@2001-01-05, Linestring(3 3,1 1)@2001-01-06]}'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(3 3,2 2)@2001-01-04,
  POINT(3 3)@2001-01-05, LINESTRING(3 3,1 1)@2001-01-06]} */
```

- Merge the temporal values

```
merge(ttype, ttype) → ttype
```

```
merge(ttype[]) → ttype
```

```
merge(ttype) → ttype
```

The temporal values may only intersect at their boundary. In that case, for all temporal types excepted `tgeometry` and `tgeography`, their base values at the common timestamps must be the same, otherwise an error is raised. For the `tgeometry` and `tgeography` types, if the value at their common timestamps is different, the resulting value after the `merge` will be the spatial union of the values. The reason for a different behaviour for these types is that they are the only temporal types that are not *scalar* types, that is, their value at a given timestamp is not a single element of the domain of the base type, but rather a subset of their domain.

The third version of the function above is an *aggregate* function that returns the result of successively appending a set of rows of temporal values. This means that it operates in the same way the SUM() and AVG() functions do and like most aggregates, it also ignores NULL values.

```
SELECT merge(tint '1@2001-01-01', tint '1@2001-01-02');
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(tbigint '[1@2001-01-01, 2@2001-01-02]', tbigint '[2@2001-01-02, 1@2001-01-03] ←
  ');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 1@2001-01-04]}
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[1@2001-01-02, 2@2001-01-03]');
-- ERROR: The temporal values have different value at their common timestamp 2001-01-02
SELECT asText(merge(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint '{[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]}'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
  [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
  POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(merge(tgeometry 'Linestring(1 1,5 1)@2001-01-01',
  tgeometry '[Linestring(5 1,10 1)@2001-01-01, Point(3 3)@2001-01-02]}'));
-- {LINESTRING(1 1,5 1,10 1)@2001-01-01, POINT(3 3)@2001-01-02}
```

```

SELECT merge(ARRAY[tint '1@2001-01-01', '1@2001-01-02']);
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{2@2001-01-02, 3@2001-01-03}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{3@2001-01-03, 4@2001-01-04}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04}
SELECT merge(ARRAY[tbigint '[1@2001-01-01, 2@2001-01-02]', '[2@2001-01-02, 1@2001-01-03]' ↔
]);
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(ARRAY[tbigint '[1@2001-01-01, 2@2001-01-02]', '[3@2001-01-03, 4@2001-01-04]' ↔
]);
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 4@2001-01-04]}
SELECT asText(merge(ARRAY[tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
[Point(3 3)@2001-01-03, Point(4 4)@2001-01-04]}', '{[Point(4 4)@2001-01-04,
Point(3 3)@2001-01-05], [Point(6 6)@2001-01-06, Point(7 7)@2001-01-07]}']));
/* {[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02], [Point(3 3)@2001-01-03,
Point(4 4)@2001-01-04, Point(3 3)@2001-01-05],
[Point(6 6)@2001-01-06, Point(7 7)@2001-01-07]} */
SELECT asText(merge(ARRAY[tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]}',
'{[Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- [Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]
SELECT asText(merge(ARRAY[tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]}',
'{[Linestring(2 2,1 1)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- {[POINT(1 1)@2001-01-01, LINESTRING(2 2,1 1)@2001-01-02, POINT(1 1)@2001-01-03]}

WITH temp(inst) AS (
SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
SELECT tfloat '5@2001-01-05' )
SELECT merge(inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}

```

5.2 Restrictions

There are three complementary sets of restriction functions. The first set functions restricts the temporal value with respect to a value or a time extent. Examples are `atValues` or `atTime`. The second set functions restricts the temporal value with respect to the *complement* of a value or a time extent. Examples are `minusValues` or `minusTime`. Finally, the third set functions restricts the temporal value before or after a timestamp `tz`. Examples are `beforeTimestamp` or `afterTimestamp`.

- Restrict to (the complement of) a value or a set of values, or, for a temporal number, a span or a span set of values

```

atValue(ttype,value) → ttype
minusValue(ttype,value) → ttype
atValues(ttype,valueSet) → ttype
minusValues(ttype,valueSet) → ttype
atSpan(tnumber,span) → tnumber
minusSpan(tnumber,span) → tnumber
atSpanSet(tnumber,spanSet) → tnumber
minusSpanSet(tnumber,spanSet) → tnumber

```

```

SELECT atValue(tint '[1@2001-01-01, 1@2001-01-15]', 1);
-- [1@2001-01-01, 1@2001-01-15]
SELECT atValues(tfloat '[1@2001-01-01, 4@2001-01-4]', floatset '{1, 3, 5}');
-- {[1@2001-01-01], [3@2001-01-03]}
SELECT atSpan(tfloat '[1@2001-01-01, 4@2001-01-4]', floatspan '[1,3]');

```

```
-- [1@2001-01-01, 3@2001-01-03]
SELECT atSpanset(tfloat '[1@2001-01-01, 5@2001-01-05]',
  floatspanset '{[1,2], [3,4]}');
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 4@2001-01-04]}
SELECT asText(atValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03]',
  geometry 'Point(1 1 1)'));
-- {[POINT Z (1 1 1)@2001-01-02]}
SELECT asText(atValues(tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03]',
  geomset '{"Point(0 0)", "Point(1 1)"}'));
-- {[POINT(0 0)@2001-01-01], [POINT(1 1)@2001-01-02]}
SELECT asText(atValue(tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,2 2)@2001-01-02,
  Linestring(0 0,2 2)@2001-01-03]', geometry 'Linestring(0 0,2 2)'));
-- {[LINESTRING(0 0,2 2)@2001-01-02, LINESTRING(0 0,2 2)@2001-01-03]}
```

```
SELECT minusValue(tint '[1@2001-01-01, 2@2001-01-02, 2@2001-01-03]', 1);
-- {[2@2001-01-02, 2@2001-01-03]}
SELECT minusValues(tfloat '[1@2001-01-01, 4@2001-01-4]', floatset '{2, 3}');
/* {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03),
  (3@2001-01-03, 4@2001-01-04)} */
SELECT minusSpan(tfloat '[1@2001-01-01, 4@2001-01-4]', floatspan '[2,3]');
-- {[1@2001-01-01, 2@2001-01-02), (3@2001-01-03, 4@2001-01-04)}
SELECT minusSpanset(tfloat '[1@2001-01-01, 5@2001-01-05]',
  floatspanset '{[1,2], [3,4]}');
-- {[2@2001-01-02, 3@2001-01-03), (4@2001-01-04, 5@2001-01-05)}
SELECT asText(minusValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03]',
  geometry 'Point(1 1 1)'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
  (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03)} */
SELECT asText(minusValues(tgeompoint '[Point(0 0 0)@2001-01-01, Point(3 3 3)@2001-01-04]',
  geomset '{"Point(1 1 1)", "Point(2 2 2)"}'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
  (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03),
  (POINT Z (2 2 2)@2001-01-03, POINT Z (3 3 3)@2001-01-04)} */
SELECT asText(minusValue(tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,2 2)@2001-01-02,
  Linestring(0 0,2 2)@2001-01-03]', geometry 'Linestring(0 0,2 2)'));
-- {[POINT(0 0)@2001-01-01, POINT(0 0)@2001-01-02]}
```

- Restrict to (the complement of) a time value

atTime(ttype,times) → ttype

minusTime(ttype,times) → ttype

```
SELECT atTime(tfloat '[1@2001-01-01, 5@2001-01-05]', timestamptz '2001-01-02');
-- 2@2001-01-02
SELECT atTime(tbigint '[1@2001-01-01, 1@2001-01-15]',
  tstzset '{2001-01-01, 2001-01-03}');
-- {1@2001-01-01, 1@2001-01-03}
SELECT atTime(tfloat '{[1@2001-01-01, 3@2001-01-03), [3@2001-01-04, 1@2001-01-06)}',
  tstzspan '[2001-01-02,2001-01-05]');
-- {[2@2001-01-02, 3@2001-01-03), [3@2001-01-04, 2@2001-01-05)}
SELECT atTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05]}');
-- {[1@2001-01-01, 1@2001-01-03), [1@2001-01-04, 1@2001-01-05)}
```

```
SELECT minusTime(tfloat '[1@2001-01-01, 5@2001-01-05]', timestamptz '2001-01-02');
-- {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 5@2001-01-05)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzset '{2001-01-02, 2001-01-03}');
/* {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03),
  (1@2001-01-03, 1@2001-01-15)} */
```



```
SELECT minusTime(tfloat '{[1@2001-01-01, 3@2001-01-03], [3@2001-01-04, 1@2001-01-06]}',
  tstzspan '[2001-01-02,2001-01-05]');
-- {[1@2001-01-01, 2@2001-01-02), [2@2001-01-05, 1@2001-01-06)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzspanset '{[2001-01-02, 2001-01-03], [2001-01-04, 2001-01-05]}');
/* {[1@2001-01-01, 1@2001-01-02), [1@2001-01-03, 1@2001-01-04),
  [1@2001-01-05, 1@2001-01-15]} */
```

- Keep the instants of a temporal value that are before/after or equal a timestamp

beforeTimestamp(ttype,timestamp,tz,strict bool=TRUE) → ttype

afterTimestamp(ttype,timestamp,tz,strict bool=TRUE) → ttype

```
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-02');
-- [1@2001-01-01, 1@2001-01-02)
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-01');
-- NULL
SELECT beforeTimestamp(tbigint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-01',
  false);
-- [1@2001-01-01]
SELECT afterTimestamp(tfloat '[1@2001-01-01, 3@2001-01-03]', timestamptz '2001-01-02',
  false);
-- [2@2001-01-02, 3@2001-01-03]
SELECT asText(afterTimestamp(tgeopoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  timestamptz '2001-01-03'));
/* {[POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05]} */
```

5.3 Bounding Box Operators

These operators test whether the bounding boxes of their arguments satisfy the predicate and result in a Boolean value. As stated in Chapter 4, the bounding box associated to a temporal type depends on the base type: It is the `tstzspan` type for the `tbool` and `ttext` types, the `tbox` type for the `tint`, `tbigint`, and `tfloat` types, and the `stbox` type for the `tgeopoint` and `tgeogpoint` types. Furthermore, as seen in Section 3.3, many PostgreSQL, PostGIS, or MobilityDB types can be converted to the `tbox` and `stbox` types. For example, numeric and span types can be converted to type `tbox`, types `geometry` and `geography` can be converted to type `stbox`, and time types and temporal types can be converted to types `tbox` and `stbox`.

A first set of operators consider the topological relationships between the bounding boxes. There are five topological operators: overlaps (`&&`), contains (`@>`), contained (`<@`), same (`~=`), and adjacent (`-|-`). The arguments of these operators can be a base type, a box, or a temporal type and the operators verify the topological relationship taking into account the value and/or the time dimension depending on the type of the arguments.

Another set of operators consider the relative position of the bounding boxes. The operators `<<`, `>>`, `&<`, and `&>` consider the value dimension for the `tint`, `tbigint`, and `tfloat` types and the X coordinates for the `tgeopoint` and `tgeogpoint` types, the operators `<<|`, `|>>`, `&<|`, and `|&>` consider the Y coordinates for the `tgeopoint` and `tgeogpoint` types, the operators `<</`, `/>>`, `&</`, and `/&>` consider the Z coordinates for the `tgeopoint` and `tgeogpoint` types, and the operators `<<#`, `#>>`, `#&<`, and `#&>` consider the time dimension for all temporal types.

Finally, it is worth noting that the bounding box operators allow to mix 2D/3D geometries but in that case, the computation is only performed on 2D.

We refer to Section 3.9 for the bounding box operators.

5.4 Comparisons

5.4.1 Traditional Comparisons

The traditional comparison operators (`=`, `<`, and so on) require that the left and right operands be of the same base type. Excepted equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed

on temporal types. These operators compare the bounding periods (see Section 2.10), then the bounding boxes (see Section 3.10) and if those are equal, then the comparison depends on the subtype. For instant values, they compare first the timestamps and if those are equal, compare the values. For sequence values, they compare the first N instants, where N is the minimum of the number of composing instants of both values. Finally, for sequence set values, they compare the first N sequence values, where N is the minimum of the number of composing sequences of both values.

The equality and inequality operators consider the equivalent representation for different subtypes as shown next.

```
SELECT tint '1@2001-01-01' = tint '{1@2001-01-01}';
-- true
SELECT tfloat '1.5@2001-01-01' = tfloat '[1.5@2001-01-01]';
-- true
SELECT ttext 'AAA@2001-01-01' = ttext '{[AAA@2001-01-01]}';
-- true
SELECT tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}' =
  tgeompoint '{[Point(1 1)@2001-01-01], [Point(2 2)@2001-01-02]}';
-- true
SELECT tgeography '{Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02}' =
  tgeography '{[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]}';
-- true
```

- Traditional comparisons

ttype {=, <>, <, >, <=, >=} ttype → boolean

cmp(ttype,ttype) → int

Function cmp returns -1, 0, or 1 depending on whether the first argument is, respectively, less than, equal to, or greater than the second one.

```
SELECT tint '[1@2001-01-01, 1@2001-01-04]' = tint '[2@2001-01-03, 2@2001-01-05]';
-- false
SELECT tint '[1@2001-01-01, 1@2001-01-04]' <> tint '[2@2001-01-03, 2@2001-01-05]';
-- true
SELECT tint '[1@2001-01-01, 1@2001-01-04]' < tint '[2@2001-01-03, 2@2001-01-05]';
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-04]' > tfloat '[2@2001-01-03, 2@2001-01-05]';
-- false
SELECT tbigint '[1@2001-01-01, 1@2001-01-04]' <= tbigint '[2@2001-01-03, 2@2001-01-05]';
-- true
SELECT tbigint '[1@2001-01-01, 1@2001-01-04]' >= tbigint '[2@2001-01-03, 2@2001-01-05]';
-- false
SELECT cmp(tfloat '[1@2001-01-01, 1@2001-01-04]', '[2@2001-01-03, 2@2001-01-05]');
-- -1
```

5.4.2 Ever and Always Comparisons

A possible generalization of the traditional comparison operators (=, <>, <, <=, etc.) to temporal types consists in determining whether the comparison is ever or always true. In this case, the result is a Boolean value. MobilityDB provides operators to test whether the comparison of a temporal value and a value of the base type or two temporal values is ever or always true. These operators are denoted by prefixing the traditional comparison operators with, respectively, ? (ever) and % (always). Some examples are ?=, %<>, or ?<=, Ever and always equality and non-equality are available for all temporal types, while ever and always inequalities are only available for temporal types whose base type has a total order defined, that is, tint, tfloat, or ttext. The ever and always comparisons are inverse operators: for example, ?= is the inverse of %<>, and ?> is the inverse of %<=.

- Ever and always comparisons

{base,ttype} {?=?, ?<>, ?<, ?>, ?<=, ?>=} {base,ttype} → boolean

{base,ttype} {%=, %<>, %<, %>, %<=, %>=} {base,ttype} → boolean

The operators do not take into account whether the bounds are inclusive or not.

```

SELECT tint '[1@2001-01-01, 3@2001-01-04]' ?= 2;
-- false
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,1 1)@2001-01-04]' ?=
  geometry 'Linestring(1 1,0 0)';
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-04)' %= 1;
-- true
SELECT tgeometry '[Linestring(0 0,1 1)@2001-01-01, Linestring(1 1,0 0)@2001-01-04)' %=
  geometry 'Linestring(0 0,1 1)';
-- true

```

```

SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' ?<> 2;
-- true
SELECT tgeopoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' ?<>
  geometry 'Point(1 1)';
-- false
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' %<> 2;
-- false
SELECT tgeopoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' %<>
  geography 'Point(2 2)';
-- true

```

```

SELECT tint '[1@2001-01-01, 4@2001-01-04]' ?< 2;
-- true
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' %< 2;
-- false

```

```

SELECT tint '[1@2001-01-03, 1@2001-01-05]' ?> 0;
-- true
SELECT tfloat '[1@2001-01-03, 1@2001-01-05)' %> 1;
-- false

```

```

SELECT tint '[1@2001-01-01, 1@2001-01-05]' ?<= 2;
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-05)' %<= 4;
-- true

```

```

SELECT ttext ' [{AAA@2001-01-01, AAA@2001-01-03), [BBB@2001-01-04, BBB@2001-01-05})'
  ?> 'AAA'::text;
-- true
SELECT ttext ' [{AAA@2001-01-01, AAA@2001-01-03), [BBB@2001-01-04, BBB@2001-01-05})'
  %> 'AAA'::text;
-- false

```

5.4.3 Temporal Comparisons

Another possible generalization of the traditional comparison operators ($=$, $<>$, $<$, $<=$, etc.) to temporal types consists in determining whether the comparison is true or false at each instant. In this case, the result is a temporal Boolean. The temporal comparison operators are denoted by prefixing the traditional comparison operators with $\#$. Some examples are $\#=$ or $\#<=$. Temporal equality and non-equality are available for all temporal types, while temporal inequalities are only available for temporal types whose base type has a total order defined, that is, tint , tfloat , or ttext .

- Temporal comparisons

```
{base,ttype} {#= $\#<>$ , #<, #>, #<=, #>=} {base,ttype}  $\rightarrow$  boolean
```

```

SELECT tfloat '[1@2001-01-01, 2@2001-01-04]' #= 3;
-- {[f@2001-01-01, f@2001-01-04]}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #= tfloat '[4@2001-01-02, 1@2001-01-05]';
-- {[f@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04)}
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03]' #=
  geometry 'Point(1 1)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(1 1,2 2)@2001-01-03]' #=
  geometry 'Linestring(2 2,1 1)';
-- {[f@2001-01-01, t@2001-01-03]}

```

```

SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #<> 2;
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, 2001-01-04)}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #<> tfloat '[2@2001-01-02, 2@2001-01-05]';
-- {[f@2001-01-02], (t@2001-01-02, t@2001-01-04)}
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(1 1,2 2)@2001-01-03]' #<>
  geometry 'Linestring(2 2,1 1)';
-- {[t@2001-01-01, f@2001-01-03]}

```

```

SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #< 2;
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-04]}
SELECT 1 #> tint '[1@2001-01-03, 1@2001-01-05]';
-- [f@2001-01-03, f@2001-01-05)
SELECT tfloat '[1@2001-01-01, 1@2001-01-05]' #<= tfloat '{2@2001-01-03, 3@2001-01-04}';
-- {t@2001-01-03, t@2001-01-04}
SELECT 'AAA'::text #> ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}';
-- {[f@2001-01-01, f@2001-01-03), [t@2001-01-04, t@2001-01-05)}

```

5.5 Miscellaneous

- Version of the MobilityDB extension

mobilitydb_version() → text

```

SELECT mobilitydb_version();
-- MobilityDB 1.3.0

```

- Versions of the MobilityDB extension and its dependencies

mobilitydb_full_version() → text

```

SELECT mobilitydb_full_version();
/* MobilityDB 1.3.0, PostgreSQL 17.2, PostGIS 3.5.2, GEOS 3.12.0-CAPI-1.18.0, PROJ 9.2.0,
  JSON-C 0.13.1, GSL 2.5 */

```

Chapter 6

Temporal Alphanumeric Types

6.1 Notation

We presented in Section 4.5 the notation used for defining the signature of the functions and operators for temporal alphanumeric types. We extend next this notations for temporal alphanumeric types.

- `torder` represents any temporal type whose base type has a total order defined, that is, `tint`, `tfloat`, or `ttext`,
- `talpha` represents any temporal alphanumeric type, such as, `tint` or `ttext`,
- `tnumber` represents any temporal number type, that is, `tint`, `tbigint`, or `tfloat`,
- `number` represents any number base type, that is, `integer`, `bigint`, or `float`,
- `numspan` represents any number span type, that is, either `intspan`, `bigintspan`, or `floatspan`,

6.2 Conversions

A temporal value can be converted into a compatible type using the notation `CAST(ttype1 AS ttype2)` or `ttype1::ttype2`.

- Convert a temporal number to a span or to a temporal bounding box

```
tnumber::{span,tbox}
```

```
valueSpan(tnumber) → numspan
```

```
timeSpan(tnumber) → tstzspan
```

```
tbox(tnumber) → tbox
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]':::intspan;
-- [1, 3]
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05)':::floatspan;
-- (1, 3]
SELECT valueSpan(tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05)');
-- (1, 3]
SELECT timeSpan(tfloat 'Interp=Step;(1@2001-01-01, 3@2001-01-03, 2@2001-01-05)');
-- (2001-01-01, 2001-01-05]
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]':::tbox;
-- TBOXINT XT((1,2),[2001-01-01,2001-01-03])
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05)':::tbox;
-- TBOXFLOAT XT((1,3),[2001-01-01,2001-01-05])
```


- Return the instant with the minimum or maximum value

`minInstant(torder) → base`

`maxInstant(torder) → base`

The function does not take into account whether the bounds are inclusive or not. If several instants have the minimum value, the first one is returned.

```
SELECT minInstant(tfloat '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- 1@2001-01-01
SELECT maxInstant(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}');
-- 5@2001-01-05
```

6.4 Transformations

- Shift and/or the value span of a temporal number by one or two numbers

`shiftValue(tnumber,base) → tnumber`

`scaleValue(tnumber,width) → tnumber`

`shiftScaleValue(tnumber,base,base) → tnumber`

For scaling, if the value span of the temporal value is a single value (for example, for a temporal instant), the result is the temporal value. Furthermore, the given width must be strictly greater than zero.

```
SELECT shiftValue(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', 1);
-- {2@2001-01-02, 3@2001-01-04, 2@2001-01-06}
SELECT shiftValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}', -1);
-- {[0@2001-01-01, 1@2001-01-02], [2@2001-01-03, 3@2001-01-04]}
SELECT scaleValue(tint '1@2001-01-01', 1);
-- 1@2001-01-01
SELECT scaleValue(tfloat '{[1@2001-01-01,2@2001-01-02], [3@2001-01-03,4@2001-01-04]}', 6);
-- {[1@2001-01-01, 3@2001-01-03], [5@2001-01-05, 7@2001-01-07]}
SELECT scaleValue(tint '1@2001-01-01', -1);
-- ERROR: The value must be strictly positive: -1
SELECT shiftScaleValue(tint '1@2001-01-01', 1, 1);
-- 2@2001-01-01
SELECT shiftScaleValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}',
-1, 6);
-- {[0@2001-01-01, 2@2001-01-02], [4@2001-01-03, 6@2001-01-04]}
```

- Extract from a temporal float with linear interpolation the subsequences where the values stay within a span of a given width for at least a given duration

`stops(tfloat,maxDist=0.0,minDuration='0 minutes') → tfloat`

If `maxDist` is not given, a value 0.0 is assumed by default and thus, the function extracts the constant segments of the temporal float.

```
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]');
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]', 0.0, '2 days');
-- NULL
```

6.5 Restrictions

- Restrict to (the complement of) the minimum or maximum value

`atMin(torder) → torder`

```
atMax(torder) → torder
minusMin(torder) → torder
minusMax(torder) → torder
```

The functions returns a NULL value if the minimum value only happens at exclusive bounds.

```
SELECT atMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {1@2001-01-01, 1@2001-01-05}
SELECT atMin(tint '{(1@2001-01-01, 3@2001-01-03)}');
-- {(1@2001-01-01, 1@2001-01-03)}
SELECT atMin(tfloat '{(1@2001-01-01, 3@2001-01-03)}');
-- NULL
SELECT atMin(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05]}');
-- {(AA@2001-01-01, AA@2001-01-03), [AA@2001-01-05]}
SELECT atMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {3@2001-01-05}
SELECT atMax(tfloat '{(1@2001-01-01, 3@2001-01-03)}');
-- NULL
SELECT atMax(tfloat '{(2@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05]}');
-- {[2@2001-01-03, 2@2001-01-05]}
SELECT atMax(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05]}');
-- {"BB"@2001-01-03, "BB"@2001-01-05}
```

```
SELECT minusMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {2@2001-01-03}
SELECT minusMin(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMin(tfloat '{(1@2001-01-01, 3@2001-01-03)}');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMin(tint '{[1@2001-01-01, 1@2001-01-03], (1@2001-01-03, 1@2001-01-05]}');
-- NULL
SELECT minusMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {1@2001-01-01, 2@2001-01-03}
SELECT minusMax(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {[1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '{(1@2001-01-01, 3@2001-01-03)}');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '{[2@2001-01-01, 1@2001-01-03], [2@2001-01-03, 2@2001-01-05]}');
-- {(2@2001-01-01, 1@2001-01-03)}
SELECT minusMax(tfloat '{[1@2001-01-01, 3@2001-01-03], (3@2001-01-03, 1@2001-01-05]}');
-- {[1@2001-01-01, 3@2001-01-03), (3@2001-01-03, 1@2001-01-05)}
```

- Restrict to (the complement of) a tbox

```
atTbox(tnumber, tbox) → tnumber
minusTbox(tnumber, tbox) → tnumber
```

Cuando el cuadro delimitador tiene dimensiones tanto de valores como temporales, las funciones restringen el número temporal con respecto al valor y las extensiones temporales del cuadro.

```
SELECT atTbox(tfloat '[0@2001-01-01, 3@2001-01-04]',
  tbox 'TBOXFLOAT XT((0,2), [2001-01-02, 2001-01-04])');
-- {[1@2001-01-02, 2@2001-01-03]}
```

```
SELECT minusTbox(tfloat '[1@2001-01-01, 4@2001-01-04]',
  'TBOXFLOAT XT((1,4), [2001-01-03, 2001-01-04])');
-- {[1@2001-01-01, 3@2001-01-03)}
WITH temp(temp, box) AS (
  SELECT tfloat '[1@2001-01-01, 4@2001-01-04]',
    tbox 'TBOXFLOAT XT((1,2), [2001-01-03, 2001-01-04])' )
SELECT minusSpan(minusTime(temp, box::tstzspan), box::floatspan) FROM temp;
-- {[1@2001-01-01], [2@2001-01-02, 3@2001-01-03]}
```


6.6 Boolean Operations

- Temporal and, temporal or

$\{\text{boolean}, \text{tbool}\} \{\&, |\} \{\text{boolean}, \text{tbool}\} \rightarrow \text{tbool}$

```
SELECT tbool '[true@2001-01-03, true@2001-01-05)' &
  tbool '[false@2001-01-03, false@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
SELECT tbool '[true@2001-01-03, true@2001-01-05)' &
  tbool '[false@2001-01-03, false@2001-01-04),
  [true@2001-01-04, true@2001-01-05)';
-- {[f@2001-01-03, t@2001-01-04, t@2001-01-05)}
SELECT tbool '[true@2001-01-03, true@2001-01-05)' |
  tbool '[false@2001-01-03, false@2001-01-05)';
-- [t@2001-01-03, t@2001-01-05)
```

- Temporal not

$\sim \text{tbool} \rightarrow \text{tbool}$

```
SELECT ~tbool '[true@2001-01-03, true@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
```

- Return the time when a temporal Boolean takes the value true

$\text{whenTrue}(\text{tbool}) \rightarrow \text{tstzspanset}$

```
SELECT whenTrue(tfloat '[1@2001-01-01, 4@2001-01-04, 1@2001-01-07]' #> 2);
-- {(2001-01-02, 2001-01-06)}
SELECT whenTrue(tdwithin(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04,
  Point(1 1)@2001-01-07]', geometry 'Point(1 1)', sqrt(2)));
-- {[2001-01-01, 2001-01-02], [2001-01-06, 2001-01-07]}
```

6.7 Mathematical Operations

- Temporal addition, subtraction, multiplication, and division

$\{\text{number}, \text{tnumber}\} \{+, -, *, /\} \{\text{number}, \text{tnumber}\} \rightarrow \text{tnumber}$

The temporal division will raise an error if the denominator is ever equal to zero during the common timespan of the arguments.

```
SELECT tint '[2@2001-01-01, 2@2001-01-04)' + 1;
-- [3@2001-01-01, 3@2001-01-04)
SELECT tfloat '[2@2001-01-01, 2@2001-01-04)' + tfloat '[1@2001-01-01, 4@2001-01-04)';
-- [3@2001-01-01, 6@2001-01-04)
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' +
  tfloat '[{1@2001-01-01, 2@2001-01-02), [1@2001-01-02, 2@2001-01-04)}';
-- {[2@2001-01-01, 4@2001-01-04), [3@2001-01-02, 6@2001-01-04)}
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-04)' - tint '[2@2001-01-03, 2@2001-01-05)';
-- [-1@2001-01-03, -1@2001-01-04)
SELECT tfloat '[3@2001-01-01, 6@2001-01-04)' - tfloat '[2@2001-01-01, 2@2001-01-04)';
-- [1@2001-01-01, 4@2001-01-04)
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04)' * 2;
-- [2@2001-01-01, 8@2001-01-04)
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' * tfloat '[2@2001-01-01, 2@2001-01-04)';
-- [2@2001-01-01, 8@2001-01-04)
SELECT tfloat '[1@2001-01-01, 3@2001-01-03)' * '[3@2001-01-01, 1@2001-01-03)';
-- {[3@2001-01-01, 4@2001-01-02, 3@2001-01-03)}
```

```

SELECT 2 / tfloat '[1@2001-01-01, 3@2001-01-04]';
-- [2@2001-01-01, 0.6666666666666667@2001-01-04]
SELECT tfloat '[1@2001-01-01, 5@2001-01-05]' / tfloat '[5@2001-01-01, 1@2001-01-05]';
-- {[0.2@2001-01-01, 1@2001-01-03, 2001-01-03, 5@2001-01-03, 2001-01-05]}
SELECT 2 / tfloat '[-1@2001-01-01, 1@2001-01-02]';
-- ERROR: Division by zero
SELECT tfloat '[-1@2001-01-04, 1@2001-01-05]' / tfloat '[-1@2001-01-01, 1@2001-01-05]';
-- [-2@2001-01-04, 1@2001-01-05]

```

- Return the absolute value of the temporal number

`abs(tnumber) → tnumber`

```

SELECT abs(tfloat '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03, 0@2001-01-04, 1@2001-01-05]
SELECT abs(tint '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 1@2001-01-05]

```

- Round up or down to the nearest integer

`floor(tfloat) → tfloat`

`ceil(tfloat) → tfloat`

```

SELECT floor(tfloat '[0.5@2001-01-01, 1.5@2001-01-02]');
-- [0@2001-01-01, 1@2001-01-02]
SELECT ceil(tfloat '[0.5@2001-01-01, 0.6@2001-01-02, 0.7@2001-01-03]');
-- [1@2001-01-01, 1@2001-01-03]

```

- Round to a number of decimal places

`round(tfloat, integer=0) → tfloat`

```

SELECT round(tfloat '[0.785398163397448@2001-01-01, 2.356194490192345@2001-01-02]', 2);
-- [0.79@2001-01-01, 2.36@2001-01-02]

```

- Convert to degrees or radians

`degrees({float, tfloat}, normalize=false) → tfloat`

`radians(tfloat) → tfloat`

The additional parameter in the `degrees` function can be used to normalize the values between 0 and 360 degrees.

```

SELECT degrees(pi() * 5);
-- 900
SELECT degrees(pi() * 5, true);
-- 180
SELECT round(degrees(tfloat '[0.785398163397448@2001-01-01, 2.356194490192345@2001-01-02]' ↵
));
-- [45@2001-01-01, 135@2001-01-02]
SELECT radians(tfloat '[45@2001-01-01, 135@2001-01-02]');
-- [0.785398163397448@2001-01-01, 2.356194490192345@2001-01-02]

```

- Return the value difference between consecutive instants of the temporal number

`deltaValue(tnumber) → tnumber`

```

SELECT deltaValue(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, -1@2001-01-02, -1@2001-01-03]
SELECT deltaValue(tfloat '[1.5@2001-01-01, 2@2001-01-02, 1@2001-01-03],
[2@2001-01-04, 2@2001-01-05]]');
/* Interp=Step; {[0.5@2001-01-01, -1@2001-01-02, -1@2001-01-03],
[0@2001-01-04, 0@2001-01-05]} */

```

- Return the trend of a temporal float with linear interpolation, which states whether its value is increasing, constant, or decreasing, represented, respectively, by 1, 0, and -1.

trend(tfloat) → tint

Note that the trend is NULL for instantaneous sequences.

```
SELECT trend(tfloat '[1@2001-01-01, 2@2001-01-02, 4@2001-01-03, 4@2001-01-04, 3@2001-01-05, 2@2001-01-06]');
-- [1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]
SELECT trend(tfloat '[1@2001-01-01]');
-- NULL
```

- Return the derivative over time of a temporal float in units per second

derivative(tfloat) → tfloat

The temporal float must have linear interpolation. Note that this function corresponds to the **speed** function for temporal points.

```
SELECT derivative(tfloat '{[0@2001-01-01, 10@2001-01-02, 5@2001-01-03], [1@2001-01-04, 0@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[-10@2001-01-01, 5@2001-01-02, 5@2001-01-03], [1@2001-01-04, 1@2001-01-05]} */
SELECT derivative(tfloat 'Interp=Step;[0@2001-01-01, 10@2001-01-02, 5@2001-01-03]');
-- ERROR: The temporal value must have linear interpolation
```

- Return the area under the curve

integral(tnumber) → float

```
SELECT integral(tint '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1
SELECT integral(tfloat '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1.5
```

- Return the time-weighted average

twAvg(tnumber) → float

```
SELECT twAvg(tfloat '{[1@2001-01-01, 2@2001-01-03], [2@2001-01-04, 2@2001-01-06]}');
-- 1.75
```

- Return the natural logarithm and the base 10 logarithm of a temporal float

ln(tfloat) → tfloat

log10(tfloat) → tfloat

The temporal float cannot be zero or negative

```
SELECT ln(tfloat '{[1@2001-01-01, 10@2001-01-02, 5@2001-01-03], [1@2001-01-04, 1@2001-01-05]}');
/* {[0@2001-01-01, 2.302585092994046@2001-01-02, 1.6094379124341@2001-01-03], [0@2001-01-04, 0@2001-01-05]} */
SELECT log10(tfloat 'Interp=Step;[-10@2001-01-01, 10@2001-01-02]');
-- ERROR: Cannot take logarithm of zero or a negative number
```

- Return the exponential (e raised to the given power) of a temporal float

exp(tfloat) → tfloat

```
SELECT exp(tfloat '{[1@2001-01-01, 10@2001-01-02], [1@2001-01-04, 1@2001-01-05]}');
/* {[2.718281828459045@2001-01-01, 22026.465794806718@2001-01-02], [2.718281828459045@2001-01-04, 2.718281828459045@2001-01-05]} */
SELECT exp(tfloat '{-10@2001-01-01, 0@2001-01-02, 10@2001-01-03}');
-- {0.000045399929762@2001-01-01, 1@2001-01-02, 22026.465794806718@2001-01-03}
```

- Return the sine, cosine, or tangent of a temporal float (argument in radians)

`sin(tfloat) → tfloat`

`cos(tfloat) → tfloat`

`tan(tfloat) → tfloat`

```
SELECT round(sin(tfloat '[0@2001-01-01, 6.283185307@2001-01-02]'), 6);
-- [0@2001-01-01, -0.000000@2001-01-02]
SELECT round(cos(tfloat '[0@2001-01-01, 3.141592654@2001-01-02]'), 6);
-- [1@2001-01-01, -1@2001-01-02]
SELECT round(tan(tfloat '{0@2001-01-01, 0.785398163@2001-01-02}'), 6);
-- {0@2001-01-01, 1@2001-01-02}
```

6.8 Text Operations

- Text concatenation

`{text,ttext} || {text,ttext} → ttext`

```
SELECT ttext '[AA@2001-01-01, AA@2001-01-04]' || text 'B';
-- ["AAB"@2001-01-01, "AAB"@2001-01-04]
SELECT ttext '[AA@2001-01-01, AA@2001-01-04]' || ttext '[BB@2001-01-02, BB@2001-01-05]';
-- ["AABB"@2001-01-02, "AABB"@2001-01-04]
SELECT ttext '[A@2001-01-01, B@2001-01-03, C@2001-01-04]' ||
  ttext '{[D@2001-01-01, D@2001-01-02), [E@2001-01-02, E@2001-01-04)}';
-- [{"AD"@2001-01-01, "AE"@2001-01-02, "BE"@2001-01-03, "BE"@2001-01-04}]
```

- Transform in lowercase, uppercase, or initcap

`upper(ttext) → ttext`

`lower(ttext) → ttext`

`initcap(ttext) → ttext`

```
SELECT lower(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["aa"@2001-01-01, "bb"@2001-01-02]
SELECT upper(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["AA"@2001-01-01, "BB"@2001-01-02]
SELECT initcap(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["Aa"@2001-01-01, "Bb"@2001-01-02]
```

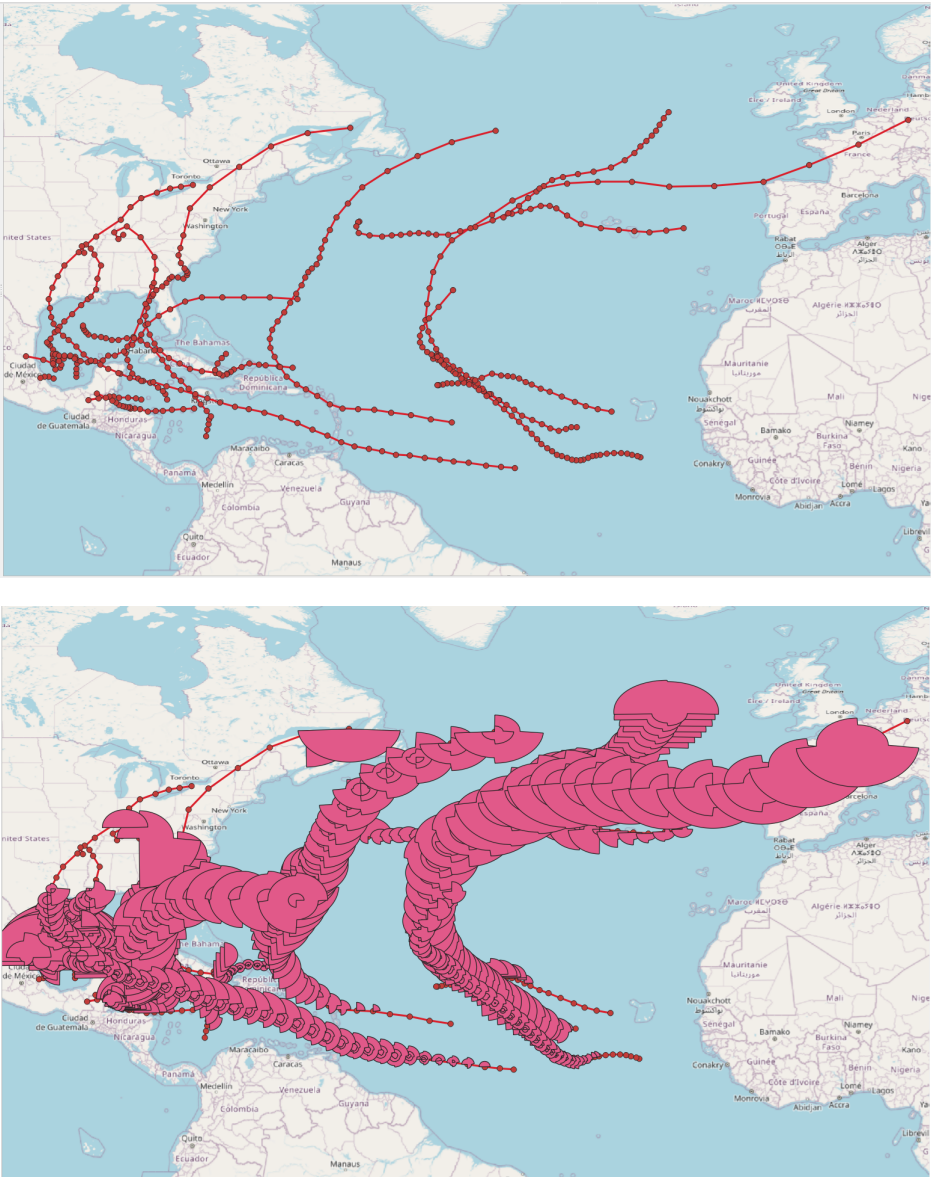



Figure 7.2: Illustration of the use of the `tgeompoint` (top) and the `tgeometry` (bottom) types for modelling the trajectory and the wind swath of tropical storms in 2024.

- `tgeo` represents a temporal geometry/geography type, that is, `tgeometry`, `tgeography`, `tgeompoint`, or `tgeogpoint`,
- `tgeom` represents a temporal geometry type that is, `tgeometry` or `tgeompoint`,
- `tpoint` represents a temporal point type, that is, `tgeompoint` or `tgeogpoint`,
- `spatial` represents a spatial base type, such as, `geometry`, `geography`, `pose`, or `npoint`,
- `geo` represents the types `geometry` or `geography`,
- `geompoint` represents the type `geometry` restricted to a point.
- `point` represents the types `geometry` or `geography` restricted to a point.
- `lines` represents the types `geometry` or `geography` restricted to a (multi)line.

In the following, we specify with the symbol  that the function supports 3D geometries and with the symbol  that the function supports geographies.

7.3 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation  

`asText({tspatial,tspatial[],spatial[]}) → {text,text[]}`

`asEWKT({tspatial,tspatial[],spatial[]}) → {text,text[]}`

```
SELECT asText(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- [POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02]
SELECT asText(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
/* {"[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
  "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"} */
SELECT asText(ARRAY[geometry 'Point(0 0)', 'Point(1 1)']);
-- {"POINT(0 0)","POINT(1 1)"}
```

```
SELECT asEWKT(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- SRID=4326;[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02]
SELECT asEWKT(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
-- {"SRID=4326;[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
  "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"}
SELECT asEWKT(ARRAY[geometry 'SRID=5676;Point(0 0)', 'SRID=5676;Point(1 1)']);
-- {"SRID=5676;POINT(0 0)","SRID=5676;POINT(1 1)"}
```

- Return the Moving Features JSON representation  

`asMFJSON(tspatial,options=0,flags=0,maxdecdigits=15) → bytea`

The `options` argument can be used to add BBOX and/or CRS in MFJSON output:

- 0: means no option (default value)
- 1: MFJSON BBOX
- 2: MFJSON Short CRS (e.g., EPSG:4326)
- 4: MFJSON Long CRS (e.g., urn:ogc:def:crs:EPSG::4326)

The `flags` argument can be used to customize the JSON output, for example, to produce an easy-to-read (for human readers) JSON output. Refer to the documentation of the `json-c` library for the possible values. Typical values are as follows:

- 0: means no option (default value)

- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

The `maxdecdigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

```
SELECT asMFJSON(tgeompoint 'Point(1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingPoint","coordinates":[[1,2]],"datetimes":["2019-01-01T17:00:00.15+01"],
   "interpolation":"None"} */
SELECT asMFJSON(tgeometry 'SRID=3812;Linestring(1 1,1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingGeometry",
   "crs":{"type":"Name","properties":{"name":"EPSG:3812"}},
   "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]}],
   "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"} */
SELECT asMFJSON(tgeompoint 'SRID=4326;
   Point(50.813810 4.384260)@2019-01-01 18:00:00.15+02', 3, 0, 2);
/* {"type":"MovingPoint","crs":{"type":"name","properties":{"name":"EPSG:4326"}},
   "stBoundedBy":{"bbox":[50.81,4.38,50.81,4.38],
   "period":{"begin":"2019-01-01 17:00:00.15+01","end":"2019-01-01 17:00:00.15+01"}},
   "coordinates":[[50.81,4.38]],"datetimes":["2019-01-01T17:00:00.15+01"],
   "interpolations":"None"} */
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB) representation, or the Hexadecimal Extended Well-Known Binary (EWKB) representation  

`asBinary(tspatial, endian text="")` → bytea

`asEWKB(tspatial, endian text="")` → bytea

`asHexEWKB(tspatial, endian text="")` → text

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(tgeompoint 'Point(1 2 3)@2001-01-01');
-- \x012e001100000000000000f03f000000000000040000000000000840009c57d3c11c0000
SELECT asEWKB(tgeogpoint 'SRID=7844;Point(1 2 3)@2001-01-01');
-- \x012f0071a41e00000000000000f03f000000000000040000000000000840009c57d3c11c0000
SELECT asHexEWKB(tgeompoint 'SRID=3812;Point(1 2 3)@2001-01-01');
-- 012E0051E40E0000000000000000F03F000000000000040000000000000840009C57D3C11C0000
```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation  

`tspatialFromText(text)` → tspatial

`tspatialFromEWKT(text)` → tspatial

In the above functions, `tspatial` replaces any spatial type, such as `tgeompoint`, `tgeometry`, or `tpose`.

```
SELECT asEWKT(tgeompointFromText(text '[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]'));
-- [POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
SELECT asEWKT(tgeographyFromText(text '[Point(1 2)@2001-01-01, Linestring(1 2,3 4)@2001-01-02]'));
-- SRID=4326;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
```

```
SELECT asEWKT(tgeompointFromEWKT(text 'SRID=3812;[Point(1 2)@2001-01-01,
   Point(3 4)@2001-01-02]'));
-- SRID=3812;[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
SELECT asEWKT(tgeographyFromEWKT(text 'SRID=7844;[Point(1 2)@2001-01-01,
   Linestring(1 2,3 4)@2001-01-02]'));
-- SRID=7844;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
```


- Input from the Moving Features JSON representation  

`tspatialFromMFJSON(text) → tspatial`

In the above function, `tspatial` replaces any spatiotemporal type, for example, `tgeopoint`, `tgeometry`, or `tpose`

```
SELECT asEWKT(tgeopointFromMFJSON(text '{"type":"MovingPoint","crs":{"type":"name",
"properties":{"name":"EPSG:4326"}},'coordinates":[[50.81,4.38]],
"datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15+01
SELECT asEWKT(tgeogpointFromMFJSON(text '{"type":"MovingPoint","crs":{"type":"name",
"properties":{"name":"EPSG:4326"}},'coordinates":[[50.81,4.38]],
"datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15+01
SELECT asEWKT(tgeographyFromMFJSON(text '{"type":"MovingGeometry",
"crs":{"type":"Name","properties":{"name":"EPSG:7844"}},
"values":[{"type":"LineString","coordinates":[[1,1],[1,2]]}],
"datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=7844;LINESTRING(1 1,1 2)@2019-01-01 17:00:00.15+01
```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation  

`tspatialFromBinary(bytea) → tspatial`

`tspatialFromEWKB(bytea) → tspatial`

`tspatialFromHexEWKB(text) → tspatial`

In the previous functions, `tspatial` replaces any spatiotemporal type, for example, `tgeopoint`, `tgeometry`, or `tpose`

```
SELECT asEWKT(tgeopointFromBinary(
'\x012e0011000000000000f03f00000000000040000000000000840009c57d3c11c0000'));
-- POINT Z (1 2 3)@2001-01-01
SELECT asEWKT(tgeogpointFromEWKB(
'\x012f0071a41e00000000000000f03f00000000000040000000000000840009c57d3c11c0000'));
-- SRID=7844;POINT Z (1 1 1)@2001-01-01
SELECT asEWKT(tgeopointFromHexEWKB(
'012E0051E40E00000000000000F03F00000000000040000000000000840009C57D3C11C0000'));
-- SRID=3812;POINT(1 2 3)@2001-01-01
```

7.4 Conversions

- Convert a spatiotemporal value to a spatiotemporal box

`tspatial::stbox`

`stbox(tspatial)`

```
SELECT tgeopoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]::stbox;
-- STBOX XT((1,1),(3,3)), [2001-01-01, 2001-01-03])
SELECT tgeography '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03]::stbox;
-- SRID=4326;GEODSTBOX ZT((1,1,1),(3,3,3)), [2001-01-01, 2001-01-03])
```

- Convert between a temporal geometry and a temporal geography

`tgeometry::tgeography`

`tgeography::tgeometry`

`tgeopoint::tgeogpoint`

`tgeogpoint::tgeopoint`

```
SELECT asText((tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02]')::tgeogpoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02]
SELECT asText((tgeography 'Linestring(0 0,1 1)@2001-01-01')::tgeometry);
-- LINESTRING(0 0,1 1)@2001-01-01
```

A common way to store temporal points in PostGIS is to represent them as geometries of type `LINESTRING M` and use the `M` dimension to encode timestamps as seconds since 1970-01-01 00:00:00. These time-enhanced geometries, called **trajectories**, can be validated with the function `ST_IsValidTrajectory` to verify that the `M` value is growing from each vertex to the next. Trajectories can be manipulated with the functions `ST_ClosestPointOfApproach`, `ST_DistanceCPA`, and `ST_CPAWithin`. Temporal point values can be converted to/from PostGIS trajectories.

- Convert between a temporal point and a PostGIS trajectory

```
tpoint::geo
```

```
geo::tpoint
```

```
SELECT ST_AsText((tgeompoint 'Point(0 0)@2001-01-01')::geometry);
-- POINT M (0 0 978307200)
SELECT ST_AsText((tgeompoint '{Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 1)@2001-01-03}')::geometry);
-- MULTIPOINT M (0 0 978307200,1 1 978393600,1 1 978480000) "
SELECT ST_AsText((tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02]')::geometry);
-- LINESTRING M (0 0 978307200,1 1 978393600)
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02],
  [Point(1 1)@2001-01-03, Point(1 1)@2001-01-04],
  [Point(1 1)@2001-01-05, Point(0 0)@2001-01-06]}'::geometry);
/* MULTILINESTRING M ((0 0 978307200,1 1 978393600),(1 1 978480000,1 1 978566400),
  (1 1 978652800,0 0 978739200)) */
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02],
  [Point(1 1)@2001-01-03],
  [Point(1 1)@2001-01-05, Point(0 0)@2001-01-06]}'::geometry);
/* GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
  POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200)) */

SELECT asText(geometry 'LINESTRING M (0 0 978307200,0 1 978393600,
  1 1 978480000)'::tgeompoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02, POINT(1 1)@2001-01-03];
SELECT asText(geometry 'GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
  POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200))'::tgeompoint);
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(1 1)@2001-01-03],
  [POINT(1 1)@2001-01-05, POINT(0 0)@2001-01-06]} */
```

7.5 Accessors

- Return the trajectory or the traversed area  

```
trajectory(tpoint, unary_union=false) → geo
```

```
traversedArea(tgeo, unary_union=false) → geo
```

This function is equivalent to **getValues** for temporal alphanumeric values. The last argument states whether the PostGIS function `ST_UnaryUnion` is applied to remove redundant geometries in the result. Notice that setting this argument to true is computationally expensive.

```
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
  Point(0 0)@2001-01-03]'));
-- LINESTRING(0 0,0 1,0 0)
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
```

```

    Point(0 0)@2001-01-03]', true));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajjectory(tgeompoint '{[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02],
    [Point(0 1)@2001-01-03, Point(0 0)@2001-01-04]}'));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajjectory(tgeompoint 'Interp=Step;{[Point(0 0)@2001-01-01,
    Point(0 1)@2001-01-02], [Point(0 1)@2001-01-03, Point(1 1)@2001-01-04]}'));
-- MULTIPOINT((0 0),(1 1),(0 1))

SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
    Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]'));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(1 1,2 2))
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
    Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]', true));
-- LINESTRING(1 1,2 2)

```

- Return the centroid as a temporal point 

centroid(tgeo) → tpoint

```

SELECT asText(centroid(tgeometry '[Point(1 1)@2000-01-01, Linestring(1 1,3 3)@2000-01-02,
    Polygon((1 1,4 4,7 1,1 1))@2000-01-03]'));
-- Interp=Step;[POINT(1 1)@2000-01-01, POINT(2 2)@2000-01-02, POINT(4 2)@2000-01-03]
SELECT asText(centroid(tgeography '[MultiPoint(1 1,4 4,7 1)@2000-01-01,
    Polygon((1 1,4 4,7 1,1 1))@2000-01-02]', 6));
-- Interp=Step;[POINT(4 2.001727)@2000-01-01, POINT(4 2.001727)@2000-01-02]

```

- Return the X/Y/Z coordinate values as a temporal float  

getX(tpoint) → tfloat

getY(tpoint) → tfloat

getZ(tpoint) → tfloat

```

SELECT getX(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
    Point(5 6)@2001-01-03}');
-- {1@2001-01-01, 3@2001-01-02, 5@2001-01-03}
SELECT getX(tgeogpoint 'Interp=Step;[Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02,
    Point(7 8 9)@2001-01-03]');
-- Interp=Step;[1@2001-01-01, 4@2001-01-02, 7@2001-01-03]
SELECT getY(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
    Point(5 6)@2001-01-03}');
-- {2@2001-01-01, 4@2001-01-02, 6@2001-01-03}
SELECT getY(tgeogpoint 'Interp=Step;[Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02,
    Point(7 8 9)@2001-01-03]');
-- Interp=Step;[2@2001-01-01, 5@2001-01-02, 8@2001-01-03]
SELECT getZ(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
    Point(5 6)@2001-01-03}');
-- The temporal point must have Z dimension
SELECT getZ(tgeogpoint 'Interp=Step;[Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02,
    Point(7 8 9)@2001-01-03]');
-- Interp=Step;[3@2001-01-01, 6@2001-01-02, 9@2001-01-03]

```

- Return true if the temporal point does not spatially self-intersect 

isSimple(tpoint) → boolean

Notice that a temporal sequence set point is simple if every composing sequence is simple.

```

SELECT isSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
    Point(0 0)@2001-01-03]');
-- false

```

```
SELECT isSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]');
-- false
SELECT isSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
[Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}');
-- true
```

- Return the length traversed by the temporal point  

length(tpoint) → float

```
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- 1.73205080756888
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
Point(0 0 0)@2001-01-03]');
-- 3.46410161513775
SELECT length(tgeompoint 'Interp=Step;[Point(0 0 0)@2001-01-01,
Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- 0
```

- Return the cumulative length traversed by the temporal point  

cumulativeLength(tpoint) → tfloatSeq

```
SELECT round(cumulativeLength(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}'), 6);
-- {[0@2001-01-01, 1.414214@2001-01-02, 2.414214@2001-01-03],
[2.414214@2001-01-04, 3.414214@2001-01-05]}
SELECT cumulativeLength(tgeompoint 'Interp=Step;[Point(0 0 0)@2001-01-01,
Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- Interp=Step;[0@2001-01-01, 0@2001-01-03]
```

- Return the speed of the temporal point in units per second  

speed(tpoint) → tfloatSeqSet

The temporal point must have linear interpolation

```
SELECT speed(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[1.4142135623731@2001-01-01, 1@2001-01-02, 1@2001-01-03],
[1@2001-01-04, 1@2001-01-05]} */
SELECT speed(tgeompoint 'Interp=Step;[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
Point(1 0)@2001-01-03]');
-- ERROR: The temporal value must have linear interpolation
```

- Return the time-weighted centroid 

twCentroid(tgeompoint) → point

```
SELECT ST_AsText(twCentroid(tgeompoint '{[Point(0 0 0)@2001-01-01,
Point(0 1 1)@2001-01-02, Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}'));
-- POINT Z (0 0.666666666666667 0.666666666666667)
```

- Return the direction, that is, the azimuth between the start and end locations  

direction(tpoint) → float

The result is expressed in radians. It is NULL if there is only one location or if the start and end locations are equal.

```
SELECT round(degrees(direction(tgeompoint '[Point(0 0)@2001-01-01,
Point(-1 -1)@2001-01-02, Point(1 1)@2001-01-03]'))::numeric, 6);
-- 45.000000
```

```
SELECT direction(tgeompoint '{[Point(0 0 0)@2001-01-01,
  Point(0 1 1)@2001-01-02, Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}');
-- NULL
```

- Return the temporal azimuth  

azimuth(tpoint) → tfloat

The result is expressed in radians. The azimuth is undefined when two successive locations are equal and in this case a temporal gap is added.

```
SELECT round(degrees(azimuth(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-02, Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]')));
-- Interp=Step;{[45@2001-01-01, 45@2001-01-02], [225@2001-01-03, 225@2001-01-04]}
```

- Return the temporal angular difference 

angularDifference(tpoint) → tfloat

The result is expressed in degrees.

```
SELECT round(angularDifference(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]'), 3);
-- {0@2001-01-01, 180@2001-01-02, 0@2001-01-03}
SELECT round(degrees(angularDifference(tgeompoint '{[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02], [Point(2 2)@2001-01-03, Point(1 1)@2001-01-04]}'))), 3);
-- {0@2001-01-01, 0@2001-01-02, 0@2001-01-03, 0@2001-01-04}
```

- Return the temporal bearing  

bearing({tpoint,point},{tpoint,point}) → tfloat

Notice that this function does not accept two temporal geographic points.

```
SELECT degrees(bearing(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]',
  geometry 'Point(2 2)'));
-- [45@2001-01-01, 0@2001-01-02, 225@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]',
  tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]')), 3);
-- [63.435@2001-01-01, 0@2001-01-02, 296.565@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]')), 3);
-- [243.435@2001-01-01, 116.565@2001-01-03]
```

7.6 Transformations

- Round the coordinate values to a number of decimal places  

round(tspatial,integer=0) → tspatial

```
SELECT asText(round(tgeompoint '{Point(1.12345 1.12345 1.12345)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(1.12345 1.12345 1.12345)@2001-01-03}', 2));
/* {POINT Z (1.12 1.12 1.12)@2001-01-01, POINT Z (2 2 2)@2001-01-02,
  POINT Z (1.12 1.12 1.12)@2001-01-03} */
SELECT asText(round(tgeography 'Linestring(1.12345 1.12345,2.12345 2.12345)@2001-01-01',
  2));
-- LINESTRING(1.12 1.12,2.12 2.12)@2001-01-01
```

- Return an array of fragments of the temporal point which are simple 

makeSimple(tpoint) → tgeompoint[]

```

SELECT asText (makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
Point(0 0)@2001-01-03]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02]",
"[POINT(1 1)@2001-01-02, POINT(0 0)@2001-01-03]"} */
SELECT asText (makeSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]'));
/* {"[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 0 2)@2001-01-03,
POINT Z (0 0 0)@2001-01-04]"} */
SELECT asText (makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
Point(0 1)@2001-01-03, Point(1 0)@2001-01-04]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02, POINT(0 1)@2001-01-03]",
"[POINT(0 1)@2001-01-03, POINT(1 0)@2001-01-04]"} */
SELECT asText (makeSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
[Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}'));
/* {"{[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02],
[POINT Z (1 1 1)@2001-01-03, POINT Z (0 0 0)@2001-01-04]}"} */

```

- Construct a geometry/geography with M measure from a temporal point and a temporal float  

`geoMeasure(tpoint, tfloat, segmentize=false) → geo`

The last argument `segmentize` states whether the resulting value is a either `Linestring M` or a `MultiLinestring M` where each component is a segment of two points.

```

SELECT ST_AsText (geoMeasure(tgeompoint '{Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02}', '{5@2001-01-01, 5@2001-01-02}'));
-- MULTIPOINT ZM (1 1 1 5,2 2 2 5)
SELECT ST_AsText (geoMeasure(tgeogpoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
[Point(1 1)@2001-01-03, Point(1 1)@2001-01-04]}',
'{[5@2001-01-01, 5@2001-01-02],[7@2001-01-03, 7@2001-01-04]}'));
-- GEOMETRYCOLLECTION M (POINT M (1 1 7),LINESTRING M (1 1 5,2 2 5))
SELECT ST_AsText (geoMeasure(tgeompoint '[Point(1 1)@2001-01-01,
Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]',
'[5@2001-01-01, 7@2001-01-02, 5@2001-01-03]', true));
-- MULTILINESTRING M ((1 1 5,2 2 5),(2 2 7,1 1 7))

```

A typical visualization for mobility data is to show on a map the trajectory of the moving object using different colors according to the speed. Figure 7.3 shows the result of the query below using a color ramp in QGIS.

```

WITH Temp(t) AS (
  SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-05,
Point(2 0)@2001-01-08, Point(3 1)@2001-01-10, Point(4 0)@2001-01-11]' )
SELECT ST_AsText (geoMeasure(t, round(speed(t) * 3600 * 24, 2), true))
FROM Temp;
/* MULTILINESTRING M ((0 0 0.35,1 1 0.35),(1 1 0.47,2 0 0.47),(2 0 0.71,3 1 0.71),
(3 1 1.41,4 0 1.41)) */

```

The following expression is used in QGIS to achieve this. The `scale_linear` function transforms the `M` value of each composing segment to the range `[0, 1]`. This value is then passed to the `ramp_color` function.

```

ramp_color('RdYlBu', scale_linear(
  m(start_point(geometry_n($geometry,@geometry_part_num)),
  0, 2, 0, 1) )

```

- Return the 3D affine transform of a temporal geometry to translate, rotate, and/or scale it in a single step 

`affine(tgeo, float a, float b, float c, float d, float e, float f, float g,`

`float h, float i, float xoff, float yoff, float zoff) → tgeo`

`affine(tgeo, float a, float b, float d, float e, float xoff, float yoff) → tgeo`



Figure 7.3: Visualizing the speed of a moving object using a color ramp in QGIS.

```
-- Rotate a 3D temporal point 180 degrees about the z axis
SELECT asEWKT(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), 0, 0, 0, 1,
0, 0, 0))
FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01, POINT(1 4 3)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
SELECT asEWKT(rotate(temp, pi()))
FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01, POINT(1 4 3)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
-- Rotate a 3D temporal point 180 degrees in both the x and z axis
SELECT asEWKT(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), -sin(pi()),
0, sin(pi()), cos(pi()), 0, 0, 0))
FROM (SELECT tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02]' AS temp) t;
-- [POINT(-1 -1)@2001-01-01, LINESTRING(-1 -1,-2 -2)@2001-01-02]
```

- Return the temporal point rotated counter-clockwise about the origin point

`rotate(tgeo, float radians) → tgeo`

`rotate(tgeo, float radians, float x0, float y0) → tgeo`

`rotate(tgeo, float radians, geometry origin) → tgeo`

```
-- Rotate a temporal point 180 degrees
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()), 6);
-- [POINT(-5 -10)@2001-01-01, POINT(-5 -5)@2001-01-02, POINT(-10 -5)@2001-01-03]
-- Rotate 30 degrees counter-clockwise at x=5, y=10
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()/6, 5, 10), 6);
-- [POINT(5 10)@2001-01-01, POINT(7.5 5.67)@2001-01-02, POINT(11.83 8.17)@2001-01-03]
-- Rotate 60 degrees clockwise from centroid
SELECT asEWKT(rotate(temp, -pi()/3, ST_Centroid(traversedArea(temp))), 2)
FROM (SELECT tgeometry '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Linestring(5 5,10 5)@2001-01-03]' AS temp) AS t;
/* [POINT(10.58 9.67)@2001-01-01, POINT(6.25 7.17)@2001-01-02,
  LINESTRING(6.25 7.17,8.75 2.83)@2001-01-03] */
```

- Return a temporal point scaled by given factors 

`scale(tgeo, float Xfactor, float Yfactor, float Zfactor) → tgeo`

`scale(tgeo, float Xfactor, float Yfactor) → tgeo`

`scale(tgeo, geometry factor) → tgeo`

`scale(tgeo, geometry factor, geometry origin) → tgeo`

```
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
0.5, 0.75, 0.8));
-- [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
0.5, 0.75));
-- [POINT Z (0.5 1.5 3)@2001-01-01, POINT Z (0.5 0.75 1)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
```

```

geometry 'Point(0.5 0.75 0.8)'));
-- [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
geometry 'Point(2 2)', geometry 'Point(1 1)'));
-- [POINT(1 1)@2001-01-01, LINESTRING(1 1,3 3)@2001-01-02]

```

- Transform a temporal geometric point into the coordinate space of a Mapbox Vector Tile 

`asMVTGeom(tpoint, bounds, extent=4096, buffer=256, clip=true) → (geom, times)`

The result is a couple composed of a geometry value and an array of associated timestamp values encoded as Unix epoch. The parameters are as follows:

- `tpoint` is the temporal point to transform
- `bounds` is an `stbox` defining the geometric bounds of the tile contents without buffer
- `extent` is the tile extent in tile coordinate space
- `buffer` is the buffer distance in tile coordinate space
- `clip` is a Boolean that determines if the resulting geometries and timestamps should be clipped or not

```

SELECT ST_AsText((mvt).geom), (mvt).times
FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01, Point(100 100)@2001-01-02]',
stbox 'STBOX X((40,40),(60,60))') AS mvt ) AS t;
-- LINESTRING(-256 4352,4352 -256) | {946714680,946734120}
SELECT ST_AsText((mvt).geom), (mvt).times
FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01, Point(100 100)@2001-01-02]',
stbox 'STBOX X((40,40),(60,60))', clip:=false) AS mvt ) AS t;
-- LINESTRING(-8192 12288,12288 -8192) | {946681200,946767600}

```

- Extract from a temporal point with linear interpolation the subsequences where the point stays within an area with a specified maximum size for at least the given duration 

`stops(tpoint, maxDist=0.0, minDuration='0 minutes') → tpoint`

The size of the area is computed as the diagonal of the minimum rotated rectangle of the points in the subsequence. If `maxDist` is not given it is assumed 0.0 and thus, the function extracts the constant segments of the given temporal point. The distance is computed in the units of the coordinate system. Note that even though the function accepts 3D geometries, the computation is always performed in 2D.

```

SELECT asText(stops(tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-02,
Point(2 2)@2001-01-03, Point(2 2)@2001-01-04]'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02), [POINT(2 2)@2001-01-03,
POINT(2 2)@2001-01-04]} */
SELECT asText(stops(tgeompoint '[Point(1 1 1)@2001-01-01, Point(1 1 1)@2001-01-02,
Point(2 2 2)@2001-01-03, Point(2 2 2)@2001-01-04]', 1.75));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03,
POINT Z (2 2 2)@2001-01-04]} */

```


Chapter 8

Temporal Geometry Types (Part 2)

8.1 Restrictions

- Restrict to (the complement of) a geometry 

`atGeometry(tgeom, geometry) → tgeom`

`minusGeometry(tgeom, geometry) → tgeom`

The geometry must be in 2D and the computation with respect to it is done in 2D. The result preserves the Z dimension of the temporal point, if it exists.

```
SELECT asText(atGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {"[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]"}
SELECT astext(atGeometry(tgeompoint '[Point(0 0 0)@2001-01-01, Point(4 4 4)@2001-01-05]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {[POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03]}
SELECT asText(atGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (2 1 1)@2001-01-02, POINT Z (3 1 1)@2001-01-03, POINT Z (3 1 3)@2001-01-05]}
SELECT asText(atGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(1 1,5 1)@2001-01-01
```

```
SELECT asText(minusGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02), (POINT(2 2)@2001-01-03,
  POINT(3 3)@2001-01-04)} */
SELECT astext(minusGeometry(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(4 4 4)@2001-01-05]', geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
  (POINT Z (2 2 2)@2001-01-03, POINT Z (4 4 4)@2001-01-05)} */
SELECT asText(minusGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 1 1)@2001-01-02)}
SELECT asText(minusGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(5 1,10 1)@2001-01-01
```

- Restrict to (the complement of) an stbox 

`atStbox(tgeom, stbox, borderInc bool=true) → tgeompoint`

`minusStbox(tgeom, stbox, borderInc bool=true) → tgeompoint`

The third optional argument is used for multidimensional tiling (see Section 9.6) to exclude the upper border of the tiles when a temporal value is split in multiple tiles, so that all fragments of the temporal geometry are exclusive.

```
SELECT asText(atStbox(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]',
  stbox 'STBOX XT(((0,0),(2,2)), [2001-01-02, 2001-01-04])'));
-- {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]}
SELECT asText(atStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2),(3,3,3))'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03, POINT Z (3 3 2)@2001-01-04,
  POINT Z (3 3 3)@2001-01-05]} */
SELECT asText(atStbox(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-03,
  Point(2 2)@2001-01-04, Linestring(3 3,4 4)@2001-01-09]', stbox 'STBOX X((2,2),(3,3))'));
-- {[LINESTRING(2 2,3 3)@2001-01-03, POINT(2 2)@2001-01-04, POINT(3 3)@2001-01-09]}
```

```
SELECT asText(minusStbox(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04]',
  stbox 'STBOX XT(((1,1),(2,2)), [2001-01-03,2001-01-04])'));
-- {(POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03)}
SELECT asText(minusStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2),(3,3,3))'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02,
  (POINT Z (3 3 3)@2001-01-05, POINT Z (3 3 7)@2001-01-09)} */
SELECT asText(minusStbox(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,3 3)@2001-01-03, Point(2 2)@2001-01-04,
  Linestring(1 1,4 4)@2001-01-09]', stbox 'STBOX X((2,2),(3,3))'));
/* {[POINT(1 1)@2001-01-01, LINESTRING(1 1,2 2)@2001-01-03,
  LINESTRING(1 1,2 2)@2001-01-04), [MULTILINESTRING((1 1,2 2),(3 3,4 4))@2001-01-09]} */
```

- Restrict to (the complement of) an elevation span 

atElevation(tgeom, zspan) → tgeom

minusElevation(tgeom, zspan) → tgeom

```
SELECT astext(atElevation(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03],
  [POINT Z (3 3 3)@2001-01-05, POINT Z (2 2 2)@2001-01-06]} */
SELECT astext(minusElevation(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03],
  [POINT Z (3 3 3)@2001-01-05, POINT Z (2 2 2)@2001-01-06]} */
```

8.2 Spatial Reference System

- Return or set the spatial reference identifier  

SRID(tspatial) → integer

setSRID(tspatial) → tspatial

```
SELECT SRID(tgeompoint 'Point(0 0)@2001-01-01');
-- 0
SELECT asEWKT(setSRID(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02]', 4326));
-- SRID=4326; [POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02]
```

- Transform to a spatial reference identifier  

transform(tspatial, integer) → tspatial

transformPipeline(tspatial, pipeline text, to_srid integer, is_forward bool=true) → tspatial

The `transform` function specifies the transformation with a target SRID. An error is raised when the input temporal point has an unknown SRID (represented by 0).

The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input temporal point is ignored, and the SRID of the output temporal point will be set to zero unless a value is provided via the optional `to_srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to `false`, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(tgeompoint 'SRID=4326;Point(4.35 50.85)@2001-01-01', 3812));
-- SRID=3812;POINT(648679.018035303 671067.055638114)@2001-01-01
```

```
WITH test(tgeo, pipeline) AS (
  SELECT tgeompoint 'SRID=4326;{Point(4.3525 50.846667 100.0)@2001-01-01,
    Point(-0.1275 51.507222 100.0)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
  SELECT asEWKT(transformPipeline(transformPipeline(tgeo, pipeline, 4326),
    pipeline, 4326, false), 6)
FROM test;
/* SRID=4326;{POINT Z (4.3525 50.846667 100)@2001-01-01,
  POINT Z (-0.1275 51.507222 100)@2001-01-02} */
```

8.3 Bounding Box Operations

- Return the spatiotemporal bounding box expanded in the spatial dimension by a float value  

`expandSpace({spatial,tspatial},float) → stbox`

```
SELECT expandSpace(geography 'Linestring(0 0,1 1)', 2);
-- SRID=4326;GEODSTBOX X((-2,-2),(3,3))
SELECT expandSpace(tgeompoint 'Point(0 0)@2001-01-01', 2);
-- STBOX XT((-2,-2),(2,2)), [2001-01-01,2001-01-01])
```

8.4 Distance Operations

- Return the smallest distance ever  

`{geo,tgeo} |=| {geo,tgeo} → float`

```
SELECT tgeompoint '[Point(0 0)@2001-01-02, Point(1 1)@2001-01-04, Point(0 0)@2001-01-06]'
  |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03, Point(0 0)@2001-01-05]'
  |=| tgeompoint '[Point(2 0)@2001-01-02, Point(1 1)@2001-01-04, Point(2 2)@2001-01-06]';
-- 0.5
SELECT tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05]' |=| tgeompoint '[Point(2 0 0)@2001-01-02,
  Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]';
-- 0.5
SELECT tgeometry '(Point(1 1)@2001-01-01, Linestring(3 1,1 1)@2001-01-03)' |=|
  geometry 'Linestring(1 3,2 2,3 3)';
-- 1
```

The operator `|=|` can be used for doing nearest neighbor searches using a GiST or an SP-GiST index (see Section 10.2). This operator corresponds to the PostGIS function `ST_DistanceCPA`, although the latter requires both arguments to be a trajectory.

```
SELECT ST_DistanceCPA(
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
    Point(0 0 0)@2001-01-05]':geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
    Point(2 2 2)@2001-01-06]':geometry);
-- 0.5
```

- Return the instant of the first temporal point at which the two arguments are at the nearest distance  

`nearestApproachInstant({geo,tgeo},{geo,tgeo}) → tgeo`

The function will only return the first instant that it finds if there are more than one. The resulting instant may be at an exclusive bound.

```
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(2 1)@2001-01-02
SELECT asText(nearestApproachInstant(tgeompoint 'Interp=Step;(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-03]', tgeompoint '(Point(1 1)@2001-01-01, Point(4 1)@2001-01-03]'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,1 1 1)@2001-01-02
```

Function `nearestApproachInstant` generalizes the PostGIS function `ST_ClosestPointOfApproach`. First, the latter function requires both arguments to be trajectories. Second, function `nearestApproachInstant` returns both the point and the timestamp of the nearest point of approach while the PostGIS function only provides the timestamp as shown next.

```
SELECT to_timestamp(ST_ClosestPointOfApproach(
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
    Point(0 0 0)@2001-01-05]':geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
    Point(2 2 2)@2001-01-06]':geometry));
-- 2001-01-03 12:00:00+00
```

- Return the minimum spatial distance between two temporal geometries over the union of their temporal extents

`minDistance({tgeo,geo},{tgeo,geo}) → float`

`minDistance(tgeo[],tgeo[]) → float`

`minDistance(tgeo,tgeo) → float (aggregate)`

The scalar form returns the minimum distance reached between the two arguments over the intersection of their temporal extents, equivalent to the minimum value of the temporal distance $\leftrightarrow$. The array form generalises this to two sets of temporal geometries and returns the minimum across all pairs. The aggregate form, used with `GROUP BY`, returns the minimum distance reached over all aggregated pairs in the group.

```
SELECT minDistance(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]',
  geometry 'Point(0 1)');
-- 0.707106781186548
SELECT minDistance(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]',
    tgeompoint '[Point(5 5)@2001-01-01, Point(6 6)@2001-01-03]'],
  ARRAY[tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03]']);
-- 0
SELECT g, minDistance(t1.Trip, t2.Trip)
FROM Trips t1, Trips t2, Groups g
```

```
WHERE t1.GroupId = g AND t2.GroupId = g AND t1.TripId < t2.TripId
GROUP BY g;
```

The aggregate form is well suited to BerlinMOD-style queries that compute the minimum approach distance between trip pairs grouped by license or vehicle class. On a single-row group the aggregate degenerates to the scalar minimum distance between the pair.

- Return the qualifying index pairs of two arrays of temporal geometries, generalising the scalar spatial relationships to a set-set spatial join

```
eDwithinPairs(tgeo[],tgeo[],float) → setof(i integer, j integer)
aDwithinPairs(tgeo[],tgeo[],float) → setof(i integer, j integer)
eIntersectsPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
aIntersectsPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
eDisjointPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
aDisjointPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
eTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer, j integer)
aTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer, j integer)
tDwithinPairs(tgeo[],tgeo[],float) → setof(i integer, j integer, periods tstzspanset)
tIntersectsPairs(tgeo[],tgeo[]) → setof(i integer, j integer, periods tstzspanset)
tDisjointPairs(tgeo[],tgeo[]) → setof(i integer, j integer, periods tstzspanset)
tTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer, j integer, periods tstzspanset)
```

Each function resolves the pruned cross product of the two input arrays inside a single call and returns the one-based indexes *i* and *j* of the qualifying pairs. The *e*-prefixed functions return the pairs for which the relationship ever holds, the *a*-prefixed functions the pairs for which it always holds, and the *t*-prefixed functions additionally return the periods during which it holds. Each input's spatiotemporal box is used as a bounding-box prefilter so that the exact relationship is computed only on the surviving candidate pairs, and pairs whose temporal extents do not overlap are excluded, matching the scalar relationships. The functions are defined for temporal points and temporal geometries in both the planar and geodetic cases, except the touches relationship, which is defined for planar geometries only. The indexes map back to the original identities through a parallel `array_agg(identity ORDER BY ...)`.

```
SELECT i, j FROM eDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]',
    tgeompoint '[Point(50 50)@2001-01-01, Point(50 60)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- 1 | 1
SELECT i, j, periods FROM tDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- one pair, with the period during which it is within the distance
SELECT i, j FROM aDisjointPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(5 0)@2001-01-01, Point(5 10)@2001-01-02]']);
-- 1 | 1
```

- Return the line connecting the nearest approach point  

```
shortestLine({geo,tgeo},{geo,tgeo}) → geo
```

The function will only return the first line that it finds if there are more than one.

```
SELECT ST_AsText(shortestLine(tgeompoint '(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(2 1,2 2)
SELECT ST_AsText(shortestLine(tgeompoint 'Interp=Step;(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(1 1,2 2)
```

```
SELECT ST_AsText(shortestLine(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,2 0 0)
```

Function `shortestLine` can be used to obtain the result provided by the PostGIS function `ST_CPAWithin` when both arguments are trajectories as shown next.

```
SELECT ST_Length(shortestLine(
  tgeopoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
    Point(0 0 0)@2001-01-05]',
  tgeopoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
    Point(2 2 2)@2001-01-06]')) <= 0.5;
-- true
SELECT ST_CPAWithin(
  tgeopoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
    Point(0 0 0)@2001-01-05]':geometry,
  tgeopoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
    Point(2 2 2)@2001-01-06]':geometry, 0.5);
-- true
```

The temporal distance operator, denoted $\leftrightarrow$, computes the distance at each instant of the intersection of the temporal extents of their arguments and results in a temporal float. Computing temporal distance is useful in many mobility applications. For example, a moving cluster (also known as convoy or flock) is defined as a set of objects that move close to each other for a long time interval. This requires to compute temporal distance between two moving objects.

The temporal distance operator accepts a geometry/geography restricted to a point or a temporal point as arguments. Notice that the temporal types only consider linear interpolation between values, while the distance is a root of a quadratic function. Therefore, the temporal distance operator gives a linear approximation of the actual distance value for temporal sequence points. In this case, the arguments are synchronized in the time dimension, and for each of the composing line segments of the arguments, the spatial distance between the start point, the end point, and the nearest point of approach is computed, as shown in the examples below.

- Return the temporal distance  

$\{\text{geo}, \text{tgeo}\} \leftrightarrow \{\text{geo}, \text{tgeo}\} \rightarrow \text{tfloat}$

```
SELECT tgeopoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]' <->
  geometry 'Point(0 1)';
-- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03)
SELECT tgeopoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]' <->
  tgeopoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03]';
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03)
SELECT tgeopoint '[Point(0 1)@2001-01-01, Point(0 0)@2001-01-03]' <->
  tgeopoint '[Point(0 0)@2001-01-01, Point(1 0)@2001-01-03]';
-- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03)
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,1 1)@2001-01-02]' <->
  tgeometry '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-02]';
-- Interp=Step;[1@2001-01-01, 1@2001-01-02]
```

8.5 Spatial Relationships

The topological relationships such as `ST_Intersects` and the distance relationships such as `ST_DWithin` can be generalized for temporal geometries. The arguments of these generalized functions are either a temporal geometry (that is, a `tgeometry`, a `tgeography`, a `tgeopoint`, or a `tgeogpoint`) and a base type (that is, a `geometry` or a `geography`) or two temporal geometries. Furthermore, both arguments must be of the same base type or the same temporal type, for example, these functions do not allow to mix a `tgeometry` and a `geography` or a `tgeometry` and a `tgeopoint`.

There are three versions of the relationships:

- The *ever* relationships determine whether the topological or distance relationship is ever satisfied (see Section 5.4.2) and returns a boolean. Examples are the `eIntersects` and `eDwithin` functions.
- The *always* relationships determine whether the topological or distance relationship is always satisfied (see Section 5.4.2) and returns a boolean. Examples are the `aIntersects` and `aDwithin` functions.
- The *temporal* relationships compute the topological or distance relationship at each instant and results in a `tbool`. Examples are the `tIntersects` and `tDwithin` functions.

For example, the following query

```
SELECT eIntersects(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 2)@2001-01-01, Point(4 2)@2001-01-05)'];
-- t
```

tests whether the temporal point ever intersects the geometry. In this case, the query is equivalent to the following one

```
SELECT ST_Intersects(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  geometry 'Linestring(0 2,4 2)');
```

where the second geometry is obtained by applying the `trajectory` function to the temporal point.

In contrast, the query

```
SELECT tIntersects(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 2)@2001-01-01, Point(4 2)@2001-01-05)'];
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-04], (f@2001-01-04, f@2001-01-05)}
```

computes at each instant whether the temporal point intersects the geometry. Similarly, the following query

```
SELECT eDwithin(tgeompoint '[Point(3 1)@2001-01-01, Point(5 1)@2001-01-03)',
  tgeompoint '[Point(3 1)@2001-01-01, Point(1 1)@2001-01-03)', 2);
-- t
```

tests whether the distance between the temporal points was ever less than or equal to 2, while the following query

```
SELECT tDwithin(tgeompoint '[Point(3 1)@2001-01-01, Point(5 1)@2001-01-03)',
  tgeompoint '[Point(3 1)@2001-01-01, Point(1 1)@2001-01-03)', 2);
-- {[t@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
```

computes at each instant whether the distance between the temporal points is less than or equal to 2.

The *ever* or *always* relationships are sometimes used in combination with a spatiotemporal index when computing the temporal relationships. For example, the following query

```
SELECT T.TripId, R.RegionId, tIntersects(T.Trip, R.Geom)
FROM Trips T, Regions R
WHERE eIntersects(T.Trip, R.Geom)
```

which verifies whether a trip `T` (which is a temporal point) intersects a region `R` (which is a geometry), will benefit from a spatiotemporal index on the column `T.Trip` since the `eIntersects` function will automatically perform the bounding box comparison `T.Trip && R.Geom`. This is further explained later in this document.

Not all spatial relationships available in PostGIS have been generalized for temporal geometries, only those derived from the following functions: `ST_Contains`, `ST_Covers`, `ST_Disjoint`, `ST_Intersects`, `ST_Touches`, and `ST_DWithin`. These functions only support 2D geometries and only the functions `ST_Covers`, `ST_Intersects`, and `ST_DWithin` support geographies. Consequently, the same applies for the MobilityDB functions derived from them, excepted that they support 3D for temporal points, that is, `tgeompoint`, and `tgeogpoint`. As stated above, each of the above PostGIS functions, such as `ST_Contains`, has three generalized versions in MobilityDB, namely `eContains`, `aContains`, and `tContains`. Furthermore, not all combinations of parameters are meaningful for the generalized functions. For example, `tContains(tpoint, geometry)` is meaningful only when the geometry is a single point, and `tContains(tpoint, tpoint)` is equivalent to `tintersects(tpoint, geometry)`.

8.5.1 Ever and Always Relationships

We present next the ever and always relationships. These relationships automatically include a bounding box comparison that makes use of any spatial indexes that are available on the arguments.

- Ever or always contains

`eContains({geometry,tgeom},{geometry,tgeom}) → boolean`

`aContains({geometry,tgeom},{geometry,tgeom}) → boolean`

This function returns true if the temporal geometry and the geometry ever or always intersect at their interior. Recall that a geometry does not contain things in its boundary and thus, polygons and lines do not contain lines and points lying in their boundary. Please refer to the documentation of the [ST_Contains](#) function in PostGIS.

```
SELECT eContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Ever or always covers

`eCovers({geometry,tgeom},{geometry,tgeom}) → boolean`

`aCovers({geometry,tgeom},{geometry,tgeom}) → boolean`

Please refer to the documentation of the [ST_Contains](#) and the [ST_Covers](#) function in PostGIS for detailed explanations about the difference between the two functions.

```
SELECT eCovers(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Is ever or always disjoint

`eDisjoint({geo,tgeo},{geo,tgeo}) → boolean`

`aDisjoint({geo,tgeo},{geo,tgeo}) → boolean`

```
SELECT eDisjoint(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]');
-- false
SELECT eDisjoint(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeometry '[Linestring(1 1 1,2 2 2)@2001-01-01, Point(2 2 2)@2001-01-03]');
-- true
```


- Is ever or always at distance within  

`eDwithin({geo,tgeo},{geo,tgeo},float) → boolean`

`aDwithin({geometry,tgeom},{geometry,tgeom},float) → boolean`

```
SELECT eDwithin(geometry 'Point(1 1 1)',
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 0)@2001-01-02]', 1);
-- true
SELECT eDwithin(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 2 2)@2001-01-01,Point(2 2 2)@2001-01-02]', 1);
-- false
```

- Ever or always intersects  

`eIntersects({geo,tgeo},{geo,tgeo}) → boolean`

`aIntersects({geometry,tgeom},{geometry,tgeom}) → boolean`

```
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03]');
-- false
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03]');
-- true
```

- Ever or always touches

`eTouches({geometry,tgeom},{geometry,tgeom}) → boolean`

`aTouches({geometry,tgeom},{geometry,tgeom}) → boolean`

```
SELECT eTouches(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-03]');
-- true
```

8.5.2 Spatiotemporal Relationships

- Temporal contains

`tContains(geometry,tgeom) → tbool`

```
SELECT tContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(1 4)@2001-01-01, Point(4 1)@2001-01-04]');
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, f@2001-01-03, f@2001-01-04)}
```

- Temporal disjoint  

`tDisjoint({geo,tgeo},{geo,tgeo}) → tbool`

The function only supports 3D or geographies for two temporal points

```

SELECT tDisjoint(geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)');
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-03], (t@2001-01-03, t@2001-01-04]}
SELECT tDisjoint(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05)');
-- {[t@2001-01-01, f@2001-01-03], (t@2001-01-03, t@2001-01-05)}

```

- Temporal distance within

`tDwithin({geo,tgeo},{geo,tgeo},float) → tbool`

```

SELECT tDwithin(geometry 'Point(1 1)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03]', sqrt(2));
-- {[t@2001-01-01, t@2001-01-03]}
SELECT tDwithin(tgeompoint '[Point(1 0)@2001-01-01, Point(1 4)@2001-01-05]',
  tgeompoint 'Interp=Step;[Point(1 2)@2001-01-01, Point(1 3)@2001-01-05]', 1);
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-04], (f@2001-01-04, t@2001-01-05)}

```

- Temporal intersects

`tIntersects({geo,tgeo},{geo,tgeo}) → tbool`

The function only supports 3D or geographies for two temporal points

```

SELECT tIntersects(geometry 'MultiPoint(1 1,2 2)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)');
/* {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, t@2001-01-03],
  (f@2001-01-03, f@2001-01-04]} */
SELECT tIntersects(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05)');
-- {[f@2001-01-01, t@2001-01-03], (f@2001-01-03, f@2001-01-05)}

```

- Temporal touches

`tTouches({geometry,tgeom},{geometry,tgeom}) → tbool`

```

SELECT tTouches(geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 0)@2001-01-04)');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04)}

```

Chapter 9

Temporal Types: Analytics Operations

9.1 Simplification

- Return a temporal float or a temporal point simplified ensuring that consecutive values are at least a certain distance or time interval apart  

`minDistSimplify({tfloat,tpoint},mindist float) → {tfloat,tpoint}`

`minTimeDeltaSimplify({tfloat,tpoint},mint interval) → {tfloat,tpoint}`

In the case of temporal points, the distance is specified in the units of the coordinate system. Notice that simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given temporal value is returned.

```
SELECT minDistSimplify(tfloat '[1@2001-01-01,2@2001-01-02,3@2001-01-04,4@2001-01-05]', 1);
-- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05]', sqrt(3)));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05]
SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(4 4 4)@2001-01-05]', sqrt(3)));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (4 4 4)@2001-01-05]
```

```
SELECT minTimeDeltaSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-04,
4@2001-01-05]', '1 day');
-- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
SELECT asText(minTimeDeltaSimplify(tgeogpoint '[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05]', '1 day'));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05]
```

- Return a temporal float or a temporal point simplified using the **Douglas-Peucker algorithm** 

`maxDistSimplify({tfloat,tgeompoint},maxdist float,syncdist=true) →
{tfloat,tgeompoint}`

`douglasPeuckerSimplify({tfloat,tgeompoint},maxdist float,syncdist=true) →
{tfloat,tgeompoint}`

The difference between the two functions is that `maxDistSimplify` uses a single-pass version of the algorithm whereas `douglasPeuckerSimplify` uses the standard recursive algorithm.

The function removes values or points that are less than or equal to the distance passed as second argument. In the case of temporal points, the distance is specified in the units of the coordinate system. The third argument applies only for temporal points and specifies whether the spatial or the synchronized distance is used. Notice that simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given temporal value is returned.

```
-- Only synchronous distance for temporal floats
SELECT maxDistSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 3@2001-01-04,
1@2001-01-05]', 1, false);
-- [1@2001-01-01, 1@2001-01-03, 3@2001-01-04, 1@2001-01-05]
-- Synchronous distance by default for temporal points
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-05, POINT(5 1)@2001-01-06]
-- Spatial distance
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2, false));
-- [POINT(1 1)@2001-01-01, POINT(5 1)@2001-01-06]

-- Spatial vs synchronized Euclidean distance
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, false));
-- [POINT(1 1)@2001-01-01, POINT(7 4)@2001-01-07]
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, true));
-- [POINT(1 1)@2001-01-01, POINT(6 1)@2001-01-06, POINT(7 4)@2001-01-07]
```

The difference between the spatial and the synchronized distance is illustrated in the last two examples above and in Figure 9.1. In the first example, which uses the spatial distance, the second instant is removed since the perpendicular distance between POINT(6 1) and the line defined by POINT(1 1) and POINT(7 4) is equal to 2.23. On the contrary, in the second example the second instant is kept since the projection of Point(6 2) at timestamp 2001-01-06 over the temporal line segment results in Point(6 3.5) and the distance between the original point and its projection is 2.5.

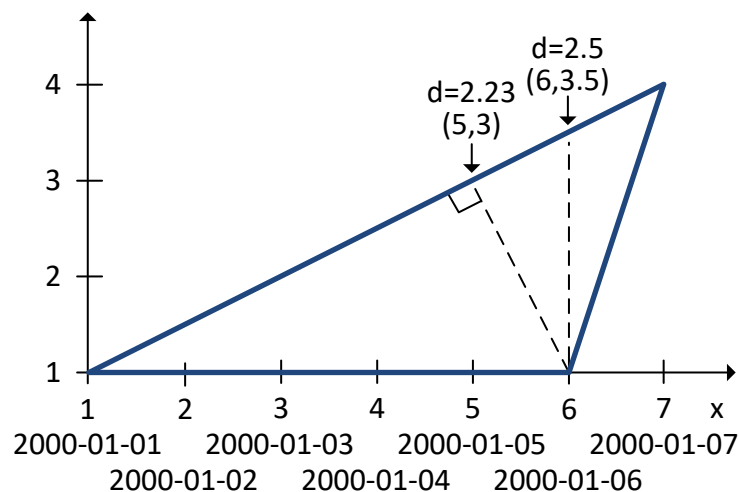


Figure 9.1: Difference between the spatial and the synchronous distance.

A typical use for the `douglasPeuckerSimplify` function is to reduce the size of a dataset, in particular for visualization purposes. If the visualization is static, then the spatial distance should be preferred, if the visualization is dynamic or animated, the synchronized distance should be preferred.

9.2 Reduction

- Sample a temporal value with respect to an interval

```
tsample(temporal, duration interval, torigin timestampz='2000-01-03',
interp='discrete') → temporal
```

If the origin is not specified, it is set by default to Monday, January 3, 2000. The given interval must be strictly greater than zero.

```
SELECT tsample(tbool '[true@2001-01-01,false@2001-01-02]', '12 hours', '2001-01-01', 'step ← ');
-- {t@2001-01-01, f@2001-01-02}
SELECT tsample(tint '{1@2001-01-01,5@2001-01-05}', '3 days', '2001-01-01');
-- {1@2001-01-01}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '1 day', '2001-01-01');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01');
-- {1@2001-01-01, 4@2001-01-04}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01', 'linear');
-- {1@2001-01-01, 4@2001-01-04}
```

```
SELECT asText(tsample(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05]', '2 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03, POINT(5 5)@2001-01-05}
SELECT asText(tsample(tgeompoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05], [Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-07}
SELECT asText(tsample(tgeompoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05], [Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01', 'step'));
-- Interp=Step;[POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-07]
```

Figure 9.2 illustrates the sampling of temporal floats with various interpolations. As shown in the figure, the sampling operation is best suited for temporal values with continuous interpolation.

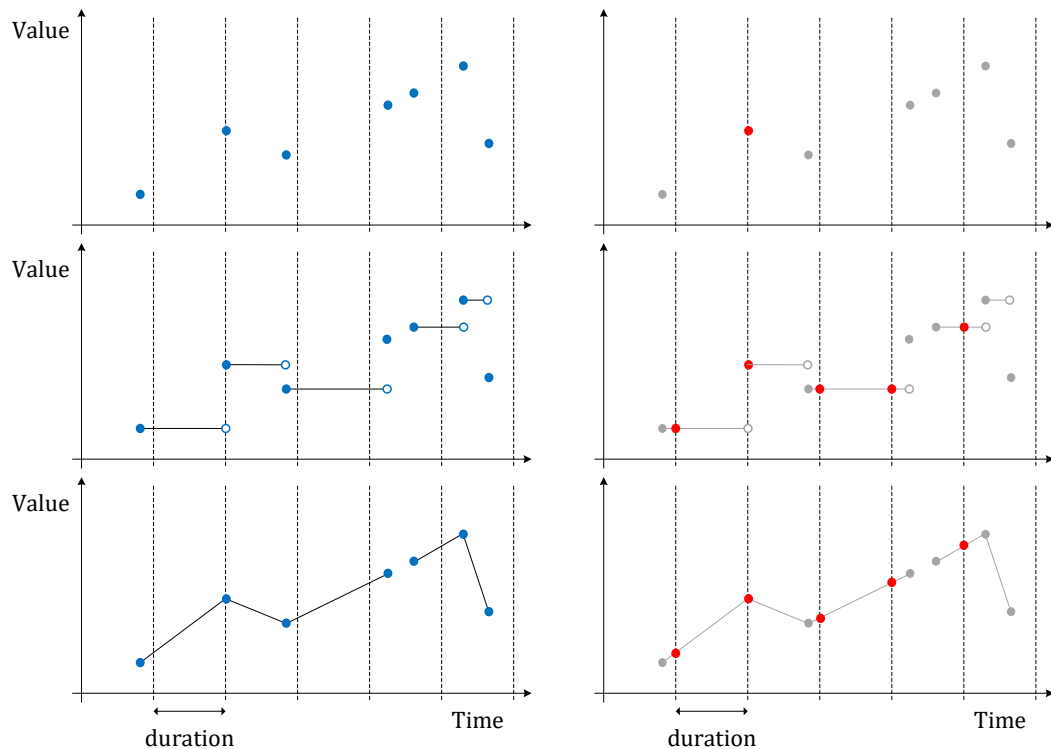


Figure 9.2: Sampling of temporal floats with discrete, step, and linear interpolation.

- Reduce the temporal precision of a temporal value with respect to an interval computing the time-weighted average/centroid in each time bin

```
tprecision({tnumber,tgeompoint,tcbuffer,tpose,tgeo,trgeometry},duration interval,torigin
timestampz='2000-01-03')
```

```
→ {tnumber,tpoint,tcbuffer,tpose,tgeo,trgeometry}
```

If the origin is not specified, it is set by default to Monday, January 3, 2000. The given interval must be strictly greater than zero.

```
SELECT tprecision(tint '[1@2001-01-01,5@2001-01-05,1@2001-01-09]', '1 day',
'2001-01-01');
-- Interp=Step;[1@2001-01-01, 5@2001-01-05, 1@2001-01-09]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]', '1 day',
'2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]', '1 day',
'2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08, 1@2001-01-09]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]', '2 days',
'2001-01-01');
-- [2@2001-01-01, 4@2001-01-03, 4@2001-01-05, 2@2001-01-07]
```

```
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
Point(1 1)@2001-01-09]', '1 day', '2001-01-01'));
/* [POINT(1.5 1.5)@2001-01-01, POINT(4.5 4.5)@2001-01-04, POINT(4.5 4.5)@2001-01-05,
POINT(1.5 1.5)@2001-01-08] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
Point(1 1)@2001-01-09]', '2 days', '2001-01-01'));
/* [POINT(2 2)@2001-01-01, POINT(4 4)@2001-01-03, POINT(4 4)@2001-01-05,
POINT(2 2)@2001-01-07] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
Point(1 1)@2001-01-09]', '4 days', '2001-01-01'));
-- [POINT(3 3)@2001-01-01, POINT(3 3)@2001-01-05]
```

Changing the precision of a temporal value is akin to changing its *temporal granularity*, for example, from timestamps to hours or days, although the precision can be set to an arbitrary interval, such as 2 hours and 15 minutes. Figure 9.3 illustrates a change of temporal precision for temporal floats with various interpolations.

9.3 Similarity

- Return the discrete **Hausdorff distance**, the discrete **Fréchet distance**, the **Dynamic Time Warp** (DTW) distance, the average Hausdorff distance, or the **Longest Common SubSequence** (LCSS) distance between two temporal values  

```
hausdorffDistance({tnumber, tgeo}, {tnumber, tgeo}) → float
```

```
frechetDistance({tnumber, tgeo}, {tnumber, tgeo}) → float
```

```
dynTimeWarpDistance({tnumber, tgeo}, {tnumber, tgeo}) → float
```

```
averageHausdorffDistance({tnumber, tgeo}, {tnumber, tgeo}) → float
```

```
lcssDistance({tnumber, tgeo}, {tnumber, tgeo}, float) → float
```

The `lcssDistance` function requires an additional parameter that specifies the distance threshold under which two values are considered to match.

These functions have a linear space complexity since only two rows of the distance matrix are allocated in memory. Nevertheless, their time complexity is quadratic in the number of instants of the temporal values. Therefore, the functions require considerable time for temporal values with large number of instants.

```
SELECT hausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(hausdorffDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
```

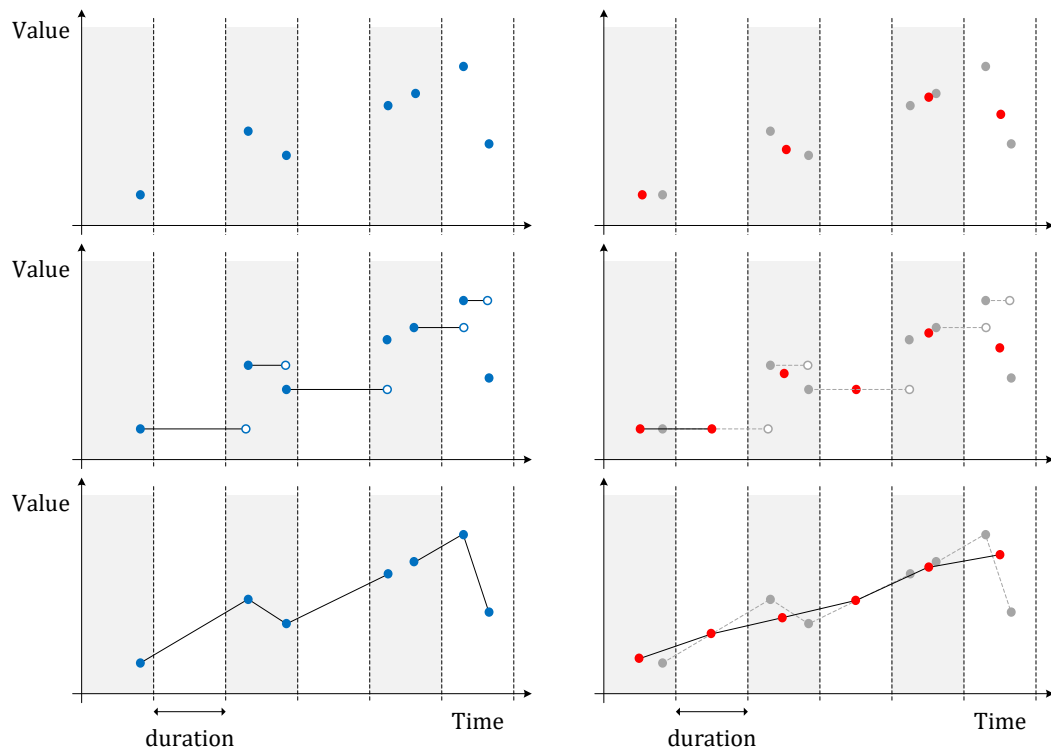


Figure 9.3: Changing the precision of temporal floats with discrete, step, and linear interpolation.

```
Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
Point(1.5 2)@2001-01-05']::numeric, 6);
-- 1.414214
```

```
SELECT frechetDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(frechetDistance(tgeopoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
Point(1 1)@2001-01-05]', tgeopoint '[Point(1.1 1.1)@2001-01-01,
Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
```

```
SELECT dynTimeWarpDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 2
SELECT round(dynTimeWarpDistance(tgeopoint '[Point(1 1)@2001-01-01,
Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
tgeopoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 3.380776
```

```
SELECT round(averageHausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]')
::numeric, 6);
-- 0.283333
SELECT round(averageHausdorffDistance(tgeopoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
Point(1 1)@2001-01-05]', tgeopoint '[Point(1.1 1.1)@2001-01-01,
Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
```

```
Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 0.385218
```

```
SELECT lcssDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]', ←
  1.0);
-- 3
SELECT round(lcssDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]', 1.0)::numeric, 6);
-- 2.000000
```

- Return the correspondence pairs between two temporal values with respect to the discrete Fréchet distance or the Dynamic Time Warp Distance   $\{\}$

$\text{frechetDistancePath}(\{\text{tnumber}, \text{tgeo}\}, \{\text{tnumber}, \text{tgeo}\}) \rightarrow \{(i, j)\}$

$\text{dynTimeWarpPath}(\{\text{tnumber}, \text{tgeo}\}, \{\text{tnumber}, \text{tgeo}\}) \rightarrow \{(i, j)\}$

The result is a set of pairs (i, j) . This function requires to allocate in memory a distance matrix whose size is quadratic in the number of instants of the temporal values. Therefore, the function will fail for temporal values with large number of instants depending on the available memory.

```
SELECT frechetDistancePath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT frechetDistancePath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```

```
SELECT dynTimeWarpPath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT dynTimeWarpPath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```


9.4 Extended Kalman Filter

- Return a smoothed temporal float or temporal point obtained by filtering a noisy trajectory with an **Extended Kalman Filter** 

```
extendedKalmanFilter(tfloat, gate float, q float, variance float, to_drop boolean DEFAULT TRUE) → tfloat
```

```
extendedKalmanFilter(tgeompoint, gate float, q float, variance float, to_drop boolean DEFAULT TRUE) → tgeompoint
```

The filter models the underlying motion of a moving object and separates it from measurement noise and occasional outliers. The `gate` parameter controls how strict the outlier detection is, `q` is the process-noise scale, `variance` is the measurement-noise variance, and `to_drop` chooses whether outliers are removed (`true`) or replaced by the predicted trajectory (`false`).

These functions are intended for trajectories with at least three instants and are typically applied to temporal values with linear interpolation.

```
SELECT asEWKT(
  extendedKalmanFilter(
    tgeompoint '[Point(1 1)@2001-01-01,
                  Point(50 50)@2001-01-02,
                  Point(2 2)@2001-01-03]',
    1e-5, 5e-10, 5e-10, true));
-- Drop mode: the outlier at 2001-01-02 is removed

SELECT asEWKT(
  extendedKalmanFilter(
    tgeompoint '[Point(1 1)@2001-01-01,
                  Point(50 50)@2001-01-02,
                  Point(2 2)@2001-01-03]',
    1e-5, 5e-10, 5e-10, false));
-- Fill mode: the outlier is replaced by a point predicted from the surrounding motion
```

9.5 Splitting Operations

When creating indexes for temporal types, what is stored in the index is not the actual value but instead, a bounding box that *represents* the value. In this case, the index will provide a list of candidate values that *may* satisfy the query predicate, and a second step is needed to filter out candidate values by computing the query predicate on the actual values.

However, when the bounding boxes have a large empty space not covered by the actual values, the index will generate many candidate values that do not satisfy the query predicate, which reduces the efficiency of the index. In these situations, it may be better to represent a value not with a *single* bounding box, but instead with *multiple* bounding boxes. This increases considerably the efficiency of the index, provided that the index is able to manage multiple bounding boxes per value. The following functions are used for generating multiple bounding boxes for a single temporal value.

- Return an array of N time spans obtained by merging the instants or segments of a temporal value  

```
splitNSpans(temp, integer) → tstzspan[]
```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of output spans. If the number of instants or segments is less than or equal to the given number, the resulting array will have one span per instant or segment of the temporal value. Otherwise, the given number of spans will be obtained by merging consecutive instants or segments.

```
SELECT splitNSpans(ttext '{A@2000-01-01, B@2000-01-02, A@2000-01-03, B@2000-01-04,
  A@2000-01-05}', 1);
-- {"[2000-01-01, 2000-01-05]"}

SELECT splitNSpans(tfloat '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 2@2000-01-04,
  1@2000-01-05}', 2);
-- {"[2000-01-01, 2000-01-03]", "[2000-01-04, 2000-01-05]"}
```

```

SELECT splitNSpans(tgeompoint '[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
  Point(1 1)@2000-01-03, Point(2 2)@2000-01-04, Point(1 1)@2000-01-05]', 2);
-- {"[2000-01-01, 2000-01-03]", "[2000-01-03, 2000-01-05]"}
SELECT splitNSpans(tgeogpoint '{[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02],
  [Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04], [Point(1 1 1)@2000-01-05]}', 2);
-- {"[2000-01-01, 2000-01-04]", "[2000-01-05, 2000-01-05]"}
SELECT splitNSpans(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 2@2000-01-04,
  2@2000-01-05}', 6);
/* {"[2000-01-01, 2000-01-01]", "[2000-01-02, 2000-01-02]", "[2000-01-03, 2000-01-03]",
  "[2000-01-04, 2000-01-04]", "[2000-01-05, 2000-01-05]"} */
SELECT splitNSpans(ttext '[A@2000-01-01, B@2000-01-02, A@2000-01-03, B@2000-01-04,
  A@2000-01-05]', 6);
/* {"[2000-01-01, 2000-01-02]", "[2000-01-02, 2000-01-03]",
  "[2000-01-03, 2000-01-04]", "[2000-01-04, 2000-01-05]"} */

```

- Return an array of time spans obtained by merging N consecutive instants or segments of a temporal value  

`splitEachNSpans(temp, integer) → tstzspan[]`

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of input instants or segments that are merged to produce an output span. If the number of input elements is less than or equal to the given number, the resulting array will have a single span per sequence. Otherwise, the given number of consecutive instants or segments will be merged into each output span. Notice that, contrary to the `splitNSpans` function, the number of spans in the result depends on the number of input instants or segments.

```

SELECT splitEachNSpans(ttext '{A@2000-01-01, B@2000-01-02, A@2000-01-03, B@2000-01-04,
  A@2000-01-05}', 1);
/* {"[2000-01-01, 2000-01-01]", "[2000-01-02, 2000-01-02]", "[2000-01-03, 2000-01-03]",
  "[2000-01-04, 2000-01-04]", "[2000-01-05, 2000-01-05]"} */
SELECT splitEachNSpans(tfloat '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 2@2000-01-04,
  1@2000-01-05}', 2);
-- {"[2000-01-01, 2000-01-03]", "[2000-01-03, 2000-01-05]"}
SELECT splitEachNSpans(tgeompoint '{[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02],
  [Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04], [Point(1 1 1)@2000-01-05]}', 2);
-- {"[2000-01-01, 2000-01-02]", "[2000-01-03, 2000-01-04]", "[2000-01-05, 2000-01-05]"}
SELECT splitEachNSpans(tgeogpoint '[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02,
  Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04, Point(1 1 1)@2000-01-05]', 6);
-- {"[2000-01-01, 2000-01-01]", "[2000-01-01, 2000-01-05]"}
SELECT splitEachNSpans(tgeogpoint '{[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02],
  [Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04], [Point(1 1 1)@2000-01-05]}', 6);
-- {"[2000-01-01, 2000-01-02]", "[2000-01-03, 2000-01-04]", "[2000-01-05, 2000-01-05]"}

```

- Return an array of N temporal boxes obtained by merging the instants or segments of a temporal number

`splitNTboxes(tnumber, integer) → tbox[]`

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of output boxes. If the number of input instants or segments is less than or equal to the given number, the resulting array will have one box per instant or segment of the temporal number. Otherwise, the specified number of boxes will be obtained by merging consecutive instants or segments.

```

SELECT splitNTboxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
  1@2000-01-05}', 1);
-- {"TBOXINT XT([1, 5], [2000-01-01, 2000-01-05])"}
SELECT splitNTboxes(tfloat '{[1@2000-01-01, 2@2000-01-02], [1@2000-01-03, 4@2000-01-04],
  [1@2000-01-05]}', 2);
/* {"TBOXFLOAT XT([1, 4], [2000-01-01, 2000-01-04])",
  "TBOXFLOAT XT([1, 1], [2000-01-05, 2000-01-05])"} */
SELECT splitNTboxes(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
  1@2000-01-05]', 3);
/* {"TBOXFLOAT XT([1, 2], [2000-01-01, 2000-01-03])",
  "TBOXFLOAT XT([1, 4], [2000-01-03, 2000-01-04])",
  "TBOXFLOAT XT([1, 4], [2000-01-04, 2000-01-05])"} */

```

```

SELECT splitNTboxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 6);
/* {"TBOXINT XT([1, 2],[2000-01-01, 2000-01-01])",
   "TBOXINT XT([2, 3],[2000-01-02, 2000-01-02])",
   "TBOXINT XT([1, 2],[2000-01-03, 2000-01-03])",
   "TBOXINT XT([4, 5],[2000-01-04, 2000-01-04])",
   "TBOXINT XT([1, 2],[2000-01-05, 2000-01-05])"} */
SELECT splitNTboxes(tint '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 6);
/* {"TBOXINT XT([1, 3],[2000-01-01, 2000-01-02])",
   "TBOXINT XT([1, 3],[2000-01-02, 2000-01-03])",
   "TBOXINT XT([1, 5],[2000-01-03, 2000-01-04])",
   "TBOXINT XT([1, 5],[2000-01-04, 2000-01-05])"} */

```

- Return an array of temporal boxes obtained by merging N consecutive instants or segments of a temporal number

`splitEachNTboxes(tnumber, integer) → tbox[]`

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of input instants or segments that are merged to produce an output box. If the number of input instants or segments is less than or equal to the given number, the resulting array will have a single box per sequence. Otherwise, the specified number of consecutive instants or segments will be merged into each output box. Notice that, contrary to the `splitNTboxes` function, the number of boxes in the result depends on the number of input instants or segments.

```

SELECT splitEachNTboxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 1);
/* {"TBOXINT XT([1, 2],[2000-01-01, 2000-01-01])",
   "TBOXINT XT([2, 3],[2000-01-02, 2000-01-02])",
   "TBOXINT XT([1, 2],[2000-01-03, 2000-01-03])",
   "TBOXINT XT([4, 5],[2000-01-04, 2000-01-04])"} */
SELECT splitEachNTboxes(tint '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 1);
/* {"TBOXINT XT([1, 3],[2000-01-01, 2000-01-02])",
   "TBOXINT XT([1, 3],[2000-01-02, 2000-01-03])",
   "TBOXINT XT([1, 5],[2000-01-03, 2000-01-04])",
   "TBOXINT XT([1, 5],[2000-01-04, 2000-01-05])"} */
SELECT splitEachNTboxes(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 3);
/* {"TBOXFLOAT XT([1, 4],[2000-01-01, 2000-01-04])",
   "TBOXFLOAT XT([1, 4],[2000-01-04, 2000-01-05])"} */
SELECT splitEachNTboxes(tfloat '{[1@2000-01-01, 2@2000-01-02], [1@2000-01-03, 4@2000-01-04],
[1@2000-01-05]}', 6);
/* {"TBOXFLOAT XT([1, 2],[2000-01-01, 2000-01-02])",
   "TBOXFLOAT XT([1, 4],[2000-01-03, 2000-01-04])",
   "TBOXFLOAT XT([1, 1],[2000-01-05, 2000-01-05])"} */

```

- Return either an array of N spatial boxes obtained by merging the segments of a (multi)line or an array of N spatiotemporal boxes obtained by merging the instants or segments of a temporal point  

`splitNSTboxes(lines, integer) → stbox[]`

`splitNSTboxes(tpoint, integer) → stbox[]`

For temporal points, the choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of output boxes. If the number of instants or segments is less than or equal to the given number, the resulting array will have either one box per segment of the (multi)line or one box per instant or segment of the temporal point. Otherwise, the specified number of boxes will be obtained by merging consecutive instants or segments.

```

SELECT splitNSTboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
-- {"STBOX X((1,1),(5,2))"}
SELECT splitNSTboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}

```

```
SELECT splitNSTboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
/* {"SRID=4326;GEODSTBOX Z((1,1,1),(2,2,1))",
    "SRID=4326;GEODSTBOX Z((2,1,1),(3,2,1))",
    "SRID=4326;GEODSTBOX Z((3,1,1),(4,2,1))",
    "SRID=4326;GEODSTBOX Z((4,1,1),(5,2,1))"} */
SELECT splitNSTboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitNSTboxes(tgeompoint '{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
    Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05}', 1);
-- {"STBOX XT(((1,1),(5,2)), [2000-01-01, 2000-01-05])"}
SELECT splitNSTboxes(tgeompoint '[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
    Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05]');
/* {"STBOX XT(((1,1),(2,2)), [2000-01-01, 2000-01-02])",
    "STBOX XT(((2,1),(3,2)), [2000-01-02, 2000-01-03])",
    "STBOX XT(((3,1),(4,2)), [2000-01-03, 2000-01-04])",
    "STBOX XT(((4,1),(5,2)), [2000-01-04, 2000-01-05])"} */
SELECT splitNSTboxes(tgeompoint '{[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02],
    [Point(3 1)@2000-01-03, Point(4 2)@2000-01-04], [Point(5 1)@2000-01-05]}', 2);
/* {"STBOX XT(((1,1),(4,2)), [2000-01-01, 2000-01-04])",
    "STBOX XT(((5,1),(5,1)), [2000-01-05, 2000-01-05])"} */
SELECT splitNSTboxes(tgeogpoint '{Point(1 1 1)@2000-01-01, Point(2 2 1)@2000-01-02,
    Point(3 1 1)@2000-01-03, Point(4 2 1)@2000-01-04, Point(5 1 1)@2000-01-05}', 6);
/* {"SRID=4326;GEODSTBOX ZT(((1,1,1),(1,1,1)), [2000-01-01, 2000-01-01])",
    "SRID=4326;GEODSTBOX ZT(((2,2,1),(2,2,1)), [2000-01-02, 2000-01-02])",
    "SRID=4326;GEODSTBOX ZT(((3,1,1),(3,1,1)), [2000-01-03, 2000-01-03])",
    "SRID=4326;GEODSTBOX ZT(((4,2,1),(4,2,1)), [2000-01-04, 2000-01-04])",
    "SRID=4326;GEODSTBOX ZT(((5,1,1),(5,1,1)), [2000-01-05, 2000-01-05])"} */
SELECT splitNSTboxes(tgeompoint '[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
    Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05]', 6);
/* {"STBOX XT(((1,1),(2,2)), [2000-01-01, 2000-01-02])",
    "STBOX XT(((2,1),(3,2)), [2000-01-02, 2000-01-03])",
    "STBOX XT(((3,1),(4,2)), [2000-01-03, 2000-01-04])",
    "STBOX XT(((4,1),(5,2)), [2000-01-04, 2000-01-05])"} */
```

- Return either an array of spatial boxes obtained by merging N consecutive segments of a (multi)line or an array of spatiotemporal boxes obtained by merging N consecutive instants or segments of a temporal point  

`splitEachNSTboxes(lines, integer) → stbox[]`

`splitEachNSTboxes(tpoint, integer) → stbox[]`

For temporal points, the choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of input instants or segments that are merged to produce an output box. If the number of instants or segments is less than or equal to the given number, the resulting array will have a single box per sequence. Otherwise, the specified number of consecutive instants or segments will be merged into each output box. Notice that, contrary to the `splitNSTboxes` function, the number of boxes in the result depends on the number of input instants or segments.

```
SELECT splitEachNSTboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
-- {"STBOX X((1,1),(5,2))"}
SELECT splitEachNSTboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}
SELECT splitEachNSTboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
/* {"SRID=4326;GEODSTBOX Z((1,1,1),(2,2,1))",
    "SRID=4326;GEODSTBOX Z((2,1,1),(3,2,1))",
    "SRID=4326;GEODSTBOX Z((3,1,1),(4,2,1))",
    "SRID=4326;GEODSTBOX Z((4,1,1),(5,2,1))"} */
SELECT splitEachNSTboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitEachNSTboxes(tgeompoint '{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
    Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05}', 1);
```

```

/* {"STBOX XT((1,1), (1,1)), [2000-01-01, 2000-01-01]",
   "STBOX XT((2,2), (2,2)), [2000-01-02, 2000-01-02]",
   "STBOX XT((3,1), (3,1)), [2000-01-03, 2000-01-03]",
   "STBOX XT((4,2), (4,2)), [2000-01-04, 2000-01-04]",
   "STBOX XT((5,1), (5,1)), [2000-01-05, 2000-01-05]} */
SELECT splitEachNStboxes(tgeompoint '[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05]', 1);
/* {"STBOX XT((1,1), (2,2)), [2000-01-01, 2000-01-02]",
   "STBOX XT((2,1), (3,2)), [2000-01-02, 2000-01-03]",
   "STBOX XT((3,1), (4,2)), [2000-01-03, 2000-01-04]",
   "STBOX XT((4,1), (5,2)), [2000-01-04, 2000-01-05]} */
SELECT splitEachNStboxes(tgeogpoint '[{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02],
[Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, [Point(5 1)@2000-01-05}]]', 2);
/* {"SRID=4326;GEODSTBOX XT((1,1), (2,2)), [2000-01-01, 2000-01-02]",
   "SRID=4326;GEODSTBOX XT((3,1), (4,2)), [2000-01-03, 2000-01-04]",
   "SRID=4326;GEODSTBOX XT((5,1), (5,1)), [2000-01-05, 2000-01-05]} */

```

9.6 Multidimensional Tiling

Multidimensional tiling is the mechanism used to partition the domain of temporal values in bins or tiles of varying number of dimensions. In the case of a single dimension, the domain can be partitioned by value or by time using bins of the same size or duration, respectively. For temporal numbers, the domain can be partitioned in two-dimensional tiles of the same size for the value dimension and the same duration for the time dimension. For temporal points, the domain can be partitioned in space in two- or three-dimensional tiles, depending on the number of dimensions of the spatial coordinates. Finally, for temporal points, the domain can be partitioned in space and time using three- or four-dimensional tiles.

Multidimensional tiling can be used for various purposes. It can be used for computing multidimensional histograms, where the temporal values are aggregated according to the underlying partition of the domain. On the other hand, multidimensional tiling can also be used for indexing purposes, where the bounding box of a temporal value can be fragmented into multiple boxes in order to improve the efficiency of the index. Finally, multidimensional tiling can be used for fragmenting temporal values according to a multidimensional grid defined over the underlying domain. This enables the distribution of a dataset across a cluster of servers, where each server contains a partition of the dataset. The advantage of this partition mechanism is that it preserves proximity in value/space and time, unlike the traditional hash-based partition mechanisms.

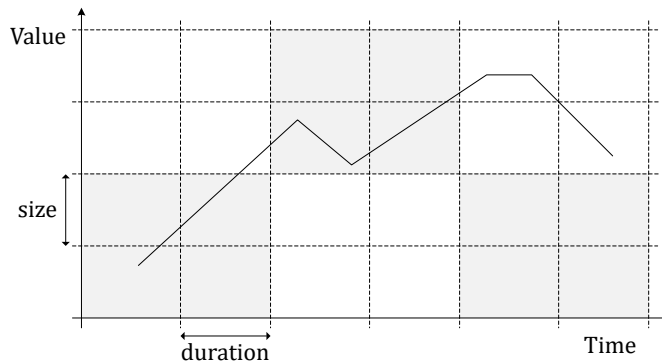


Figure 9.4: Multidimensional tiling for temporal floats.

Figure 9.4 illustrates multidimensional tiling for temporal floats. The two-dimensional domain is split into tiles having the same size for the value dimension and the same duration for the time dimension. Suppose that this tiling scheme is used for distribute a dataset across a cluster of six servers, as suggested by the gray pattern in the figure. In this case, the values are fragmented so each server will receive the data of contiguous tiles. This implies in particular that four nodes will receive one fragment of the temporal float depicted in the figure. One advantage of this distribution of data based on multidimensional tiling is that it reduces the data that needs to be exchanged between nodes when processing queries, a process typically referred to as *reshuffling*.

Many of the functions in this section are *set-returning functions* (also known as *table functions*) since they typically return more than one value. In this case, the functions are marked with the `{}` symbol.

9.6.1 Bin Operations

- Return an array of bins that cover a value or time span with bins of the same size or duration

`bins(numspans, size number, origin number=0) → span[]`

`bins(datespans, duration interval, origin date='2000-01-03') → span[]`

`bins(tstzspans, duration interval, origin timestamptz='2000-01-03') → span[]`

If the origin is not specified, it is set by default to 0 for value spans and Monday, January 3, 2000 for time spans.

```
SELECT bins(floatspan '[-10, -1]', 2.5, -7);
-- {"[-12, -9.5)", "[-9.5, -7)", "[-7, -4.5)", "[-4.5, -2)", "[-2, 0.5)"}
SELECT bins(intspan '[15, 25]', 2);
-- {"[14, 16)", "[16, 18)", "[18, 20)", "[20, 22)", "[22, 24)", "[24, 26)", "[26, 28)"}
SELECT index, span
FROM unnest(bins(datespan '[2001-01-15, 2001-01-25]', '2 days'))
  WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-17)
-- 2 | [2001-01-17, 2001-01-19)
-- 3 | [2001-01-19, 2001-01-21)
-- ...
SELECT index, span
FROM unnest(bins(tstzspanset '{{[2001-01-15, 2001-01-17],[2001-01-22, 2001-01-25]}}',
  '2 days', '2001-01-02')) WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-16)
-- 2 | [2001-01-16, 2001-01-17)
-- 3 | [2001-01-22, 2001-01-24)
-- 4 | [2001-01-24, 2001-01-25]
```

- Return the bin that contains a number or a timestamp

`getBin(number, size number, origin number=0) → span`

`getBin(date, duration interval, origin date='2000-01-03') → datespan`

`getBin(timestamptz, duration interval, origin timestamptz='2000-01-03') → tstzspan`

If the origin is not specified, it is set by default to 0 for value bins and to Monday, January 3, 2000 for time bins.

```
SELECT getBin(2, 2);
-- [2, 4)
SELECT getBin(2, 2.5, 1.5);
-- [1.5, 4)
SELECT getBin('2001-01-04', interval '1 week');
-- [2001-01-01, 2001-01-08)
SELECT getBin('2001-01-04', interval '1 week', '2001-01-07');
-- [2000-12-31, 2001-01-07)
```

9.6.2 Tile Operations

- Return the set of tiles that covers a temporal box with tiles of the same size and/or duration `{}`

`valueTiles(tbox, vsize float, vorigin float=0) → {(index, tile)}`

`timeTiles(tbox, duration interval, torigin timestamptz='2000-01-03') →`
`{(index, tile)}`

`valueTimeTiles(tbox, vsize float, duration interval, vorigin float=0,`
`torigin timestamptz='2000-01-03') → {(index, tile)}`

The result is a set of pairs (index, tile). If the origin of the value and/or time dimension is not specified, it is set by default to 0 and to Monday, January 3, 2000, respectively.

```
SELECT (gr).index, (gr).tile
FROM (SELECT valueTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, 2.0) AS gr) t;
-- 1 | TBOXFLOAT X([14, 16))
-- 2 | TBOXFLOAT X([16, 18))
-- 3 | TBOXFLOAT X([18, 20))
-- ...
SELECT (gr).index, (gr).tile
FROM (SELECT timeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, '2 days') AS gr) t;
-- 1 | TBOX T([2001-01-15, 2001-01-17))
-- 2 | TBOX T([2001-01-17, 2001-01-19))
-- 3 | TBOX T([2001-01-19, 2001-01-21))
-- ...
SELECT (gr).index, (gr).tile
FROM (SELECT valueTimeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, 2.0, '2 days')
      AS gr) t;
-- 1 | TBOX XT([14,16),[2001-01-15,2001-01-17))
-- 2 | TBOX XT([16,18),[2001-01-15,2001-01-17))
-- 3 | TBOX XT([18,20),[2001-01-15,2001-01-17))
-- ...
SELECT valueTimeTiles(tfloat '[15@2001-01-15,25@2001-01-25]'::tbox, 2.0, '2 days', 11.5);
-- (1,"TBOX XT([13.5,15.5),[2001-01-15,2001-01-17))")
-- (2,"TBOX XT([15.5,17.5),[2001-01-15,2001-01-17))")
-- (3,"TBOX XT([17.5,19.5),[2001-01-15,2001-01-17))")
-- ...
```

- Return the set of tiles that covers a spatiotemporal box with tiles of the same size and/or duration  {}

```
spaceTiles({stbox,tgeompoint},xsize float,[ysize float,zsize float,]
  sorigin geompoint='Point(0 0 0)',borderInc bool=true) → {(index,tile)}
timeTiles({stbox,tgeompoint},duration interval,torigin timestamptz='2000-01-03',
  borderInc bool=true) → {(index,tile)}
spaceTimeTiles({stbox,tgeompoint},xsize float,[ysize float,zsize float,]duration interval
  sorigin geompoint='Point(0 0 0)',torigin timestamptz='2000-01-03',
  borderInc bool=true) → {(index,tile)}
```

The result is a set of pairs (index, tile). If the origin of the space and/or time dimension is not specified, it is set by default to 'Point(0 0 0)' and to Monday, January 3, 2000, respectively. The optional argument borderInc states whether the upper border of the extent is included and thus, extra tiles containing the border are generated. When the first argument is a tgeompoint, the bounding spatiotemporal box is derived implicitly via stbox(temp).

In the case of a spatiotemporal grid, ysize and zsize are optional, the size for the missing dimensions is assumed to be equal to xsize. The SRID of the tile coordinates is determined by the input box and the sizes are given in the units of the SRID. If the origin for the spatial coordinates is given, which must be a point, its dimensionality and SRID should be equal to the one of box, otherwise an error is raised.

```
SELECT spaceTiles(tgeompoint '[Point(3 3)@2001-01-15,
  Point(15 15)@2001-01-25]', 2.0);
-- (1,"STBOX X((2,2),(4,4))")
-- (2,"STBOX X((4,2),(6,4))")
-- (3,"STBOX X((6,2),(8,4))")
-- ...
SELECT timeTiles(tgeompoint '[Point(3 3)@2001-01-15,
  Point(15 15)@2001-01-25]', '2 days');
-- (1,"STBOX T([2001-01-15, 2001-01-17))")
-- (2,"STBOX T([2001-01-17, 2001-01-19))")
-- (3,"STBOX T([2001-01-19, 2001-01-21))")
-- ...
```

```

SELECT spaceTiles(tgeompoint 'SRID=3812;[Point(3 3)@2001-01-15,
    Point(15 15)@2001-01-25]':::stbox, 2.0, geometry 'Point(3 3)');
-- (1,"SRID=3812;STBOX X((3,3),(5,5))")
-- (2,"SRID=3812;STBOX X((5,3),(7,5))")
-- (3,"SRID=3812;STBOX X((7,3),(9,5))")
-- ...
SELECT spaceTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
    Point(15 15 15)@2001-01-25]':::stbox, 2.0, geometry 'Point(3 3 3)');
-- (1,"STBOX Z((3,3,3),(5,5,5))")
-- (2,"STBOX Z((5,3,3),(7,5,5))")
-- (3,"STBOX Z((7,3,3),(9,5,5))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3)@2001-01-15,
    Point(15 15)@2001-01-25]', 2.0, interval '2 days');
-- (1,"STBOX XT((2,2),(4,4)), [2001-01-15,2001-01-17))")
-- (2,"STBOX XT((4,2),(6,4)), [2001-01-15,2001-01-17))")
-- (3,"STBOX XT((6,2),(8,4)), [2001-01-15,2001-01-17))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
    Point(15 15 15)@2001-01-25]':::stbox, 2.0, interval '2 days',
    'Point(3 3 3)', '2001-01-15');
-- (1,"STBOX ZT((3,3,3),(5,5,5)), [2001-01-15,2001-01-17))")
-- (2,"STBOX ZT((5,3,3),(7,5,5)), [2001-01-15,2001-01-17))")
-- (3,"STBOX ZT((7,3,3),(9,5,5)), [2001-01-15,2001-01-17))")
-- ...

```

- Return the temporal tile that covers a value and/or a timestamp 

getValueTile(value float, vsize float, vorigin float=0.0,) → tbox

getTboxTimeTile(time timestamptz, duration interval,
torigin timestamptz='2000-01-03') → tbox

getValueTimeTile(value float, time timestamptz, size float, duration interval,
vorigin float=0.0, toorigin timestamptz='2000-01-03') → tbox

If the origin of the value and/or time dimension is not specified, it is set by default to 0 and to Monday, January 3, 2000, respectively.

```

SELECT getValueTile(15, 2);
-- TBOX ([14,16))
SELECT getTboxTimeTile('2001-01-15', interval '2 days');
-- TBOX ([2001-01-15,2001-01-17))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days');
-- TBOX XT([14,16), [2001-01-15,2001-01-17))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days', 1, '2001-01-02');
-- TBOX XT([15,17), [2001-01-14,2001-01-16))

```

- Return the spatiotemporal tile that covers a point and/or a timestamp 

getSpaceTile(point geometry, xsize float, [ysize float, zsize float],
sorigin geompoint='Point(0 0 0)') → stbox

getStboxTimeTile(time timestamptz, duration interval,
torigin timestamptz='2000-01-03') → stbox

getSpaceTimeTile(point geometry, time timestamptz, xsize float,
[ysize float, zsize float,]duration, interval, sorigin geompoint='Point(0 0 0)',
torigin timestamptz='2000-01-03') → stbox

If the origin of the space and/or time dimension is not specified, it is set by default to Point(0 0 0) and to Monday, January 3, 2000, respectively.

In the case of a spatiotemporal grid, `ysize` and `zsize` are optional, the size for the missing dimensions is assumed to be equal to `xsize`. The SRID of the tile coordinates is determined by the input point and the sizes are given in the units of the SRID. If the origin for the spatial coordinates is given, which must be a point, its dimensionality and SRID should be equal to the one of `box`, otherwise an error is raised.

```
SELECT getSpaceTile(geometry 'Point(1 1 1)', 2.0);
-- STBOX Z((0,0,0), (2,2,2))
SELECT getStboxTimeTile(timestampz '2001-01-01', interval '2 days');
-- STBOX T([2001-01-01,2001-01-03))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days');
-- STBOX XT((0,0), (2,2)), [2001-01-01, 2001-01-03))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days',
    'Point(1 1)', '2001-01-02');
-- STBOX XT(((1,1), (3,3)), [2000-12-31, 2001-01-02))
```

9.6.3 Bounding Box Operations

These functions fragment the bounding box of a temporal value with respect to a multidimensional grid. They provide an alternative to the functions in Section 9.5 to split the bounding boxes by specifying the maximum size of the boxes in the various dimensions.

- Return an array of value or time spans obtained from the instants or segments of a temporal number with respect to a value or time grid

`valueBins(tnumber, vsize number, vorigin number=0) → numspan[]`

`timeBins(temp, duration interval, torigin timestampz='2000-01-03') → tstzspan[]`

The choice between instants or segments depends on whether the interpolation is discrete or continuous. If the origin of values or time is not specified, it is set by default to 0 for value spans or to Monday, January 3, 2000 for time spans.

```
SELECT valueBins(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
    1@2000-01-05}', 3);
-- {"[1, 3]", "[4, 5]"}
SELECT valueBins(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
    1@2000-01-05]', 3, 1);
-- {"[1, 4]", "[4, 4]"}

```

```
SELECT timeBins(ttext '{AAA@2000-01-01, BBB@2000-01-02, AAA@2000-01-03, CCC@2000-01-04,
    AAA@2000-01-05}', '3 days');
-- {"[2000-01-01, 2000-01-02]", "[2000-01-03, 2000-01-05]"}
SELECT timeBins(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
    1@2000-01-05]', '3 days', '2000-01-01');
-- {"[2000-01-01, 2000-01-04]", "[2000-01-04, 2000-01-05]"}

```

- Return an array of temporal boxes obtained from the instants or segments of a temporal number with respect to a value and/or time grid

`valueBoxes(tnumber, size number, vorigin number=0) → tbox[]`

`timeBoxes(tnumber, duration interval, torigin timestampz='2000-01-03') → tbox[]`

`valueTimeBoxes(tnumber, size number, duration interval, vorigin number=0,
 torigin timestampz='2000-01-03') → tbox[]`

The choice between instants or segments depends on whether the interpolation is discrete or continuous. If the origin of value and/or time dimension is not specified, it is set by default to 0 and to Monday, January 3, 2000, respectively.

```
SELECT valueBoxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
    1@2000-01-05}', 3);
/* {"TBOXINT XT([1, 3], [2000-01-01, 2000-01-05])",
    "TBOXINT XT([4, 5], [2000-01-04, 2000-01-04])"} */

```

```

SELECT timeBoxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 3, '3 days');
/* {"TBOXINT XT([1, 3],[2000-01-01, 2000-01-02])",
    "TBOXINT XT([1, 5],[2000-01-03, 2000-01-05])"} */
SELECT valueTimeBoxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 3, '3 days');
/* {"TBOXINT XT([1, 3],[2000-01-01, 2000-01-02])",
    "TBOXINT XT([1, 2],[2000-01-03, 2000-01-05])",
    "TBOXINT XT([4, 5],[2000-01-04, 2000-01-04])"} */
SELECT valueTimeBoxes(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 3, '3 days', 1, '2000-01-01');
/* {"TBOXFLOAT XT([1, 4],[2000-01-01, 2000-01-04])",
    "TBOXFLOAT XT([1, 4],[2000-01-04, 2000-01-05])",
    "TBOXFLOAT XT([4, 4],[2000-01-04, 2000-01-04])"} */

```

- Return an array of spatiotemporal boxes obtained from the instants or segments of a temporal point with respect to a space and/or time grid 

```

spaceBoxes(tgeompoint, xsize float, [ysize float, zsize float,]
  sorigin geompoint='Point(0 0 0)', borderInc bool=true) → stbox[]
timeBoxes(tgeompoint, duration interval, torigin timestampz='2000-01-03',
  borderInc bool=true) → stbox[]
spaceTimeBoxes(tgeompoint, xsize float, [ysize float, zsize float,] duration interval,
  sorigin geompoint='Point(0 0 0)', torigin timestampz='2000-01-03',
  borderInc bool=true) → stbox[]

```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The arguments `ysize` and `zsize` are optional, the size for the missing dimensions is assumed to be equal to `xsize`. The SRID of the tile coordinates is determined by the temporal point and the sizes are given in the units of the SRID. If the origin for the spatial coordinates is given, which must be a point, its dimensionality and SRID should be equal to the one of temporal point, otherwise an error is raised. If the origin of space and/or time dimension is not specified, it is set by default to 'Point(0 0 0)' and to Monday, January 3, 2000, respectively. The optional argument `borderInc` states whether the upper border of the extent is included and thus, extra tiles containing the border are generated.

```

SELECT spaceBoxes(tgeompoint '{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
Point(1 1)@2000-01-03, Point(4 4)@2000-01-04, Point(1 1)@2000-01-05}', 3);
/* {"STBOX XT((1,1),(2,2)), [2000-01-01, 2000-01-05])",
    "STBOX XT((4,4),(4,4)), [2000-01-04, 2000-01-04])"} */
SELECT timeBoxes(tgeompoint '{Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02,
Point(1 1 1)@2000-01-03, Point(4 4 4)@2000-01-04, Point(1 1 1)@2000-01-05}',
  interval '2 days', '2000-01-01');
/* {"STBOX ZT(((1,1,1),(2,2,2)), [2000-01-01, 2000-01-02])",
    "STBOX ZT(((1,1,1),(4,4,4)), [2000-01-03, 2000-01-04])",
    "STBOX ZT(((1,1,1),(1,1,1)), [2000-01-05, 2000-01-05])"} */
SELECT spaceTimeBoxes(tgeompoint '{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
Point(1 1)@2000-01-03, Point(4 4)@2000-01-04, Point(1 1)@2000-01-05}', 3, '3 days');
/* {"STBOX XT((1,1),(2,2)), [2000-01-01, 2000-01-02])",
    "STBOX XT((1,1),(1,1)), [2000-01-03, 2000-01-05])",
    "STBOX XT((4,4),(4,4)), [2000-01-04, 2000-01-04])"} */
SELECT spaceTimeBoxes(tgeompoint '[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02,
Point(1 1 1)@2000-01-03, Point(4 4 4)@2000-01-04, Point(1 1 1)@2000-01-05]',
  3, interval '3 days', 'Point(1 1 1)', '2000-01-01');
/* {"STBOX ZT(((1,1,1),(4,4,4)), [2000-01-01, 2000-01-04])",
    "STBOX ZT(((1,1,1),(4,4,4)), [2000-01-04, 2000-01-05])",
    "STBOX ZT(((4,4,4),(4,4,4)), [2000-01-04, 2000-01-04])"} */

```

9.6.4 Splitting Operations

These functions split a temporal value with respect to a sequence of bins (see Section 9.6.1) or tiles (see Section 9.6.2).

- Split a temporal number with respect to value and/or time bins {}

valueSplit(tnumber, size number, origin number=0) → {(number, tnumber)}

timeSplit(ttype, duration interval, origin timestamp tz='2000-01-03') →
{(time, temp)}

valueTimeSplit(tnumber, size number, duration interval, vorigin number=0,
torigin timestamp tz='2000-01-03') → {(number, time, tnumber)}

The result is a set of pairs or a set of triples. If the origin of the value and/or the time dimension is not specified, they are set by default to 0 and to Monday, January 3, 2000, respectively.

```
SELECT (sp).number, (sp).tnumber
FROM (SELECT valueSplit(tint '1@2001-01-01, 2@2001-01-02, 5@2001-01-05, 10@2001-01-10]',
  2) AS sp) t;
-- 0 | {[1@2001-01-01, 1@2001-01-02)}
-- 2 | {[2@2001-01-02, 2@2001-01-05)}
-- 4 | {[5@2001-01-05, 5@2001-01-10)}
-- 10 | {[10@2001-01-10)}
SELECT valueSplit(tfloat '1@2001-01-01, 10@2001-01-10]', 2.0, 1.0);
-- (1,"{[1@2001-01-01, 3@2001-01-03)}")
-- (3,"{[3@2001-01-03, 5@2001-01-05)}")
-- (5,"{[5@2001-01-05, 7@2001-01-07)}")
-- (7,"{[7@2001-01-07, 9@2001-01-09)}")
-- (9,"{[9@2001-01-09, 10@2001-01-10 00:00:00+01)}")
```

```
SELECT (ts).time, (ts).temp
FROM (SELECT timeSplit(tfloat '1@2001-02-01, 10@2001-02-10]', '2 days') AS ts) t;
-- 2001-01-31 | [1@2001-02-01, 2@2001-02-02)
-- 2001-02-02 | [2@2001-02-02, 4@2001-02-04)
-- 2001-02-04 | [4@2001-02-04, 6@2001-02-06)
-- ...
SELECT (ts).time, astext((ts).temp) AS temp
FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  '2 days', '2001-02-01') AS ts) AS t;
-- 2001-02-01 | [POINT(1 1)@2001-02-01, POINT(3 3)@2001-02-03)
-- 2001-02-03 | [POINT(3 3)@2001-02-03, POINT(5 5)@2001-02-05)
-- 2001-02-05 | [POINT(5 5)@2001-02-05, POINT(7 7)@2001-02-07)
-- ...
```

Notice that we can split a temporal value in cyclic (instead of linear) time bins. The following two examples show how to split a temporal value by hour and by day of the week.

```
SELECT (ts).time::time AS hour, merge((ts).temp) AS temp
FROM (SELECT timeSplit(tfloat '1@2001-01-01, 10@2001-01-03]', '1 hour') AS ts) t
GROUP BY hour ORDER BY hour;
/* 00:00:00 | {[1@2001-01-01 00:00:00+01, 1.1875@2001-01-01 01:00:00+01),
  [5.5@2001-01-02 00:00:00+01, 5.6875@2001-01-02 01:00:00+01)} */
/* 01:00:00 | {[1.1875@2001-01-01 01:00:00+01, 1.375@2001-01-01 02:00:00+01),
  [5.6875@2001-01-02 01:00:00+01, 5.875@2001-01-02 02:00:00+01)} */
/* 02:00:00 | {[1.375@2001-01-01 02:00:00+01, 1.5625@2001-01-01 03:00:00+01),
  [5.875@2001-01-02 02:00:00+01, 6.0625@2001-01-02 03:00:00+01)} */
/* 03:00:00 | {[1.5625@2001-01-01 03:00:00+01, 1.75@2001-01-01 04:00:00+01),
  [6.0625@2001-01-02 03:00:00+01, 6.25@2001-01-02 04:00:00+01)} */
/* ... */
SELECT EXTRACT(DOW FROM (ts).time) AS dow_no, TO_CHAR((ts).time, 'Dy') AS dow,
  asText(round(merge((ts).temp), 2)) AS temp
FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-01-01, Point(10 10)@2001-01-14]',
  '1 hour') AS ts) t
GROUP BY dow, dow_no ORDER BY dow_no;
/* 0 | Sun | [POINT(1 1)@2001-01-01, POINT(1.69 1.69)@2001-01-02),
  [POINT(5.85 5.85)@2001-01-08, POINT(6.54 6.54)@2001-01-09)} */
```

```

/* 1 | Mon | {[POINT(1.69 1.69)@2001-01-02, POINT(2.38 2.38)@2001-01-03),
           [POINT(6.54 6.54)@2001-01-09, POINT(7.23 7.23)@2001-01-10]} */
/* 2 | Tue | {[POINT(2.38 2.38)@2001-01-03, POINT(3.08 3.08)@2001-01-04),
           [POINT(7.23 7.23)@2001-01-10, POINT(7.92 7.92)@2001-01-11]} */
/* ... */

SELECT (sp).number, (sp).time, (sp).tnumber
FROM (SELECT valueTimeSplit(tint '1@2001-02-01, 2@2001-02-02, 5@2001-02-05,
10@2001-02-10]', 5, '5 days') AS sp) t;
-- 0 | 2001-02-01 | {[1@2001-02-01, 2@2001-02-02, 2@2001-02-05]}
-- 5 | 2001-02-01 | {[5@2001-02-05, 5@2001-02-06]}
-- 5 | 2001-02-06 | {[5@2001-02-06, 5@2001-02-10]}
-- 10 | 2001-02-06 | {[10@2001-02-10]}
SELECT (sp).number, (sp).time, (sp).tnumber
FROM (SELECT valueTimeSplit(tfloat '1@2001-02-01, 10@2001-02-10]', 5.0, '5 days', 1.0,
'2001-02-01') AS sp) t;
-- 1 | 2001-01-01 | [1@2001-01-01, 6@2001-01-06]
-- 6 | 2001-01-06 | [6@2001-01-06, 10@2001-01-10]

```

- Split a temporal point with respect to a spatial grid  

```

spaceSplit(tgeompoint, xsize float, [ysize float, zsize float,]
origin geompoint='Point(0 0 0)', bitmatrix boolean=true, borderInc bool=true) →
{(point, tpoint)}

timeSplit(tpoint, duration interval, torigin timestamptz='2000-01-03') → {(time, tpoint)}

spaceTimeSplit(tgeompoint, xsize float, [ysize float, zsize float,]
duration interval, sorigin geompoint='Point(0 0 0)',
torigin timestamptz='2000-01-03', bitmatrix boolean=true, borderInc boolean=true) →
{(point, time, tpoint)}

```

The result is a set of pairs or a set of triples. If the origin of the space and/or time dimension is not specified, it is set by default to 'Point(0 0 0)' and to Monday, January 3, 2000, respectively. The arguments *ysize* and *zsize* are optional, the size for the missing dimensions is assumed to be equal to *xsize*. If the argument *bitmatrix* is not specified, then the computation will use a bit matrix to speed up the process. The optional argument *borderInc* states whether the upper border of the extent is included and thus extra tiles containing the border are generated.

```

SELECT ST_AsText((sp).point) AS point, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1)@2001-03-01, Point(10 10)@2001-03-10]',
2.0) AS sp) t;
-- POINT(0 0) | {[POINT(1 1)@2001-03-01, POINT(2 2)@2001-03-02]}
-- POINT(2 2) | {[POINT(2 2)@2001-03-02, POINT(4 4)@2001-03-04]}
-- POINT(4 4) | {[POINT(4 4)@2001-03-04, POINT(6 6)@2001-03-06]}
-- ...
SELECT ST_AsText((sp).point) AS point, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1 1)@2001-03-01,
Point(10 10 10)@2001-03-10]', 2.0, geometry 'Point(1 1 1)') AS sp) t;
-- POINT Z(1 1 1) | {[POINT Z(1 1 1)@2001-03-01, POINT Z(3 3 3)@2001-03-03]}
-- POINT Z(3 3 3) | {[POINT Z(3 3 3)@2001-03-03, POINT Z(5 5 5)@2001-03-05]}
-- POINT Z(5 5 5) | {[POINT Z(5 5 5)@2001-03-05, POINT Z(7 7 7)@2001-03-07]}
-- ...

```

```

SELECT (sp).time, astext((sp).temp) AS tpoint
FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
interval '2 days') AS sp) t;
-- 2001-01-31 | [POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02]
-- 2001-02-02 | [POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04]
-- 2001-02-04 | [POINT(4 4)@2001-02-04, POINT(6 6)@2001-02-06]
-- ...

```

```
SELECT ST_AsText((sp).point) AS point, (sp).time, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  2.0, interval '2 days') AS sp) t;
-- POINT(0 0) | 2001-01-31 | {[POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02]}
-- POINT(2 2) | 2001-01-31 | {[POINT(2 2)@2001-02-02]}
-- POINT(2 2) | 2001-02-02 | {[POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04]}
-- ...
SELECT ST_AsText((sp).point) AS point, (sp).time, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1 1)@2001-02-01,
  Point(10 10 10)@2001-02-10]', 2.0, interval '2 days', 'Point(1 1 1)',
  '2001-03-01') AS sp) t;
-- POINT Z(1 1 1) | 2001-02-01 | {[POINT Z(1 1 1)@2001-02-01, POINT Z(3 3 3)@2001-02-03]}
-- POINT Z(3 3 3) | 2001-02-01 | {[POINT Z(3 3 3)@2001-02-03]}
-- POINT Z(3 3 3) | 2001-02-03 | {[POINT Z(3 3 3)@2001-02-03, POINT Z(5 5 5)@2001-02-05]}
-- ...
```

Chapter 10

Temporal Types: Aggregation and Indexing

10.1 Aggregation

The temporal aggregate functions generalize the traditional aggregate functions. Their semantics is that they compute the value of the function at every instant in the *union* of the temporal extents of the values to aggregate. In contrast, recall that all other functions manipulating temporal types compute the value of the function at every instant in the *intersection* of the temporal extents of the arguments.

The temporal aggregate functions are the following ones:

- For all temporal types, the function `tCount` generalize the traditional function `count`. The temporal count can be used to compute at each point in time the number of available objects (for example, number of cars in an area).
- For all temporal types, function `extent` returns a bounding box that encloses a set of temporal values. Depending on the base type, the result of this function can be a `tstzspan`, a `tbox` or an `stbox`.
- For the temporal Boolean type, the functions `tAnd` and `tOr` generalize the traditional functions `and` and `or`.
- For temporal numeric types, there are two types of temporal aggregate functions. The functions `tMin`, `tMax`, `tSum`, and `tAvg` generalize the traditional functions `min`, `max`, `sum`, and `avg`. Furthermore, the functions `wMin`, `wMax`, `wCount`, `wSum`, and `wAvg` are window (or cumulative) versions of the traditional functions that, given a time interval `w`, compute the value of the function at an instant `t` by considering the values during the interval `[t-w, t]`. All window aggregate functions are available for temporal integers, while for temporal floats only window minimum and maximum are meaningful.
- For the temporal text type, the functions `tMin` y `tMax` generalize the traditional functions `min` and `max`.
- Finally, for temporal point types, the function `tCentroid` generalizes the function `ST_Centroid` provided by PostGIS. For example, given set of objects that move together (that is, a convoy or a flock) the temporal centroid will produce a temporal point that represents at each instant the geometric center (or the center of mass) of all the moving objects.

Note

The names `tMin`, `tMax`, `merge`, `appendInstant`, and `appendSequence` each denote both a scalar function and an aggregate. The canonical name of the aggregate form carries an `Agg` suffix (`tMinAgg`, `tMaxAgg`, `mergeAgg`, `appendInstantAgg`, and `appendSequenceAgg`), so that the scalar and aggregate names are distinct. The bare aggregate names remain available as backward-compatibility aliases and are deprecated; they will be removed in a future major release. The scalar forms (the box accessors `tMin`/`tMax` and the binary builder `merge`) keep their names.

In the examples that follow, we suppose the tables `Department` and `Trip` contain the two tuples introduced in Section 4.2.

- Temporal count

```
tCount(ttype) → {tintSeq,tintSeqSet}
```

```
SELECT tCount (NoEmps) FROM Department;
-- {[1@2001-01-01, 2@2001-02-01, 1@2001-08-01, 1@2001-10-01]}
```

- Bounding box extent

extent (temp) → {tstzspan, tbox, stbox}

```
SELECT extent (noEmps) FROM Department;
-- TBOX XT((4,12), [2001-01-01, 2001-10-01])
SELECT extent (Trip) FROM Trips;
-- STBOX XT(((0,0), (3,3)), [2001-01-01 08:00:00+01, 2001-01-01 08:20:00+01])
```

- Temporal and, temporal or

tOr (tbool) → tbool

tAnd (tbool) → tbool

```
SELECT tAnd (NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-04-01, f@2001-10-01]}
SELECT tOr (NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-08-01, f@2001-10-01]}
```

- Temporal minimum, maximum, sum, and average

tMin (ttype) → ttype

tMax (ttype) → ttype

tSum (tnumber) → {tnumberSeq, tnumberSeqSet}

tAvg (tnumber) → {tfloatSeq, tfloatSeqSet}

```
SELECT tMin (NoEmps) FROM Department;
-- {[10@2001-01-01, 4@2001-02-01, 6@2001-06-01, 6@2001-10-01]}
SELECT tMax (NoEmps) FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-01, 6@2001-10-01]}
SELECT tSum (NoEmps) FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01, 18@2001-06-01, 6@2001-08-01,
    6@2001-10-01]} */
SELECT tAvg (NoEmps) FROM Department;
/* {[10@2001-01-01, 10@2001-02-01), [7@2001-02-01, 7@2001-04-01),
    [8@2001-04-01, 8@2001-06-01), [9@2001-06-01, 9@2001-08-01),
    [6@2001-08-01, 6@2001-10-01]} */
```

- Window count

wCount (tnumber, interval) → {tintSeq, tintSeqSet}

```
SELECT wCount (NoEmps, interval '2 days') FROM Department;
/* {[1@2001-01-01, 2@2001-02-01, 3@2001-04-01, 2@2001-04-03, 3@2001-06-01, 2@2001-06-03,
    1@2001-08-03, 1@2001-10-03]} */
```

- Window minimum, maximum, sum, and average

wMin (tnumber, interval) → {tnumberSeq, tnumberSeqSet}

wMax (tnumber, interval) → {tnumberDiscSeq, tnumberSeqSet}

wSum (tint, interval) → {tintSeq, tintSeqSet}

wAvg (tint, interval) → {tfloatSeq, tfloatSeqSet}

```

SELECT wMin(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 4@2001-04-01, 6@2001-06-03, 6@2001-10-03]}
SELECT wMax(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-03, 6@2001-10-03]}
SELECT wSum(NoEmps, interval '2 days') FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 26@2001-04-01, 16@2001-04-03, 22@2001-06-01,
    18@2001-06-03, 6@2001-08-03, 6@2001-10-03]} */
SELECT round(wAvg(NoEmps, interval '2 days'), 3) FROM Department;
/* Interp=Step;{[10@2001-01-01, 7@2001-02-01, 8.667@2001-04-01, 8@2001-04-03,
    7.333@2001-06-01, 9@2001-06-03, 6@2001-08-03, 6@2001-10-03]} */

```

- Temporal centroid

tCentroid(tgeompoint) → tgeompoint

```

SELECT tCentroid(Trip) FROM Trips;
/* {[POINT(0 0)@2001-01-01 08:00:00+00, POINT(1 0)@2001-01-01 08:05:00+00),
    [POINT(0.5 0)@2001-01-01 08:05:00+00, POINT(1.5 0.5)@2001-01-01 08:10:00+00,
    POINT(2 1.5)@2001-01-01 08:15:00+00),
    [POINT(2 2)@2001-01-01 08:15:00+00, POINT(3 3)@2001-01-01 08:20:00+00)} */

```

10.2 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal types. The GiST index implements an R-tree and the SP-GiST index implements an n-dimensional quad-tree. Examples of index creation are as follows:

```

CREATE INDEX Department_NoEmps_Gist_Idx ON Department USING Gist(NoEmps);
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);

```

The GiST and SP-GiST indexes store the bounding box for the temporal types. As explained in Chapter 4, these are

- the `tstzspan` type for the `tbool` and `ttext` types,
- the `tbox` type for the `tint` and `tfloat` types,
- the `stbox` type for the `tgeompoint`, `tgeogpoint`, `tgeometry` and `tgeography` types.

A GiST or SP-GiST index can accelerate queries involving the following operators (see Section 5.3 for more information):

- `<<`, `&<`, `&>`, `>>`, which only consider the value dimension in temporal alphanumeric types,
- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, `&</`, `<</`, `/>>`, and `/&>`, which only consider the spatial dimension in temporal point types,
- `&<#`, `<<#`, `#>>`, `#&>`, which only consider the time dimension for all temporal types,
- `&&`, `@>`, `<@`, `~=>`, and `|=>`, which consider as many dimensions as they are shared by the indexed column and the query argument. These operators work on bounding boxes (that is, `tstzspan`, `tbox`, or `stbox`), not the entire values.

For example, given the index defined above on the `Department` table and a query that involves a condition with the `&&` (overlaps) operator, if the right argument is a temporal float then both the value and the time dimensions are considered for filtering the tuples of the relation, while if the right argument is a float value, a float span, or a time type, then either the value or the time dimension will be used for filtering the tuples of the relation. Furthermore, a bounding box can be constructed from a value/span and/or a timestamp/period, which can be used for filtering the tuples of the relation. Examples of queries using the index on the `Department` table defined above are given next.


```

SELECT * FROM Department WHERE NoEmps && intspan '[1, 5)';
SELECT * FROM Department WHERE NoEmps && tstzspan '[2001-04-01, 2001-05-01)';
SELECT * FROM Department WHERE NoEmps &&
    tbox(intspan '[1, 5)', tstzspan '[2001-04-01, 2001-05-01)');
SELECT * FROM Department WHERE NoEmps &&
    tfloat '{[1@2001-01-01, 1@2001-02-01), [5@2001-04-01, 5@2001-05-01)}';

```

Similarly, examples of queries using the index on the Trips table defined above are given next.

```

SELECT * FROM Trips WHERE Trip && geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))';
SELECT * FROM Trips WHERE Trip && timestampz '2001-01-01';
SELECT * FROM Trips WHERE Trip && tstzspan '[2001-01-01, 2001-01-05)';
SELECT * FROM Trips WHERE Trip &&
    stbox(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))', tstzspan '[2001-01-01, 2001-01-05)');
SELECT * FROM Trips WHERE Trip &&
    tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02, Point(1 1)@2001-01-05)}';

```

Finally, B-tree indexes can be created for table columns of all temporal types. For this index type, the only useful operation is equality. There is a B-tree sort ordering defined for values of temporal types, with corresponding `<`, `<=`, `>`, `>=` and operators, but the ordering is rather arbitrary and not usually useful in the real world. B-tree support for temporal types is primarily meant to allow sorting internally in queries, rather than creation of actual indexes.

In order to speed up several of the functions for temporal types, we can add in the `WHERE` clause of queries a bounding box comparison that make uses of the available indexes. For example, this would be typically the case for the functions that project the temporal types to the value/spatial and/or time dimensions. This will filter out the tuples with an index as shown in the following query.

```

SELECT atTime(T.Trip, tstzspan '[2001-01-01, 2001-01-02)')
FROM Trips T
-- Bouding box index filtering
WHERE T.Trip && tstzspan '[2001-01-01, 2001-01-02)';

```

In the case of temporal points, all spatial relationships with the ever and always semantics (see Section 8.5) automatically include a bounding box comparison that will make use of any indexes that are available on the temporal points. For this reason, the first version of the relationships is typically used for filtering the tuples with the help of an index when computing the temporal relationships as shown in the following query.

```

SELECT tIntersects(T.Trip, R.Geom)
FROM Trips T, Regions R
-- Bouding box index filtering
WHERE eIntersects(T.Trip, R.Geom);

```

10.3 Statistics and Selectivity

10.3.1 Statistics Collection

The PostgreSQL planner relies on statistical information about the contents of tables in order to generate the most efficient execution plan for queries. These statistics include a list of some of the most common values in each column and a histogram showing the approximate data distribution in each column. For large tables, a random sample of the table contents is taken, rather than examining every row. This enables large tables to be analyzed in a small amount of time. The statistical information is gathered by the `ANALYZE` command and stored in the `pg_statistic` catalog table. Since different kinds of statistics may be appropriate for different kinds of data, the table only stores very general statistics (such as number of null values) in dedicated columns. Everything else is stored in five “slots”, which are couples of array columns that store the statistics for a column of an arbitrary type.

The statistics collected for time types and temporal types are based on those collected by PostgreSQL for scalar types and span types. For scalar types, such as `float`, the following statistics are collected:

1. fraction of null values,
2. average width, in bytes, of non-null values,
3. number of different non-null values,
4. array of most common values and array of their frequencies,
5. histogram of values, where the most common values are excluded,
6. correlation between physical and logical row ordering.

For span types, like `tstzspan`, three additional histograms are collected:

7. length histogram of non-empty spans,
8. histograms of lower and upper bounds.

For geometries, in addition to (1)–(3), the following statistics are collected:

9. number of dimensions of the values, N-dimensional bounding box, number of rows in the table, number of rows in the sample, number of non-null values,
10. N-dimensional histogram that divides the bounding box into a number of cells and keeps the proportion of values that intersects with each cell.

The statistics collected for columns of the time and span types `tstzset`, `tstzspan`, `tstzspanset`, `intspan`, and `floatspan` replicate those collected by PostgreSQL for the `tstzrange`. This is clear for the span types in MobilityDB, which are more efficient versions of the range types in PostgreSQL. For the `tstzset` and the `tstzspanset` types, a value is converted into its bounding period, then the statistics for the `tstzspan` type are collected.

The statistics collected for columns of temporal types depend on their subtype and their base type. In addition to statistics (1)–(3) that are collected for all temporal types, statistics are collected for the time and the value dimensions independently. More precisely, the following statistics are collected for the time dimension:

- For columns of instant subtype, the statistics (4)–(6) are collected for the timestamps.
- For columns of other subtype, the statistics (7)–(8) are collected for the (bounding box) periods.

The following statistics are collected for the value dimension:

- For columns of temporal types with step interpolation (that is, `tbool`, `ttext`, or `tint`):
 - For the instant subtype, the statistics (4)–(6) are collected for the values.
 - For all other subtypes, the statistics (7)–(8) are collected for the values.
- For columns of the temporal float type (that is, `tfloat`):
 - For the instant subtype, the statistics (4)–(6) are collected for the values.
 - For all other subtype, the statistics (7)–(8) are collected for the (bounding) value spans.
- For columns of temporal point types (that is, `tgeompoint` and `tgeogpoint`) the statistics (9)–(10) are collected for the points.

10.3.2 Selectivity Estimation

Boolean operators in PostgreSQL can be associated with two selectivity functions, which compute how likely a value of a given type will match a given criterion. These selectivity functions rely on the statistics collected. There are two types of selectivity functions. The *restriction* selectivity functions try to estimate the percentage of the rows in a table that satisfy a WHERE-clause condition of the form `column OP constant`. On the other hand, the *join* selectivity functions try to estimate the percentage of the rows in a table that satisfy a WHERE-clause condition of the form `table1.column1 OP table2.column2`.

MobilityDB defines 23 classes of Boolean operators (such as `=`, `<`, `&&`, `<<`, etc.), each of which can have as left or right arguments a PostgreSQL type (such as `integer`, `timestampz`, etc.) or a MobilityDB type (such as `tstzspan`, `tint`, etc.). As a consequence, there is a very high number of operators with different arguments to be considered for the selectivity functions. The approach taken was to group these combinations into classes corresponding to the value and time dimensions. The classes correspond to the type of statistics collected as explained in the previous section.

MobilityDB estimates both restriction and join selectivity for time, span, and temporal types.

Chapter 11

Temporal Poses

The `pose` type is used to represent the *location* and *orientation* of geometric objects within coordinate systems anchored to the earth's surface or within other astronomical coordinate systems. The location is represented by a 2D or 3D point. For 2D poses, the orientation is defined by a rotation angle in $(-\pi, \pi]$ expressed in radians. For 3D poses, the orientation is defined by four float values W, X, Y, Z , representing a unit **quaternion** $Q = \langle W, X, Y, Z \rangle$ where $\|Q\| = \sqrt{W^2 + X^2 + Y^2 + Z^2} = 1$.

The pose constructor accepts quaternions whose norm is within $1e-3$ of unit (a tolerance wide enough to absorb the integrator drift typical of sensor-fusion clients such as IMUs and AR/VR runtimes), then renormalizes to exactly unit norm before storing. Way-off norms (e.g., the obvious-bug case $(1, 1, 1, 1)$ where $|q| = 2$), zero quaternions, and quaternions with NaN/Inf components are rejected up front. The on-disk representation is therefore independent of the caller's floating-point hygiene, and downstream comparison, hashing, SLERP, and Euler-decomposition code can rely on the unit-norm invariant.

The **GeoPose Standards Working Group** (SWG), working under the auspices of the **Open Geospatial Consortium**, has defined a standard for exchanging pose information across different users, devices, and platforms. More information about the standard can be found in the **GitHub repository** of the GeoPose SWG.

The `pose` type serves as base type for defining the temporal pose type `tpose`. The `tpose` type has similar functionality as the temporal point type `tgeompoint`. Thus, most functions and operators described before for the `tgeompoint` type are also applicable for the `tpose` type. In addition, there are specific functions defined for the `tpose` type. We cover these functions in this chapter.

The `tpose` type is used for defining the type `trgeometry` (that is, temporal rigid geometry) defined in the next chapter. The implementation of the these types in MobilityDB has been studied in the following PhD thesis.

- Maxime Schoemans, *Managing Rigid Temporal Geometries in Moving Object Databases*, PhD thesis, Université libre de Bruxelles, 2025.

11.1 Static Poses

A 2D pose is a couple of the form $(\text{point2D}, \text{radius})$ where `point2D` is a 2D geometric point and `radius` is a float value representing a rotation angle in $(-\pi, \pi]$ expressed in radians. A 3D pose is a tuple of the form $(\text{point3D}, W, Y, X, Z)$ where `point3D` is a 3D geometric point, and `W`, and `X`, `Y`, and `Z` are four floats representing a *unit* quaternion. Examples of input of pose values are as follows:

```
SELECT pose 'Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

An SRID can be specified for a pose either at the beginning of the pose literal or before the point literal as shown below.

```
SELECT pose 'SRID=3812;Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(SRID=5676;Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

As for other spatial types, a pose can be either planar (geometric) or geodetic (geographic). A geodetic pose is denoted by the `GeodPose` keyword, in the same way that a geodetic `stbox` is denoted by `GEODSTBOX`.

```
SELECT pose 'GeodPose(Point(1 1), 0.5)';
SELECT pose 'SRID=4326;GeodPose(Point Z(1 1 1), 1, 0, 0, 0)';
```

Values of the pose type must satisfy several constraints so that they are well defined. Examples of incorrect pose type values are as follows.

```
-- Empty point
select pose 'Pose(Point empty, 0.5)';
-- Incorrect point value
SELECT pose 'Pose(Linestring(1 1,2 2), 1.0)';
-- Incorrect radius value
SELECT pose 'Pose(Point(1 1), -10.0)';
-- Incorrect 3D point
SELECT pose 'Pose(Point Z(1 1), 1.0)';
-- Incomplete 3D orientation
SELECT pose 'Pose(Point Z(1 1 1), 1.0)';
```

We give next the functions and operators for the pose type.

11.1.1 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

`asText({pose,pose[]}) → {text,text[]}`

`asEWKT({pose,pose[]}) → {text,text[]}`

```
SELECT asText(pose 'SRID=4326;Pose(Point(0 0),1)');
-- Pose(POINT(0 0),1)
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(POINT(0 0),1)","Pose(POINT(1 1),2)"}
SELECT asEWKT(pose 'SRID=4326;Pose(Point(0 0),1)');
-- SRID=4326;Pose(Point(0 0),1)
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)', 'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2)"}

```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

`asBinary(pose, endian text="") → bytea`

`asEWKB(pose, endian text="") → bytea`

`asHexEWKB(pose, endian text="") → text`

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(pose 'Pose(Point(1 2),1)');
-- \x0101000000000000f03f0000000000004000000000000f03f
SELECT asEWKB(pose 'SRID=7844;Pose(Point(1 2),1)');
-- \x0141a41e00000000000000f03f000000000000400000000000f03f
SELECT asHexEWKB(pose 'SRID=3812;Pose(Point(1 2),1)');
-- 0141E40E0000000000000000F03F000000000000400000000000F03F

```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

`poseFromText(text) → pose`

`poseFromEWKT(text) → pose`

```
SELECT asEWKT(poseFromText(text 'Pose(Point(1 2),1)'));
-- Pose(POINT(1 2),1)
SELECT asEWKT(poseFromEWKT(text 'SRID=3812;Pose(Point(1 2),1)'));
-- SRID=3812;Pose(Point(1 2),1)
```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

poseFromBinary(bytea) → pose

poseFromEWKB(bytea) → pose

poseFromHexEWKB(text) → pose

```
SELECT asEWKT(poseFromBinary(
  '\x0101000000000000f03f00000000000004000000000000f03f'));
-- Pose(POINT(1 2),1)
SELECT asEWKT(poseFromEWKB(
  '\x0141a41e00000000000000f03f00000000000004000000000000f03f'));
-- SRID=7844;Pose(Point(1 2),1)
SELECT asEWKT(poseFromHexEWKB(
  '0141E40E00000000000000f03F00000000000004000000000000F03F'));
-- SRID=3812;Pose(POINT(1 2),1)
```

11.1.2 Constructors

- Constructor for poses

pose(geompoint2D,float) → pose

pose(geompoint3D,float,float,float,float) → pose

```
SELECT asText(pose(ST_Point(1,1), radians(45)), 6);
-- Pose(POINT(1 1),0.785398)
SELECT asEWKT(pose(ST_Point(1,1,3812), radians(45)), 6);
-- SRID=3812;Pose(POINT(1 1),0.785398)
SELECT asText(pose(ST_PointZ(1,1,1), 1, 0, 0, 0));
-- Pose(POINT Z (1 1 1),1,0,0,0)
```

11.1.3 Conversions

Values of the pose type can be converted to the geometry point type using an explicit CAST or using the :: notation as shown below.

- Convert a pose and, optionally, a timestamp or a period, into a spatiotemporal box

pose::stbox

stbox(pose) → stbox

stbox(pose,{timestampz,tstzspan}) → stbox

```
SELECT stbox(pose 'SRID=5676;Pose(Point(1 1),0.3)');
-- SRID=5676;STBOX X((1,1),(1,1))
SELECT stbox(pose 'Pose(Point(1 1),0.3)', timestampz '2001-01-01');
-- STBOX XT(((1,1),(1.3,1.3)), [2001-01-01, 2001-01-01])
SELECT stbox(pose 'Pose(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((1,1),(1.3,1.3)), [2001-01-01, 2001-01-02])
```

- Convert a pose into geometry point

pose::geompoint

```
SELECT ST_AsText(pose(ST_Point(1, 1), 1)::geometry);
-- Point(1 1)
SELECT ST_AsEWKT(pose(ST_PointZ(1, 1, 1, 5676), 1, 0, 0, 0)::geometry);
-- SRID=5676;POINT(1 1 1)
```

11.1.4 Accessors

- Return the point

point(pose) → geompoint

```
SELECT ST_AsText(point(pose 'Pose(Point(1 1), 0.3)'));
-- Point(1 1)
```

- Return the rotation

rotation(pose2D) → float

```
SELECT rotation(pose 'Pose(Point(1 1), 0.3)');
-- 0.3
```

- Return the orientation

orientation(pose3D) → (X,W,Z,T)

```
SELECT orientation(pose 'Pose(Point Z(1 1 1), 0, 0, 0, 1)');
-- (0, 0, 0, 1)
```

11.1.5 Transformations

- Round the point and the orientation of the pose to the number of decimal places

round(pose, integer=0) → pose

```
SELECT asText(round(pose(ST_Point(1.123456789, 1.123456789), 0.123456789), 6));
-- Pose(POINT(1.123457 1.123457), 0.123457)
```

11.1.6 Spatial Reference System

- Return or set the spatial reference identifier

srid(pose) → integer

setSRID(pose) → pose

```
SELECT SRID(pose 'Pose(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(pose 'Pose(Point(0 0), 1)', 4326));
-- SRID=4326;Pose(POINT(0 0), 1)
```

- Transform to a spatial reference identifier

transform(pose, integer) → pose

transformPipeline(pose, pipeline text, to_srid integer, is_forward bool=true) → pose

The transform function specifies the transformation with a target SRID. An error is raised when the input pose has an unknown SRID (represented by 0).

The transformPipeline function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

```
urn:ogc:def:coordinateOperation:AUTHORITY::CODE
```

The SRID of the input pose is ignored, and the SRID of the output pose will be set to zero unless a value is provided via the optional `to_srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(pose 'SRID=4326;Pose(Point(4.35 50.85),1)', 3812), 6);
-- SRID=4326;Pose(POINT(648679.018035 671067.055638),1)
```

```
WITH test(pose, pipeline) AS (
  SELECT pose 'Pose(SRID=4326;Point(4.3525 50.846667),1)',
         text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(pose, pipeline, 4326), pipeline,
4326, false), 6)
FROM test;
-- SRID=4326;Pose(POINT(4.3525 50.846667),1)
```

11.1.7 Comparisons

The comparison operators (`=`, `<`, and so on) are available for poses. Excepted the equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on poses.

- Traditional comparisons

```
pose {=, <>, <, >, <=, >=} pose
```

```
SELECT pose 'Pose(Point(3 3), 0.5)' = pose 'Pose(Point(3 3), 0.5)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' <> pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' < pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.6)' > pose 'Pose(Point(2 2), 0.6)';
-- true
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)' <= pose 'Pose(Point Z(2 2 2), 0.5, 0.5, 0.5, 0.5)';
-- true
SELECT pose 'Pose(Point(1 1), 0.6)' >= pose 'Pose(Point(1 1), 0.5)';
-- true
```

- Are the poses approximately equal with respect to an epsilon value?

```
pose ~= pose → boolean
```

```
SELECT pose 'Pose(SRID=5676;Point(1 1), 0.3)' ~=
pose 'Pose(SRID=5676;Point(1 1.0000001), 0.30000001)';
-- true
```

11.1.8 OGC GeoPose v1.0 support

MobilityDB's pose type implements the data model behind the [OGC GeoPose v1.0 standard \(21-056r10\)](#): a position plus an orientation in a known reference frame, where the orientation is a unit quaternion in Hamilton convention or, equivalently, a yaw / pitch / roll triple under the ZYX intrinsic Tait-Bryan convention. This section groups the SQL surface that exposes that interoperability — JSON I/O for the standard's Basic conformance classes, an explicit renormalization helper for callers running long compositions, and the Euler-angle accessors that Basic-YPR consumers expect.

11.1.8.1 JSON input and output

- Convert to or from the OGC GeoPose v1.0 JSON encoding (Basic-Quaternion or Basic-YPR conformance class)

`asGeoPose(pose, conformance int4=0, maxdecimaldigits int4=-1) → text`

`poseFromGeoPose(text) → pose`

The `conformance` argument selects the output class: 0 = Basic-Quaternion (default, lossless), 1 = Basic-YPR (yaw, pitch, roll in degrees, ZYX intrinsic Tait-Bryan). The `maxdecimaldigits` argument is the number of significant digits to keep in the JSON numbers; -1 uses json-c's lossless default. The input function auto-detects the conformance class from the JSON keys (quaternion member → Basic-Quaternion, angles member → Basic-YPR). The Basic conformance classes mandate a geographic outer frame; the implementation accepts SRID 4326 (or 0, treated as geographic) and rejects projected SRIDs at the conversion boundary.

```
SELECT asGeoPose(pose 'Pose(Point(8 47 1500), 0.7071067811865476, 0, 0, ↵
0.7071067811865475)', 0, 6);
-- {"position":{"lat":47,"lon":8,"h":1500},"quaternion":{"x":0,"y":0,"z":0.707107,"w ↵
":0.707107}}
SELECT asGeoPose(pose 'Pose(Point(8 47 1500), 0.7071067811865476, 0, 0, ↵
0.7071067811865475)', 1, 6);
-- {"position":{"lat":47,"lon":8,"h":1500},"angles":{"yaw":90,"pitch":0,"roll":0}}
SELECT asEWKT(poseFromGeoPose(
'{"position":{"lat":47,"lon":8,"h":1500},"angles":{"yaw":90,"pitch":0,"roll":0}}'));
-- Pose(POINT Z (8 47 1500),0.7071067811865476,0,0,0.7071067811865475)
```

11.1.8.2 Quaternion renormalization

- Return a pose whose orientation quaternion has been brought back to unit norm. A 2D pose is returned unchanged since its orientation is a single angle that does not drift. A 3D pose has its quaternion divided by its Euclidean norm, so long compositions of SLERPs (or any other path that accumulates floating-point drift in $|q|$) can be brought back to the $|q| = 1$ invariant required by the SLERP and Euler-decomposition code.

`poseNormalize(pose) → pose`

```
SELECT asText(poseNormalize(pose 'Pose(Point(1 1 1), 0.5, 0.5, 0.5, 0.5)'));
-- Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)
```

11.1.8.3 Euler-angle accessors

- Return the yaw, pitch, or roll angle of a pose, in radians, under the ZYX intrinsic Tait-Bryan convention required by the OGC GeoPose Basic-YPR conformance class

`yaw(pose) → float`

`pitch(pose) → float`

`roll(pose) → float`

For a 2D pose `yaw` returns the stored rotation `theta` — by convention the yaw of the body frame — and `pitch` and `roll` return 0. For a 3D pose the three values come from the ZYX intrinsic Tait-Bryan decomposition of the orientation quaternion. The `asin` pitch term is clamped to $[-1, 1]$ to absorb the small numeric drift that long quaternion compositions can introduce.

```
SELECT yaw(pose 'Pose(Point(1 1), 0.5)');
-- 0.5
SELECT pitch(pose 'Pose(Point(1 1), 0.5)');
-- 0
SELECT roll(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 0
SELECT yaw(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 1.5707963267948966
```

- Lift the Euler-angle accessors through a temporal pose, returning a temporal float (radians) per instant under the ZYX intrinsic Tait-Bryan convention

`yaw(tpose) → tfloat`

`pitch(tpose) → tfloat`

`roll(tpose) → tfloat`

```
SELECT asText(yaw(tpose '[Pose(Point(0 0), 0.0)@2000-01-01, Pose(Point(1 1), 0.5)@2000-01-02]'));
-- [0@2000-01-01, 0.5@2000-01-02]
```

11.1.8.4 Frame metadata registry

The OGC GeoPose v1.0 standard distinguishes the *outer frame* (the global reference, e.g., WGS-84 geographic or ECEF) from the *inner frame* (the body frame whose orientation is the pose's quaternion). The Basic conformance classes mandate WGS-84 geographic as the outer frame and an implicit right-handed body-axes inner frame; the Advanced class supports stacks of named frames.

In MobilityDB v1 the `pose` type encodes the outer frame implicitly via its SRID and uses the conventional right-handed body-axes inner frame. The `geopose_frames` table documents this mapping and seeds a registry parallel to `pgPointCloud`'s `pointcloud_formats`, so that future Advanced-class support only needs to extend the catalog instead of redesigning it.

```
SELECT frame_id, authority, code, name, is_geographic
FROM geopose_frames ORDER BY frame_id;
-- 1 | EPSG | 4326 | WGS-84 geographic (lat/lon/h) | t
-- 2 | EPSG | 4978 | WGS-84 ECEF (Earth-Centred Earth-Fixed) | f
-- 3 | OGC | LTP | Local Tangent Plane (East-North-Up) | f
-- 4 | OGC | BODY | Right-handed body axes (default inner frame) | f
```

Three SQL helpers provide a stable lookup interface for code that doesn't want to hard-code the table layout:

- `geopose_frame_srid(int) → int` — the PostGIS SRID for a frame, or `NULL` if the frame is parametric (LTP, BODY).
- `geopose_frame_name(int) → text` — the human-readable name of the frame.
- `geopose_frame_is_geographic(int) → boolean` — `true` for lat/lon/h frames, `false` for Cartesian or projected.

Users can register custom frames by inserting into `geopose_frames`; the catalog is marked as a configuration table so `pg_dump` preserves user rows.

11.1.8.5 Body↔world rigid transform

The `applyPose` function applies the rigid-body transform encoded by a pose (the OGC GeoPose *body→world* mapping) to a body-frame geometry, producing the corresponding world-frame geometry. The transform is

$$\text{world} = R(q) \cdot \text{body} + p$$

where (p, q) are the pose's position and orientation. The static form takes a single pose and a static geometry; the temporal form lifts the per-instant rigid transform of a `tpose` across its instants, producing a `tgeompoint` world-frame trajectory of the body geometry. v1 supports point and multipoint body geometries; lines and polygons are deferred. Linear interpolation of the resulting trajectory is the chord on each segment, which approximates the true rigid-body trajectory (a circular arc under SLERP) — the same trade-off MobilityDB already takes for spatial trajectories.

`applyPose(geometry, pose) → geometry`

`applyPose(geometry, tpose) → tgeompoint`

```
-- A body sensor offset 1 unit along the body X axis, traced through a
-- tpose that ends 90 degrees yawed and translated to (10, 20).
SELECT asText(applyPose(ST_Point(1,0),
  tpose '[Pose(Point(0 0), 0)@2026-01-01,
        Pose(Point(10 20), 1.5707963267948966)@2026-01-02]'));
-- [POINT(1 0)@2026-01-01, POINT(10 21)@2026-01-02]
```

11.1.8.6 Cross-SRID frame transforms

The `transform(pose, srid)` function carries pose values across SRIDs. Workstream #6 of the temporal-GeoPose plan extends the pre-existing *position-only* behaviour with the orientation correction required when the source and target frames have different bases at the pose's point. v1 implements the canonical OGC GeoPose case — WGS-84 geographic (EPSG:4326) ↔ WGS-84 ECEF (EPSG:4978) — using the standard East-North-Up basis at the geographic point as the rotation pivot. For SRID pairs whose orientation correction is not yet defined, `transform` emits a NOTICE and passes the orientation through unchanged.

Round-tripping through ECEF and back lands exactly at the input pose:

```
SELECT asEWKT(round(
  transform(transform(pose 'SRID=4326;Pose(Point(8 47 0), 1, 0, 0, 0)', 4978),
    4326), 6));
-- SRID=4326;Pose(POINT Z (8 47 0),1,0,0,0)

-- At the equator-meridian (lat=lon=0), the body identity quaternion in
-- the ECEF basis becomes the canonical ENU->ECEF rotation:
SELECT asEWKT(round(
  transform(pose 'SRID=4326;Pose(Point(0 0 0), 1, 0, 0, 0)', 4978), 6));
-- SRID=4978;Pose(POINT Z (6378137 0 0),0.5,-0.5,-0.5,-0.5)
```

The implementation also fixes a pre-existing position-only bug where `transform` did not update the result's SRID field, so the returned pose now carries the target SRID as expected.

11.1.8.7 Temporal JSON I/O

- Convert a temporal pose to or from a TemporalGeoPose JSON envelope. Each per-instant payload is a strictly OGC-valid Basic-class GeoPose document plus a `validTime` member; the envelope records the conformance class, the interpolation, and (for sequences and sequence sets) the period bounds inclusion. A non-MEOS GeoPose consumer can iterate `instants[]` (or each `sequences[].instants[]`) and consume each element as a static GeoPose document.

`asGeoPose(tpose, conformance int4=0, maxdecimaldigits int4=-1) → text`

`tposeFromGeoPose(text) → tpose`

The envelope shape is

```
{
  "type":          "TemporalGeoPose",
  "version":       "1.0",
  "conformance":   "Basic-Quaternion" | "Basic-YPR",
  "interpolation": "None" | "Discrete" | "Step" | "Linear",
  "instants":      [...]              // for TInstant + TSequence
  "lower_inc":     true|false,         // for TSequence only
  "upper_inc":     true|false,         // for TSequence only
  "sequences":     [{...}, ...]       // for TSequenceSet only
}
```

```
SELECT asGeoPose(tpose '[Pose(Point(8 47), 0)@2026-01-01,
                        Pose(Point(9 48), 0.5)@2026-01-02]',
  1, 4);
```

```
-- {"type":"TemporalGeoPose","version":"1.0","conformance":"Basic-YPR",
--  "interpolation":"Linear","lower_inc":true,"upper_inc":true,
--  "instants":[
--    {"position":{"lat":47,"lon":8,"h":0},
--     "angles":{"yaw":0,"pitch":0,"roll":0},
--     "validTime":"2026-01-01 00:00:00+01"},
--    {"position":{"lat":48,"lon":9,"h":0},
--     "angles":{"yaw":28.65,"pitch":0,"roll":0},
--     "validTime":"2026-01-02 00:00:00+01"}]}
```

11.1.8.8 Speed and angular speed

- Return the speed of a temporal pose as a temporal float — the magnitude of the position-component velocity (distance travelled per unit time). The orientation is not part of this scalar; for angular velocity see `angularSpeed`.

`speed(tpose) → tfloat`

```
SELECT asText(speed(tpose '[Pose(Point(0 0), 0)@2000-01-01, Pose(Point(1 1), 0.5)@2000-01-02]'));
-- Interp=Step;[1.6368e-5@2000-01-01, 1.6368e-5@2000-01-02]
-- (sqrt(2) units / 86400 seconds -- units depend on the SRID's CRS)
```

- Return the angular speed of a temporal pose as a step-interpolated temporal float (radians per unit time). For 2D poses this is the per-segment shortest-arc theta delta divided by the segment duration; for 3D poses it is the SLERP arc angle $2 \cdot \arccos(|q_1 \cdot q_2|)$ divided by the segment duration. SLERP is by construction constant-angular-velocity along a segment, so the result is piecewise-constant.

`angularSpeed(tpose) → tfloat`

```
-- 90-degree yaw rotation over one day -> pi/2 rad / 86400 s
SELECT asText(angularSpeed(tpose
  '[Pose(Point(0 0 0), 1, 0, 0, 0)@2000-01-01,
    Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)@2000-01-02]'));
-- Interp=Step;[1.8181e-5@2000-01-01, 1.8181e-5@2000-01-02]
```

11.2 Temporal Poses

The temporal pose type `tpose` allows to represent the evolution in time of the position of objects and their orientation. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tpose` values in these subtypes are given next.

```
SELECT tpose 'Pose(Point(1 1), 0.5)@2001-01-01';
SELECT tpose '{Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03}';
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.4)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03]';
SELECT tpose '{[Pose(Point(1 1), 1)@2001-01-01, Pose(Point(2 2), 1)@2001-01-02],
  [Pose(Point(2 2), 2)@2001-01-04, Pose(Point(2 2), 3)@2001-01-05]}';
```

As for temporal point types, the temporal pose type accepts type modifiers (or `typmod` in PostgreSQL terminology) to specify the subtype, the dimensionality of the point, and/or the spatial reference identifier (SRID). The possible values for the subtype are `Instant`, `Sequence`, and `SequenceSet` and the possible values for the geometry type are either `Point` or `PointZ`. The three arguments are optional and if any of them is not specified for a column, values of any subtype, dimensionality, and/or SRID are allowed.

```
SELECT asEWKT(tpose(Sequence,5676) 'SRID=5676;[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-03]
```

```
SELECT tpose(Sequence) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
SELECT tpose(PointZ) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- Column has Z dimension but the tpose value does not
SELECT tpose(5676) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal pose SRID (0) does not match column SRID (5676)
```

Temporal pose values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into a normal form are as follows.

```
SELECT asText(tpose '[[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(2 2), 0.4)@2001-01-02, Pose(Point(3 3), 0.6)@2001-01-03)']');
-- [[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(3 3),0.6)@2001-01-03]
SELECT asText(tpose '[[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(2 2), 0.3)@2001-01-02, Pose(Point(2 2), 0.5)@2001-01-03),
  [Pose(Point(2 2), 0.5)@2001-01-03, Pose(Point(2 2), 0.7)@2001-01-04)]']');
/* {[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.3)@2001-01-02,
  Pose(Point(2 2),0.7)@2001-01-04)} */
```

11.3 Validity of Temporal Poses

Temporal pose values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tpose 'NULL@2001-01-01 08:05:00';
SELECT tpose 'Point(0 0)@NULL';
-- Base type is not a pose
SELECT tpose 'Point(0 0)@2001-01-01 08:05:00';
```

We give next the functions and operators for temporal poses. Most functions and operators for temporal types and spatiotemporal types described in the previous chapters can be applied for temporal pose types. Therefore, in the signatures of the functions, the type `pose` can be used whenever the types `base` and `spatial` are specified, and the type `tpose` can be used whenever the types `ttype` and `tpatial` are specified. To avoid redundancy, we only present some examples of these functions and operators for temporal poses and we refer to previous chapters for detailed explanations about them.

11.4 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

```
asText({tpose, tpose[]}) → {text, text[]}
```

```
asEWKT({tpose, tpose[]}) → {text, text[]}
```

```
SELECT asText(tpose 'SRID=4326;[Pose(Point(0 0),1)@2001-01-01,
  Pose(Point(1 1),2)@2001-01-02)']');
-- [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02]
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(POINT(0 0),1)","Pose(POINT(1 1),2)"}
SELECT asEWKT(tpose 'SRID=4326;[Pose(Point(0 0),1)@2001-01-01,
  Pose(Point(1 1),2)@2001-01-02)']');
-- SRID=4326;[Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02]
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)',
  'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2)"}

```

- Return the Moving Features JSON (MF-JSON) representation

asMFJSON(tpose) → text

```
SELECT asMFJSON(tpose 'Pose(Point(1 2),0.5)@2001-01-01', 3);
/* {"type":"MovingPose","bbox":[[1,2],[1,2]],"period":{"begin":"2001-01-01T00:00:00+01",
  "end":"2001-01-01T00:00:00+01","lower_inc":true,"upper_inc":true},
  "values":[{"position":{"lat":2,"lon":1},"rotation":0.5}],
  "datetimes":["2001-01-01T00:00:00+01"],"interpolation":"None"} */
SELECT asMFJSON(tpose 'Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3);
/* {"type":"MovingPose","bbox":[[1,2,3],[1,2,3]],
  "period":{"begin":"2001-01-01T00:00:00+01","end":"2001-01-01T00:00:00+01",
  "lower_inc":true,"upper_inc":true},
  "values":[{"position":{"lat":2,"lon":1,"h":3},"quaternion":{"x":0,"y":0,"z":0,"w":1}}],
  "datetimes":["2001-01-01T00:00:00+01"],"interpolation":"None"} */
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB) representation, or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

asBinary(tpose, endian text='') → bytea

asEWKB(tpose, endian text='') → bytea

asHexEWKB(tpose, endian text='') → text

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(tpose 'Pose(Point(1 2),1)@2001-01-01');
-- \x013a0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asEWKB(tpose 'SRID=7844;Pose(Point(1 2),1)@2001-01-01');
-- \x013a0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asHexEWKB(tpose 'SRID=3812;Pose(Point(1 2),1)@2001-01-01');
-- 013A0001000000000000F03F000000000000400000000000F03F009C57D3C11C0000
```

- Input from the Well-Known Text (WKT) representation or from the Extended Well-Known Text (EWKT) representation

tposeFromText(text) → tpose

tposeFromEWKT(text) → tpose

```
SELECT asEWKT(tposeFromText(text '[Pose(Point(1 2),1)@2001-01-01,
  Pose(Point(3 4),2)@2001-01-02]'));
-- [Pose(POINT(1 2),1)@2001-01-01, Pose(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tposeFromEWKT(text 'SRID=3812;[Pose(Point(1 2),1)@2001-01-01,
  Pose(Point(3 4),2)@2001-01-02]'));
-- SRID=3812;[Pose(Point(1 2),1)@2001-01-01, Pose(Point(3 4),2)@2001-01-02]
```

- Input from the Moving Features JSON (MF-JSON) representation

tposeFromMFJSON(bytea) → tpose

```
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose
  'SRID=3812;Pose(Point(1 2),0.5)@2001-01-01', 3)));
-- SRID=3812;Pose(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose
  'SRID=5676;Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3)));
-- SRID=5676;Pose(POINT Z(1 2 3),1,0,0,0)@2001-01-01
```

- Input from the Well-Known Binary (WKB) representation, from the Extended Well-Known Binary (EWKB) representation, or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

tposeFromBinary(bytea) → tpose

tposeFromEWKB(bytea) → tpose

tposeFromHexEWKB(text) → tpose

```

SELECT asEWKT(tposeFromBinary(
  '\x013a0001000000000000f03f00000000000004000000000000f03f009c57d3c11c0000'));
-- Pose(Point(1 2),1)@2001-01-01
SELECT asEWKT(tposeFromEWKB(
  '\x013a0001000000000000f03f00000000000004000000000000f03f009c57d3c11c0000'));
-- SRID=7844;Pose(Point(1 1),2)@2001-01-01
SELECT asEWKT(tposeFromHexEWKB(
  '013A0041E40E0000E40E00000000000000F03F...40000000000000F03F009C57D3C11C0000'));
-- SRID=3812;Pose(Point(1 2),1)@2001-01-01

```

11.5 Constructors

- Constructor for temporal poses having a constant value

`tpose(pose,timestamp_tz) → tposeInst`

`tpose(pose,tstzset) → tposeDiscSeq`

`tpose(pose,tstzspan,interp='linear') → tposeContSeq`

`tpose(pose,tstzspanset,interp='linear') → tposeSeqSet`

```

SELECT asText(tpose('Pose(Point(1 1), 0.5)', timestamp_tz '2001-01-01'));
-- Pose(Point(1 1),0.5)@2001-01-01
SELECT asText(tpose('Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)',
  tstzset '{2001-01-01, 2001-01-05}'));
/* {Pose(Point Z(1 1 1),0.5,0.5,0.5,0.5)@2001-01-01,
  Pose(Point Z(1 1 1),0.5,0.5,0.5,0.5)@2001-01-05} */
SELECT asText(tpose('Pose(Point(1 1), 0.5)',
  tstzspan '[2001-01-01, 2001-01-02]'));
-- [Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(1 1),0.5)@2001-01-02]
SELECT asText(tpose('Pose(Point(1 1), 0.2)',
  tstzspanset '{[2001-01-01, 2001-01-03]}', 'step'));
-- Interp=Step;{[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(1 1),0.2)@2001-01-03]}

```

- Constructor for temporal poses of sequence subtype

`tposeSeq(tposeInst[],interp={'step','linear'},leftInc bool=true,
rightInc bool=true) → tposeSeq`

```

SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.3)@2001-01-01',
  'Pose(Point(2 2), 0.5)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* {Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02,
  Pose(Point(1 1),0.5)@2001-01-03} */
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.2)@2001-01-01',
  'Pose(Point(1 1), 0.4)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* {Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(1 1),0.4)@2001-01-02,
  Pose(Point(1 1),0.5)@2001-01-03} */

```

- Constructor for temporal poses of sequence set subtype

`tposeSeqSet(tpose[]) → tposeSeqSet`

`tposeSeqSetGaps(tposeInst[],maxt=NULL,maxdist=NULL,interp='linear') → tposeSeqSet`

```

SELECT asText(tposeSeqSet(ARRAY[tpose
  '[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.4)@2001-01-02]',
  '[Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.8)@2001-01-04]']));
/* {[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.4)@2001-01-02],
  [Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.6)@2001-01-04]} */
SELECT asText(tposeSeqSetGaps(ARRAY[tpose 'Pose(Point(1 1),0.1)@2001-01-01',
  'Pose(Point(1 1),0.3)@2001-01-03', 'Pose(Point(1 1),0.5)@2001-01-05', '1 day']));

```

```
/* {[Pose(Point(1 1),0.1)@2001-01-01], [Pose(Point(1 1),0.3)@2001-01-03],
   [Pose(Point(1 1),0.5)@2001-01-05]} */
```

- Construct a 2D temporal pose from a temporal geometry point and a temporal float

`tpose(tgeompoint, tfloat) → tpose`

The time frame of the result is the intersection of the time frames of the temporal point and the temporal float. If they do not intersect, a null value is returned

```
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
  tfloat '[1@2001-01-01, 2@2001-01-02]'));
-- [Pose(Point(1 1),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
  tfloat '[2@2001-01-03, 3@2001-01-04]'));
-- NULL
```

11.6 Conversions

A temporal pose value can be converted to and from a temporal point, which is a temporal geometry point for a planar pose and a temporal geography point for a geodetic pose. This can be done using an explicit CAST or using the `::` notation. A null value is returned if any of the composing geometry point values cannot be converted into a pose value.

- Convert a temporal pose into a temporal geometry point

`tpose::tgeompoint`

```
SELECT asText((tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02]')::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
SELECT asEWKT((tpose '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
  Pose(Point Z(2 2 2), 1, 0, 0, 0)@2001-01-02]')::tgeompoint);
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02)
```

- Convert a geodetic temporal pose into a temporal geography point

`tpose::tgeogpoint`

```
SELECT asText((tpose '[GeodPose(Point(1 1), 0.2)@2001-01-01,
  GeodPose(Point(1 1), 0.3)@2001-01-02]')::tgeogpoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
```

11.7 Accessors

- Return the values

`getValues(tpose) → poseset`

```
SELECT asText(getValues(tpose '{[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]}'));
-- {"Pose(Point(1 1),0.3)","Pose(Point(1 1),0.5)"}
SELECT asEWKT(getValues(tpose 'SRID=5676;{[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02]}'));
-- SRID=5676;{"Pose(Point(1 1),0.3)"}
```

- Return the points

`points(tpose) → geomset`


```
SELECT asEWKT(points(tpose 'SRID=5676;{Pose(Point(3 3), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02}'));
-- SRID=5676;{Point(1 1), Point(3 3)}
```

- Return the rotation

rotation(tpose2d) → tfloat

```
SELECT rotation(tpose 'SRID=5676;{[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.3)@2001-01-02]}');
-- {[0.3@2001-01-01, 0.3@2001-01-02]}
```

- Return the value at a timestamp

valueAtTimestamp(tpose, timestamptz) → pose

```
SELECT asText(valueAtTimestamp(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(3 3), 0.5)@2001-01-03]', '2001-01-02'));
-- Pose(Point(2 2),0.4)
SELECT asText(valueAtTimestamp(tpose
'[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
Pose(Point Z(3 3 3), 1, 0, 0, 0)@2001-01-03]', timestamptz '2001-01-02'), 6);
-- Pose(POINT Z (2 2 2),0.866025,0.288675,0.288675,0.288675)
```

11.8 Transformations

- Transform to another subtype

tposeInst(tpose) → tposeInst

tposeSeq(tpose) → tposeSeq

tposeSeqSet(tpose) → tposeSeqSet

```
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Pose(Point(1 1),0.5)@2001-01-01}
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01'));
-- [Pose(Point(1 1),0.5)@2001-01-01]
SELECT asText(tposeSeqSet(tpose 'Pose(PointZ(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01'));
-- {[Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-01]}
```

- Transform to another interpolation

setInterp(tpose, interp) → tpose

```
SELECT asText(setInterp(tpose 'Pose(Point(1 1),0.2)@2001-01-01','linear'));
-- [Pose(Point(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tpose '{[Pose(Point Z(1 1 1),1,0,0,0)@2001-01-01],
[Pose(Point Z(2 2 2),0,0,0,1)@2001-01-02]}', 'discrete'));
-- {Pose(POINT Z (1 1 1),1,0,0,0)@2001-01-01, Pose(POINT Z (2 2 2),0,0,0,1)@2001-01-02}
```

- Round the points and the orientations of the temporal pose to the number of decimal places

round(tpose, integer=0) → tpose

```
SELECT asText(round(tpose '{[Pose(Point(1 1.123456789),0.123456789)@2001-01-01,
Pose(Point(1 1),0.5)@2001-01-02]}', 3));
-- {[Pose(POINT(1 1.123),0.123)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-02]}
```

11.9 Modifications

- Append a temporal instant or a temporal sequence

`appendInstant(tpose, tposeInst) → tpose`

`appendSequence(tpose, tposeSeq) → tpose`

```
SELECT asText(appendInstant(tpose 'Pose(Point(1 1), 0.5)@2001-01-01', 'Pose(Point(2 2), 1) ←
    @2001-01-02'));
-- [Pose(POINT(1 1),0.5)@2001-01-01, Pose(POINT(2 2),1)@2001-01-02]
SELECT asText(appendSequence(tpose '[Pose(Point(1 1), 0.5)@2001-01-01]', '[Pose(Point(2 2) ←
    , 1)@2001-01-02]'));
-- {[Pose(POINT(1 1),0.5)@2001-01-01], [Pose(POINT(2 2),1)@2001-01-02]}
```

- Merge the temporal poses

`merge(tpose, tpose) → tpose`

`merge(tpose[]) → tpose`

`merge(tpose) → tpose`

`mergeAgg(tpose) → tpose`

```
SELECT asText(merge(tpose
    '[Pose(Point(1 1 1),1,0,0,0)@2001-01-01, Pose(Point(2 2 2),0,1,0,0)@2001-01-02]',
    '[Pose(Point(2 2 2),0,1,0,0)@2001-01-02, Pose(Point(3 3 3),0,0,0,1)@2001-01-03]'));
/* [Pose(POINT Z (1 1 1),1,0,0,0)@2001-01-01, Pose(POINT Z (2 2 2),0,1,0,0)@2001-01-02,
    Pose(POINT Z (3 3 3),0,0,0,1)@2001-01-03] */
SELECT asText(merge(ARRAY[tpose
    '{[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(2 2),0.2)@2001-01-02],
    [Pose(Point(3 3),0.3)@2001-01-03, Pose(Point(4 4),0.4)@2001-01-04]}',
    '[Pose(Point(4 4),0.4)@2001-01-04, Pose(Point(5 5),0.5)@2001-01-05]']));
/* {[Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.2)@2001-01-02],
    [Pose(POINT(3 3),0.3)@2001-01-03, Pose(POINT(5 5),0.5)@2001-01-05]} */
WITH temp(inst) AS (
    SELECT tpose 'Pose(Point(1 1),0.1)@2001-01-01' UNION
    SELECT tpose 'Pose(Point(2 2),0.2)@2001-01-02' UNION
    SELECT tpose 'Pose(Point(3 3),0.3)@2001-01-03' )
SELECT asText(merge(inst)) FROM temp;
/* {Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.2)@2001-01-02,
    Pose(POINT(3 3),0.3)@2001-01-03} */
```

11.10 Restrictions

- Restrict to (the complement of) a value or a set of values

`atValue(tpose, value) → tpose`

`minusValue(tpose, value) → tpose`

`atValues(tpose, valueset) → tpose`

`minusValues(tpose, valueset) → tpose`

```
SELECT atValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
    Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
-- {[Pose(Point(2 2),0.5)@2001-01-02]}
SELECT minusValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
    Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
/* {[Pose(Point(2 2),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02),
    (Pose(Point(2 2),0.5)@2001-01-02, Pose(Point(2 2),0.7)@2001-01-03)} */
```

- TODO Restrict to (the complement of) a geometry

`atGeometry(tpose, geometry) → tpose`

`minusGeometry(tpose, geometry) → tpose`

```
SELECT atGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-03]',
'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-03]',
'Polygon((40 40,40 50,50 50,50 40,40 40))');
/* {(Pose(Point(2 2),0.342593)@2001-01-01 05:06:40.364673+01,
Pose(Point(2 2),0.7)@2001-01-03)} */
```

11.11 Spatial Reference System

- Return or set the spatial reference identifier

`SRID(tpose) → integer`

`setSRID(tpose) → tpose`

```
SELECT SRID(tpose 'SRID=5676;[Pose(Point(0 0),1)@2001-01-01,
Pose(Point(1 1),2)@2001-01-02)');
-- 5676
SELECT asEWKT(setSRID(tpose '[Pose(Point(0 0),1)@2001-01-01,
Pose(Point(1 1),2)@2001-01-02]', 5676));
-- SRID=5676;Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
```

- Transform to a spatial reference identifier

`transform(tpose, integer) → tpose`

`transformPipeline(tpose, pipeline text, to_srid integer, is_forward bool=true) → tpose`

```
SELECT asEWKT(transform(tpose 'SRID=4326;Pose(Point(4.35 50.85),1)@2001-01-01',
3812));
-- SRID=3812;Pose(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tpose, pipeline) AS (
SELECT tpose 'SRID=4326;{Pose(Point(4.3525 50.846667),1)@2001-01-01,
Pose(Point(-0.1275 51.507222),2)@2001-01-02}',
text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tpose, pipeline, 4326), pipeline,
4326, false), 6)
FROM test;
/* SRID=4326;{Pose(POINT(4.3525 50.846667),1)@2001-01-01,
Pose(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

11.12 Bounding Box Operations

- Topological operators

`{tstzspan, stbox, tpose} {&&, <@, @>, ~=, -|-} {tstzspan, stbox, tpose} → boolean`

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' && tstzspan '[2001-01-02,2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
```

```

Pose(Point(1 1), 0.5)@2001-01-02]' @> stbox(pose 'Pose(Point(1 1), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' ~= tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.35)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true

```

- Position operators

```

{stbox,tpose} {<<, &<, >>, &>} {stbox,tpose} → boolean
{stbox,tpose} {<<|, &<|, |>>, |&>} {stbox,tpose} → boolean
{stbox,tpose} {<</, &</, />>, /&>} {stbox,tpose} → boolean
{tstzspan,stbox,tpose} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tpose} → boolean

```

```

SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' << stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' <<| stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1 1), 1,0,0,0)@2001-01-01,
Pose(Point(1 1 1), 1,0,0,0)@2001-01-02]' <</ stbox(pose 'Pose(Point(2 2 2), 1,0,0,0)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03,
Pose(Point(1 1), 0.5)@2001-01-05]' #>>
tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02]';
-- true

```

11.13 Distance Operations

We present next the functions that compute distance operations for temporal poses. Notice that for these operations only the point component is considered, not the orientation.

- Return the smallest distance ever

```

{geo,pose,tpose} |=| {geo,pose,tpose} → float

```

```

SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01, Pose(Point(2 2), 0.7)@2001-01-02]' |=|
geometry 'Linestring(50 50,55 55)';
-- 67.88225099390856
SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01, Pose(Point(2 2), 0.7)@2001-01-02]' |=|
pose 'Pose(Point(1 1), 0.5)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' |=| pose 'Pose(Point(2 2), 0.2)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1

```

The operator `|=|` can be used for doing nearest neighbor searches using GiST or SP-GiST indexes (see Section 10.2).

- Return the instant of the first temporal pose at which the two arguments are at the nearest distance

```

nearestApproachInstant({geo,pose,tpose},{geo,pose,tpose}) → tpose

```

```
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- Pose(Point(2 2),0.3)@2001-01-01
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- Pose(Point(2 2),0.3)@2001-01-01
```

- Return the line connecting the nearest approach point

```
shortestLine({geo,pose,tpose},{geo,pose,tpose}) → geometry
```

The function will only return the first line that it finds if there are more than one.

```
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(2 2,50 50)
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- LINESTRING(2 2,1 1)
```

- Return the temporal distance

```
tpose <-> tpose → tfloat
```

```
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' <-> pose 'Pose(Point(2 2), 0.2)', 6);
-- [1.414214@2001-01-01, 1.414214@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)', 6);
-- [69.296465@2001-01-01, 69.296465@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(2 2), 0.5)@2001-01-03]' <->
tpose '[Pose(Point(1 2), 0.3)@2001-01-02, Pose(Point(2 1), 0.5)@2001-01-04]', 6);
-- [0.707107@2001-01-02, 0.5@2001-01-02 12:00:00+01, 0.707107@2001-01-03]
```

11.14 Spatial Relationships

The topological and distance relationships described in Section 8.5 such as `eIntersects`, `aDwithin`, or `tContains` only consider the location of the geometries, not their orientation. To be able to apply these spatial relationships to the temporal poses, they must be transformed to temporal geometry points. This can be easily performed using the functions `geometry` and `tgeompoint` or using an explicit casting `::` as shown in the examples below.

- Ever and always relationships

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03]'));
-- true
SELECT eDisjoint(geometry(pose 'Pose(Point(2 2), 0.0)'),
tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03]'));
-- true
SELECT eIntersects(
tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03]':tgeompoint,
tpose '[Pose(Point(2 2),0.0)@2001-01-01, Pose(Point(2 2),1)@2001-01-03]':tgeompoint);
-- false
```

- Spatiotemporal relationships

```

SELECT tContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03]'));
-- {[t@2001-01-01 00:00:00+01, t@2001-01-03 00:00:00+01]}
SELECT tDisjoint(geometry(pose 'Pose(Point(2 2), 0.0)'),
  tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03]'));
-- [t@2001-01-01, t@2001-01-03]
SELECT tDwithin(
  tpose '[Pose(Point(1 1),0.3)@2001-01-01,Pose(Point(1 1),0.5)@2001-01-03]::tgeompoint,
  tpose '[Pose(Point(1 1),0.5)@2001-01-01,Pose(Point(3 3),0.3)@2001-01-03]::tgeompoint,1);
/* {[t@2001-01-01 00:00:00+01, t@2001-01-01 16:58:14.025894+01],
  (f@2001-01-01 16:58:14.025894+01, f@2001-01-03 00:00:00+01)} */

```

11.15 Comparisons

- Traditional comparisons

`tpose {=, <>, <, >, <=, >=}` `tpose` → boolean

```

SELECT tpose ' {[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02),
  [Pose(Point(1 1), 0.3)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]} ' =
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tpose ' {[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]} ' <>
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.6)@2001-01-03]';
-- true

```

- Ever and always comparisons

`tpose {?=?, %=}` `tpose` → boolean

```

SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-04]' ?= Pose(Point(1 1), 0.3);
-- true
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.2)@2001-01-04]' &= Pose(Point(1 1), 0.2);
-- true

```

- Temporal comparisons

`tpose {#=, #<>}` `tpose` → tbool

```

SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-03]' #= pose 'Pose(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.8)@2001-01-03]' #<>
  tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.7)@2001-01-03]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}

```

11.16 Aggregations

The three aggregate functions for temporal poses are illustrated next.

- Temporal count

`tCount(tpose) → {tintSeq,tintSeqSet}`

```
WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
    Pose(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
    Pose(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03,
    Pose(Point(1 1), 0.5)@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}
```

- Window count

`wCount(tpose) → {tintSeq,tintSeqSet}`

```
WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
    Pose(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
    Pose(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03,
    Pose(Point(1 1), 0.5)@2001-01-05]' )
SELECT wCount(Temp, '1 day')
FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
  1@2001-01-06]} */
```

11.17 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal poses. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal poses, which is an `stbox`, as all spatiotemporal types.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal poses,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal poses,
- `&&`, `@>`, `<@`, `~=-`, `-|-`, and `|=|`, which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

Chapter 12

Temporal Network Points

The temporal points that we have considered so far represent the movement of objects that can move freely on space since it is assumed that they can change their position from one location to the next one without any motion restriction. This is the case for animals and for flying objects such as planes or drones. However, in many cases, objects do not move freely in space but rather within spatially embedded networks such as routes or railways. In this case, it is necessary to take the embedded networks into account while describing the movements of these moving objects. Temporal network points account for these requirements.

Compared with the free-space temporal points, network-based points have the following advantages:

- Network points provide road constraints that reflect the real movements of moving objects.
- The geometric information is not stored with the moving point, but once and for all in the fixed networks. In this way, the location representations and interpolations are more precise.
- Network points are more efficient in terms of data storage, location update, formulation of query, as well as indexing. These are discussed later in this document.

Temporal network points are based on [pgRouting](#), a PostgreSQL extension for developing network routing applications and doing graph analysis. Therefore, temporal network points assume that the underlying network is defined in a table named `ways`, which has at least three columns: `gid` containing the unique route identifier, `length` containing the route length, and `the_geom` containing the route geometry.

There are two static network types, `npoint` (short for network point) and `nsegment` (short for network segment), which represent, respectively, a point and a segment of a route. An `npoint` value is composed of a route identifier and a float number in the range `[0,1]` determining a relative position of the route, where 0 corresponds to the beginning of the route and 1 to the end of the route. An `nsegment` value is composed of a route identifier and two float numbers in the range `[0,1]` determining the start and end relative positions. A `nsegment` value whose start and end positions are equal corresponds to an `npoint` value.

The `npoint` type serves as base type for defining the temporal network point type `tnpoint`. The `tnpoint` type has similar functionality as the temporal point type `tgeompoint` with the exception that it only considers two dimensions. Thus, all functions and operators described before for the `tgeompoint` type are also applicable for the `tnpoint` type. In addition, there are specific functions defined for the `tnpoint` type.

Operational guidance on choosing between `tnpoint` and `tgeompoint`; the SRID-inheritance contract (the SRID is read from the route registered in `ways`, not carried explicitly on the value); the numerical hazards production workloads should anticipate (position-parameter clamping at `[0, 1]`, mixed-SRID handling, route-not-registered errors, cross-network operations); and the durability and storage conventions are collected in [Section 12.19](#).

12.1 Static Network Types

An `npoint` value is a couple of the form `(rid, position)` where `rid` is a `bigint` value representing a route identifier and `position` is a `float` value in the range `[0,1]` indicating its relative position. The values 0 and 1 of `position` denote,

respectively, the starting and the ending position of the route. The road distance between an `npoint` value and the starting position of route with identifier `rid` is computed by multiplying position by length, where length is the route length. Examples of input of network point values are as follows:

```
SELECT npoint 'Npoint(76, 0.3)';
SELECT npoint 'Npoint(64, 1.0)';
```

The constructor function for network points has one argument for the route identifier and one argument for the relative position. An example of a network point value defined with the constructor function is as follows:

```
SELECT npoint(76, 0.3);
```

An `nsegment` value is a triple of the form `(rid, startPosition, endPosition)` where `rid` is a bigint value representing a route identifier and `startPosition` and `endPosition` are float values in the range `[0,1]` such that `startPosition ≤ endPosition`. Semantically, a network segment represents a set of network points `(rid, position)` with `startPosition ≤ position ≤ endPosition`. If `startPosition=0` and `endPosition=1`, the network segment is equivalent to the entire route. If `startPosition=endPosition`, the network segment represents into a single network point. Examples of input of network point values are as follows:

```
SELECT nsegment 'Nsegment(76, 0.3, 0.5)';
SELECT nsegment 'Nsegment(64, 0.5, 0.5)';
SELECT nsegment 'Nsegment(64, 0.0, 1.0)';
SELECT nsegment 'Nsegment(64, 1.0, 0.0)';
-- converted to nsegment 'Nsegment(64, 0.0, 1.0)';
```

As can be seen in the last example, the `startPosition` and `endPosition` values will be inverted to ensure that the condition `startPosition ≤ endPosition` is always satisfied. The constructor function for network segments has one argument for the route identifier and two optional arguments for the start and end positions. Examples of network segment values defined with the constructor function are as follows:

```
SELECT nsegment(76, 0.3, 0.3);
SELECT nsegment(76); -- start and end position assumed to be 0 and 1 respectively
SELECT nsegment(76, 0.5); -- end position assumed to be 1
```

Values of the `npoint` type can be converted to the `nsegment` type using an explicit `CAST` or using the `::` notation as shown next.

```
SELECT npoint(76, 0.33)::nsegment;
```

Values of static network types must satisfy several constraints so that they are well defined. These constraints are given next.

- The route identifier `rid` must be found in column `gid` of table `ways`.
- The position, `startPosition`, and `endPosition` values must be in the range `[0,1]`. An error is raised whenever one of these constraints are not satisfied.

Examples of incorrect static network type values are as follows.

```
-- incorrect rid value
SELECT npoint 'Npoint(87.5, 1.0)';
-- incorrect position value
SELECT npoint 'Npoint(87, 2.0)';
-- rid value not found in the ways table
SELECT npoint 'Npoint(99999999, 1.0)';
```

We give next the functions and operators for the static network types.

12.1.1 Constructors

- Constructors for network points and network segments

`npoint(bigint, float) → npoint`

`nsegment(bigint, float, float) → nsegment`

```
SELECT npoint(76, 0.3);
SELECT nsegment(76, 0.3, 0.5);
```

12.1.2 Conversions

Values of the `npoint` and `nsegment` types can be converted to the `geometry` type using an explicit `CAST` or using the `::` notation as shown below. Similarly, `geometry` values of subtype `point` or `linestring` (restricted to two points) can be converted, respectively, to `npoint` and `nsegment` values. For this, the route that intersects the given points must be found, where a tolerance of 0.00001 units (depending on the coordinate system) is assumed so a point and a route that are close are considered to intersect. If no such route is found, a null value is returned.

- Convert between a network point or segment and a geometry

`{npoint, nsegment}::geometry`

`geometry::{npoint, nsegment}`

```
SELECT ST_AsText(npoint(76, 0.33)::geometry);
-- POINT(21.6338731332283 50.0545869554067)
SELECT ST_AsText(nsegment(76, 0.33, 0.66)::geometry);
-- LINESTRING(21.6338731332283 50.0545869554067,30.7475989651999 53.9185062927473)
SELECT ST_AsText(nsegment(76, 0.33, 0.33)::geometry);
-- POINT(21.6338731332283 50.0545869554067)
```

```
SELECT geometry 'SRID=5676;Point(279.269156511873 811.497076880187)::npoint;
-- NPoint(3,0.781413)
SELECT geometry 'SRID=5676;LINESTRING(406.729536784738 702.58583437902,
383.570801314823 845.137059419277)::nsegment;
-- NSegment(3,0.6,0.9)
SELECT geometry 'SRID=5676;Point(279.3 811.5)::npoint;
-- NULL
SELECT geometry 'SRID=5676;LINESTRING(406.7 702.6,383.6 845.1)::nsegment;
-- NULL
```

- Construct a spatiotemporal box from a network point and, optionally, a timestamp or a period

`stbox(npoint) → stbox`

`stbox(npoint, {timestampz, tstzspan}) → stbox`

```
SELECT stbox(npoint 'NPoint(1,0.3)');
-- STBOX X((48.711754,20.92568),(48.711758,20.925682))
SELECT stbox(npoint 'NPoint(1,0.3)', timestampz '2001-01-01');
-- STBOX XT(((62.786633,80.143555),(62.786636,80.143562)), [2001-01-01,2001-01-01])
SELECT stbox(npoint 'NPoint(1,0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((62.786633,80.143555),(62.786636,80.143562)), [2001-01-01,2001-01-02])
```

12.1.3 Accessors

- Return the route identifier

`route({npoint, nsegment}) → bigint`

```
SELECT route(npoint 'Npoint(63, 0.3)');
-- 63
SELECT route(nsegment 'Nsegment(76, 0.3, 0.3)');
-- 76
```

- Return the position

getPosition(npoint) → float

```
SELECT getPosition(npoint 'Npoint(63, 0.3)');
-- 0.3
```

- Return the start/end position

startPosition(nsegment) → float

endPosition(nsegment) → float

```
SELECT startPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.3
SELECT endPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.5
```

12.1.4 Transformations

- Round the position(s) of the network point or the network segment to the number of decimal places

round({npoint, nsegment}, integer=0) → {npoint, nsegment}

```
SELECT round(npoint(76, 0.123456789), 6);
-- NPoint(76,0.123457)
SELECT round(nsegment(76, 0.123456789, 0.223456789), 6);
-- NSegment(76,0.123457,0.223457)
```

12.1.5 Spatial Operations

- Return the spatial reference identifier

SRID({npoint, nsegment}) → integer

```
SELECT SRID(npoint 'Npoint(76, 0.3)');
-- 5676
SELECT SRID(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 5676
```

Two npoint values may have different route identifiers but may represent the same spatial point at the intersection of the two routes. Function same is used for testing spatial similarity of network points.

- Spatial similarity

equals(npoint, npoint)::Boolean

```
WITH inter(geom) AS (
  SELECT st_intersection(t1.the_geom, t2.the_geom)
  FROM ways t1, ways t2 WHERE t1.gid = 1 AND t2.gid = 2),
fractions(f1, f2) AS (
  SELECT ST_LineLocatePoint(t1.the_geom, i.geom), ST_LineLocatePoint(t2.the_geom, i.geom)
  FROM ways t1, ways t2, inter i WHERE t1.gid = 1 AND t2.gid = 2)
SELECT equals(npoint(1, f1), npoint(2, f2)) FROM fractions;
-- true
```

12.1.6 Comparisons

The comparison operators (=, <, and so on) for static network types require that the left and right arguments be of the same type. Excepted the equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on static network types.

- Traditional comparisons

```
npoint {=, <>, <, >, <=, >=} npoint
nsegment {=, <>, <, >, <=, >=} nsegment
```

```
SELECT npoint 'Npoint(3, 0.5)' = npoint 'Npoint(3, 0.5)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' <> nsegment 'Nsegment(3, 0.5, 0.5)';
-- false
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' < nsegment 'Nsegment(3, 0.5, 0.6)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' > nsegment 'Nsegment(2, 0.5, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.5)' <= npoint 'Npoint(2, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.6)' >= npoint 'Npoint(1, 0.5)';
-- true
```

12.2 Temporal Network Points

The temporal network point type `tnpoint` allows to represent the movement of objects over a network. It corresponds to the temporal point type `tgeompoint` restricted to two-dimensional coordinates. As all the other temporal types it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tnpoint` values in these subtypes are given next.

```
SELECT tnpoint 'Npoint(1, 0.5)@2001-01-01';
SELECT tnpoint '{Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03}';
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint '[{Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03}, [Npoint(2, 0.6)@2001-01-04, Npoint(2, 0.6)@2001-01-05}]';
```

The temporal network point type accepts type modifiers (or `typmod` in PostgreSQL terminology). The possible values for the type modifier are `Instant`, `Sequence`, and `SequenceSet`. If no type modifier is specified for a column, values of any subtype are allowed.

```
SELECT tnpoint(Sequence) '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint(Sequence) 'Npoint(1, 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Temporal network point values of sequence subtype and linear or step interpolation must be defined on a single route. Therefore, a value of sequence set subtype is needed for representing the movement of an object that traverses several routes, even if there is no temporal gap. For example, in the following value

```
SELECT tnpoint '[{NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.5)@2001-01-03},
  [NPoint(2, 0.4)@2001-01-03, NPoint(2, 0.6)@2001-01-04}]';
```

the network point changes its route at 2001-01-03.

Temporal network point values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into a normal form are as follows.

```
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
  NPoint(1, 0.6)@2001-01-03]';
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.6)@2001-01-03]
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.3)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03), [NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04)]}';
-- {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02, NPoint(1,0.7)@2001-01-04]}
```

12.3 Validity of Temporal Network Points

Temporal network point values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tnpoint 'NULL@2001-01-01 08:05:00';
SELECT tnpoint 'Point(0 0)@NULL';
-- Base type is not a network point
SELECT tnpoint 'Point(0 0)@2001-01-01 08:05:00';
-- Multiple routes in a continuous sequence
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01 09:00:00, Npoint(2, 0.2)@2001-01-01 09:05:00]';
```

We present next the operation for temporal network point types. Most functions for temporal types described in the previous chapters can be applied for temporal network point types. Therefore, in the signatures of the functions, the notation base also represents an npoint and the notations ttype, tpoint, and tgeompoint also represent a tnpoint. Furthermore, the functions that have an argument of type geometry accept in addition an argument of type npoint. To avoid redundancy, we only present next some examples of these functions and operators for temporal network points.

12.4 Constructors

- Constructor for temporal network points having a constant value

```
tnpoint(npoint,timestamp_tz) → tnpointInst
tnpoint(npoint,tstzset) → tnpointDiscSeq
tnpoint(npoint,tstzspan,interp='linear') → tnpointContSeq
tnpoint(npoint,tstzspanset,interp='linear') → tnpointSeqSet
```

```
SELECT tnpoint('Npoint(1, 0.5)', timestamp_tz '2001-01-01');
-- NPoint(1,0.5)@2001-01-01
SELECT tnpointSeq('Npoint(1, 0.3)', tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.3)@2001-01-03, NPoint(1,0.3)@2001-01-05}
SELECT tnpointSeq('Npoint(1, 0.5)', tstzspan '[2001-01-01, 2001-01-02]');
-- [NPoint(1,0.5)@2001-01-01, NPoint(1,0.5)@2001-01-02]
SELECT tnpointSeqSet('Npoint(1, 0.2)', tstzspanset '{[2001-01-01, 2001-01-03]}', 'step');
-- Interp=Step;{[NPoint(1,0.2)@2001-01-01, NPoint(1,0.2)@2001-01-03]}
```

- Constructor for temporal network points of sequence subtype

```
tnpointSeq(tnpointInst[],interp={'step','linear'},leftInc bool=true,
  rightInc bool=true) → tnpointSeq
```

```
SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.3)@2001-01-01',
  'Npoint(1, 0.5)@2001-01-02', 'Npoint(1, 0.5)@2001-01-03']);
-- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.5)@2001-01-02, NPoint(1,0.5)@2001-01-03}
SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.2)@2001-01-01',
  'Npoint(1, 0.4)@2001-01-02', 'Npoint(1, 0.5)@2001-01-03']);
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03]
```

- Constructor for temporal network points of sequence set subtype

```
tnpointSeqset(tnpoint[]) → tnpointSeqset
```

```
tnpointSeqsetGaps(tnpointInst[], maxt=NULL, maxdist=NULL, interp='linear') →  
tnpointSeqset
```

```
SELECT tnpointSeqset(ARRAY[tnpoint '[Npoint(1,0.2)@2001-01-01, Npoint(1,0.4)@2001-01-02,  
    Npoint(1,0.5)@2001-01-03]', '[Npoint(2,0.6)@2001-01-04, Npoint(2,0.6)@2001-01-05]']]);  
/* {[Npoint(1,0.2)@2001-01-01, Npoint(1,0.4)@2001-01-02, Npoint(1,0.5)@2001-01-03],  
    [Npoint(2,0.6)@2001-01-04, Npoint(2,0.6)@2001-01-05]} */  
SELECT tnpointSeqsetGaps(ARRAY[tnpoint 'NPoint(1,0.1)@2001-01-01',  
    'NPoint(1,0.3)@2001-01-03', 'NPoint(1,0.5)@2001-01-05'], '1 day');  
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.3)@2001-01-03], [NPoint(1,0.5)@2001-01-05]}
```

12.5 Conversions

A temporal network point value can be converted to and from a temporal geometry point. This can be done using an explicit CAST or using the `::` notation. A null value is returned if any of the composing geometry point values cannot be converted into a npoint value.

- Convert between a temporal network point and a temporal geometry point

```
tnpoint::tgeompoint
```

```
tgeompoint::tnpoint
```

```
SELECT asText((tnpoint '[NPoint(1, 0.2)@2001-01-01,  
    NPoint(1, 0.3)@2001-01-02]')::tgeompoint);  
/* [POINT(23.057077727326 28.7666335767956)@2001-01-01,  
    POINT(48.7117553116406 20.9256801894708)@2001-01-02] */  
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,  
    POINT(48.7117553116406 20.9256801894708)@2001-01-02]':::tnpoint  
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02]  
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,  
    POINT(48.7117553116406 20.9)@2001-01-02]':::tnpoint  
-- NULL
```

12.6 MF-JSON Input and Output

A temporal network point is written to and read from Moving-Features JSON. Each instant renders its network point as `{"route":<route id>,"position":<position>}` — the route id as a JSON number and the position as a float in `[0, 1]` — matching the SQL accessor surface (`route`, `getPosition`).

- Output a temporal network point as MF-JSON and read it back; the round trip is lossless. The output takes the options, flags, and decimal-digits arguments shared with the other temporal types.

```
asMFJSON(tnpoint, integer, integer, integer) → text
```

```
tnpointFromMFJSON(text) → tnpoint
```

```
SELECT tnpointFromMFJSON(asMFJSON(tnpoint 'NPoint(1, 0.5)@2001-01-01'))  
= tnpoint 'NPoint(1, 0.5)@2001-01-01';  
-- true
```

12.7 Accessors

- Return the values

getValues(tnpoint) → npointset

```
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]}');
-- {"NPoint(1,0.3)","NPoint(1,0.5)"}
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02]}');
-- {"NPoint(1,0.3)"}

```

- Return the road identifiers

routes(tnpoint) → bigintset

```
SELECT routes(tnpoint '{NPoint(3, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02}');
-- {1, 3}

```

- Return the value at a timestamp

valueAtTimestamp(tnpoint,timestamp) → npoint

```
SELECT valueAtTimestamp(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]',
    '2001-01-02');
-- NPoint(1,0.4)

```

- Return the length traversed by the temporal network point

length(tnpoint) → float

```
SELECT length(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]');
-- 54.3757408468784

```

- Return the cumulative length traversed by the temporal network point

cumulativeLength(tnpoint) → tfloat

```
SELECT cumulativeLength(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02,
    NPoint(1, 0.5)@2001-01-03], [NPoint(1, 0.6)@2001-01-04, NPoint(1, 0.7)@2001-01-05]}');
/* {[0@2001-01-01, 54.3757408468784@2001-01-02, 54.3757408468784@2001-01-03],
    [54.3757408468784@2001-01-04, 81.5636112703177@2001-01-05]} */

```

- Return the speed of the temporal network point in units per second

speed({tnpointSeq, tnpointSeqSet}) → tfloatSeqSet

```
SELECT speed(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
    NPoint(1, 0.6)@2001-01-03]') * 3600 * 24;
/* Interp=Step;[21.4016800272077@2001-01-01, 14.2677866848051@2001-01-02,
    14.2677866848051@2001-01-03] */

```

12.8 Transformations

- Transform a temporal network point to another subtype

tnpointInst(tnpoint) → tnpointInst

tnpointSeq(tnpoint) → tnpointSeq

tnpointSeqSet(tnpoint) → tnpointSeqSet

```

SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01', 'discrete');
-- {NPoint(1,0.5)@2001-01-01}
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- [NPoint(1,0.5)@2001-01-01]
SELECT tnpointSeqSet(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- {[NPoint(1,0.5)@2001-01-01]}

```

- Transform a temporal network point to another interpolation

setInterp(tnpoint, interp) → tnpoint

```

SELECT setInterp(tnpoint 'NPoint(1,0.2)@2001-01-01', 'linear');
-- [NPoint(1,0.2)@2001-01-01]
SELECT setInterp(tnpoint '{[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}',
  'discrete');
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02}

```

- Round the fraction of the temporal network point to the number of decimal places

round(tnpoint, integer) → tnpoint

```

SELECT round(tnpoint '{[NPoint(1,0.123456789)@2001-01-01, NPoint(1,0.5)@2001-01-02]}', 6);
-- {[NPoint(1,0.123457)@2001-01-01 00:00:00+01, NPoint(1,0.5)@2001-01-02 00:00:00+01]}

```

12.9 Modifications

- Append a temporal instant or a temporal sequence

appendInstant(tnpoint, tnpointInst) → tnpoint

appendSequence(tnpoint, tnpointSeq) → tnpoint

```

SELECT asText(appendInstant(tnpoint 'Npoint(1, 0.1)@2001-01-01', 'Npoint(1, 0.2)@2001-01-02'));
-- [NPoint(1,0.1)@2001-01-01 00:00:00+01, NPoint(1,0.2)@2001-01-02 00:00:00+01]
SELECT asText(appendSequence(tnpoint '[Npoint(1, 0.1)@2001-01-01]', '[Npoint(1, 0.2)@2001-01-02]'));
-- {[NPoint(1,0.1)@2001-01-01 00:00:00+01], [NPoint(1,0.2)@2001-01-02 00:00:00+01]}

```

- Merge the temporal network points

merge(tnpoint, tnpoint) → tnpoint

merge(tnpoint[]) → tnpoint

mergeAgg(tnpoint) → tnpoint

```

SELECT asText(merge(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  '[Npoint(1, 0.3)@2001-01-03, Npoint(1, 0.5)@2001-01-05]'));
-- [NPoint(1,0.1)@2001-01-01 00:00:00+01, NPoint(1,0.5)@2001-01-05 00:00:00+01]
SELECT asText(merge(ARRAY[tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.2)@2001-01-02]' ←
  ,
  '[Npoint(1, 0.2)@2001-01-02, Npoint(1, 0.3)@2001-01-03]']],
  '[Npoint(1,0.1)@2001-01-01 00:00:00+01, NPoint(1,0.3)@2001-01-03 00:00:00+01]
WITH temp(inst) AS (
  SELECT tnpoint 'Npoint(1, 0.1)@2001-01-01' UNION
  SELECT tnpoint 'Npoint(1, 0.2)@2001-01-02' UNION
  SELECT tnpoint 'Npoint(1, 0.3)@2001-01-03' )
SELECT asText(mergeAgg(inst)) FROM temp;
-- {NPoint(1,0.1)@2001-01-01 00:00:00+01, NPoint(1,0.2)@2001-01-02 00:00:00+01, NPoint(1,0.3)@2001-01-03 00:00:00+01}

```


12.10 Restrictions

- Restrict to (the complement of) a value or a set of values

`atValue(tnpoint, value) → tnpoint`

`minusValue(tnpoint, value) → tnpoint`

`atValues(tnpoint, valueset) → tnpoint`

`minusValues(tnpoint, valueset) → tnpoint`

```
SELECT atValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
-- {[NPoint(2,0.5)@2001-01-02]}
SELECT minusValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
/* {[NPoint(2,0.3)@2001-01-01, NPoint(2,0.5)@2001-01-02],
  (NPoint(2,0.5)@2001-01-02, NPoint(2,0.7)@2001-01-03)} */
```

- Restrict to (the complement of) a geometry

`atGeometry(tnpoint, geometry) → tnpoint`

`minusGeometry(tnpoint, geometry) → tnpoint`

```
SELECT atGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
/* {(NPoint(2,0.342593)@2001-01-01 05:06:40.364673+01,
  NPoint(2,0.7)@2001-01-03 00:00:00+01)} */
```

12.11 Distance Operations

- Return the smallest distance ever

`{geo, npoint, tnpoint} |=| {geo, npoint, tnpoint} → float`

The operator `|=|` can be used for doing nearest neighbor searches using a GiST or an SP-GiST index (see Section 10.2).

```
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
  geometry 'SRID=5676;Linestring(50 50,55 55)';
-- 31.69220882252415
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
  npoint 'NPoint(1, 0.5)';
-- 19.49691305292373
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-02]';
-- 5.231180723735304
```

- Return the instant of the first temporal network point at which the two arguments are at the nearest distance

`nearestApproachInstant({geo, npoint, tnpoint}, {geo, npoint, tnpoint}) → tnpoint`

```
SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)');
-- NPoint(2,0.349928)@2001-01-01 02:59:44.402905+01
SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)');
-- NPoint(2,0.592181)@2001-01-01 17:31:51.080405+01
```

- Return the line connecting the nearest approach point between the two arguments

`shortestLine({geo,npoint,tpoint},{geo,npoint,tpoint}) → geometry`

The function will only return the first line that it finds if there are more than one

```
SELECT ST_AsText(shortestLine(tpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(50.7960725266492 48.8266286733015,50 50)
SELECT ST_AsText(shortestLine(tpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)'));
-- LINESTRING(77.0902838115125 66.6659083092593,90.8134936900394 46.4385792121146)
```

- Return the temporal distance

`tnpoint <-> tnpoin → tfloat`

```
SELECT tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
  npoint 'NPoint(1, 0.2)';
-- [2.34988300875063@2001-01-02 00:00:00+01, 2.34988300875063@2001-01-03 00:00:00+01]
SELECT tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
  geometry 'Point(50 50)';
-- [25.0496666945044@2001-01-01 00:00:00+01, 26.4085688426232@2001-01-03 00:00:00+01]
SELECT tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
  tnpoin '[NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-04]';
-- [2.34988300875063@2001-01-02 00:00:00+01, 2.34988300875063@2001-01-03 00:00:00+01]
```

12.12 Spatial Operations

- Return the time-weighted centroid

`twCentroid(tpoint) → geometry(Point)`

```
SELECT ST_AsText(twCentroid(tpoint '{[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-02, NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04]}'));
-- POINT(79.9787466444847 46.2385558051041)
```

12.13 Bounding Box Operations

- Topological operators

`{tstzspan,stbox,tnpoint} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tnpoint} → boolean`

```
SELECT tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' &&
  tstzspan '[2001-01-02,2001-01-03]';
-- true
SELECT tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @>
  stbox(npoin 'NPoint(1, 0.5)');
-- true
SELECT tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' ~=
  tnpoin '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.35)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03]';
-- true
```

- Position operators

`{stbox,tnpoint} {<<, &<, >>, &>} {stbox,tnpoint} → boolean`

`{stbox,tnpoint} {<<|, &<|, |>>, |&>} {stbox,tnpoint} → boolean`

`{tstzspan,stbox,tnpoint} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tnpoint} → boolean`

```

SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' <<
  stbox(npoint 'NPoint(1, 0.2)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' <<|
  stbox(npoint 'NPoint(1, 0.5)');
-- false
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' #&>
  tstzspan '[2001-01-01,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.3)@2001-01-05]' #>>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02]';
-- true

```

12.14 Spatial Relationships

The topological and distance relationships described in Section 8.5 such as `eIntersects`, `aDwithin`, or `tContains` are defined over the geographical space while the temporal network points are defined over the network space. To be able to apply these relationships to the temporal network points, they must be transformed to temporal geometry points. This can be easily performed using the functions `geometry` and `tgeompoint` or using an explicit casting `::` as shown in the examples below.

- Ever and always spatial relationships

```

SELECT eContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tgeompoint(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]'));
-- false
SELECT eDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tgeompoint(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]'));
-- true
SELECT eIntersects(
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]::tgeompoint,
  tnpoint '[NPoint(2, 0.0)@2001-01-01, NPoint(2, 1)@2001-01-03]::tgeompoint);
-- false

```

- Spatiotemporal relationships

```

SELECT tContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tgeompoint(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]'));
-- [f@2001-01-01 00:00:00+01, f@2001-01-03 00:00:00+01)
SELECT tDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tgeompoint(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]'));
-- [t@2001-01-01 00:00:00+01, t@2001-01-03 00:00:00+01)
SELECT tDwithin(
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]::tgeompoint,
  tnpoint '[NPoint(1, 0.5)@2001-01-01, NPoint(1, 0.3)@2001-01-03]::tgeompoint, 1);
/* {[t@2001-01-01 00:00:00+01, t@2001-01-01 22:35:55.379053+01],
  (f@2001-01-01 22:35:55.379053+01, t@2001-01-02 01:24:04.620946+01,
  t@2001-01-03 00:00:00+01)} */

```

12.15 Comparisons

- Traditional comparisons

```
tnpoint {=, <>, <, >, <=, >=} tnpoint → boolean
```

```

SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-02],
[NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-03]}' =
tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- true
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]}' <>
tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- false
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <
tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.6)@2001-01-03]';
-- true

```

- Ever and always comparisons

{npoint,tnpoint} {?=?, %=} {npoint,tnpoint} → boolean

```

SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-04]' ?= Npoint(1, 0.3);
-- true
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.2)@2001-01-04]' &= Npoint(1, 0.2);
-- true

```

- Temporal comparisons

{npoint,tnpoint} {#=#, #<>} {npoint,tnpoint} → tbool

```

SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-03]' #=
npoint 'NPoint(1, 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.8)@2001-01-03]' #<>
tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.7)@2001-01-03]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}

```

12.16 Aggregations

The three aggregate functions for temporal network points are illustrated next.

- Temporal count

tCount(tnpoint) → {tintSeq,tintSeqSet}

```

WITH Temp(temp) AS (
  SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]' UNION
  SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04]' UNION
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}

```

- Window count

wCount(tnpoint) → {tintSeq,tintSeqSet}

```

WITH Temp(temp) AS (
  SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]' UNION
  SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04]' UNION
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' )
SELECT wCount(Temp, '1 day')
FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
1@2001-01-06]} */

```

- Temporal centroid

`tCentroid(tnpoint) → tgeompoint`

```
WITH Temp(temp) AS (
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]' UNION
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-03]' UNION
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' )
SELECT astext(tCentroid(Temp))
FROM Temp;
/* {[POINT(72.451531682218 76.5231414472853)@2001-01-01,
POINT(55.7001249027598 72.9552602410653)@2001-01-03]} */
```

12.17 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal networks points. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal network points, which is an `stbox` and thus stores the absolute coordinates of the underlying space.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<, &<, &>, >>, <<|, &<|, |&>, |>>`, which only consider the spatial dimension in temporal network points,
- `<<#, &<#, #&>, #>>`, which only consider the time dimension in temporal network points,
- `&&, @>, <@, ~=, -|-`, and `|=|`, which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

12.18 Aggregation

- Return the spatiotemporal bounding box covering every materialised position in a set of network points. The aggregate is polymorphic with the extent aggregates on the other spatial temporal types, so one query can compute a single box across a heterogeneous set of `tgeompoint`, `tgeometry`, and `tnpoint` columns. Each instant's spatial extent is resolved through the ways registry, which dominates the cost over a large input.

`extent(tnpoint) → stbox`

```
SELECT extent(temp) FROM tbl_tnpoint;
```

12.19 Production guidance

This section captures the operational knowledge production workloads need on top of the function reference: when `npoint` / `tnpoint` are the right type, how SRIDs flow from the network registry, the numerical hazards that bite network-constrained tracking workloads, and the durability story for serialised values. Most of these points are surfaced separately throughout the chapter; the goal here is one place a new contributor can read end-to-end before instrumenting an application.

12.19.1 When to use `tnpoint` vs `tgeompoint`

A `tgeompoint` tracks a position in free space; a `tnpoint` tracks a position constrained to a known network of routes. Each network-point value is a pair (`rid`, `pos`) where `rid` is the route identifier and `pos` is the relative position along that route, normalised to `[0, 1]` (start of route to end of route).

Use `tgeompoint` when the moving object is not constrained to a network, when the network topology is unknown, or when only the trajectory geometry matters — vehicle GPS tracks at the raw-feed level, animal movement, AIS feeds.

Use `tnpoint` when motion is on a known network and you want network-relative semantics: distances measured along the route, not in free space; aggregations grouped by route segment; spatial joins constrained to "same network". Canonical workloads are train-network and road-network tracking, where map-matched feeds become `tnpoint` values.

The position component of a `tnpoint` is recoverable as a `tgeompoint` via the standard cast (or as a `geometry` via `geometry(npoint)`); a `geometry` can be projected back onto the loaded network with `npoint(geometry)`. Consequently a single `tnpoint` column subsumes a map-matched `tgeompoint` column when network-relative semantics are needed.

12.19.2 SRID inheritance from the network registry

An `npoint` value does not carry its own SRID; it inherits the SRID of the route referenced by its `rid` field. The set of known routes — the *ways* registry — is loaded into a per-process cache (see `meos/src/npoint/ways_meos.c`) the first time a network function runs, and is keyed by route gid. Every `npoint` accessor that needs the route geometry (`npoint_to_geompoint`, `npoint_to_stbox`, all spatial predicates and distance functions) consults this cache.

In the PostgreSQL extension the cache is populated automatically from the `public.ways` table. In standalone MEOS the application must call `meos_register_ways(...)` with the route table before any `npoint` operation; `meos_finalize_ways()` is idempotent and can be called multiple times safely. Cross-type operations between an `npoint` and a `geometry` validate that the route's SRID matches the geometry's SRID; mismatches raise an explicit error rather than silently re-projecting.

12.19.3 Numerical hazards

Four hazards reliably bite real network-constrained tracking workloads. The implementation mitigates each as follows:

- *Position-parameter clamping at `[0, 1]`.* A position outside `[0, 1]` has no meaning on a route that has not been extended.
Mitigation: `npoint_make` rejects `pos < 0` or `pos > 1` at construction with the message "The relative position must be a real number between 0 and 1", and the same check is applied at the WKT, WKB, and MFJSON parser entries. The endpoints `0.0` (start of route) and `1.0` (end of route) are valid and supported — the test suite exercises both. Corrupt or unclipped feed data fails loudly rather than poisoning a downstream query.
- *Mixed-SRID handling.* A `tnpoint` inherits its SRID from its route, while a `geometry` argument carries its SRID explicitly.
Mitigation: every cross-type operator (`tnpoint <-> geometry`, `tnpoint && geometry`, `distance(tnpoint, geometry)`) validates the two SRIDs match through `ensure_same_srid`; mismatches raise an explicit error rather than silently re-projecting one side onto the other.
- *Route-not-registered.* An `npoint` referencing a route gid not present in the ways cache cannot resolve to a geometry.
Mitigation: `npoint_make` calls `ensure_route_exists` at construction; the WKT, WKB, MFJSON, and `npoint(...)` SQL constructors all flow through this check and raise "There is no route with gid value <n> in table ways" on miss. Standalone MEOS users who skip the `meos_register_ways(...)` bootstrap will see this error on the first network operation, not later as a silent zero-result query.
- *Cross-network operations.* Two `tnpoint` values whose routes belong to different networks (or different connected components of the same network) cannot be related through network distance.
Mitigation: `distance` and spatial-relationship operators between two `tnpoint` values lower both sides to `tgeompoint` first and compute the result as free-space distance / topology. This is documented behaviour: callers who need true network distance must restrict their query to a single connected component upstream.

12.19.4 Durability and storage

The on-disk representation of an `npoint` is a 16-byte tuple: an 8-byte route id followed by an 8-byte position. The byte layout is stable: `asBinary(tnpoint) / tnpointFromBinary(bytea)` round-trip preserves the value bit-for-bit, and the WKB / HexEWKB serialisations follow the standard temporal-types endian-flag-then-payload pattern. The `asMFJSON(tnpoint) / tnpointFromMFJSON(text)` pair is bidirectional, so MovingFeatures-JSON is a viable interchange format on equal footing with WKB. Each instant renders as `{"route":<route>, "position":<position>}`, with the field names matching the SQL accessor surface (`route`, `getPosition`) on the natural-language descriptor convention.

The route id is emitted as a JSON number, which round-trips losslessly between MEOS encoders and decoders (both retain full `int64` precision via `INT64_FORMAT / strtoll`). It is not, however, lossless across JavaScript / ECMAScript consumers: the `Number` type has a 53-bit mantissa, so route ids beyond 2^{53} ($\sim 9 \times 10^{15}$) lose precision when parsed by a generic browser-side `JSON.parse`. For OSM-derived networks this matters in practice — OSM way ids crossed 2^{53} in 2025 — and the recommended mitigation on the JS side is to parse with a library that materialises numbers as `BigInt` (for example `json-bigint`) rather than rely on the default. This is a deliberate design choice, not an oversight: the alternative (emitting the route as a JSON string, as `th3index` does for its `int64` cell payload) would diverge from the JSON shape that all numeric accessors return for this type and break the round-trip with naive int-parsing on every consumer that *does* handle 64-bit integers correctly.

The `pg_dump` output of a table containing `npoint` or `tnpoint` columns uses the WKT representation by default; `pg_dump --binary-upgrade` uses the WKB representation. Both are round-trip-stable, but a dump is only restorable into a database whose `public.ways` table contains all referenced route ids: the route registry is logically a foreign key on every `npoint` column, even though it is not enforced as one.

Chapter 13

Temporal Circular Buffers

A static circular buffer type `cbuffer` represents a 2D point and a radius greater than or equal to 0 around the point. The radius represent the “impact” of the point on its surroundings. This impact could be interpreted as noise, pollution, or similar aspects depending on application requirements.

The `cbuffer` type serves as base type for defining the temporal circular buffer type `tcbuffer`. The `tcbuffer` type has similar functionality as the temporal point type `tgeompoint` with the exception that it only considers two dimensions. Thus, most functions and operators described before for the `tgeompoint` type are also applicable for the `tcbuffer` type. In addition, there are specific functions defined for the `tcbuffer` type.

Figure 13.1 illustrates the use of the `tcbuffer` type for modelling the circular buffer around the wind swath of tropical storms in 2024. The data is obtained from the National Oceanic and Atmospheric Administration (NOAA). As illustrated in the figure, the types `tgeompoint` and `tcbuffer` allow *linear* interpolation, while as we have seen in Figure 7.2, the type `tgeometry` only allows *step* interpolation.

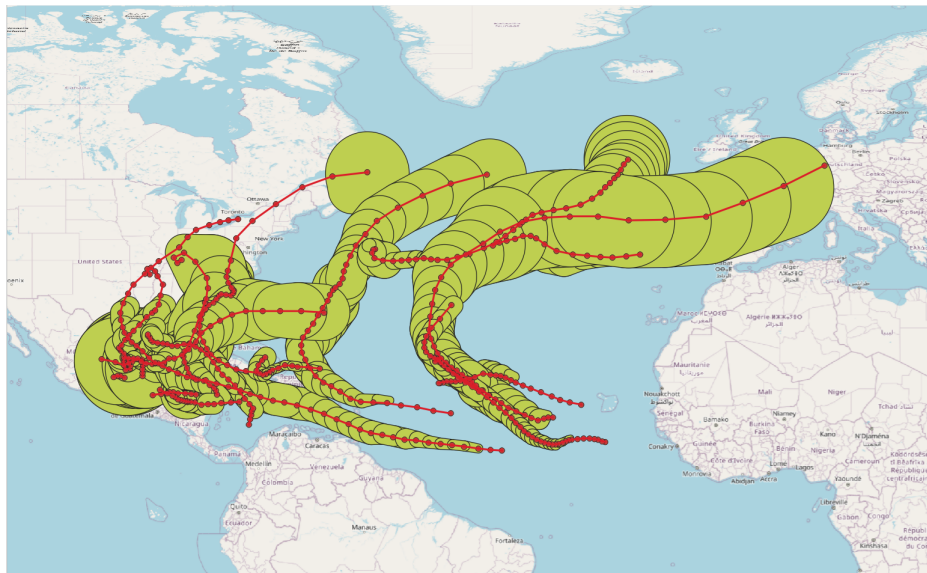


Figure 13.1: Illustration of the use of the `tcbuffer` type for modelling the circular buffer around the wind swath of tropical storms in 2024.

13.1 Static Circular Buffers

A `cbuffer` value is a couple of the form `(point, radius)` where `point` is a geometric point and `radius` is a float value greater than or equal to zero representing a radius around the point expressed using the units of the SRID of the point. Examples of input of circular buffer values are as follows:

```
SELECT cbuffer 'Cbuffer(Point(1 1), 0.3)';
SELECT cbuffer 'Cbuffer(Point(1 1), 10.0)';
```

The `cbuffer` type corresponds to the result of the PostGIS function `ST_MinimumBoundingRadius`.

Values of the circular buffer type must satisfy several constraints so that they are well defined. Examples of incorrect circular buffer type values are as follows.

```
-- incorrect point value
SELECT cbuffer 'Cbuffer(Linestring(1 1,2 2), 1.0)';
-- incorrect 3D point
SELECT cbuffer 'Cbuffer(Point Z(1 1 1), 1.0)';
-- incorrect radius value
SELECT cbuffer 'Cbuffer(Point(1 1), -2.0)';
```

We give next the functions and operators for the circular buffer type.

13.1.1 Constructors

- Constructor for circular buffers

`cbuffer(geometry, float) → cbuffer`

```
SELECT asText(cbuffer(ST_Point(1,1), 0.3));
-- Cbuffer(POINT(1 1),0.3)
```

13.1.2 Conversions

Values of the `cbuffer` type can be converted to the `geometry` type using an explicit `CAST` or using the `::` notation as shown below. Similarly, a `geometry` can be converted to a `cbuffer` value using the `ST_MinimumBoundingRadius` function in PostGIS.

- Convert between a circular buffer and a geometry

`geometry::cbuffer`

`cbuffer::geometry`

```
SELECT ST_AsText(cbuffer(ST_Point(1,1), 1)::geometry);
-- CURVEPOLYGON(CIRCULARSTRING(0 1,2 1,0 1))
SELECT asText(geometry 'SRID=5676;CIRCULARSTRING(0 1,2 1,0 1)::cbuffer);
-- Cbuffer(POINT(1 1),1)
SELECT asText(geometry 'SRID=5676;LINESTRING(0 0,2 2)::cbuffer);
-- Cbuffer(POINT(1 1),1.414213562373095)
```

- Convert a circular buffer and, optionally, a timestamp or a period, to a spatiotemporal box

`stbox(cbuffer) → stbox`

`stbox(cbuffer, {timestamp, tstzspan}) → stbox`

```
SELECT stbox(cbuffer 'SRID=5676;Cbuffer(Point(1 1),0.3)');
-- SRID=5676;STBOX X((0.7,0.7),(1.3,1.3))
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', timestamp '2001-01-01');
-- STBOX XT(((0.7,0.7),(1.3,1.3)),[2001-01-01,2001-01-01])
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((0.7,0.7),(1.3,1.3)),[2001-01-01,2001-01-02])
```

13.1.3 Accessors

- Return the point

`point(cbuffer) → geompoint`

```
SELECT ST_AsText(point(cbuffer 'Cbuffer(Point(1 1), 0.3)'));
-- Point(1 1)
```

- Return the radius

`radius(cbuffer) → float`

```
SELECT radius(cbuffer 'Cbuffer(Point(1 1), 0.3)');
-- 0.3
```

13.1.4 Transformations

- Round the point and the radius of the circular buffer to the number of decimal places

`round(cbuffer, integer=0) → cbuffer`

```
SELECT asText(round(cbuffer(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
-- Cbuffer(POINT(1.123457 1.123457),0.123457)
```

13.1.5 Spatial Operations

- Return or set the spatial reference identifier

`SRID(cbuffer) → integer`

`setSRID(cbuffer) → cbuffer`

```
SELECT SRID(cbuffer 'Cbuffer(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(cbuffer 'Cbuffer(Point(0 0),1)', 4326));
-- SRID=4326;Cbuffer(POINT(0 0),1)
```

- Transform to a spatial reference identifier

`transform(cbuffer, integer) → cbuffer`

`transformPipeline(cbuffer, pipeline text, to_srid integer, is_forward bool=true) → cbuffer`

The `transform` function specifies the transformation with a target SRID. An error is raised when the input circular buffer has an unknown SRID (represented by 0).

The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input circular buffer is ignored, and the SRID of the output circular buffer will be set to zero unless a value is provided via the optional `to_srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(cbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)', 3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)
```

```
WITH test(cbuffer, pipeline) AS (
  SELECT cbuffer 'Cbuffer(SRID=4326;Point(4.3525 50.846667),1)',
         text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
  SELECT asEWKT(transformPipeline(transformPipeline(cbuffer, pipeline, 4326), pipeline,
    4326, false), 6)
  FROM test;
-- SRID=4326;Cbuffer(POINT(4.3525 50.846667),1)
```

13.1.6 Comparisons

The comparison operators (=, <, and so on) are available for circular buffers. They compare first the points and then the radius of the arguments. Excepted the equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on circular buffers.

- Traditional comparisons

`cbuffer {=, <>, <, >, <=, >} cbuffer`

```
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' = cbuffer 'Cbuffer(Point(3 3), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' <> cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' < cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.6)' > cbuffer 'Cbuffer(Point(2 2), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.5)' <= cbuffer 'Cbuffer(Point(2 2), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.6)' >= cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- true
```

13.2 Temporal Circular Buffers

The temporal circular buffer type `tcbuffer` allows to represent the movement of objects together with a circular radius around them. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tcbuffer` values in these subtypes are given next.

```
SELECT tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01';
SELECT tcbuffer '{Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02,
  Cbuffer(Point(1 1), 0.5)@2001-01-03}';
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01, Cbuffer(Point(1 1), 0.4)@2001-01-02,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]';
SELECT tcbuffer '{[Cbuffer(Point(1 1), 1)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-02],
  [Cbuffer(Point(2 2), 2)@2001-01-04, Cbuffer(Point(2 2), 3)@2001-01-05]}';
```

The temporal circular buffer type accepts type modifiers (or `typmod` in PostgreSQL terminology) to specify the subtype and/or the spatial reference identifier (SRID). The possible values for the subtype are `Instant`, `Sequence`, and `SequenceSet`. The two arguments are optional and if any of them is not specified for a column, values of any subtype and/or SRID are allowed.

```
SELECT asEWKT(tcbuffer(Sequence,5676) 'SRID=5676;[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-03]
SELECT tcbuffer(Sequence) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
SELECT tcbuffer(5676) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal circular buffer SRID (0) does not match column SRID (5676)
```

Temporal circular buffer values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into a normal form are as follows.

```
SELECT asText(tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(2 2), 0.4)@2001-01-02, Cbuffer(Point(3 3), 0.6)@2001-01-03)');
-- [Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(3 3),0.6)@2001-01-03)
SELECT asText(tcbuffer '{[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(2 2), 0.3)@2001-01-02, Cbuffer(Point(2 2), 0.5)@2001-01-03),
  Cbuffer(Point(2 2), 0.5)@2001-01-03, Cbuffer(Point(2 2), 0.7)@2001-01-04)}');
/* {[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.3)@2001-01-02,
  Cbuffer(Point(2 2),0.7)@2001-01-04)} */
```

13.3 Validity of Temporal Circular Buffers

Temporal circular buffer values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tcbuffer 'NULL@2001-01-01 08:05:00';
SELECT tcbuffer 'Point(0 0)@NULL';
-- Base type is not a circular buffer
SELECT tcbuffer 'Point(0 0)@2001-01-01 08:05:00';
```

We give next the functions and operators for temporal circular buffer. Most functions and operators for temporal types described in the previous chapters can be applied for temporal circular buffer types. Therefore, in the signatures of the functions, the notation `base` represents a `cbuffer` and the notations `ttype`, `tpoint`, and `tgeompoint` also represent a `tcbuffer`. Furthermore, the functions that have an argument of type `geometry` accept in addition an argument of type `cbuffer`. To avoid redundancy, we only present next some examples of these functions and operators for temporal circular buffers.

13.4 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

$$\text{asText}(\{tcbuffer, tcbuffer[], cbuffer[]\}) \rightarrow \{text, text[]\}$$

```
asEWKT({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
```

```
SELECT asText(tcbuffer 'SRID=4326;[Cbuffer(Point(0 0),1)@2001-01-01,
    Cbuffer(Point(1 1),2)@2001-01-02)');
-- [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)
SELECT asText(ARRAY[cbuffer 'Cbuffer(Point(0 0),1)', 'Cbuffer(Point(1 1),2)']);
-- {"Cbuffer(POINT(0 0),1)","Cbuffer(POINT(1 1),2)"}
SELECT asEWKT(tcbuffer 'SRID=4326;[Cbuffer(Point(0 0),1)@2001-01-01,
    Cbuffer(Point(1 1),2)@2001-01-02)');
-- SRID=4326;[Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)
SELECT asEWKT(ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(0 0),1)',
    'Cbuffer(SRID=5676;Point(1 1),2)']);
-- {"Cbuffer(SRID=5676;POINT(0 0),1)","Cbuffer(SRID=5676;POINT(1 1),2)"}

```

- Return the Moving Features JSON (MF-JSON) representation

```
asMFJSON(tcbuffer) → text
```

A circular buffer is always planar 2D. Each value is an object with a `point` member, an `[x, y]` coordinate array, and a numeric `radius` member, mirroring the `point` and `radius` accessors.

```

SELECT asMFJSON(tcbuffer 'Cbuffer(Point(1 2),0.5)@2001-01-01', 3);
/* {"type":"MovingCircularBuffer","bbox":[[0.5,1.5],[1.5,2.5]],
   "period":{"begin":"2001-01-01T00:00:00+01","end":"2001-01-01T00:00:00+01",
   "lower_inc":true,"upper_inc":true},
   "values":[{"point":[1,2],"radius":0.5}],
   "datetimes":["2001-01-01T00:00:00+01"],"interpolation":"None"} */
SELECT asMFJSON(tcbuffer 'SRID=4326;Cbuffer(Point(1 1),0.2)@2001-01-01,
   Cbuffer(Point(2 2),0.4)@2001-01-02)', 3);
/* {"type":"MovingCircularBuffer","crs":{"type":"Name",
   "properties":{"name":"EPSG:4326"}}, "bbox":[[0.8,0.8],[2.4,2.4]],
   "period":{"begin":"2001-01-01T00:00:00+01","end":"2001-01-02T00:00:00+01",
   "lower_inc":true,"upper_inc":false},
   "values":[{"point":[1,1],"radius":0.2}, {"point":[2,2],"radius":0.4}],
   "datetimes":["2001-01-01T00:00:00+01","2001-01-02T00:00:00+01"],
   "lower_inc":true,"upper_inc":false,"interpolation":"Linear"} */

```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB) representation, or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

`asBinary(tcbuffer, endian text="")` → `bytea`

`asEWKB(tcbuffer, endian text="")` → `bytea`

`asHexEWKB(tcbuffer, endian text="")` → `text`

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```

SELECT asBinary(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
-- \x013b0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asEWKB(tcbuffer 'SRID=7844;Cbuffer(Point(1 2),1)@2001-01-01');
-- \x013b0041a41e00000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asHexEWKB(tcbuffer 'SRID=3812;Cbuffer(Point(1 2),1)@2001-01-01');
-- 013B0041E40E00000000000000F03F000000000000400000000000F03F009C57D3C11C0000

```

- Input from the Well-Known Text (WKT) representation or from the Extended Well-Known Text (EWKT) representation

`tcbufferFromText(text)` → `tcbuffer`

`tcbufferFromEWKT(text)` → `tcbuffer`

```

SELECT asEWKT(tcbufferFromText(text 'Cbuffer(Point(1 2),1)@2001-01-01,
   Cbuffer(Point(3 4),2)@2001-01-02)');
-- [Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tcbufferFromEWKT(text 'SRID=3812;Cbuffer(Point(1 2),1)@2001-01-01,
   Cbuffer(Point(3 4),2)@2001-01-02)');
-- SRID=3812;Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]

```

- Input from the Moving Features JSON (MF-JSON) representation

`tcbufferFromMFJSON(text)` → `tcbuffer`

```

SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer
   'SRID=3812;Cbuffer(Point(1 2),0.5)@2001-01-01', 3)));
-- SRID=3812;Cbuffer(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer
   'SRID=5676;Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02', 3)));
-- SRID=5676;Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]

```

- Input from the Well-Known Binary (WKB) representation, from the Extended Well-Known Binary (EWKB) representation, or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

`tcbufferFromBinary(bytea)` → `tcbuffer`

`tcbufferFromEWKB(bytea)` → `tcbuffer`

`tcbufferFromHexEWKB(text)` → `tcbuffer`

```

SELECT asEWKT(tcbufferFromBinary(
  '\x013b0001000000000000f03f00000000000004000000000000f03f009c57d3c11c0000'));
-- Cbuffer(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tcbufferFromEWKB(
  '\x013b0041a41e00000000000000f03f00000000000004000000000000f03f009c57d3c11c0000'));
-- SRID=7844;Cbuffer(Point(1 1),2)@2001-01-01
SELECT asEWKT(tcbufferFromHexEWKB(
  '013B0041E40E00000000000000F03F00000000000004000000000000F03F009C57D3C11C0000'));
-- SRID=3812;Cbuffer(POINT(1 2),1)@2001-01-01

```

13.5 Constructors

- Constructor for temporal circular buffers having a constant value

`tcbuffer(cbuffer,timestamptz) → tcbufferInst`

`tcbuffer(cbuffer,tstzset) → tcbufferDiscSeq`

`tcbuffer(cbuffer,tstzspan,interp='linear') → tcbufferContSeq`

`tcbuffer(cbuffer,tstzspanset,interp='linear') → tcbufferSeqSet`

```

SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)', timestamptz '2001-01-01'));
-- Cbuffer(Point(1 1),0.5)@2001-01-01
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.3)',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}'));
/* {Cbuffer(Point(1 1),0.3)@2001-01-01, Cbuffer(Point(1 1),0.3)@2001-01-03,
  Cbuffer(Point(1 1),0.3)@2001-01-05} */
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)',
  tstzspan '[2001-01-01, 2001-01-02]'));
-- [Cbuffer(Point(1 1),0.5)@2001-01-01, Cbuffer(Point(1 1),0.5)@2001-01-02]
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.2)',
  tstzspanset '{[2001-01-01, 2001-01-03]}', 'step'));
-- Interp=Step;{[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(1 1),0.2)@2001-01-03]}

```

- Constructor for temporal circular buffers of sequence subtype

`tcbufferSeq(tcbufferInst[],interp='linear',leftInc bool=true,`

`rightInc bool=true) → tcbufferSeq`

```

SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
  'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03']));
/* [Cbuffer(Point(1 1),0.3)@2001-01-01, Cbuffer(Point(2 2),0.5)@2001-01-02,
  Cbuffer(Point(1 1),0.5)@2001-01-03] */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
  'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03',
  'discrete']));
/* {Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02,
  Cbuffer(POINT(1 1),0.5)@2001-01-03} */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.2)@2001-01-01',
  'Cbuffer(Point(1 1), 0.4)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03', 'step']));
/* Interp=Step;[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(1 1),0.4)@2001-01-02,
  Cbuffer(Point(1 1),0.5)@2001-01-03] */

```

- Constructor for temporal circular buffers of sequence set subtype

`tcbufferSeqSet(tcbuffer[]) → tcbufferSeqSet`

`tcbufferSeqSetGaps(tcbufferInst[],maxt=NULL,maxdist=NULL,interp='linear') →
tcbufferSeqSet`

```

SELECT asText (tcbufferSeqSet (ARRAY[tcbuffer
  '[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.4)@2001-01-02]',
  '[Cbuffer(Point(2 2),0.6)@2001-01-03, Cbuffer(Point(2 2),0.8)@2001-01-04]']));
/* {[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.4)@2001-01-02],
  [Cbuffer(Point(2 2),0.6)@2001-01-03, Cbuffer(Point(2 2),0.6)@2001-01-04]} */
SELECT asText (tcbufferSeqSetGaps (ARRAY[tcbuffer 'Cbuffer(Point(1 1),0.1)@2001-01-01',
  'Cbuffer(Point(1 1),0.3)@2001-01-03', 'Cbuffer(Point(1 1),0.5)@2001-01-05]', '1 day']));
/* {[Cbuffer(Point(1 1),0.1)@2001-01-01], [Cbuffer(Point(1 1),0.3)@2001-01-03],
  [Cbuffer(Point(1 1),0.5)@2001-01-05]} */

```

- Construct a temporal circular buffer from a temporal geometry point and a temporal float

`tcbuffer(tgeompoint, tfloat) → tcbuffer`

The time frame of the result is the intersection of the time frames of the temporal point and the temporal float. If they do not intersect, a null value is returned

```

SELECT asText (tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
  tfloat '[1@2001-01-01, 2@2001-01-02]'));
-- [Cbuffer(Point(1 1),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)
SELECT asText (tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
  tfloat '[2@2001-01-03, 3@2001-01-04]'));
-- NULL

```

13.6 Conversions

A temporal circular buffer value can be converted to and from a temporal geometry point. This can be done using an explicit CAST or using the `::` notation. A null value is returned if any of the composing geometry point values cannot be converted into a cbuffer value.

- Convert a temporal circular buffer to a temporal geometry point

`tcbuffer::tgeompoint`

```

SELECT asText((tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02]')::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)

```

- Convert a temporal circular buffer to a temporal float

`tcbuffer::tfloat`

```

SELECT (tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02]')::tfloat;
-- [0.2@2001-01-01, 0.3@2001-01-02)

```

13.7 Accessors

- Return the values

`getValues(tcbuffer) → cbufferset`

```

SELECT asText (getValues(tcbuffer '{[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]}'));
-- {"Cbuffer(Point(1 1),0.3)","Cbuffer(Point(1 1),0.5)"}
SELECT asEWKT (getValues(tcbuffer 'SRID=5676;{[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02]}'));
-- SRID=5676;{"Cbuffer(Point(1 1),0.3)"}

```

- Return the points or the radii

points(tcbuffer) → geomset
radius(tcbuffer) → floatset

```
SELECT asEWKT(points(tcbuffer 'SRID=5676;{Cbuffer(Point(3 3), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02}'));
-- SRID=5676;{Point(1 1), Point(3 3)}
SELECT radius(tcbuffer 'SRID=5676;{Cbuffer(Point(3 3), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02}');
-- {0.3, 0.5}
```

- Return the value at a timestamp

valueAtTimestamp(tcbuffer,timestamp) → cbuffer

```
SELECT asText(valueAtTimestamp(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]', '2001-01-02'));
-- Cbuffer(Point(2 2),0.4)
```

13.8 Transformations

- Transform a temporal circular buffer to another subtype

tcbufferInst(tcbuffer) → tcbufferInst
tcbufferSeq(tcbuffer) → tcbufferSeq
tcbufferSeqSet(tcbuffer) → tcbufferSeqSet

```
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Cbuffer(Point(1 1),0.5)@2001-01-01}
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- [Cbuffer(Point(1 1),0.5)@2001-01-01]
SELECT asText(tcbufferSeqSet(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- {[Cbuffer(Point(1 1),0.5)@2001-01-01]}
```

- Transform a temporal circular buffer to another interpolation

setInterp(tcbuffer, interp) → tcbuffer

```
SELECT asText(setInterp(tcbuffer 'Cbuffer(Point(1 1),0.2)@2001-01-01', 'linear'));
-- [Cbuffer(Point(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tcbuffer '[Cbuffer(Point(1 1),0.1)@2001-01-01,
  [Cbuffer(Point(1 1),0.2)@2001-01-02]]', 'discrete'));
-- {Cbuffer(Point(1 1),0.1)@2001-01-01, Cbuffer(Point(1 1),0.2)@2001-01-02}
```

- Round the points and the radii of the temporal circular buffer to the number of decimal places

round(tcbuffer, integer) → tcbuffer

```
SELECT asText(round(tcbuffer '[Cbuffer(Point(1 1.123456789),0.123456789)@2001-01-01,
  Cbuffer(Point(1 1),0.5)@2001-01-02]', 3));
-- {[Cbuffer(Point(1 1.123),0.123)@2001-01-01, Cbuffer(Point(1 1),0.5)@2001-01-02]}
```

13.9 Modifications

- Append a temporal instant or a temporal sequence

appendInstant(tcbuffer,tcbufferInst) → tcbuffer
appendSequence(tcbuffer,tcbufferSeq) → tcbuffer


```

SELECT asText (appendInstant (tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01', 'Cbuffer(Point(
  (2 2), 0.2)@2001-01-02')));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01 00:00:00+01, Cbuffer(POINT(2 2),0.2)@2001-01-02 00:00:00+01]
SELECT asText (appendSequence (tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01]', '[Cbuffer(
  Point(2 2), 0.2)@2001-01-02]'));
-- {[Cbuffer(POINT(1 1),0.1)@2001-01-01 00:00:00+01], [Cbuffer(POINT(2 2),0.2)@2001-01-02 00:00:00+01]}

```

- Merge the temporal circular buffers

`merge(tcbuffer,tcbuffer) → tcbuffer`

`merge(tcbuffer[]) → tcbuffer`

`mergeAgg(tcbuffer) → tcbuffer`

```

SELECT asText (merge(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1),
  0.3)@2001-01-03]',
  '[Cbuffer(Point(1 1), 0.3)@2001-01-03, Cbuffer(Point(1 1), 0.5)@2001-01-05]'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01 00:00:00+01, Cbuffer(POINT(1 1),0.5)@2001-01-05 00:00:00+01]
SELECT asText (merge (ARRAY[tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1
  1), 0.2)@2001-01-02]',
  '[Cbuffer(Point(1 1), 0.2)@2001-01-02, Cbuffer(Point(1 1), 0.3)@2001-01-03]']));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01 00:00:00+01, Cbuffer(POINT(1 1),0.3)@2001-01-03 00:00:00+01]
WITH temp(inst) AS (
  SELECT tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01' UNION
  SELECT tcbuffer 'Cbuffer(Point(2 2), 0.2)@2001-01-02' )
SELECT asText (mergeAgg(inst)) FROM temp;
-- {Cbuffer(POINT(1 1),0.1)@2001-01-01 00:00:00+01, Cbuffer(POINT(2 2),0.2)@2001-01-02 00:00:00+01}

```

13.10 Spatial Operations

- Return or set the spatial reference identifier

`SRID(tcbuffer) → integer`

`setSRID(tcbuffer) → tcbuffer`

```

SELECT SRID(tcbuffer 'SRID=5676;[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02)');
-- 5676
SELECT asEWKT(setSRID(tcbuffer '[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02)', 5676));
-- SRID=5676;[Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)

```

- Transform to a spatial reference identifier

`transform(tcbuffer,integer) → tcbuffer`

`transformPipeline(tcbuffer,pipeline text,to_srid integer,is_forward bool=true) → tcbuffer`

The `transform` function specifies the transformation with a target SRID. An error is raised when the input temporal circular buffer has an unknown SRID (represented by 0).

The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format: `urn:ogc:def:coordinateOperation:AUTHORITY::CODE`. The SRID of the input temporal circular buffer is ignored, and the SRID of the output temporal circular buffer will be set to zero unless a

value is provided via the optional `to_srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to `false`, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(tcbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)@2001-01-01',
3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tcbuffer, pipeline) AS (
  SELECT tcbuffer 'SRID=4326;{Cbuffer(Point(4.3525 50.846667),1)@2001-01-01,
    Cbuffer(Point(-0.1275 51.507222),2)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
  SELECT asEWKT(transformPipeline(transformPipeline(tcbuffer, pipeline, 4326), pipeline,
    4326, false), 6)
FROM test;
/* SRID=4326;{Cbuffer(POINT(4.3525 50.846667),1)@2001-01-01,
  Cbuffer(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

- Return the area traversed by the temporal circular buffer

`traversedArea(tcbuffer, unary_union=false) → geometry`

```
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]'));
-- CURVEPOLYGON(CIRCULARSTRING(0.7 1,1.3 1,0.7 1))
```

- Return the temporal geometric point given by the centers of a temporal circular buffer

`centroid(tcbuffer) → tgeompoint`

```
SELECT asText(centroid(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]'));
-- [POINT(1 1)@2001-01-01 00:00:00+01, POINT(3 3)@2001-01-03 00:00:00+01]
```

- Return the convex hull of the area traversed by the temporal circular buffer

`convexHull(tcbuffer) → geometry`

The convex hull is computed on the linearized traversed area: the circular arcs bounding the swept region are first stroked into straight segments and the hull is then computed with GEOS. The result is therefore a linear `POLYGON` that approximates the circular boundary, not a curved `CURVEDPOLYGON`. Use `traversedArea` to obtain the exact curved swept region.

13.11 Distance Operations

- Return the smallest distance ever

`{geo, cbuffer, tcbuffer} || {geo, cbuffer, tcbuffer} → float`

The operator `||` that can be used for doing nearest neighbor searches using a GiST or an SP-GiST index (see Section 10.2).

```
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' || geometry 'Linestring(50 50,55 55)';
-- 67.58225099390856
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' || cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- 0.6142135623730951
```

- Return the instant of the first temporal circular buffer at which the two arguments are at the nearest distance

`nearestApproachInstant({geo, cbuffer, tcbuffer}, {geo, cbuffer, tcbuffer}) → tcbuffer`

```
SELECT nearestApproachInstant (tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
    Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)');
-- Cbuffer(Point(2 2),0.349928)@2001-01-01 02:59:44.402905+01
SELECT nearestApproachInstant (tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
    Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- Cbuffer(Point(2 2),0.592181)@2001-01-01 17:31:51.080405+01
```

- Return the line connecting the nearest approach point between the two arguments

```
shortestLine({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → geometry
```

The function will only return the first line that it finds if there are more than one

```
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
    Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(50 50,2.212132034355964 2.212132034355964)
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
    Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)'));
-- LINESTRING(1.353553390593274 1.353553390593274,1.787867965644036 1.787867965644036)
```

- Return the temporal distance

```
{geo,cbuffer,tcbuffer} <-> {geo,cbuffer,tcbuffer} → tfloat
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- [2.34988300875063@2001-01-02 00:00:00+01, 2.34988300875063@2001-01-03 00:00:00+01]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)';
-- [25.0496666945044@2001-01-01 00:00:00+01, 26.4085688426232@2001-01-03 00:00:00+01]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-03]' <->
    tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-04]';
-- [2.34988300875063@2001-01-02 00:00:00+01, 2.34988300875063@2001-01-03 00:00:00+01]
```

- Return the minimum spatial distance, ignoring time

```
minDistance(tcbuffer,{geo,cbuffer}) → float
```

```
minDistance(tcbuffer,tcbuffer) → float (aggregate)
```

The result is the spatial distance between the areas swept by the two arguments over their whole lifespans, equal to `ST_Distance(traversedArea(...))`. The two-argument temporal-circular-buffer form is an aggregate so that a cross join grouped by a key yields the minimum over every pair in the group.

```
SELECT minDistance(tcbuffer '[Cbuffer(Point(0 0), 0.5)@2001-01-01,
    Cbuffer(Point(10 0), 0.5)@2001-01-02]', geometry 'Point(5 5)');
-- 4.5
SELECT minDistance(tcbuffer '[Cbuffer(Point(0 0), 0.5)@2001-01-01,
    Cbuffer(Point(10 0), 0.5)@2001-01-02]', cbuffer 'Cbuffer(Point(5 5), 1)');
-- 3.5
SELECT g, minDistance(t1.Noise, t2.Noise)
FROM Vessel v1, VesselTrip t1, Vessel v2, VesselTrip t2
WHERE v1.MMSI = t1.MMSI AND v2.MMSI = t2.MMSI
GROUP BY t1.ShipTypeLabel, t2.ShipTypeLabel;
```

13.12 Restrictions

- Restrict to (the complement of) a value or a set of values

```
atValue(tcbuffer,value) → tcbuffer
```

```
minusValue(tcbuffer,value) → tcbuffer
atValues(tcbuffer,valueSet) → tcbuffer
minusValues(tcbuffer,valueSet) → tcbuffer
```

```
SELECT asText(atValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
-- {[Cbuffer(Point(2 2),0.5)@2001-01-02]}
SELECT asText(minusValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
/* {[Cbuffer(Point(2 2),0.3)@2001-01-01, Cbuffer(Point(2 2),0.5)@2001-01-02),
(Cbuffer(Point(2 2),0.5)@2001-01-02, Cbuffer(Point(2 2),0.7)@2001-01-03]} */
```

- Restrict to (the complement of) a geometry

```
atGeometry(tcbuffer,geometry) → tcbuffer
minusGeometry(tcbuffer,geometry) → tcbuffer
```

```
SELECT asText(atGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0,0 2,2 2,2 0,0 0))'));
-- {[Cbuffer(Point(2 2),0.3)@2001-01-01, Cbuffer(Point(2 2),0.7)@2001-01-03]}
SELECT asText(minusGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0,0 2,2 2,2 0,0 0))'));
/* {[Cbuffer(Point(2 2),0.342593)@2001-01-01 05:06:40.364673+01,
Cbuffer(Point(2 2),0.7)@2001-01-03 00:00:00+01]} */
```

- Restrict a temporal circular buffer to (the complement of) a spatiotemporal box

```
atStbox(tcbuffer,stbox,borderInc bool=true) → tcbuffer
minusStbox(tcbuffer,stbox,borderInc bool=true) → tcbuffer
```

```
SELECT asText(atStbox(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2000-01-01,
Cbuffer(Point(3 3), 0.5)@2000-01-03]', stbox 'STBOX X((0,0),(2,2))'));
-- {[Cbuffer(Point(1 1),0.5)@2000-01-01 00:00:00+01, Cbuffer(Point(2 2),0.5)@2000-01-02 00:00:00+01]} ↔
```

13.13 Comparisons

- Traditional comparisons

```
tcbuffer {=, <>, <, >, <=, >=} tcbuffer → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
Cbuffer(Point(1 1), 0.3)@2001-01-02,
[Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]]' =
tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
Cbuffer(Point(1 1), 0.5)@2001-01-03]]' <>
tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
Cbuffer(Point(1 1), 0.5)@2001-01-03]]' <
tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.6)@2001-01-03]';
-- false
```

- Ever and always comparisons

```
{tcbuffer,cbuffer} {?=, %=} {tcbuffer,cbuffer} → boolean
```

```

SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03]' ?= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03]' ?=
  tcbuffer '[Cbuffer(Point(1 1), 0.4)@2001-01-01, Cbuffer(Point(1 1), 0.2)@2001-01-03]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-04]' %= cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- true

```

- Temporal comparisons

tcbuffer {#=, #<>} tcbuffer → tbool

```

SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03]' #= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.8)@2001-01-03]' #<>
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.7)@2001-01-03]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}

```

13.14 Bounding Box Operations

- Topological operators

{tstzspan, stbox, tcbuffer} {&&, <@, @>, ~=, -|} {tstzspan, stbox, tcbuffer} → boolean

```

SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' && cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' @> stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.5)::geometry <@
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' ~= tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.35)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true

```

- Position operators

{stbox, tcbuffer} {<<, &<, >>, &>} {stbox, tcbuffer} → boolean

{stbox, tcbuffer} {<<|, &<|, |>>, |&>} {stbox, tcbuffer} → boolean

{tstzspan, stbox, tcbuffer} {<<#, &<#, #>>, #&>} {tstzspan, stbox, tcbuffer} → boolean

```

SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' << cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' <<| stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' &> cbuffer 'Cbuffer(Point(1 1), 0.3)::geometry';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,

```

```
Cbuffer(Point(1 1), 0.5)@2001-01-02]' >>#
tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03, Cbuffer(Point(1 1), 0.5)@2001-01-05]'
-- true
```

13.15 Spatial Relationships

- The *ever* and *always* relationships determine whether the topological or distance relationship is ever or always satisfied (see Section 5.4.2) and return a boolean. Examples are the `eIntersects` and `aIntersects` functions.

```
eContains(geometry,tcbuffer) → boolean
aContains(geometry,tcbuffer) → boolean
eCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
aDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
eIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
```

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)'];
-- false
SELECT eDisjoint(cbuffer 'Cbuffer(Point(2 2), 0.0)',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)'];
-- true
SELECT eIntersects(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03]',
  tcbuffer '[Cbuffer(Point(2 2), 0.0)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-03)'];
-- false
```

- The *temporal* relationships compute the topological or distance relationship at each instant and result in a `tbool`. Examples are the `tIntersects` and `tDwithin` functions.

```
tContains(geometry,tcbuffer) → boolean
tDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
tDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
tIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
tTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
```

```
SELECT tDisjoint(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)'];
-- {[t@2001-01-01 00:00:00+01, t@2001-01-03 00:00:00+01]}
SELECT tDwithin(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]', tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03]', 1);
/* {[t@2001-01-01 00:00:00+01, t@2001-01-01 22:35:55.379053+01],
  (f@2001-01-01 22:35:55.379053+01, t@2001-01-02 01:24:04.620946+01,
  t@2001-01-03 00:00:00+01)} */
```

13.16 Aggregations

The three aggregate functions for temporal circular buffers are illustrated next.

- Temporal count

`tCount(tcbuffer) → {tintSeq,tintSeqSet}`

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}
```

- Window count

`wCount(tcbuffer) → {tintSeq,tintSeqSet}`

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05]' )
SELECT wCount(Temp, '1 day')
FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
  1@2001-01-06]} */
```

13.17 Simplification

- Return a temporal circular buffer simplified ensuring that consecutive centers are at least a certain distance or time interval apart

`minDistSimplify(tcbuffer,mindist float) → tcbuffer`

`minTimeDeltaSimplify(tcbuffer,mint interval) → tcbuffer`

The simplification operates on the center trajectory of the temporal circular buffer and preserves the radius at each surviving instant. Simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given value is returned.

- Return a temporal circular buffer simplified using the [Douglas-Peucker algorithm](#)

`maxDistSimplify(tcbuffer,maxdist float,syncdist=true) → tcbuffer`

`douglasPeuckerSimplify(tcbuffer,maxdist float,syncdist=true) → tcbuffer`

The difference between the two functions is that `maxDistSimplify` uses a single-pass version of the algorithm whereas `douglasPeuckerSimplify` uses the standard recursive algorithm. As for the functions above, the simplification operates on the center trajectory and preserves the radius at each surviving instant.

```
SELECT asText(douglasPeuckerSimplify(tcbuffer '[Cbuffer(Point(1 1),0.5)@2000-01-01,
  Cbuffer(Point(2 2),0.5)@2000-01-02, Cbuffer(Point(3 1),0.5)@2000-01-03,
  Cbuffer(Point(4 4),0.5)@2000-01-04]', 2.0));
-- {Cbuffer(POINT(1 1),0.5)@2000-01-01 00:00:00+01, Cbuffer(POINT(4 4),0.5)@2000-01-04 00:00:00+01}
```

13.18 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal circular buffers. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist (Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal circular buffers, which is an `stbox`, as all spatiotemporal types.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal circular buffers,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal circular buffers,
- `&&`, `@>`, `<@`, `~=`, `-|-`, and `|=|`, which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

Chapter 14

Temporal Rigid Geometries

A temporal rigid geometry (`trgeometry`) is a static 2D reference geometry — typically a polygon describing the shape of a moving body — paired with a temporal pose (`tpose`) describing how that geometry translates and rotates over time. The materialised geometry at time t is the reference geometry rotated and translated according to the pose interpolated at t .

This is the natural representation for moving objects whose *shape* matters, not just their position. The motivating example is AIS maritime traffic, where each ship's hull (a pentagon derived from the antenna offsets A, B, C, D) follows a pose path defined by lat/lon and heading reports. The materialised geometry at any instant is the actual ship outline at that moment.

The `trgeometry` type builds on Chapter 11. Most accessor, comparison, topological, position, and indexing operators have the same shape as the corresponding `tpose` functions; this chapter covers the `trgeometry`-specific surface — pose-to-shape materialisation, traversed area, materialised distance, and spatial restrictions on the centroid trajectory.

Operational guidance on choosing between `trgeometry`, `tpose`, and `tgeometry`; the frame conventions the implementation enforces (2D-only reference shape, single shared reference geometry per row, same-SRID requirement, centroid-based spatial-restriction semantics); the numerical hazards production workloads should anticipate (sampling caveat for `traversedArea` over rotation segments, adaptive sub-sampling tolerance in materialised distance, mixed-SRID rejection); and the durability and storage conventions are collected in Section 14.18.

14.1 Input and Output

A `trgeometry` literal pairs a reference geometry with a temporal pose. The reference geometry comes first, separated from the pose by a semicolon; the pose follows the standard `tpose` syntax (instant, sequence, or sequence set).

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.5)@2001-01-01';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0)@2001-01-01, Pose( ←
  Point(5 0), 0.5)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose( ←
  Point(10 0), 1.5)@2001-01-02]';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{[Pose(Point(0 0), 0.0)@2001-01-01, Pose( ←
  Point(5 0), 0.0)@2001-01-02], [Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(2 0), 0.0) ←
  @2001-01-05]}';
```

The reference geometry can be any 2D polygon or polyhedral surface. The pose interpolation is linear in translation and angular-shortest-path in rotation. SRIDs are inherited from the reference geometry and the pose; they must agree.

An SRID can be specified for a `trgeometry` either at the beginning of the literal or before the reference geometry as shown below.

```
SELECT trgeometry 'SRID=25832;Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.5)@2001 ←
  -01-01';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));SRID=25832;Pose(Point(0 0), 0.5)@2001 ←
  -01-01';
```

Step interpolation can also be specified in the same way as for other temporal types:

```
SELECT trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
```

The standard wire formats are supported.

- Return the Well-Known Text (WKT) representation

```
asText({trgeometry,trgeometry[]} [, maxdecimaldigits int=15]) → text
```

```
SELECT asText(trgeometry 'Polygon((1 1,2 2,3 1,1 1));Pose(Point(1.123456789 1.123456789), 0.5)@2000-01-01', 6);
-- POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1.123457 1.123457),0.5)@Sat Jan 01 00:00:00 2000 PST
```

- Return the Extended Well-Known Text (EWKT) representation, prefixed with the SRID

```
asEWKT({trgeometry,trgeometry[]} [, maxdecimaldigits int=15]) → text
```

```
SELECT asEWKT(trgeometry 'SRID=25832;Polygon((1 1,2 2,3 1,1 1));Pose(Point(1 1),0.5)@2000-01-01');
-- SRID=25832;POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1 1),0.5)@Sat Jan 01 00:00:00 2000 PST
```

- Return the Moving Features JSON representation

```
asMFJSON(trgeometry [, options int=0 [, flags int=0 [, maxdecimaldigits int=15]]) → text
```

```
SELECT asMFJSON(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0),0)@2001-01-01');
```

- Return the Extended Well-Known Binary (EWKB) representation

```
asEWKB(trgeometry [, endian text]) → bytea
```

Companion `asHexEWKB` returns the same bytes as a hex-encoded text.

- Each output format has a companion parser:

```
trgeometryFromText(text) → trgeometry
```

```
trgeometryFromEWKT(text) → trgeometry
```

```
trgeometryFromMFJSON(text) → trgeometry
```

```
trgeometryFromEWKB(bytea) → trgeometry
```

```
trgeometryFromHexEWKB(text) → trgeometry
```

14.2 Constructors

A `trgeometry` can be built directly from its component types — a reference geometry plus a temporal pose:

- Construct a temporal rigid geometry from a reference geometry and a temporal pose

```
trgeometry(geometry, tpose) → trgeometry
```

```
SELECT trgeometry(
  geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
  tpose 'Pose(Point(0 0), 0.0)@2001-01-01';
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(0 0),0)@Mon Jan 01 00:00:00 2001 PST
```

Both arguments must share the same SRID. The reference geometry may be a polygon or a polyhedral surface; the pose may be of any temporal subtype.

- Append a single instant to an existing trgeometry, growing the underlying pose path

`appendInstant(trgeometry, trgeometry [, maxdist float [, maxt interval]]) → trgeometry`

```
SELECT appendInstant (
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 1,0 0));Pose(Point(5 0), 0.0)@2001-01-02');
```

- Append an entire sequence to an existing trgeometry

`appendSequence(trgeometry, trgeometry) → trgeometry`

14.3 Conversions

The `trgeometry` can be projected to its component types or to a pure-point trajectory.

- Return the underlying temporal pose

`tpose(trgeometry) → tpose`

```
SELECT asText(tpose(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.5)@2001-01-02]'));
-- [Pose(POINT(0 0),0)@..., Pose(POINT(10 0),0.5)@...]
```

- Return the centroid (antenna) trajectory as a temporal point

`tgeompoint(trgeometry) → tgeompoint`

```
SELECT asText(tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- [POINT(0 0)@..., POINT(10 0)@...]
```

14.4 Accessors

The standard temporal-type accessors apply: each returns the materialised geometry at the requested timestamp (the reference shape rotated and translated according to the interpolated pose), or a metadata value derived from the pose path.

- Return the materialised geometry at the start, end, or a chosen timestamp

`startValue(trgeometry) → geometry`

`endValue(trgeometry) → geometry`

`valueAtTimestamp(trgeometry, timestampt) → geometry`

```
SELECT asText(startValue(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 1,0 0))

SELECT asText(valueAtTimestamp(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  timestampt '2001-01-01 12:00:00'));
-- POLYGON((5 0,6 0,6 1,5 1,5 0))
```

Note that the rotation interpolates linearly along the angular shortest path; a 90° rotation between two instants placed one day apart returns a 45°-rotated polygon at the midpoint.

- Return the number of distinct instants, sequences, or timestamps

`numInstants(trgeometry) → integer`

`numSequences(trgeometry) → integer`

`numTimestamps(trgeometry) → integer`

- Return the array of constituent instants, sequences, or inter-instant segments

`instants(trgeometry) → trgeometry[]`

`sequences(trgeometry) → trgeometry[]`

`segments(trgeometry) → trgeometry[]`

```
SELECT array_length(
  instants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0)@2001-01-01, ←
    Pose(Point(5 0), 0.5)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}'),
  1);
-- 3
```

- Return the set of distinct centroid (antenna) points seen along the trgeometry

`points(trgeometry) → geomset`

```
SELECT asText(points(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0) ←
  @2001-01-01, Pose(Point(5 5), 0.0)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}'));
-- {"POINT(0 0)", "POINT(5 5)"}
```

- Return the rotation angle as a temporal float

`rotation(trgeometry) → tfloat`

```
SELECT asText(rotation(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0) ←
  @2001-01-01, Pose(Point(0 0), 1.5)@2001-01-02]'));
-- [0@..., 1.5@...]
```

- Return the static reference geometry

`geom(trgeometry) → geometry`

```
SELECT ST_AsText(geom(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.5) ←
  @2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))
```

14.5 Traversed Area

The `traversedArea` function returns the union of every materialised geometry over the trgeometry's time domain — the spatial footprint of the moving body.

- Return the union of the materialised polygons across the trgeometry's time domain

`traversedArea(trgeometry [, unary_union boolean = true]) → geometry`

```
SELECT ST_AsText(traversedArea(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point ←
    (10 0), 0.0)@2001-01-02]'));
-- POLYGON((0 0, 11 0, 11 1, 0 1, 0 0))
```

With `unary_union = false` the function returns a `GeometryCollection` of the per-instant materialised polygons without dissolving them — useful when the caller wants to do its own post-processing.

Sampling caveat: the implementation samples at the input instants, so pure-translation segments are exact but the swept ribbon between two instants where a rotation happens is approximated by the convex hull of the two endpoint polygons. Adding intermediate sub-samples is a documented future refinement.

14.6 Spatial Functions

These functions derive a spatial value from the moving body. `centroid` and `bodyPointTrajectory` return temporal points tracking a position over time; `convexHull` returns a static geometry.

- Return the centroid of the moving body as a temporal point

`centroid(trgeometry) → tgeompoint`

```
SELECT asText(centroid(
  trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(↔
    Point(2 0),0)@2001-01-02]')');
-- Interp=Step;[POINT(0.5 0.5)@2001-01-01, POINT(2.5 0.5)@2001-01-02]
```

- Return the convex hull of the area traversed by the moving body

`convexHull(trgeometry) → geometry`

```
SELECT ST_GeometryType(convexHull(
  trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(↔
    Point(2 0),0)@2001-01-02]')');
-- ST_Polygon
```

- Return the world-frame trajectory of a body-frame point carried by the moving rigid geometry. The point is expressed in the reference shape's local frame and the time-varying pose is applied to it at each instant.

`bodyPointTrajectory(trgeometry, geometry) → tgeompoint`

```
SELECT asText(bodyPointTrajectory(
  trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(↔
    Point(2 0),0)@2001-01-02]',
  geometry 'Point(0 0)');
-- Interp=Step;[POINT(0 0)@2001-01-01, POINT(2 0)@2001-01-02]
```

14.7 Motion Metrics

These functions report the motion of a temporal rigid geometry along its centroid trajectory.

- Return the total length traversed by the centroid, or its cumulative length over time

`length(trgeometry) → float`

`cumulativeLength(trgeometry) → tfloat`

```
SELECT length(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01, ↔
  Pose(Point(3 4),0)@2001-01-02]');
-- 5
```

- Return the speed of the centroid as a temporal float

`speed(trgeometry) → tfloat`

- Return the time-weighted centroid of the centroid trajectory as a geometry

`twCentroid(trgeometry) → geometry`

```
SELECT ST_AsText(twCentroid(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0) ↔
  @2001-01-01, Pose(Point(3 4),0)@2001-01-02]')');
-- POINT(1.5 2)
```

14.8 Modifications

The standard sequence-modifying functions apply: `round`, `setInterp`, `shiftTime`, `scaleTime`, `shiftScaleTime`, `deleteTimestamp`, etc.

- Round all numeric components to a given number of decimal places

`round(trgeometry, integer) → trgeometry`

```
SELECT asText(round(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(1.123456789 1.123456789), ↵
    0.123456)@2001-01-01',
  3));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1.123 1.123),0.123)@...
```

- Convert between linear and step interpolations

`setInterp(trgeometry, text) → trgeometry`

- Shift, scale, or both shift and scale the time domain of a trgeometry

`shiftTime(trgeometry, interval) → trgeometry`

`scaleTime(trgeometry, interval) → trgeometry`

`shiftScaleTime(trgeometry, interval, interval) → trgeometry`

14.9 Spatial Restrictions

The `atGeometry`, `minusGeometry`, `atStbox`, and `minusStbox` functions restrict a trgeometry to (the complement of) a geometry or a spatiotemporal box. The semantic mirrors tpose: a temporal rigid geometry is "in" a region at time `t` iff its centroid (antenna) at `t` lies in that region.

- Restrict a trgeometry to (the complement of) a geometry

`atGeometry(trgeometry, geometry) → trgeometry`

`minusGeometry(trgeometry, geometry) → trgeometry`

```
SELECT temporalTime(atGeometry(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point ↵
    (4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))');
-- {[2001-01-02, 2001-01-04]}

SELECT temporalTime(minusGeometry(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point ↵
    (4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))');
-- {[2001-01-01, 2001-01-02], (2001-01-04, 2001-01-05]}
```

Restriction against an empty geometry returns NULL for `atGeometry` and the original input for `minusGeometry`.

- Restrict a trgeometry to (the complement of) a spatiotemporal box

`atStbox(trgeometry, stbox [, border_inc boolean = true]) → trgeometry`

`minusStbox(trgeometry, stbox [, border_inc boolean = true]) → trgeometry`

```
SELECT temporalTime(atStbox(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point ↵
    (4 0), 0.0)@2001-01-05]',
  stbox 'STBOX X((1, -1), (3, 1))');
-- {[2001-01-02, 2001-01-04]}
```

```
SELECT temporalTime(atStbox(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point ↵
    (4 0), 0.0)@2001-01-05]',
  stbox 'STBOX T([2001-01-02, 2001-01-04])');
-- {[2001-01-02, 2001-01-04]}
```

If the STBox has only a temporal component the call collapses to the standard `atTime`; if it has only a spatial component the temporal domain is preserved.

14.10 Distance Operations

Distance between a `trgeometry` and a `geometry` / `pose` / `spatiotemporal box` / another `trgeometry` is computed on the *materialised* `geometry` — the reference shape rotated and translated according to the `pose`. Distance is exact at every shared timestamp; for sequence inputs an adaptive recursive bisection emits intermediate marks where the distance trajectory deviates from a straight line.

- Return the temporal distance between two `trgeometries`

```
{geometry,trgeometry} <-> {geometry,trgeometry} → tfloat
tDistance({geometry,trgeometry}, {geometry,trgeometry}) → tfloat
```

```
SELECT round(maxValue(tDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.0)@2001-01-01',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.0)@2001-01-01')), 6);
-- 4

SELECT asText(tDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point ↵
    (0 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(2 0), 0.0)@2001-01-01, Pose(Point ↵
    (10 0), 0.0)@2001-01-02]'));
-- [1@2001-01-01 ..., 9@2001-01-02 ...]
```

The dispatcher covers every subtype combination — `TINSTANT` × `TINSTANT`, `TINSTANT` × `TSEQUENCE` asymmetric, `TSEQUENCE` × `TSEQUENCE`, `TSEQUENCE` × `TSEQUENCESET`, `TSEQUENCESET` × `TSEQUENCESET`. The adaptive sub-sampling kernel underlies every continuous combo; instant combos materialise once. Pure-translation segments converge at depth 0 (two endpoint marks); rotation-heavy segments emit up to 32 marks per inter-instant gap. The bound on the `LINEAR`-interpolated approximation between marks is $1e-3$ by construction.

- Return the smallest distance between two `trgeometries` over their shared time domain

```
{geometry,trgeometry,stbox} |=| {geometry,trgeometry,stbox} → float
nearestApproachDistance({geometry,trgeometry,stbox}, {geometry,trgeometry,stbox}) → float
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose( ↵
  Point(10 0), 0.0)@2001-01-02]'
  |=|
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(50 0), 0.0)@2001-01-01, Pose ↵
    (Point(50 0), 0.0)@2001-01-02]';
-- 39
```

The `|=|` operator can be used for nearest-neighbour searches against GiST or SP-GiST indexes (see Section 14.17).

- Return the instant at which the two arguments are at their smallest mutual distance

```
nearestApproachInstant({geometry,trgeometry}, {geometry,trgeometry}) → trgeometry
```

```
SELECT asText(nearestApproachInstant(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  geometry 'Point(5 5)'));
-- POLYGON((5 0,6 0,6 1,5 1,5 0));Pose(POINT(5 0),0)@2001-01-01 12:00:00+00
```

- Return the line connecting the two arguments at their nearest approach

```
shortestLine({geometry,trgeometry}, {geometry,trgeometry}) → geometry
```

```
SELECT ST_AsText(shortestLine(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.0)@2001-01-01',
  geometry 'Point(10 0)'));
-- LINESTRING(1 0,10 0)
```

14.11 Similarity

The similarity functions compare two temporal rigid geometries through their centroid trajectories.

- Return the discrete Hausdorff distance, the discrete Fréchet distance, or the Dynamic Time Warp distance between two temporal rigid geometries

```
hausdorffDistance(trgeometry, trgeometry) → float
```

```
frechetDistance(trgeometry, trgeometry) → float
```

```
dynTimeWarpDistance(trgeometry, trgeometry) → float
```

```
SELECT round(frechetDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(2 0),0)@2000-01-03]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(0 2),0)@2000-01-03]'), 6);
-- 2.828427
SELECT round(hausdorffDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(2 0),0)@2000-01-03]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(0 2),0)@2000-01-03]'), 6);
-- 2
```

- Return the correspondence pairs between two temporal rigid geometries with respect to the discrete Fréchet distance or the Dynamic Time Warp distance, as a set of (i, j) instant-index pairs

```
frechetDistancePath(trgeometry, trgeometry) → {(i, j)}
```

```
dynTimeWarpPath(trgeometry, trgeometry) → {(i, j)}
```

14.12 Comparisons

Equality and ever / always operators compare two trgeometries on the materialised geometry — both the reference shape and the pose path must agree at every shared instant.

- Test exact (in)equality of two trgeometries

```
trgeometry = trgeometry → boolean
```

```
trgeometry <> trgeometry → boolean
```


- Test whether the materialised geometry is ever (not) equal to the operand at any time

```
{geometry,trgeometry} ?= {geometry,trgeometry} → boolean
{geometry,trgeometry} ?<> {geometry,trgeometry} → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(↔
    Point(5 0), 0.0)@2001-01-02]';
    ?= geometry 'Polygon((5 0,6 0,6 1,5 1,5 0))';
-- t
```

- Test whether the materialised geometry is always (not) equal to the operand

```
{geometry,trgeometry} %= {geometry,trgeometry} → boolean
{geometry,trgeometry} %<> {geometry,trgeometry} → boolean
```

14.13 Bounding-Box Operators

Topological and position operators compare the spatiotemporal bounding boxes of the operands; both can be a `trgeometry`, an `stbox`, a `geometry`, or a `tstzspan`.

- Standard topological operators on bounding boxes

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(↔
    Point(10 0), 0.0)@2001-01-02]';
    &&
    stbox 'STBOX X((5, -1), (15, 1))';
-- t
```

- Spatial and temporal position operators

The full set `<<`, `>>`, `&<`, `&>`, `<<|`, `&<|`, `|>>`, `|&>`, `<<#`, `&<#`, `#>>`, `#&>` all accept `trgeometry` on either side.

14.14 Tiles

These functions return the spatiotemporal boxes of a multidimensional grid that the temporal rigid geometry intersects, computed on its centroid trajectory.

- Return the spatiotemporal boxes of a temporal rigid geometry split with respect to a spatial, temporal, or spatiotemporal grid

```
spaceBoxes(trgeometry, xsize float [, ysize float [, zsize float]], sorigin geometry
= 'Point(0 0 0)', bitmatrix bool = true, borderInc bool = true) → stbox[]
```

```
timeBoxes(trgeometry, interval, toorigin timestampz = '2000-01-03', bitmatrix bool =
true, borderInc bool = true) → stbox[]
```

```
spaceTimeBoxes(trgeometry, xsize float [, ysize float [, zsize float]], interval, sorigin
geometry = 'Point(0 0 0)', toorigin timestampz = '2000-01-03', bitmatrix bool = true,
borderInc bool = true) → stbox[]
```

```
SELECT array_length(spaceBoxes(
    trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 ↔
        0),0)@2001-01-02]',
    2.0), 1);
-- 6
```

14.15 Bounding Boxes

These functions decompose a temporal rigid geometry into the bounding boxes or time spans of its segments or of a requested number of bins.

- Return the array of spatiotemporal boxes or time spans of the segments of a temporal rigid geometry
`stboxes(trgeometry) → stbox[]`
`spans(trgeometry) → tstzspan[]`
- Return the spatiotemporal boxes or time spans of a temporal rigid geometry split into a given number of bins, or with a given number of segments per bin
`splitNStboxes(trgeometry, integer) → stbox[]`
`splitEachNStboxes(trgeometry, integer) → stbox[]`
`splitNSpans(trgeometry, integer) → tstzspan[]`
`splitEachNSpans(trgeometry, integer) → tstzspan[]`

14.16 Aggregations

- Number of overlapping trgeometries at each instant — equivalent to `tcount` on the underlying tpose
`tcount(trgeometry) → tint`
- Aggregate bounding box of a column of trgeometries
`extent(trgeometry) → stbox`

```
SELECT extent(temp) FROM tbl_trgeometry;
```

- Combine two trgeometries with the same reference geometry into a single sequence-set
`merge(trgeometry, trgeometry) → trgeometry`
`mergeAgg(trgeometry) → trgeometry`

```
SELECT asText(merge(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(5 0), 0.0)@2001-01-05]')');
```

14.17 Indexing

Three index types are available on `trgeometry` columns. Each uses the spatiotemporal bounding box of the materialised geometry and supports topological, position, and KNN distance queries.

- `GIST(temp)` — R-tree GiST.
- `SPGIST(temp trgeometry_quadtree_ops)` — quad-tree SP-GiST.
- `SPGIST(temp trgeometry_kdtree_ops)` — kd-tree SP-GiST.

```
CREATE INDEX tbl_trgeometry_gist_idx
  ON tbl_trgeometry
  USING GIST(temp);

-- topological lookup
SELECT count(*) FROM tbl_trgeometry
WHERE temp && stbox 'STBOX XT((0, 0), (10, 10)),[2001-01-01, 2001-01-02]';

-- KNN (ordered scan)
SELECT mmsi
FROM tbl_trgeometry
ORDER BY temp <-> trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(100 100), 0.0)@2001 <->
    -01-01'
LIMIT 10;
```

The kd-tree variant typically wins on workloads dominated by KNN queries with a tight ordering bound; the quad-tree variant is more uniform. The R-tree GiST is the safe default for mixed read/write workloads.

14.18 Production guidance

This section captures the operational knowledge production workloads need on top of the function reference: when `trgeometry` is the right type relative to its neighbours (`tpose` and `tgeometry`), the frame conventions the implementation enforces, and the numerical hazards real rigid-body workloads hit. Most of these are surfaced separately throughout the chapter; the goal here is one place a new contributor can read end-to-end before instrumenting an application.

14.18.1 When to use `trgeometry` vs `tpose` vs `tgeometry`

Three temporal types span the rigid-body / arbitrary-shape design space. Pick the narrowest type that fits the workload — narrower types pay less storage and produce sharper bounding boxes for the spatial index.

- `tpose` — temporal position and orientation only; the body is treated as a point with a heading. Use when the shape is irrelevant to the query (kinematic tracking, heading-only collision proxies).
- `trgeometry` — a single static reference shape that translates and rotates over time per a temporal pose. Use when the shape *matters* but is rigid: ships, vehicles, drones with fixed geometry, body-conforming exclusion zones. The reference geometry is stored once per row, not per instant, so storage is dominated by the underlying `tpose`.
- `tgeometry` — temporal arbitrary geometry whose shape itself varies over time. Use when the shape is non-rigid: storm wind swaths, expanding plumes, evolving polygons.

A `trgeometry` can be projected to its underlying `tpose` (centroid trajectory) or to a fully-materialised `tgeometry` via the conversions in Section 14.3. The reverse direction (lifting a `tgeometry` back to a `trgeometry`) is not generally well-defined since arbitrary shape changes do not factor cleanly into rotation + translation of a fixed reference.

14.18.2 Frame conventions

- *2D-only reference geometry.* The reference shape is a 2D polygon or polyhedral surface; the pose's rotation component is the 2D yaw angle from Chapter 11. There is no 3D rigid-body representation in this type — workloads needing full 3D orientation (quaternion) on a 3D shape should compose `tpose` with the materialisation step explicitly.
- *Single shared reference geometry.* Every instant of a `trgeometry` shares the same reference shape; only the pose varies. The shape is stored once per row, not per instant.
- *Same-SRID requirement.* The reference geometry and the temporal pose must share an SRID; the constructor rejects mismatches with the standard MEOS `ensure_same_srid` error. The materialised geometry inherits this SRID.

- *Centroid-based spatial-restriction semantics.* `atGeometry`, `minusGeometry`, `atStbox`, and `minusStbox` test the centroid of the materialised shape against the region — not the materialised shape itself. A `trgeometry` whose centroid lies outside a region but whose body extends into the region is *excluded*. This mirrors the underlying `tpose` semantics (a pose is a point) and is a deliberate design choice; workloads needing whole-shape containment must materialise to `tgeometry` first.

14.18.3 Numerical hazards

Four hazards reliably bite real rigid-body workloads. The implementation mitigates each as follows:

- *Mixed-SRID inputs.* Constructing a `trgeometry` from a reference geometry and a `tpose` in different SRIDs would silently produce wrong materialised positions if the SRIDs were not checked.

Mitigation: the constructor and every cross-type operator route through `ensure_same_srid`, which raises an error naming both SRIDs. The error path is loud and explicit; there is no implicit transform.

- *traversedArea sampling caveat.* The `traversedArea` function samples at the input instants and connects them with the convex hull of the two endpoint polygons. Pure-translation segments are exact, but inter-instant rotations are approximated — for a 90° rotation between two instants, the swept ribbon is wider than the convex hull captures.

Mitigation: documented in Section 14.5. Workloads needing tight swept-area bounds must up-sample the input `tpose` before constructing the `trgeometry`; the `setInterp` + sub-instant insertion pattern from the `tpose` chapter is the standard approach. Adaptive sub-sampling within `traversedArea` is a documented future refinement.

- *Adaptive sub-sampling tolerance in materialised distance.* Continuous-distance kernels (`tDistance`, `nearestApproachDistance`) materialise the rigid shape per inter-instant gap and emit intermediate marks where the distance trajectory deviates from a straight line. The depth-cap is 32 marks per gap and the LINEAR-interpolated approximation between marks is bounded by $1e-3$ (in CRS units).

Mitigation: the $1e-3$ bound is a documented design choice. Pure-translation segments converge at depth 0; rotation-heavy segments use the full 32-mark budget. Workloads that need sharper precision on a specific operator should pre-process the input poses to denser instants rather than rely on tighter recursion.

- *Centroid-based spatial restriction surprise.* `atGeometry(trgeometry, geometry)` tests the pose centroid against the region, not the materialised shape. A long ship whose stern is in a harbour but whose bow has crossed the harbour boundary will be reported as inside the harbour as long as its centroid is inside.

Mitigation: documented as a tier-D design choice in Section 14.9. Workloads needing whole-shape semantics must convert via `trgeometry::tgeometry` before applying the spatial restriction. The conversion materialises one polygon per instant and is correspondingly more expensive.

14.18.4 Durability and storage

The on-disk representation of a `trgeometry` is the reference `GSERIALIZED` followed by the temporal-pose payload. The byte layout is stable: `asBinary(trgeometry) / trgeometryFromBinary(bytea)` round-trip preserves the value bit-for-bit (including the reference shape's SRID, Z and M flags, and the underlying `tpose` subtype), and the WKB / EWKB / HexEWKB serialisations follow the standard PostGIS endian-flag-then-payload pattern.

The `asMFJSON(trgeometry) / trgeometryFromMFJSON(text)` pair is bidirectional. Each instant renders as `{"geometry": ...}`; the reference geometry uses the standard PostGIS GeoJSON shape and the pose payload follows the OGC GeoPose Basic-YPR conformance class layout (`{"position": ..., "quaternion": ...}` for 3D / `{"position": ..., "rotation": ...}` for 2D). The reference geometry is emitted once per instant in the current schema; future schema work could elevate it to a chapter-level header to avoid the per-instant repetition for long sequences.

The `pg_dump` output of a table containing `trgeometry` columns uses the WKT representation by default; `pg_dump --binary-upgrade` uses the WKB representation. Both are round-trip-stable across PostgreSQL major-version upgrades.

Chapter 15

JSON Types in MobilityDB

PostgreSQL provides two types for storing JSON data: `json` and `jsonb`. The `json` type stores an exact copy of the input text, which must be reparsed each time it is processed. On the other hand, the `jsonb` type stores the JSON data in a decomposed binary format that makes it slightly slower to input due to added conversion overhead, but significantly faster to process, since no reparsing is needed. The `jsonb` type also supports indexing.

In MobilityDB, the `textset` and the `ttext` types are used for representing, respectively, set of JSON values and temporal JSON values. Furthermore, the `jsonb` type serves as base type for defining the `jsonbset` and the `tjsonb` types. Most functions and operators described in the previous chapters for set and temporal types are also applicable for the corresponding JSON types. In addition, there are specific functions defined for these types, which are derived from the corresponding functions of the `json` and the `jsonb` types.

In this chapter, we describe the MobilityDB JSON types and its associated operations. We refer to the PostgreSQL [documentation](#) for a detailed explanation of the `json` and `jsonb` types and their functionality. We aimed at enabling the same syntax of the original PostgreSQL functions for the corresponding MobilityDB types, as illustrated in this [document](#).

Consider for example the `->` operator, which extracts a JSON object field with a given key.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Applying the same operator to a JSONB set and to a temporal JSONB value yields the following results

```
SELECT jsonbset '{"{"unit": "km", "speed": 10}',
  '{"unit": "km", "speed": 20}]' -> text 'speed';
-- {"10", "20"}
SELECT tjsonb '["unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]' -> text 'speed';
-- {[10@2001-01-01, 20@2001-01-02]}
```

which are obtained by applying the PostgreSQL operator `->` at every element of the JSONB set and to every instant of the temporal JSONB value.

15.1 JSONB Set Operations

When manipulating JSON collections of varying structure, it may be the case that an item is defined in some of the documents but not all. PostgreSQL provides the *lax mode* for this purpose, where the argument `null_value_treatment` determines the behavior in the case a function returns a NULL value. The argument can take one of the following values: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, where `'use_json_null'` is the default value. In PostgreSQL the lax mode is supported only for *update* (not *query*) operations with the function `jsonb_set_lax`. In MobilityDB, we kept the same semantics for the corresponding function `jsonbset_set_lax`, but we enabled similar behavior for *all* JSON operations that may return a null value, except that we replaced the value `'return_target'` with `'return_null'`,

since the former is not meaningful for set operations. For operators such as $\rightarrow$, the default value 'use_json_null' is used and cannot be changed, whereas for the corresponding functions `jsonbset_object_field`, the last argument specifies the behavior in the case the function returns NULL. We illustrate this behavior for function `jsonbset_object_field` below.

- Extract a JSON object field specified by a key

```
{textset,jsonbset} -> text → {textset,jsonbset}
jsonbset ->> text → textset
jsonbset_object_field(jsonbset,text,null_handle text='use_json_null') → jsonbset
jsonbset_object_field_text(jsonbset,text,null_handle text='use_json_null') → textset
```

```
SELECT textset '[[{"unit": "km", "speed": 10},
 {"unit": "km", "speed": 20}]]' -> text 'speed';
-- [{"10", "20"}]
SELECT jsonbset ' [{"position":"Point(1 1)", "speed":10},
 {"position":"Point(2 2)", "speed":20}]]' ->> text 'position';
-- [{"Point(1 1)", "Point(2 2)"}]
SELECT textset '[[{"unit": "km", "speed": 10},
 {"unit": "km"}, {"unit": "km", "speed": 10}]]' -> text 'speed';
-- [{"10", null, 10}]
```

We show below the behavior of the function `jsonbset_object_field` for the possible values of the `null_handle` argument. All JSON functions that may return a NULL value behave similarly in MobilityDB.

```
SELECT jsonbset_object_field(textset '[[{"unit": "km", "speed": 10},
 {"unit": "km"}, {"unit": "km", "speed": 10}]]', 'speed',
 'raise_exception');
-- ERROR: The lifted operation returned NULL
SELECT jsonbset_object_field(textset '[[{"unit": "km", "speed": 10},
 {"unit": "km"}, {"unit": "km", "speed": 10}]]', 'speed',
 'use_json_null');
-- [{"10", null, 10}]
SELECT jsonbset_object_field(textset '[[{"unit": "km", "speed": 10},
 {"unit": "km"}, {"unit": "km", "speed": 10}]]', 'speed',
 'delete_key');
-- [{"10", 10}, [10]]
SELECT jsonbset_object_field(textset '[[{"unit": "km", "speed": 10},
 {"unit": "km"}, {"unit": "km", "speed": 10}]]', 'speed',
 'return_null');
-- NULL
```

- Extract a JSON object field specified by a path

```
{textset,jsonbset} #> text[] → {textset,jsonbset}
jsonbset_extract_path(jsonbset,text[],null_handle text='use_json_null') → jsonbset
jsonbset_extract_path_text(jsonbset,text[],null_handle text='use_json_null') → textset
```

```
SELECT textset '[{"speed": {"unit": "km", "value": 10}},
 {"speed": {"unit": "km", "value": 20}}]' #> ARRAY[text 'speed', 'value'];
-- [{"10", "20"}]
SELECT jsonbset ' [{"position":"Point(1 1)", "PoIs":["Grand Place", "La Bourse"]},
 {"position":"Point(2 2)", "PoIs":["Palais Royal", "Manneken Pis"]}]]' #>>
ARRAY[text 'PoIs', '1'];
-- [{"La Bourse", "Manneken Pis"]}
```

- Extract an element from a JSON array

```
jsonbset_array_element(jsonbset,integer,null_handle text='use_json_null') → jsonbset
jsonbset_array_element_text(jsonbset,integer,null_handle text='use_json_null') → textset
```

```
SELECT jsonbset_array_element(textset '["Grand Place", "La Bourse"],
    ["Palais Royal", "Manneken Pis"]', 0);
-- ["Grand Place", "Palais Royal"]
SELECT jsonbset_array_element(jsonbset '["Grand Place", "La Bourse"],
    ["Palais Royal", "Manneken Pis"]', 1);
-- ["La Bourse", "Manneken Pis"]
```

- Extract a alphanumeric value from a JSONB set given by a key

```
intset(jsonbset,text,null_handle text='raise_exception') → tint
floatset(jsonbset,text,null_handle text='raise_exception') → tfloat
textset(jsonbset,text,null_handle text='raise_exception') → textset
```

Note that the value `use_json_null` cannot be used for the above functions and thus the default value `'raise_exception'` is used.

```
SELECT intset(jsonbset '{"\speed": 10, \units": \km/h\}',
    '{"speed": 20, \units": \km/h\}', 'speed');
-- {10, 20}
SELECT floatset(jsonbset '{"\speed": 10, \units": \km/h\}',
    '{"speed": 20, \units": \km/h\}', '{"speed": 25}', 'speed');
-- {10, 20, 25}
SELECT textset(jsonbset '{"\road": \Bvd Gén. Jacques\, \category": \primary\}',
    '{"road": \Bvd de la Cambre\, \category": \residential\}', 'category');
-- {primary, residential}
```

- Temporal JSONB concatenation

```
jsonbset || {jsonb,jsonbset} → jsonbset
```

```
SELECT jsonbset '[[{"speed":10}, {"speed":20}]' ||
    '{"unit":"km"}'::jsonb;
-- [{"unit": "km", "speed": 10}, {"unit": "km", "speed": 20}]
SELECT jsonbset '[[{"speed":10}, {"speed":20}]' ||
    jsonbset '[[{"Position":"Point(1 1)", {"Position":"Point(2 2)"}]]';
/* [{"speed": 10, "Position": "Point(1 1)",
    {"speed": 20, "Position": "Point(2 2)"}] */
```

- Temporal JSONB deletion

```
jsonbset - {int,text,text[]} → jsonbset
```

```
jsonbset #- text[] → jsonbset
```

```
SELECT jsonbset '[[{"unit": "km", "speed": 10},
    {"unit": "km", "speed": 20}]]' - 'unit';
-- [{"speed": 10}, {"speed": 20}]
SELECT jsonbset '["Grand Place", "La Bourse"],
    ["Palais Royal", "Manneken Pis"]' - 1;
-- ["Grand Place", ["Palais Royal"]]
SELECT jsonbset '["location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"],
    "location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]]' #-
    ARRAY[text "\PoIs\","1"];
-- ["La Bourse", "Manneken Pis"]
```

As shown below, the result of a delete operation may be an empty record or an empty array. To remove these values, the function `minusValue` can be used.

```
SELECT jsonbset '[[{"unit": "km", "speed": 10, "light": true},
    {"unit": "km", "speed": 20}]]' - ARRAY[text 'unit', 'speed'];
-- [{"light": true}, {}]
SELECT jsonbset '["Grand Place", "La Bourse"],
    ["Palais Royal"]' - 0;
-- ["La Bourse", []]
```

```
SELECT minusValues(jsonbset '[[{"unit": "km", "speed": 10, "light": true},
{"unit": "km", "speed": 20}]]' - ARRAY[text 'unit', 'speed'],
jsonbset '{"[]", "{}"}');
-- [{"light": true}, {"light": true}]
SELECT minusValues(jsonbset '["Grand Place", "La Bourse",
"Palais Royal"]' - 0, jsonbset '{"[]", "{}"}');
-- [{"La Bourse"}, {"La Bourse"}]
```

- Temporal JSONB exists

```
jsonbset ? jsonb → tbool
jsonbset ?| jsonb[] → tbool
jsonbset ?& jsonb[] → tbool
```

As in PostgreSQL, the operators `?|` and `?&` test, respectively, whether *any* or *all* of the strings in the text array exist as top-level keys or array elements.

```
SELECT jsonbset '{"\geom\": \Point(1 1)\"}' ? text 'geom';
-- t
SELECT jsonbset '{"geom": "Point(1 1)", {"geom": "Point(2 2)",
{"geom": "Point(1 1)"}}' ?| ARRAY[text 'geom'];
-- {t, t, t}
SELECT jsonbset ' [{"speed": 10, "units": "km/h"},
{"speed": 20, "units": "km/h"}]' ?& ARRAY['speed', 'units'];
-- {[t, t]}
```

- Temporal JSONB contains/contained

```
{jsonbset, jsonb} @> {jsonbset, jsonb} → tbool
{jsonbset, jsonb} <@ {jsonbset, jsonb} → tbool
```

```
SELECT jsonbset '{"\geom\": \Point(1 1)\"}' @> jsonb '{"geom": "Point(1 1)"}';
-- t
SELECT jsonbset '{"geom": "Point(1 1)", {"geom": "Point(2 2)",
{"geom": "Point(1 1)"}}' @> jsonb '{"geom": "Point(1 1)"}';
-- {t, f, t}
SELECT jsonb '{"geom": "Point(1 1)"}' <@ jsonbset '{" [{"geom": "Point(1 1)",
{"geom": "Point(1 1)", {"geom": "Point(1 1)"},
{"geom": "Point(2 2)", {"geom": "Point(2 2)"}]}]';
-- {[t, t], [f, f]}
```

- Temporal JSONB set

```
jsonbset_set(jsonbset, path text[], jsonb, create boolean=true) → jsonbset
jsonbset_set_lax(jsonbset, path text[], jsonb, create boolean=true, handle_null text) → jsonbset
```

Return the `jsonbset` value with the item specified by `path` replaced by `jsonb`, or added if `create` is true and the item does not exist. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, and `create_if_missing` is true, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive.

Function `jsonbset_set_lax` behaves identically to `jsonbset_set` if the given value is not NULL. Otherwise, it behaves according to the value of `handle_null`, which must be one of `'raise_exception'`, `'use_json_null'`, `'delete_key'`, or `'return_source'`. As stated above, we kept PostgreSQL behavior for this function enabling the value `'return_source'` whereas for all other *query* operations, this value has been replaced with `'return_null'`.

```
SELECT jsonbset_set(jsonbset ' [{"speed":10}, {"speed":20}]',
ARRAY['units'], '"km/h"::jsonb);
-- [{"speed": 10, "units": "km/h"}, {"speed": 20, "units": "km/h"}]
SELECT jsonbset_set(jsonbset ' [{"speed":10}, {"speed":20}]',
```



```

    ARRAY['units'], 'km/h'::jsonb, false);
-- [{"speed": 10}, {"speed": 20}]
SELECT jsonbset_set(jsonbset ' [{"speed": 10, "units": "km/h"},
    {"speed": 20, "units": "km/h"}]', ARRAY['units'], 'mi/h'::jsonb);
-- [{"speed": 10, "units": "mi/h"}, {"speed": 20, "units": "mi/h"}]
SELECT jsonbset_set_lax(jsonbset ' [{"speed": 10, "units": "km/h"},
    {"speed": 20}]', ARRAY['units'], 'null'::jsonb, true, 'delete_key');
-- [{"speed": 10, "units": "km/h"}, {"speed": 20}]

```

- Temporal JSONB insert

`jsonbset_insert(jsonbset, path text[], jsonb, after boolean=false) → jsonbset`

Return the `jsonbset` value with `jsonb` inserted. If the item specified by the path is an array element, the new value will be inserted before that item if `after` is false, or after it otherwise. If the item specified by the path is an object field, the new value will be inserted only if the object does not already contain that key. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive.

```

SELECT jsonbset_insert(jsonbset ' [{"speed":10}, {"speed":20}]',
    ARRAY['units'], 'km/h'::jsonb);
-- [{"speed": 10, "units": "km/h"}, {"speed": 20, "units": "km/h"}]
SELECT jsonbset_insert(jsonbset ' [{"speed":10}, {"speed":20}]',
    ARRAY['units'], 'km/h'::jsonb, false);
-- [{"speed": 10}, {"speed": 20}]
SELECT jsonbset_insert(jsonbset ' [{"speed": 10, "units": "km/h"},
    {"speed": 20, "units": "km/h"}]', ARRAY['units'], 'mi/h'::jsonb);
-- [{"speed": 10, "units": "mi/h"}, {"speed": 20, "units": "mi/h"}]

```

- Return a JSONB set without nulls

`jsonbset_strip_nulls(jsonbset, strip_in_arrays bool=false) → jsonbset`

The last argument states whether null array elements are also stripped. Bare null values are never stripped.

```

SELECT jsonbset_strip_nulls(textset ' [{"speed": 10, "lights": null},
    {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}]');
/* [{"speed": 10},
    {"PoIs": ["Grand Place", "La Bourse", null], "speed": 20}] */
SELECT jsonbset_strip_nulls(jsonbset ' [{"speed": 10, "lights": null},
    {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}]', true);
/* [{"speed": 10},
    {"PoIs": ["Grand Place", "La Bourse"], "speed": 20}] */
SELECT jsonbset_strip_nulls(jsonbset
    '{"{"road": "Bvd Gén. Jacques", "category": "primary"},
    {"road": "Bvd de la Cambre", "category": "primary"},
    {"road": "rue de l'Abbaye", "category": null}}');
/* [{"road": "Bvd Gén. Jacques", "category": "primary"},
    {"road": "Bvd de la Cambre", "category": "primary"},
    {"road": "rue de l'Abbaye"}] */

```

As shown below, the function does not strip bare null values. Function `minusValue` can be used for this purpose.

```

SELECT jsonbset_strip_nulls(textset ' [{"speed": 10}, null,
    {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}]);
/* [{"speed":10}, null,
    {"speed":20,"PoIs":["Grand Place","La Bourse"]} */
SELECT minusValues(textset ' [{"speed": 10}, null,
    {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}]', 'null');
/* [{"speed": 10}, {"speed": 10},
    [{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}]]

```

- Does the JSON path return any item for the specified JSONB set?

```
jsonbset @? jsonpath → tbool
```

```
jsonbset_path_exists(jsonbset,vars jsonb='{}',silent boolean=false) → tbool
```

```
jsonbset_path_exists_tz(jsonbset,vars jsonb='{}',silent boolean=false) → tbool
```

The operator @? suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT jsonbset ' [{"speed":10}, {"speed": 20, "units": "km/h"} ]' @?
'$ .units ? (@ == "km/h")';
-- [f, t]
SELECT jsonbset_path_exists(jsonbset
' [{"road": "Bvd Gén. Jacques", "category": "primary"},
{"road": "Bvd Gén. Jacques", "category": "primary"},
{"road": "Bvd de la Cambre", "category": "residential"} ]',
'$ .category ? (@ == "residential")');
-- {f, f, t}
-- TODO, it works without ".datetime()"
SELECT jsonbset_path_exists_tz(jsonbset
' [{"speed":10, "cameraId": "25", "lastInspection": "2000-08-01 12:00:00"},
{"speed":20, "cameraId": "35", "lastInspection": "2000-03-01 12:00:00"} ]',
'$ .lastInspection ? (@.datetime() > "2000-06-01".datetime() )');
-- [t, f]
```

- Return the result of a JSON path predicate check for a JSONB set

```
jsonbset @@ jsonpath → tbool
```

```
jsonbset_path_match(jsonbset,vars jsonb='{}',silent boolean=false) → tbool
```

```
jsonbset_path_match_tz(jsonbset,vars jsonb='{}',silent boolean=false) → tbool
```

The operator @@ suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT jsonbset ' [{"speed":10}, {"speed": 20, "units": "km/h"} ]' @@
'$ .units == "km/h"';
-- [f, t]
SELECT jsonbset_path_match(jsonbset
' [{"road": "Bvd Gén. Jacques", "category": "primary"},
{"road": "Bvd Gén. Jacques", "category": "primary"},
{"road": "Bvd de la Cambre", "category": "residential"} ]',
'$ .category == "residential"');
-- {f, f, t}
-- TODO, it works without ".datetime()"
SELECT jsonbset_path_match_tz(jsonbset
' [{"speed":10, "cameraId": "25", "lastInspection": "2000-08-01 12:00:00"},
{"speed":20, "cameraId": "35", "lastInspection": "2000-03-01 12:00:00"} ]',
'$ .lastInspection.datetime() > "2000-06-01".datetime()');
-- [t, f]
```

- Return all items returned by a JSON path from a JSONB set, as a JSON array

```
jsonbset_path_query_array(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

```
jsonbset_path_query_array_tz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

The above function can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```

SELECT jsonbset_path_query_array(jsonbset
  '[{"speed":10}, {"speed": 20, "units": "km/h"}]',
  '$ ? (@.speed >= $min && @.speed <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}], [{"speed": 20, "units": "km/h"}]]
-- TODO, it works without "_tz"
SELECT jsonbset_path_query_array_tz(jsonbset
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]},
  {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}]',
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
  @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2000-01-05", "max":"2000-01-09"}');
-- [{"2000-01-06", "2000-01-09"}, ["2000-01-07"]]

```

- Return the first item returned by a JSON path from a JSONB set

```

jsonbset_path_query_first(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbset_path_query_first_tz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset

```

The above functions can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```

SELECT jsonbset_path_query_first(jsonbset
  '[{"speed":10}, {"speed": 20, "units": "km/h"}]',
  '$.speed ? (@ >= $min && @ <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}], [{"speed": 20, "units": "km/h"}]]
-- TODO, it works without "_tz"
SELECT jsonbset_path_query_array_tz(jsonbset
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]},
  {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}]',
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2000-01-05", "max":"2000-01-09"}');
-- [{"2000-01-06", "2000-01-09"}, ["2000-01-07"]]

```

15.2 Temporal JSONB values

The temporal type `tjsonb` allows to represent the evolution in time of `jsonb` values. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tjsonb` values in these subtypes are given next.

```

-- Instant
SELECT tjsonb '{"\vehicleId\: 1, \location\: \"Point(1 1)\"}@2001-01-01';
-- Sequence with discrete interpolation
SELECT tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"@2001-01-02}';
-- Sequence with step interpolation
SELECT tjsonb '["vehicleId": 1, "location": "Point(1 1)"@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"@2001-01-02}';
-- Sequence set
SELECT tjsonb '["vehicleId": 1, "location": "Point(1 1)"@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"@2001-01-02},
  [{"vehicleId": 1, "location": "Point(2 2)"@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"@2001-01-04}]]';

```

As can be seen above, for the `Instant` subtype we need to enclose the `JSONB` value between quotes and escape the inner quotes, otherwise the opening brace of the `JSONB` value would be interpreted as the beginning of a `Sequence` subtype with discrete interpolation.

The `tjsonb` type accepts type modifiers (or `typmod` in PostgreSQL terminology) to specify the subtype. The possible values for the subtype are `Instant`, `Sequence`, and `SequenceSet`. The argument is optional and if not specified for a column, values of any subtype are allowed.

```
SELECT tjsonb(Instant) '{"\vehicleId\": 1, \location\: \"Point(1 1)\"}@2001-01-01';
-- {"location": "Point(1 1)", "vehicleId": 1}@2001-01-01
SELECT tjsonb(Sequence) '{"\vehicleId\: 1, \location\: \"Point(1 1)\"}@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Temporal JSONB values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if their JSONB values are equal. Also, two consecutive sequences that are adjacent and have the same end and start JSONB value can be merged into a single one. Examples of transformation into a normal form are as follows.

```
SELECT asText(tjsonb '["{"vehicleId": 1, "location": "Point(1 1)}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)}@2001-01-02,
{"vehicleId": 1, "location": "Point(1 1)}@2001-01-03]');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(1 1)", "vehicleId": 1}@2001-01-03] */
SELECT asText(tjsonb '["{"vehicleId": 1, "location": "Point(1 1)}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)}@2001-01-03],
{"vehicleId": 1, "location": "Point(2 2)}@2001-01-03,
{"vehicleId": 1, "location": "Point(2 2)}@2001-01-04}]');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-04}] */
```

15.3 Validity of temporal JSONB values

Temporal JSONB values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tjsonb 'NULL@2001-01-01 08:05:00';
SELECT tjsonb 'Point(0 0)@NULL';
-- Base type is not a JSONB
SELECT tjsonb 'Point(0 0)@2001-01-01 08:05:00';
```

We give next the functions and operators for temporal JSONB. Most functions and operators for temporal types described in the previous chapters can be applied for temporal JSONB types. Therefore, in the signatures of the functions given previously, the notation `base` represents a `tjsonb` value and the notation `ttype`, represents a `tjsonb` value. To avoid redundancy, we only present next some examples of these functions and operators for temporal JSONB values.

15.4 Input and Output

- Return the Well-Known Text (WKT) representation

`asText({tjsonb,tjsonb[]}) → {text,text[]}`

```
SELECT asText(tjsonb '["{"vehicleId": 1, "location": "Point(1 1)}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)}@2001-01-02]');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02] */
SELECT asText(ARRAY[tjsonb '{"\vehicleId\: 1, \location\: \"Point(1 1)\"}@2001-01-01',
'\vehicleId\: 1, \location\: \"Point(2 2)\"}@2001-01-02']);
/* [{"location\: \"Point(1 1)\", \"vehicleId\: 1}@2001-01-01",
"location\: \"Point(2 2)\", \"vehicleId\: 1}@2001-01-02"} */
```

As can be seen above, for arrays of JSONB values, the elements must be enclosed between quotes and thus, the inner quotes must be escaped.

- Return the Well-Known Binary (WKB) or the Hexadecimal Extended Well-Known Binary (HexWKB) representation

`asBinary(tjsonb, endian text = "") → bytea`

`asHexWKB(tjsonb, endian text = "") → text`

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01'};
-- \x0141000138000000000000000200002008000080090000000a000000090000106c6f63617469666e7...
SELECT asHexWKB(tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01'};
-- 0141000138000000000000000200002008000080090000000A000000090000106C6F63617469666E7...
```

- Return the Moving Features JSON (MF-JSON) representation

`asMFJSON(text) → tjsonb`

```
SELECT asMFJSON(tjsonb '{"Position": "Point(1 1)"@2001-01-01,
{"Position": "Point(2 2)"@2001-01-02}');
/* {"type": "MovingJsonb", "values": [{"Position": "Point(1 1)"}, {"Position": "Point(2 2)"}],
"datetimes": ["2001-01-01T00:00:00+01", "2001-01-02T00:00:00+01"],
"lower_inc": true, "upper_inc": true, "interpolation": "Step"} */
```

- Input from the Well-Known Text (WKT) representation

`tjsonbFromText(text) → tjsonb`

```
SELECT tjsonbFromText(text
'["vehicleId": 1, "location": "Point(1 1)"@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"@2001-01-02}');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02} */
```

- Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

`tjsonbFromBinary(bytea) → tjsonb`

`tjsonbFromHexWKB(text) → tjsonb`

```
SELECT tjsonbFromBinary(asBinary(
tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01}'));
-- [{"location": "Point(1 2)", "vehicleId": 1}@2001-01-01]
SELECT tjsonbFromHexWKB(asHexWKB(
tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01}'));
-- [{"location": "Point(1 2)", "vehicleId": 1}@2001-01-01]
```

- Input from the Moving Features JSON (MF-JSON) representation

`tjsonbFromMFJSON(text) → tjsonb`

```
SELECT tjsonbFromMFJSON('{"type": "MovingJsonb", "interpolation": "Step",
"values": [{"Position": "Point(1 1)"}, {"Position": "Point(2 2)"}],
"datetimes": ["2001-01-01T10:00:00Z", "2001-01-01T12:00:00Z"]}');
/* [{"Position": "Point(1 1)"@2001-01-01 11:00:00+01,
{"Position": "Point(2 2)"@2001-01-01 13:00:00+01} */
```

15.5 Constructors

- Constructor for temporal JSONB values having a constant value

`tjsonb(jsonb, timestamp tz) → tjsonbInst`

`tjsonb(jsonb, tstzset) → tjsonbDiscSeq`

`tjsonb(jsonb, tstzspan) → tjsonbContSeq`

`tjsonb(jsonb, tstzspanset) → tjsonbSeqSet`

```

SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}', timestampz '2001-01-01');
-- {"location": "Point(1 1)", "vehicleId": 1}@2001-01-01
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
   {"location": "Point(1 1)", "vehicleId": 1}@2001-01-03,
   {"location": "Point(1 1)", "vehicleId": 1}@2001-01-05} */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspan '[2001-01-01, 2001-01-02]');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
   {"location": "Point(1 1)", "vehicleId": 1}@2001-01-02} */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspanset '[[2001-01-01, 2001-01-02],[2001-01-03, 2001-01-04]]');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
   {"location": "Point(1 1)", "vehicleId": 1}@2001-01-02],
   [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-03,
   {"location": "Point(1 1)", "vehicleId": 1}@2001-01-04}] */

```

- Constructor for temporal JSONB values of sequence subtype

```
tjsonbSeq(tjsonbInst[], leftInc bool=true, rightInc bool=true) → tjsonbSeq
```

```

SELECT tjsonbSeq(ARRAY[tjsonb
  '{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01',
  '{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02',
  '{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03']);
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
   {"location": "Point(2 2)", "vehicleId": 1}@2001-01-02,
   {"location": "Point(1 1)", "vehicleId": 1}@2001-01-03} */

```

- Constructor for temporal JSONB values of sequence set subtype

```
tjsonbSeqset(tjsonb[]) → tjsonbSeqset
```

```
tjsonbSeqsetGaps(tjsonbInst[], maxt=NULL, maxdist=NULL) → tjsonbSeqset
```

```

SELECT tjsonbSeqset(ARRAY[tjsonb
  '{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02',
  '{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-04']);
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
   {"location": "Point(2 2)", "vehicleId": 1}@2001-01-02,
   {"location": "Point(2 2)", "vehicleId": 1}@2001-01-03,
   {"location": "Point(3 3)", "vehicleId": 1}@2001-01-04} */
SELECT tjsonbSeqsetGaps(ARRAY[tjsonb
  '{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01',
  '{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03',
  '{"vehicleId": 1, "location": "Point(3 3)"}@2001-01-05'], interval '1 day');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
   {"location": "Point(2 2)", "vehicleId": 1}@2001-01-03,
   {"location": "Point(3 3)", "vehicleId": 1}@2001-01-05} */

```

15.6 Conversions

- Convert a temporal JSONB to a tstzspan

```
tjsonb::tstzspan
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]':::tstzspan;
-- [2001-01-01, 2001-01-02]
```

- Convert between a temporal JSONB and a text

tjsonb::text

text::tjsonb

```
SELECT (text '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]')::jsonb;
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02}] */
SELECT pg_typeof(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]':::text);
-- text
```

- Convert between a temporal JSONB and a temporal text

tjsonb::ttext

ttext::tjsonb

```
SELECT (ttext '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]')::tjsonb;
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02}] */
SELECT pg_typeof(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]':::ttext);
-- ttext
```

15.7 Accessors

- Return the values

getValues(tjsonb) → jsonbset

```
SELECT getValues(tjsonb
' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02] ');
-- [{"location": "Point(1 1)", "vehicleId": 1}]
```

- Return the value at a timestamp

valueAtTimestamp(tjsonb,timestamp) → jsonb

```
SELECT valueAtTimestamp(tjsonb
' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]', '2001-01-02');
-- {"location": "Point(2 2)", "vehicleId": 1}
```

15.8 Transformations

- Transform a temporal JSONB to another subtype

tjsonbInst(tjsonb) → tjsonbInst

tjsonbSeq(tjsonb,interp='step') → tjsonbSeq

tjsonbSeqSet(tjsonb) → tjsonbSeqSet

```

SELECT tjsonbSeq(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01';
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01]
SELECT tjsonbSeq(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01',
  'discrete');
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01]
SELECT tjsonbSeqSet(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01';
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01}]

```

- Transform a temporal JSONB value to another interpolation

`setInterp(tjsonb,interp) → tjsonb`

```

SELECT setInterp(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01', ' ←
  step');
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01]
SELECT setInterp(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01],
  [{"vehicleId": 1, "location": "Point(1 1)"@2001-01-02}]', 'discrete');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
  {"location": "Point(1 1)", "vehicleId": 1}@2001-01-02} */

```

15.9 Temporal JSON Operations

In this section, we give the temporal versions of the functions for the `json` and `jsonb` types. We refer to the PostgreSQL [documentation](#) for detailed explanations of these functions. We aimed at keeping as much as possible the same syntax for the non-temporal and the temporal versions of these functions as illustrated in this [document](#).

As stated in Section 4.4, a common way to generalize the traditional operations to the temporal types is to apply the operation *at each instant*, which yields a temporal value as result. Consider for example the `->` operator that extracts a JSON object field with the given key.

```

SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10

```

Applying the same operator to a temporal JSON or JSONB value yields the following result

```

SELECT ttext ' [{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02}]' -> text 'speed';
-- [{"10"@2001-01-01, "20"@2001-01-02}]
SELECT tjsonb ' [{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02}]' -> text 'speed';
-- [{"10"@2001-01-01, "20"@2001-01-02}]

```

which is obtained by applying the PostgreSQL operator `->` at every instant of the temporal JSON or JSONB value.

When manipulating JSON collections of varying structure, it may be the case that an item is defined in some of the documents but not all. PostgreSQL provides the lax mode for this purpose, where the argument `null_value_treatment` determines the behavior in the case a function returns a NULL value. The argument can take one of the following values: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, where `'use_json_null'` is the default value. In PostgreSQL the lax mode is supported only for *update* (not *query*) operations with the function `jsonb_set_lax`. In MobilityDB, we kept the same semantics for the corresponding function `tjsonb_set_lax`, but we enabled similar behavior for *all* temporal JSON operations that may return a null value, except that we replaced the value `'return_target'` with `'return_null'`, since the former is not meaningful for temporal operations. For operators such as `->`, the default value `'use_json_null'` is used and cannot be changed, whereas for the corresponding functions `tjson_object_field` and `tjsonb_object_field`, the last argument specifies the behavior in the case the function returns NULL. We illustrate this behavior for function `tjson_object_field` below.

- Extract a temporal JSON object field specified by a key


```
{ttext,tjsonb} -> text → {ttext,tjsonb}
tjsonb ->> text → ttext
tjson_object_field(ttext,text,null_handle text='use_json_null') → ttext
tjsonb_object_field(tjsonb,text,null_handle text='use_json_null') → tjsonb
tjsonb_object_field_text(tjsonb,text,null_handle text='use_json_null') → ttext
```

```
SELECT ttext '[[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02}]' -> text 'speed';
-- [{"10"@2001-01-01, "20"@2001-01-02}]
SELECT tjsonb '[[{"position":"Point(1 1)", "speed":10}@2001-01-01,
  {"position":"Point(2 2)", "speed":20}@2001-01-03}]' ->> text 'position';
-- [{"Point(1 1)"@2001-01-01, "Point(2 2)"@2001-01-03}]
SELECT ttext '[[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]' -> text 'speed';
-- [{"10@2001-01-01, null@2001-01-02, 10@2001-01-03}]
```

We show below the behavior of the function `tjson_object_field` for the possible values of the `null_handle` argument. All JSON functions that may return a NULL value behave similarly in MobilityDB.

```
SELECT tjson_object_field(ttext '[[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'raise_exception');
-- ERROR: The lifted operation returned NULL
SELECT tjson_object_field(ttext '[[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'use_json_null');
-- [{"10@2001-01-01, null@2001-01-02, 10@2001-01-03}]
SELECT tjson_object_field(ttext '[[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'delete_key');
-- [{"10@2001-01-01, 10@2001-01-02}, {"10@2001-01-03}]
SELECT tjson_object_field(ttext '[[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'return_null');
-- NULL
```

- Extract a temporal JSON object field specified by a path

```
{ttext,tjsonb} #> text[] → {ttext,tjsonb}
tjson_extract_path(ttext,text[],null_handle text='use_json_null') → ttext
tjsonb_extract_path(tjsonb,text[],null_handle text='use_json_null') → tjsonb
tjsonb_extract_path_text(tjsonb,text[],null_handle text='use_json_null') → ttext
```

```
SELECT ttext '[[{"speed": {"unit": "km", "value": 10}}@2001-01-01,
  {"speed": {"unit": "km", "value": 20}}@2001-01-02]' #> ARRAY[text 'speed', 'value'];
-- [{"10"@2001-01-01, "20"@2001-01-02}]
SELECT tjsonb '[[{"position":"Point(1 1)", "PoIs":["Grand Place", "La Bourse"]}@2001-01-01,
  {"position":"Point(2 2)", "PoIs":["Palais Royal", "Manneken Pis"]}@2001-01-03]' #>>
  ARRAY[text 'PoIs', '1'];
-- [{"La Bourse"@2001-01-01, "Manneken Pis"@2001-01-03}]
```

- Extract an element from a temporal JSON array

```
tjson_array_element(tjsonb,integer,null_handle text='use_json_null') → tjsonb
tjsonb_array_element(tjsonb,integer,null_handle text='use_json_null') → tjsonb
tjsonb_array_element_text(tjsonb,integer,null_handle text='use_json_null') → tjsonb
```

```
SELECT tjson_array_element(ttext '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal", "Manneken Pis"]@2001-01-02]', 0);
-- ["Grand Place"@2001-01-01, "Palais Royal"@2001-01-02]
SELECT tjsonb_array_element(tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal", "Manneken Pis"]@2001-01-02]', 1);
-- ["La Bourse"@2001-01-01, "Manneken Pis"@2001-01-02]
```

- Extract a temporal alphanumeric value from a temporal JSONB value given by a key

`tbool(tjsonb,text,null_handle text='raise_exception') → tbool`

`tint(tjsonb,text,null_handle text='raise_exception') → tint`

`tfloat(tjsonb,text,interp='linear',null_handle text='raise_exception') → tfloat`

`ttext(tjsonb,text, null_handle text='raise_exception') → ttext`

Note that the value `use_json_null` cannot be used for the above functions and thus the default value `'raise_exception'` is used.

```
SELECT tbool(tjsonb '["speed": 10, "lights": "on"]@2001-01-01,
{"speed": 20, "lights": "on"}@2001-01-02]', 'lights');
-- [true@2001-01-01, true@2001-01-02]
SELECT tint(tjsonb '["speed": 10, "units": "km/h"]@2001-01-01,
{"speed": 20, "units": "km/h"}@2001-01-02]', 'speed');
-- [10@2001-01-01, 20@2001-01-02]
SELECT tfloat(tjsonb '["speed": 10, "units": "km/h"]@2001-01-01,
{"speed": 20, "units": "km/h"}@2001-01-02],[{"speed": 20}@2001-01-03,
{"speed": 20}@2001-01-04}]', 'speed', 'step');
-- Interp=Step;[10@2001-01-01, 20@2001-01-02], [20@2001-01-03, 20@2001-01-04]]
SELECT ttext(tjsonb '["road": "Bvd Gén. Jacques", "category": "primary"]@2001-01-01,
{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
{"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03]', 'category');
-- [primary@2001-01-01, residential@2001-01-03]
```

- Temporal JSONB concatenation

`tjsonb || {jsonb,tjsonb} → tjsonb`

```
SELECT tjsonb '["speed":10}@2001-01-01, {"speed":20}@2001-01-02}]' ||
{"unit":"km"}::jsonb;
-- [{"unit": "km", "speed": 10}@2001-01-01, {"unit": "km", "speed": 20}@2001-01-02}]
SELECT tjsonb '["speed":10}@2001-01-01, {"speed":20}@2001-01-03}]' ||
tjsonb '["Position":"Point(1 1)"]@2001-01-02, {"Position":"Point(2 2)"}@2001-01-04]';
/* [{"speed": 10, "Position": "Point(1 1)"}@2001-01-02,
{"speed": 20, "Position": "Point(2 2)"}@2001-01-03}] */
```

- Temporal JSONB deletion

`tjsonb - {int,text,text[]} → tjsonb`

`tjsonb #- text[] → tjsonb`

```
SELECT tjsonb '["unit": "km", "speed": 10}@2001-01-01,
{"unit": "km", "speed": 20}@2001-01-02}]' - 'unit';
-- [{"speed": 10}@2001-01-01, {"speed": 20}@2001-01-02}]
SELECT tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal", "Manneken Pis"]@2001-01-02]' - 1;
-- ["Grand Place"]@2001-01-01, ["Palais Royal"]@2001-01-02]
SELECT tjsonb '["location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]@2001-01-01,
"location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]@2001-01-02]' #-
ARRAY[text "\"PoIs\"", "1"];
-- ["La Bourse"@2001-01-01, "Manneken Pis"@2001-01-02]
```

As shown below, the result of a delete operation may be an empty record or an empty array. To remove these values, the function `minusValue` can be used.

```
SELECT tjsonb '{{{"unit": "km", "speed": 10, "light": true}@2001-01-01,
{"unit": "km", "speed": 20}@2001-01-02}}' - ARRAY[text 'unit', 'speed'];
-- [{"light": true}@2001-01-01, {}@2001-01-02]}
SELECT tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal"]@2001-01-02]' - 0;
-- [{"La Bourse"]@2001-01-01, []@2001-01-02]}
```

```
SELECT minusValues(tjsonb '{{{"unit": "km", "speed": 10, "light": true}@2001-01-01,
{"unit": "km", "speed": 20}@2001-01-02}}' - ARRAY[text 'unit', 'speed'],
jsonbset '{"[]", "{}"}');
-- [{"light": true}@2001-01-01, {"light": true}@2001-01-02)}
SELECT minusValues(tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal"]@2001-01-02]' - 0, jsonbset '{"[]", "{}"}');
-- [{"La Bourse"]@2001-01-01, ["La Bourse"]@2001-01-02)}
```

- Temporal JSONB exists

```
tjsonb ? jsonb → tbool
tjsonb ?| jsonb[] → tbool
tjsonb ?& jsonb[] → tbool
```

As in PostgreSQL, the operators `?|` and `?&` test, respectively, whether *any* or *all* of the strings in the text array exist as top-level keys or array elements.

```
SELECT tjsonb '{"\"geom\": \"Point(1 1)\"'}@2001-01-01' ? text 'geom';
-- t@2001-01-01
SELECT tjsonb '{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
{"geom": "Point(1 1)"}@2001-01-03}' ?| ARRAY[text 'geom'];
-- {t@2001-01-01, t@2001-01-02, t@2001-01-03}
SELECT tjsonb '{{"speed": 10, "units": "km/h"}@2001-01-01,
{"speed": 20, "units": "km/h"}@2001-01-02}' ?& ARRAY['speed', 'units'];
-- {t@2001-01-01, t@2001-01-02]}
```

- Temporal JSONB contains/contained

```
{tjsonb, jsonb} @> {tjsonb, jsonb} → tbool
{tjsonb, jsonb} <@ {tjsonb, jsonb} → tbool
```

```
SELECT tjsonb '{"\"geom\": \"Point(1 1)\"'}@2001-01-01' @> jsonb '{"geom": "Point(1 1)"}';
-- t@2001-01-01
SELECT tjsonb '{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
{"geom": "Point(1 1)"}@2001-01-03}' @> jsonb '{"geom": "Point(1 1)"}';
-- {t@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT jsonb '{"geom": "Point(1 1)"}' <@ tjsonb '{{{"geom": "Point(1 1)"}@2001-01-01,
{"geom": "Point(1 1)"}@2001-01-02, {"geom": "Point(1 1)"}@2001-01-03},
{"geom": "Point(2 2)"}@2001-01-04, {"geom": "Point(2 2)"}@2001-01-05}}';
-- {t@2001-01-01, t@2001-01-03, [f@2001-01-04, f@2001-01-05]}
```

- Temporal JSONB set

```
tjsonb_set(tjsonb, path text[], jsonb, create boolean=true) → tjsonb
tjsonb_set_lax(tjsonb, path text[], jsonb, create boolean=true, handle_null text) → tjsonb
```

Return the `tjsonb` value with the item specified by `path` replaced by `jsonb`, or added if `create` is true and the item does not exist. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, and `create_if_missing` is true, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive.

Function `tjsonb_set_lax` behaves identically to `tjsonb_set` if the given value is not NULL. Otherwise, it behaves according to the value of `handle_null`, which must be one of `'raise_exception'`, `'use_json_null'`, `'delete_key'`, or `'return_source'`. As stated above, we kept PostgreSQL behavior for this function enabling the value `'return_source'` whereas for all other *query* operations, this value has been replaced with `'return_null'`.

```
SELECT tjsonb_set(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"::jsonb);
-- [{"speed": 10, "units": "km/h"}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]
SELECT tjsonb_set(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"::jsonb, false);
-- [{"speed": 10}@2001-01-01, {"speed": 20}@2001-01-02]
SELECT tjsonb_set(tjsonb '{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02}', ARRAY['units'], '"mi/h"::jsonb);
-- [{"speed": 10, "units": "mi/h"}@2001-01-01, {"speed": 20, "units": "mi/h"}@2001-01-02]
SELECT tjsonb_set_lax(tjsonb '{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20}@2001-01-02}', ARRAY['units'], 'null::jsonb, true, 'delete_key');
-- [{"speed": 10, "units": "km/h"}@2001-01-01, {"speed": 20}@2001-01-02]
```

- Temporal JSONB insert

`tjsonb_insert(tjsonb, path text[], jsonb, after boolean=false) → tjsonb`

Return the `tjsonb` value with `jsonb` inserted. If the item specified by the path is an array element, the new value will be inserted before that item if `after` is false, or after it otherwise. If the item specified by the path is an object field, the new value will be inserted only if the object does not already contain that key. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive.

```
SELECT tjsonb_insert(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"::jsonb);
-- [{"speed": 10, "units": "km/h"}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]
SELECT tjsonb_insert(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"::jsonb, false);
-- [{"speed": 10}@2001-01-01, {"speed": 20}@2001-01-02]
SELECT tjsonb_insert(tjsonb '{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02}', ARRAY['units'], '"mi/h"::jsonb);
-- [{"speed": 10, "units": "mi/h"}@2001-01-01, {"speed": 20, "units": "mi/h"}@2001-01-02]
```

- Return a temporal JSON value without nulls

`tjson_strip_nulls(ttext, strip_in_arrays bool=false) → ttext`

`tjsonb_strip_nulls(tjsonb, strip_in_arrays bool=false) → tjsonb`

The last argument states whether null array elements are also stripped. Bare null values are never stripped.

```
SELECT tjson_strip_nulls(ttext '{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02}');
/* [{"speed": 10}@2001-01-01,
  {"PoIs": ["Grand Place", "La Bourse", null], "speed": 20}@2001-01-02} */
SELECT tjsonb_strip_nulls(tjsonb '{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02}', true);
/* [{"speed": 10}@2001-01-01,
  {"PoIs": ["Grand Place", "La Bourse"], "speed": 20}@2001-01-02} */
SELECT tjsonb_strip_nulls(tjsonb
  '{"{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd de la Cambre", "category": "primary"}@2001-01-02,
  {"road": "rue de l'Abbaye", "category": null}@2001-01-03}');
/* [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd de la Cambre", "category": "primary"}@2001-01-02,
  {"road": "rue de l'Abbaye"}@2001-01-03] */
```

As shown below, the function does not strip bare null values. Function `minusValue` can be used for this purpose.

```
SELECT tjson_strip_nulls(ttext ' [{"speed": 10}@2001-01-01, null@2001-01-02,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03] ');
/* [{"speed":10}@2001-01-01, null@2001-01-02,
  {"speed":20,"PoIs":["Grand Place","La Bourse"]}@2001-01-03] */
SELECT minusValues(ttext ' [{"speed": 10}@2001-01-01, null@2001-01-02,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03]', 'null');
/* [{"speed": 10}@2001-01-01, {"speed": 10}@2001-01-02),
  [{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03}]
```

- Does the JSON path return any item for the specified temporal JSONB value?

```
tjsonb @? jsonpath → tbool
```

```
tjsonb_path_exists(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

```
tjsonb_path_exists_tz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

The operator @? suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonb ' [{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]' @?
  '$.units ? (@ == "km/h")';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonb_path_exists(tjsonb
  ' [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
  '$.category ? (@ == "residential")');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
-- TODO, it works without ".datetime()"
SELECT tjsonb_path_exists_tz(tjsonb
  ' [{"speed":10, "cameraId": "25", "lastInspection": "2000-08-01 12:00:00"}@2001-01-01,
  {"speed":20, "cameraId": "35", "lastInspection": "2000-03-01 12:00:00"}@2001-01-02]',
  '$.lastInspection ? (@.datetime() > "2000-06-01".datetime() )');
-- [t@2001-01-01, f@2001-01-02]
```

- Return the result of a JSON path predicate check for a temporal JSONB value

```
tjsonb @@ jsonpath → tbool
```

```
tjsonb_path_match(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

```
tjsonb_path_match_tz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

The operator @@ suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonb ' [{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]' @@
  '$.units == "km/h"';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonb_path_match(tjsonb
  ' [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
  '$.category == "residential")';
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
-- TODO, it works without ".datetime()"
SELECT tjsonb_path_match_tz(tjsonb
  ' [{"speed":10, "cameraId": "25", "lastInspection": "2000-08-01 12:00:00"}@2001-01-01,
  {"speed":20, "cameraId": "35", "lastInspection": "2000-03-01 12:00:00"}@2001-01-02]',
  '$.lastInspection.datetime() > "2000-06-01".datetime() ');
-- [t@2001-01-01, f@2001-01-02]
```

- Return all items returned by a JSON path from a temporal JSONB value, as a JSON array

`tjsonb_path_query_array(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb`

`tjsonb_path_query_array_tz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb`

The above function can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonb_path_query_array(tjsonb
  '[{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$ ? (@.speed >= $min && @.speed <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}]@2001-01-01, [{"speed": 20, "units": "km/h"}@2001-01-02]
-- TODO, it works without "_tz"
SELECT tjsonb_path_query_array_tz(tjsonb
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]}@2001-01-01,
  {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}@2001-01-02]',
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
    @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2000-01-05", "max":"2000-01-09}');
-- [{"2000-01-06", "2000-01-09"}@2001-01-01, [{"2000-01-07"}@2001-01-02]
```

- Return the first item returned by a JSON path from a temporal JSONB value

`tjsonb_path_query_first(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb`

`tjsonb_path_query_first_tz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb`

The above functions can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonb_path_query_first(tjsonb
  '[{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$.speed ? (@ >= $min && @ <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}]@2001-01-01, [{"speed": 20, "units": "km/h"}@2001-01-02]
-- TODO, it works without "_tz"
SELECT tjsonb_path_query_array_tz(tjsonb
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]}@2001-01-01,
  {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}@2001-01-02]',
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2000-01-05", "max":"2000-01-09}');
-- [{"2000-01-06", "2000-01-09"}@2001-01-01, [{"2000-01-07"}@2001-01-02]
```

15.10 Restrictions

- Restrict to (the complement of) a value or a set of values

`atValue(tjsonb,value) → tjsonb`

`minusValue(tjsonb,value) → tjsonb`

`atValues(tjsonb,valueset) → tjsonb`

`minusValues(tjsonb,valueset) → tjsonb`

```
SELECT atValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
-- [{"location": "Point(2 2)", "vehicleId": 1}@2001-01-03]
SELECT minusValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
  {"location": "Point(1 1)", "vehicleId": 1}@2001-01-03}] */
```

15.11 Comparisons

- Traditional comparisons

`tjsonb {=, <>, <, >, <=, >=} tjsonb → boolean`

```
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02),
 [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03}]' =
tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03}]' <>
tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- false
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03}]' <
tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- true
```

- Ever and always comparisons

`{jsonb,tjsonb} {?=?, %=} {jsonb,tjsonb} → boolean`

```
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02}]' ?=
jsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}]';
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02}]' %=
jsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}]';
-- false
```

- Temporal comparisons

`{jsonb,tjsonb} {#=, #<>} {jsonb,tjsonb} → tbool`

```
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03}]' #=
jsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}]';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03}]' #<>
tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03}]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

15.12 Bounding Box Operations

- Topological operators

`{tjsonb,tstzspan} {&&, <@#, #@>, ~=, -|-} {tjsonb,tstzspan} → boolean`

The temporal contains and contained operator are marked with a # to distinguish them from the corresponding JSONB operators.

```

SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' &&
  tstzspan '[2001-01-01, 2001-03-01]';
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' #@>
  timestampz '[2001-01-02]':tstzspan;
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' -|-
  tjsonb ' [{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true

```

- Position operators

$\{tjsonb, tstzspan\} \{<<\#, \&<\#, \#>>, \#&>\} \{tjsonb, tstzspan\} \rightarrow \text{boolean}$

```

SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' <<\#
  tstzspan '[2001-01-01, 2001-03-01]';
-- false
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' \#>>
  timestampz '[2001-01-01]':tstzspan;
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' \#&>
  tjsonb ' [{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true

```

15.13 Aggregations

The aggregate functions for temporal JSONB values are illustrated next.

- Temporal count

$tCount(tjsonb) \rightarrow \{tintSeq, tintSeqSet\}$

```

WITH Temp(temp) AS (
  SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-03], (1@2001-01-03, 1@2001-01-04)}

```

- Window count

$wCount(tjsonb) \rightarrow \{tintSeq, tintSeqSet\}$

```

WITH Temp(temp) AS (
  SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT wCount(Temp, interval '2 days')
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-05], (1@2001-01-05, 1@2001-01-06)}

```


15.14 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal JSONB values. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist (Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal JSONB values, which is an `stbox`, as all spatiotemporal types.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal JSONB values,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal JSONB values,
- `&&`, `@>`, `<@`, `~`, `=`, `-|-`, and `|=|`, which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

Chapter 16

Temporal H3 Cell Indices

The temporal H3 cell index type `th3index` represents the movement of objects as a sequence of H3 hexagonal cell identifiers. **Uber's H3** is a discrete global grid system that tessellates the sphere into 122 base cells at resolution 0 and subdivides each cell into 7 children at every finer resolution (up to 15). Each cell has a 64-bit integer identifier that encodes the base cell, resolution, and hierarchical position.

The `th3index` type shares its on-disk representation with `tbigint`: both store a temporal 64-bit integer. The distinct SQL type exists purely to let the type-checker reject `h3index`-specific functions when applied to arbitrary `bigint` trajectories (and vice-versa). Casts to / from `tbigint` are *explicit* (AS ASSIGNMENT) and binary-coercion only — zero-cost at runtime but requiring an explicit `::` in SQL.

As with other temporal types, `th3index` has four subtypes: `Instant`, `discrete-sequence`, `continuous-sequence` with step interpolation (H3 cells are discrete — linear interpolation is meaningless), and `SequenceSet`.

Operational guidance on when to choose `th3index` over `tgeompoint` or `tgeogpoint`; the H3-specific hazards consumers should anticipate (int64 ordering arbitrary with respect to grid geometry, resolution mixing in operations, pentagon cells, compaction round-trip cell-order non-preservation, antimeridian and pole behaviour at high resolution); and the durability and storage conventions are collected in Section 16.17.

16.1 Input and Output

A `th3index` value is entered using the standard temporal syntax with the underlying 64-bit H3 cell identifier. The input parser accepts both the canonical hexadecimal form (as produced by the H3 CLI and by `h3_cell_to_string` upstream) and the equivalent unsigned decimal. Hexadecimal is idiomatic in the H3 ecosystem and is what the output function emits — all examples below use it. For reference, the hex value `831c02fffffffffff` used in the examples corresponds to the decimal value `590464338553208831`.

```
SELECT th3index '831c02fffffffffff@2001-01-01';
SELECT th3index '{831c02fffffffffff@2001-01-01, 831c00fffffffffff@2001-01-02}';
SELECT th3index '[831c02fffffffffff@2001-01-01, 831c00fffffffffff@2001-01-02]';
SELECT th3index ' {[831c02fffffffffff@2001-01-01, 831c00fffffffffff@2001-01-02],
  [880326b885fffff@2001-01-03, 880326b88dfffff@2001-01-04]}';
```

The type accepts type modifiers (`typmod`) to constrain a column to a specific subtype: `Instant`, `Sequence`, or `SequenceSet`.

```
SELECT th3index(Instant) '831c02fffffffffff@2001-01-01';
SELECT th3index(SequenceSet) ' {[831c02fffffffffff@2001-01-01, 831c00fffffffffff@2001-01-02]}';
-- ERROR: temporal type (Instant) does not match column type (Sequence)
SELECT th3index(Sequence) '831c02fffffffffff@2001-01-01';
```

The functions `asText`, `asBinary`, and `asHexWKB` serialize a `th3index` value, and `th3indexFromBinary` and `th3indexFromText` reconstruct it. The hexadecimal WKB form lets a temporal H3 index column round-trip through a text-based COPY.

```
SELECT asHexWKB(th3index '831c02fffffffffff@2001-01-01');
SELECT th3indexFromHexWKB(asHexWKB(th3index '831c02fffffffffff@2001-01-01'));
-- 831c02fffffffffff@2001-01-01
```

16.2 Conversions

A `th3index` value shares the same on-disk representation as `tbigint`. The binary-coercion cast between them is *explicit* — requiring `::` — so `h3index`-specific functions cannot be silently called on arbitrary `bigint` trajectories.

- Convert between a `th3index` and a `tbigint` (explicit, zero-cost)

```
tbigint::th3index
th3index::tbigint
```

```
SELECT (tbigint '590464338553208831@2001-01-01')::th3index;
-- 831c02fffffffffff@2001-01-01
SELECT ((th3index '831c02fffffffffff@2001-01-01')::tbigint)::th3index
= th3index '831c02fffffffffff@2001-01-01';
-- true
```

- Cast a temporal H3 cell to a temporal geodetic / planar point of cell centroids. The two overloads give the caller a choice between the geodetic (`tgeogpoint`) and SRID-4326 planar (`tgeompoint`) representations.

```
th3index::tgeogpoint
th3index::tgeompoint
```

```
SELECT (th3index '831c02fffffffffff@2001-01-01')::tgeogpoint;
SELECT (th3index '831c02fffffffffff@2001-01-01')::tgeompoint;
```

16.3 Static Geometry Coverage

These functions convert an ordinary (non-temporal) geometry into the H3 cell or cell set that covers it at a given resolution, and test whether a temporal H3 trajectory ever enters that cell set. The cell-set output together with the `ever-intersects` predicate form the cross-platform spatial prefilter used by the portable BerlinMOD queries.

- Return the H3 cell that contains a point geometry at the given resolution. The geometry must be a `POINT`.

```
geoToH3Cell(geometry, integer) → h3index
```

```
SELECT geoToH3Cell(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- 871fa4418ffffff
```

- Return the set of H3 cells covering a geometry at the given resolution. Every geometry type is supported: a `POINT` yields a single cell, a `LINESTRING` samples its segments, a `POLYGON` fills its interior, and `MULTI*` / `GEOMETRYCOLLECTION` values return the union of their components.

```
geoToH3IndexSet(geometry, integer) → h3indexset
```

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- {"871fa4418ffffff"}
```

- Return whether a temporal H3 trajectory ever takes one of the cells in a cell set. Combined with `geoToH3IndexSet` it answers whether a trip ever passes through a region, without materialising the trajectory geometry — the sound, conservative prefilter for the exact `eIntersects(geometry, th3index)`.

```
ever_eq(h3indexset, th3index) → boolean
```

```
h3indexset ?= th3index → boolean
```

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7) ?=
  th3index '871fa4418ffffff@2001-01-01';
-- true
```

16.4 Accessors

- Return the first or last H3 cell identifier of a temporal value

startValue(th3index) → h3index

endValue(th3index) → h3index

```
SELECT startValue(th3index '831c02fffffffff@2001-01-01');
-- 831c02fffffffff
SELECT endValue(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}');
-- 831c00fffffffff
```

- Return the nth distinct H3 cell identifier (1-indexed). Returns NULL if out of range.

valueN(th3index, integer) → h3index

```
SELECT valueN(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}', 1);
-- 831c02fffffffff
SELECT valueN(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}', 2);
-- 831c00fffffffff
SELECT valueN(th3index '831c02fffffffff@2001-01-01', 99);
-- NULL
```

- Return the set of distinct H3 cell identifiers the trajectory takes

getValues(th3index) → h3indexset

```
SELECT getValues(th3index '831c02fffffffff@2001-01-01');
-- {831c02fffffffff}
SELECT getValues(th3index
  '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02,
   880326b885fffff@2001-01-03]');
-- {831c00fffffffff, 831c02fffffffff, 880326b885fffff}
```

- Return the H3 cell identifier at a given timestamp, using step interpolation

valueAtTimestamp(th3index, timestamptz) → h3index

```
SELECT valueAtTimestamp(th3index
  '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-05]',
  '2001-01-03');
-- 831c02fffffffff
```

16.5 Index Inspection

- Return the temporal resolution level (0–15) of each cell in a trajectory

th3GetResolution(th3index) → tint

```
SELECT th3GetResolution(th3index '831c02fffffffff@2001-01-01');
-- 3@2001-01-01
SELECT th3GetResolution(th3index '880326b885fffff@2001-01-01');
-- 8@2001-01-01
```

- Return the temporal base-cell number (0–121) of each cell in a trajectory

`th3GetBaseCellNumber(th3index) → tint`

```
SELECT th3GetBaseCellNumber(th3index '831c02ffffffffff@2001-01-01');
```

- Return a temporal boolean stating at each instant whether the value is a valid H3 cell

`isValidCell(th3index) → tbool`

```
SELECT isValidCell(th3index '831c02ffffffffff@2001-01-01');
-- t@2001-01-01
SELECT isValidCell(th3index '0@2001-01-01');
-- f@2001-01-01
```

- Return a temporal boolean stating whether each cell has Class-III orientation. Class III alternates with resolution: odd resolutions are class III, even are class II.

`th3IsResClassIii(th3index) → tbool`

- Return a temporal boolean stating whether each cell is a pentagon (12 cells per resolution are pentagons, the remaining 110 at each base cell descend as hexagons)

`th3IsPentagon(th3index) → tbool`

```
SELECT th3IsPentagon(th3index '831c00ffffffffff@2001-01-01');
-- t@2001-01-01
SELECT th3IsPentagon(th3index '831c02ffffffffff@2001-01-01');
-- f@2001-01-01
```

16.6 Hierarchy

- Return the temporal parent cell at the given resolution. The no-argument form drops to the next-coarser resolution.

`th3CellToParent(th3index, integer) → th3index`

`th3CellToParent(th3index) → th3index`

```
SELECT th3GetResolution(
  th3CellToParent(th3index '8a2a1072b59ffff@2001-01-01', 5));
-- 5@2001-01-01
```

- Return the temporal center-child cell at the given resolution. The no-argument form drops to the next-finer resolution.

`th3CellToCenterChild(th3index, integer) → th3index`

`th3CellToCenterChild(th3index) → th3index`

- Return the ordinal position (0-based) of each cell among the children of its parent at the given resolution

`th3CellToChildPos(th3index, integer) → tbigint`

- Return the temporal child cell at a given ordinal position of the given parent, at the given child resolution. The two temporal operands are synchronised over their shared time axis.

`th3ChildPosToCell(tbigint, th3index, integer) → th3index`

16.7 Lat/Lng Conversions

- Return the temporal H3 cell at the given resolution that contains each point in a temporal geodetic or planar point (SRID must be 4326 for the `tgeompoint` overload)

`tgeompoint_to_th3(tgeompoint, integer) → th3index`

`tgeompoint_to_th3(tgeompoint, integer) → th3index`

```
SELECT th3GetResolution(tgeompoint_to_th3(
  tgeompoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
```

- Return the temporal geodetic centroid of each cell in a trajectory. A planar (SRID 4326) overload is available under the name `th3CellToLatlngTgeompoint`.

`th3CellToLatlng(th3index) → tgeompoint`

`th3CellToLatlngTgeompoint(th3index) → tgeompoint`

- Return the temporal polygon boundary of each cell in a trajectory, as a temporal geography

`th3CellToBoundary(th3index) → tgeography`

16.8 Directed Edges

H3 directed edges are themselves 64-bit identifiers (encoded differently from cell identifiers — `isValidDirectedEdge` distinguishes them). The MobilityDB `th3index` type carries directed edges by convention: the same type, interpreted through edge-specific accessors.

- Return a temporal boolean stating whether two temporal cells are grid neighbours at each instant. The two operands are synchronised.

`th3AreNeighborCells(th3index, th3index) → tbool`

```
SELECT th3AreNeighborCells(
  th3index '880326b885ffff@2001-01-01',
  th3index '880326b88dffff@2001-01-01');
-- t@2001-01-01
```

- Return a temporal directed-edge identifier from an origin/destination pair of temporal cells

`th3CellsToDirectedEdge(th3index, th3index) → th3index`

- Return a temporal boolean stating at each instant whether the value is a valid H3 directed edge

`isValidDirectedEdge(th3index) → tbool`

- Return the temporal origin or destination cell of a temporal directed edge

`th3GetDirectedEdgeOrigin(th3index) → th3index`

`th3GetDirectedEdgeDestination(th3index) → th3index`

- Return the temporal polygon boundary of each directed edge in a trajectory, as a temporal geography

`th3DirectedEdgeToBoundary(th3index) → tgeography`

16.9 Vertices

- Return the temporal H3 vertex at the given vertex number (0–5 for hexagons, 0–4 for pentagons)

```
th3CellToVertex(th3index, integer) → th3index
```

- Return the temporal geodetic coordinates of each vertex in a trajectory

```
th3VertexToLatlng(th3index) → tgeogpoint
```

- Return a temporal boolean stating at each instant whether the value is a valid H3 vertex

```
isValidVertex(th3index) → tbool
```

16.10 Grid Traversal

- Return the temporal grid-hop distance (number of single-cell steps) between two temporal cells. Equivalently, the <-> operator returns the same distance. The two operands are synchronised over their shared time axis.

```
th3GridDistance(th3index, th3index) → tbigint
```

```
th3index <-> th3index → tbigint
```

```
SELECT th3index '880326b885fffff@2001-01-01'
      <-> th3index '880326b88dfffff@2001-01-01';
-- 1@2001-01-01
```

- Convert between a temporal cell and a temporal local (I, J) coordinate pair relative to an origin temporal cell. The local coordinate is exposed as a tgeompoint with the I axis in the x slot and J in y.

```
th3CellToLocalIj(th3index, th3index) → tgeompoint
```

```
th3LocalIjToCell(th3index, tgeompoint) → th3index
```

16.11 Metrics

The three metric functions take an optional unit argument. For areas: km2 (default), m2, or rads2. For lengths: km (default), m, or rads. Passing an area unit to a length function (or vice-versa) raises an error.

- Return the temporal area of each cell in a trajectory, in the requested unit

```
th3CellArea(th3index, text='km2') → tfloat
```

- Return the temporal length of each directed edge in a trajectory, in the requested unit

```
th3EdgeLength(th3index, text='km') → tfloat
```

- Return the temporal great-circle distance between two temporal geodetic points (not between cells). The two operands are synchronised.

```
greatCircleDistance(tgeogpoint, tgeogpoint, text='km') → tfloat
```

16.12 Comparisons

16.12.1 Ever and Always Comparisons

The probe operand may be a bare h3index H3 cell identifier or another th3index trajectory; both are supported in either operand position.

- Return true if a temporal H3 cell is ever equal to the probe at at least one instant

```

ever_eq({h3index,th3index},th3index) → boolean
ever_eq(th3index,h3index) → boolean
{h3index,th3index} ?= th3index
th3index ?= h3index

```

```

SELECT th3index
  '[831c02fffffffffff@2001-01-01, 880326b885fffff@2001-01-05]'
  ?= 880326b885fffff::h3index;
-- true

```

- Return true if a temporal H3 cell is always equal to the probe across its entire time axis

```

always_eq({h3index,th3index},th3index) → boolean
always_eq(th3index,h3index) → boolean
{h3index,th3index} %= th3index
th3index %= h3index

```

- Return true if a temporal H3 cell is ever different from the probe

```

ever_ne({h3index,th3index},th3index) → boolean
ever_ne(th3index,h3index) → boolean
{h3index,th3index} ?<> th3index
th3index ?<> h3index

```

- Return true if a temporal H3 cell is always different from the probe

```

always_ne({h3index,th3index},th3index) → boolean
always_ne(th3index,h3index) → boolean
{h3index,th3index} %<> th3index
th3index %<> h3index

```

16.12.2 Temporal Comparisons

- Return a temporal boolean giving the per-instant equality (or inequality) between a temporal H3 cell and a probe

```

temporal_teq({h3index,th3index},th3index) → tbool
temporal_teq(th3index,h3index) → tbool
temporal_tne({h3index,th3index},th3index) → tbool
temporal_tne(th3index,h3index) → tbool
{h3index,th3index} #= th3index
th3index #= h3index
{h3index,th3index} #<> th3index
th3index #<> h3index

```

16.13 Bounding-Box Operators

The following operators compare the spatiotemporal *bounding box* of temporal H3 cells, not cell-hierarchy containment. `th3index` is a spatial temporal type: its bounding box is an `stbox` that combines the geodetic spatial extent of the cell boundaries (their longitude/latitude envelope in SRID 4326) with the trajectory's `tstzspan` time period. The operators below compare that box along the spatial and/or temporal axes. See the indexing section for the opclasses that make these operators index-capable.

Note

The @> operator tests spatiotemporal bounding-box containment (spatial extent and time together), *not* H3 hierarchical containment. To test whether one cell is the parent of another at a given resolution, use **th3CellToParent**.

- Bounding-box overlap, contains, contained-by, same, and adjacent

```
th3index && {tstzspan, stbox, th3index} → boolean
```

```
th3index @> {tstzspan, stbox, th3index} → boolean
```

```
th3index <@ {tstzspan, stbox, th3index} → boolean
```

```
th3index ~= {tstzspan, stbox, th3index} → boolean
```

```
th3index -|- {tstzspan, stbox, th3index} → boolean
```

- Relative position along the spatial axes (strictly-left, overlaps-left, overlaps-right, strictly-right; strictly-below, overlaps-below, overlaps-above, strictly-above)

```
th3index << {stbox, th3index} → boolean
```

```
th3index &< {stbox, th3index} → boolean
```

```
th3index &> {stbox, th3index} → boolean
```

```
th3index >> {stbox, th3index} → boolean
```

```
th3index <<| {stbox, th3index} → boolean
```

```
th3index &<| {stbox, th3index} → boolean
```

```
th3index |&> {stbox, th3index} → boolean
```

```
th3index |>> {stbox, th3index} → boolean
```

- Relative position along the time axis (strictly-before, overlaps-before, strictly-after, overlaps-after)

```
th3index <<# {tstzspan, stbox, th3index} → boolean
```

```
th3index &<# {tstzspan, stbox, th3index} → boolean
```

```
th3index #>> {tstzspan, stbox, th3index} → boolean
```

```
th3index #&> {tstzspan, stbox, th3index} → boolean
```

16.14 Set-Returning Helpers

The following static helpers return an `h3indexset` (or `intset` for `h3GetIcosahedronFaces`) — the finite-collection companion type of `h3index`. They operate on a static `h3index` (or an `h3indexset`) and are not lifted to `th3index`; the temporal lift is deferred pending a `tset<T>` primitive.

- All cells within `k` grid steps of the origin (inclusive of the origin at `k=0`).

```
h3GridDisk(h3index, integer) → h3indexset
```

- The cells at *exactly* `k` grid steps from the origin. Fails near pentagons.

```
h3GridRing(h3index, integer) → h3indexset
```

- The inclusive path of cells between two cells of the same resolution.

```
h3GridPathCells(h3index, h3index) → h3indexset
```

- All children of a cell at the given target resolution.

```
h3CellToChildren(h3index, integer) → h3indexset
```

- Compacted representation of a cell set — finer cells are merged into their parent whenever a full hexagonal set of siblings is present.

```
h3CompactCells(h3indexset) → h3indexset
```

- Uncompacted representation at the given resolution — the inverse of `h3CompactCells`.

```
h3UncompactCells(h3indexset, integer) → h3indexset
```

- All outgoing directed edges of a cell (6 for hexagons, 5 for pentagons).

```
h3OriginToDirectedEdges(h3index) → h3indexset
```

- All vertexes of a cell (6 for hexagons, 5 for pentagons).

```
h3CellToVertexes(h3index) → h3indexset
```

- The icosahedron face indexes (0..19) intersected by a cell.

```
h3GetIcosahedronFaces(h3index) → intset
```

16.15 Spatial Relationships

The spatial relationships evaluate a `th3index` argument through its per-instant cell-boundary geometry — the hexagonal footprint of each cell, as returned by `th3CellToBoundary` — and then apply the corresponding relationship of the temporal geometry types. Each relationship has three quantified forms: the `e`-prefixed *ever* form returns `true` when the relationship holds at some instant, the `a`-prefixed *always* form when it holds at every instant, and the `t`-prefixed *temporal* form returns a `tbool` of the instant-by-instant result. Every form accepts the three argument orderings (`geometry, th3index`), (`th3index, geometry`), and (`th3index, th3index`).

- Whether one footprint contains the other (interiors only)

```
eContains, aContains, tContains
```

- Whether one footprint covers the other (boundary included)

```
eCovers, aCovers, tCovers
```

- Whether the footprints share no point

```
eDisjoint, aDisjoint, tDisjoint
```

- Whether the footprints share at least one point

```
eIntersects, aIntersects, tIntersects
```

- Whether the footprints share a boundary point but no interior point

```
eTouches, aTouches, tTouches
```

- Whether the footprints lie within a given distance, passed as the third argument

```
eDwithin, aDwithin, tDwithin (... , dist float)
```

```
SELECT eContains(
  geometry 'SRID=4326;Polygon((4.20 50.70, 4.50 50.70, 4.50 50.90, 4.20 50.90, 4.20 50.70))' ←
  ,
  th3index '871fa44a8ffffff@2001-01-01');
-- true
```

16.16 Indexing

GiST and SP-GiST operator classes are defined so the bounding-box operators of the previous section can be used with corresponding indexes.

- GiST operator class for temporal H3 cells. Indexes an `stbox` spatiotemporal bounding box per row (the geodetic extent of the cell boundaries together with the time period).

```
CREATE INDEX ON trips USING gist(cells);           -- uses th3_rtree_ops by default
CREATE INDEX ON trips USING gist(cells th3_rtree_ops);
```

- SP-GiST operator classes for temporal H3 cells, keyed on the `stbox` spatiotemporal bounding box. `th3_quadtree_ops` is the default (quadtree partitioning); `th3_kdtree_ops` uses k-d tree partitioning and must be named explicitly.

```
CREATE INDEX ON trips USING spgist(cells);         -- uses th3_quadtree_ops by default
CREATE INDEX ON trips USING spgist(cells th3_kdtree_ops);
```

16.17 Operational notes

This section captures the operational knowledge consumers need on top of the function reference: when `th3index` is the right type, the conventions the implementation assumes, and the H3-specific hazards real workloads need to be aware of. The H3 hierarchical hexagonal grid has a few non-obvious properties that the bitwise `int64` encoding hides; this section surfaces them so they don't surprise consumers.

16.17.1 When to use `th3index`

Use `th3index` when the natural identifier for a discrete location at a given resolution is an H3 cell — vehicle trips bucketed at H3 resolution 9, environmental sensor readings binned by H3 cell, geo-fenced events keyed by their containing cell. The type stores one H3 cell per instant; resolution may vary across instants in a single trajectory but most workloads pin one resolution per column.

Choose `tgeompoint` / `tgeogpoint` over `th3index` when the actual geographic position (sub-cell precision) is the value of interest. Choose `tbigint` over `th3index` when the values happen to be 64-bit integers but carry no H3 semantics — the binary representations are identical (they cast losslessly via `th3index :: tbigint` and back) but the SQL type system uses the distinction to reject `h3index`-agnostic functions on `h3index`-specific trajectories.

16.17.2 H3-specific hazards

Five hazards reliably bite workloads using H3 cells. Each is a property of the H3 grid itself rather than a MobilityDB implementation choice; the mitigations below explain how to work with them.

- *Int64 ordering is arbitrary with respect to grid geometry.* H3 cell identifiers are 64-bit unsigned integers whose bitpattern encodes resolution, base cell, and hierarchical position. The bitwise ordering of two cells carries no geographic meaning — two cells that are bit-adjacent may be on opposite sides of the planet, and two cells that are spatially adjacent may have wildly different bitpatterns.

Mitigation: `th3index` deliberately has no `h3span` / `h3spanset` companion types (precedent: `geometry` has no `geometryspan` and the `bbox` indexes capture the cells' spatiotemporal extent (`stbox`), not the cell-id value. Value-range filtering must go through `h3_*` inspection functions (resolution, base cell, hierarchy checks) or explicit set enumeration via `h3indexset`. Code that assumes `cell_a < cell_b` reflects spatial proximity will silently produce wrong results.

- *Resolution mixing in operations.* H3 cells at different resolutions (0–15) represent different coverage areas. Mixing resolutions in a single trajectory is valid but semantically requires explicit justification — `th3CellToParent(cell, coarser_res)` coarsens, `h3CellToChildren(parent, finer_res)` refines, and `h3CompactCells` / `h3UncompactCells` round-trip correctly only when input resolutions are compatible.

Mitigation: consumers should document the resolution invariant per trajectory (e.g. "all cells are resolution 9") and validate inputs at the ingestion boundary. The `th3GetResolution` accessor lets a `CHECK` constraint enforce this.

- *Pentagon cells.* The H3 grid has 12 pentagonal (instead of hexagonal) cells per resolution — the Eisenstein duals of the icosahedron's 12 vertices. `h3GridRing` may fail near these pentagons; `h3GridPathCells` fails if the path crosses one. The error is loud (`libh3` raises an explicit failure), but workloads that expect ring / path operations to always succeed need a guard.

Mitigation: defensive code should wrap pentagon-sensitive calls and check `h3_is_pentagon_cell` on inputs and key outputs. For trajectories that pass near a pentagon, fall back to `h3GridDisk` (which is pentagon-safe) or compute path segments in pieces around the pentagon.

- *Compaction / decompaction round-trip.* `h3CompactCells` produces the most compact mixed-resolution representation of a set of cells; `h3UncompactCells` expands it back. The round-trip is lossless in spatial extent, but *cell ordering is not preserved* — `libh3` does not guarantee a sorted output. Queries that consume the output as if its order were stable will return different results before and after compaction even when the underlying spatial set is identical.

Mitigation: re-sort post-compaction if order matters (e.g. `ORDER BY cell` in SQL or `ARRAY_AGG ... ORDER BY 1` when materialising). For temporal sets, prefer the `h3indexset` set-equality semantics over array-equality semantics.

- *Antimeridian and pole behaviour.* H3's base cells are positioned on an icosahedron; at high resolutions some cells near the antimeridian or poles can have counter-intuitive geometry. `th3CellToLatLng` / `h3_latlng_to_cell` round-trip correctly, but coordinates near $\pm 180^\circ$ longitude or near the poles may resolve to cells in unexpected base-cell groups.

Mitigation: workloads at polar latitudes or that cross the antimeridian should validate representative cells against the `libh3` reference implementation directly and add fixtures for the specific edge cases the workload encounters.

16.17.3 Durability and storage

The on-disk representation of `th3index` is byte-identical to `tbigint` (each instant carries one 64-bit H3 cell ID plus a timestamp). Consequences for storage planning:

- *WKT* via `th3_in` and `th3_out`. Cells render as canonical hex strings (e.g. `'8928308280ffff'`); both decimal and hex are accepted on input. Round-trip is bit-stable.
- *WKB / EWKB / HexWKB* via the standard PostGIS endian-flag-then-payload pattern. Stable across PostgreSQL major versions.
- *MFJSON* via `asMFJSON(th3index)` and `th3indexFromMFJSON(text)`. The cell payload renders as the int64 cell identifier in the `values` array, the same representation carried by `tbigint`; consumers that need the canonical hex form apply `h3_cell_to_string`.
- *pg_dump* uses WKT in plain mode and WKB under `--binary-upgrade`; both round-trip cleanly.
- *Cross-cast with tbigint:* the bidirectional `th3index :: tbigint` casts are zero-cost bitwise reinterpretations. Use them when working with both type-safe (`th3index`) and arithmetic-friendly (`tbigint`) views of the same trajectory in a single query.

Chapter 17

Temporal QUADBIN Cell Indices

The temporal QUADBIN cell index type `tquadbin` represents the movement of objects as a sequence of QUADBIN square cell identifiers. **CARTO's QUADBIN** is a hierarchical square grid system built on the slippy-map (web-mercator) tile scheme: a single world cell at resolution 0 subdivides into four equal children at every finer resolution (up to 26). Each cell has a 64-bit integer identifier that encodes a header tag, the resolution, and the tile coordinates of the cell.

The `tquadbin` type shares its on-disk representation with `tbigint`: both store a temporal 64-bit integer. The distinct SQL type exists purely to let the type-checker reject quadbin-specific functions when applied to arbitrary bigint trajectories (and vice-versa). Casts to / from `tbigint` are *explicit* (AS ASSIGNMENT) and binary-coercion only — zero-cost at runtime but requiring an explicit `::` in SQL.

As with other temporal types, `tquadbin` has four subtypes: `Instant`, `discrete-sequence`, `continuous-sequence` with step interpolation (QUADBIN cells are discrete — linear interpolation is meaningless), and `SequenceSet`.

Operational guidance on when to choose `tquadbin` over `tgeompoint` or `tgeogpoint`; the quadbin-specific hazards consumers should anticipate (int64 ordering arbitrary with respect to grid geometry, resolution mixing in operations, web-mercator latitude limit, antimeridian behaviour); and the durability and storage conventions are collected in Section 17.12.

17.1 Input and Output

A `tquadbin` value is entered using the standard temporal syntax with the underlying 64-bit QUADBIN cell identifier. The input parser accepts both the canonical hexadecimal form and the equivalent unsigned decimal. Hexadecimal is idiomatic in the QUADBIN ecosystem and is what the output function emits — all examples below use it. For reference, the hex value `480fffffffffffffff` used in the examples is the resolution-0 world cell, equal to the decimal value 5192650370358181887.

```
SELECT tquadbin '480fffffffffffffff@2001-01-01';
SELECT tquadbin '{480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02}';
SELECT tquadbin '[480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02]';
SELECT tquadbin '{[480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02],
[48a6227afffffffffff@2001-01-03, 485623fffffffffff@2001-01-04]}';
```

The type accepts type modifiers (`typmod`) to constrain a column to a specific subtype: `Instant`, `Sequence`, or `SequenceSet`.

```
SELECT tquadbin(Instant) '480fffffffffffffff@2001-01-01';
SELECT tquadbin(SequenceSet) '{[480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02]}' ←
;
-- ERROR: temporal type (Instant) does not match column type (Sequence)
SELECT tquadbin(Sequence) '480fffffffffffffff@2001-01-01';
```

17.2 Conversions

A `tquadbin` value shares the same on-disk representation as `tbigint`. The binary-coercion cast between them is *explicit* — requiring `::` — so quadbin-specific functions cannot be silently called on arbitrary bigint trajectories.

- Convert between a `tquadbin` and a `tbigint` (explicit, zero-cost)

`tbigint::tquadbin`

`tquadbin::tbigint`

```
SELECT (tbigint '5192650370358181887@2001-01-01')::tquadbin;
-- 480fffffffffffffff@2001-01-01
SELECT ((tquadbin '480fffffffffffffff@2001-01-01')::tbigint)::tquadbin
= tquadbin '480fffffffffffffff@2001-01-01';
-- true
```

- Cast a temporal QUADBIN cell to a temporal planar (SRID 4326) point of cell centroids. The cast delegates to `tquadbinCellToPoint`

`tquadbin::tgeompoint`

```
SELECT (tquadbin '480fffffffffffffff@2001-01-01')::tgeompoint;
```

17.3 Static Cells

The companion static type `quadbin` holds a single (non-temporal) QUADBIN cell identifier; `quadbinset` holds a finite set of them. They share the same hexadecimal / decimal textual representation as the temporal type. The static type carries the standard equality / ordering operators (a `btree` and a `hash` operator class), and exposes the validity predicates below.

- Return whether a value is a structurally valid QUADBIN identifier (correct header tag and resolution field)

`isValidIndex(quadbin) → boolean`

- Return whether a value is a valid QUADBIN cell (a valid index whose tile coordinates fall within the bounds of its resolution)

`isValidCell(quadbin) → boolean`

```
SELECT isValidCell(quadbin '480fffffffffffffff');
-- true
SELECT isValidCell(quadbin '0');
-- false
```

17.4 Accessors

- Return the first or last QUADBIN cell identifier of a temporal value

`startValue(tquadbin) → quadbin`

`endValue(tquadbin) → quadbin`

```
SELECT startValue(tquadbin '480fffffffffffffff@2001-01-01');
-- 480fffffffffffffff
SELECT endValue(tquadbin '{480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02}');
-- 48427fffffffffffffff
```

- Return the *n*th distinct QUADBIN cell identifier (1-indexed). Returns `NULL` if out of range.

`valueN(tquadbin, integer) → quadbin`


```
SELECT quadbinCellToParent(quadbin '48a6227affffffff', 5);
-- 485623fffffffff
SELECT tquadbinGetResolution(
  tquadbinCellToParent(tquadbin '48a6227affffffff@2001-01-01', 5));
-- 5@2001-01-01
```

- Return the four child cells at the requested finer resolution as a quadbinset. This is a static set-returning helper on a single quadbin; it has no temporal lift (children are a per-instant set, deferred pending a tset<T> primitive).

quadbinCellToChildren(quadbin, integer) → quadbinset

```
SELECT quadbinCellToChildren(quadbin '480fffffffffffff', 1);
-- {4813fffffffffffff, 4817fffffffffffff, 481bfffffffffffff, 481fffffffffffff}
```

- Return the neighbouring cell at the same resolution in the given direction (up / down / left / right)

quadbinCellSibling(quadbin, text) → quadbin

```
SELECT quadbinCellSibling(
  quadbinCellSibling(quadbin '48427fffffffffffff', 'right'), 'left')
= quadbin '48427fffffffffffff';
-- true
```

- Return all cells within k grid steps of the origin (inclusive of the origin at k=0), as a quadbinset. On the square grid the disk at step k is the $(2k+1) \times (2k+1)$ square block of cells centred on the origin.

quadbinGridDisk(quadbin, integer) → quadbinset

```
SELECT numValues(quadbinGridDisk(quadbin '48a6227affffffff', 0)); -- 1
SELECT numValues(quadbinGridDisk(quadbin '48a6227affffffff', 1)); -- 9
SELECT numValues(quadbinGridDisk(quadbin '48a6227affffffff', 2)); -- 25
```

17.7 Geometry Conversions

- Return the QUADBIN cell at the given resolution that contains a point geometry. The geometry must be a POINT; longitudes are interpreted in SRID 4326.

geoToQuadbinCell(geometry, integer) → quadbin

```
SELECT geoToQuadbinCell(geometry 'SRID=4326;POINT(4.35 50.85)', 10)
= quadbin '48a6227affffffff';
-- true
```

- Return the centroid of a cell as an SRID-4326 point. The static form takes a single quadbin; the temporal form returns the centroid of each cell in a trajectory as a tgeompoint.

quadbinCellToPoint(quadbin) → geometry

tquadbinCellToPoint(tquadbin) → tgeompoint

- Return the square polygon boundary of a cell as an SRID-4326 geometry. The static form takes a single quadbin; the temporal form returns the boundary of each cell in a trajectory as a tgeometry.

quadbinCellToBoundary(quadbin) → geometry

tquadbinCellToBoundary(tquadbin) → tgeometry

17.8 Tiles and Quadkeys

QUADBIN is built on the slippy-map (web-mercator) tile scheme, so a cell maps directly to a tile coordinate triple (x , y , z) and to the base-4 quadkey string used by tile servers. These conversions are the QUADBIN-specific bridge to the tiling ecosystem.

- Build a cell from slippy-map tile coordinates: column x , row y , and zoom z

`quadbinTileToCell(integer, integer, integer) → quadbin`

```
SELECT quadbinTileToCell(0, 0, 0) = quadbin '480fffffffffffff';
-- true
SELECT quadbinTileToCell(3, 5, 4) = quadbin '48427fffffffffffff';
-- true
SELECT quadbinTileToCell(524, 343, 10) = quadbin '48a6227afffffffffff';
-- true
```

- Decompose a cell back into its tile coordinate triple, returned as an integer array $\{x, y, z\}$ — the inverse of `quadbinTileToCell`

`quadbinCellToTile(quadbin) → integer[]`

```
SELECT quadbinCellToTile(quadbin '480fffffffffffff');
-- {0,0,0}
SELECT quadbinCellToTile(quadbin '48a6227afffffffffff');
-- {524,343,10}
```

- Emit the base-4 slippy-tile quadkey string of a cell (one digit per zoom level, MSB-first; the resolution-0 world cell maps to the empty string). The static form takes a single `quadbin`; the temporal form returns the quadkey of each cell in a trajectory as a `ttext`.

`quadbinCellToQuadkey(quadbin) → text`

`tquadbinCellToQuadkey(tquadbin) → ttext`

```
SELECT quadbinCellToQuadkey(quadbin '480fffffffffffff');
-- (empty string)
SELECT quadbinCellToQuadkey(quadbin '48427fffffffffffff');
-- 0213
SELECT quadbinCellToQuadkey(quadbin '48a6227afffffffffff');
-- 1202021322
```

17.9 Metrics

- Return the area of a cell on the sphere, in square metres. The static form takes a single `quadbin`; the temporal form returns the area of each cell in a trajectory as a `tfloat`. Cell area shrinks toward the poles, as web-mercator squares cover progressively less ground at higher latitudes.

`quadbinCellArea(quadbin) → float8`

`tquadbinCellArea(tquadbin) → tfloat`

17.10 Comparisons

The B-tree comparison operators (`=`, `<>`, `<`, `<=`, `>`, `>=`) and the hash operator are defined on `tquadbin` through the generic temporal comparison machinery, ordering by the underlying temporal-value sequence. They support the `tquadbin_btree_ops` and `tquadbin_hash_ops` operator classes used by GROUP BY, DISTINCT, ORDER BY, and merge / hash joins. As with `tbigint`, the `int64` ordering of QUADBIN identifiers carries no geographic meaning — see Section 17.12.

17.11 Indexing

B-tree and hash operator classes are defined so the comparison operators of the previous section can be used with corresponding indexes.

- Default B-tree operator class for temporal QUADBIN cells, ordering by the underlying temporal value.

```
CREATE INDEX ON trips USING btree(cells);           -- uses tquadbin_btree_ops by default
```

- Default hash operator class for temporal QUADBIN cells, used by hash joins and hash aggregation.

```
CREATE INDEX ON trips USING hash(cells);           -- uses tquadbin_hash_ops by default
```

17.12 Operational notes

This section captures the operational knowledge consumers need on top of the function reference: when `tquadbin` is the right type, the conventions the implementation assumes, and the QUADBIN-specific hazards real workloads need to be aware of. The QUADBIN hierarchical square grid has a few non-obvious properties that the bitwise `int64` encoding hides; this section surfaces them so they don't surprise consumers.

17.12.1 When to use `tquadbin`

Use `tquadbin` when the natural identifier for a discrete location at a given resolution is a QUADBIN tile — vehicle trips bucketed at a fixed zoom level, sensor readings binned by tile, or events keyed by their containing cell, especially in pipelines that already speak the slippy-map / web-mercator tiling scheme. The type stores one QUADBIN cell per instant; resolution may vary across instants in a single trajectory but most workloads pin one resolution per column.

Choose `tgeompoint` / `tgeogpoint` over `tquadbin` when the actual geographic position (sub-cell precision) is the value of interest. Choose `tbigint` over `tquadbin` when the values happen to be 64-bit integers but carry no QUADBIN semantics — the binary representations are identical (they cast losslessly via `tquadbin :: tbigint` and back) but the SQL type system uses the distinction to reject quadbin-agnostic functions on quadbin-specific trajectories.

17.12.2 QUADBIN-specific hazards

A few hazards reliably bite workloads using QUADBIN cells. Each is a property of the QUADBIN grid itself rather than a MobilityDB implementation choice; the mitigations below explain how to work with them.

- *Int64 ordering is arbitrary with respect to grid geometry.* QUADBIN cell identifiers are 64-bit integers whose bitpattern encodes a header tag, resolution, and tile coordinates. The bitwise ordering of two cells carries no spatial-proximity meaning — two cells that are bit-adjacent may be far apart, and two cells that are spatially adjacent may have different bitpatterns.

Mitigation: `tquadbin` deliberately has no `quadbinspan` / `quadbinspanset` companion types (precedent: `geometry` has no `geometryspan`). Value-range filtering must go through the `quadbin_*` inspection functions (resolution, validity, hierarchy) or explicit set enumeration via `quadbinset`. Code that assumes `cell_a < cell_b` reflects spatial proximity will silently produce wrong results.

- *Resolution mixing in operations.* QUADBIN cells at different resolutions (0–26) represent different coverage areas. Mixing resolutions in a single trajectory is valid but semantically requires explicit justification — `quadbinCellToParent(cell, coarser_res)` coarsens and `quadbinCellToChildren(parent, finer_res)` refines.

Mitigation: consumers should document the resolution invariant per trajectory (e.g. "all cells are resolution 10") and validate inputs at the ingestion boundary. The `quadbinGetResolution` accessor lets a `CHECK` constraint enforce this.

- *Web-mercator latitude limit.* QUADBIN inherits the web-mercator projection, which is defined only between approximately $\pm 85.05^\circ$ latitude. Points outside this band have no QUADBIN cell, and cell area shrinks toward the poles — `quadbinCellArea` returns progressively smaller values at higher latitudes for cells of the same resolution.

Mitigation: workloads near the poles should clamp or reject latitudes outside the web-mercator band at the ingestion boundary; do not rely on QUADBIN cells for polar coverage.

- *Antimeridian behaviour.* Tile columns wrap at the antimeridian ($\pm 180^\circ$ longitude). A trajectory crossing the antimeridian moves between cells whose tile `x` coordinates differ by the full width of the grid at that zoom, even though they are spatially adjacent.

Mitigation: workloads that cross the antimeridian should validate representative cells against the QUADBIN reference implementation and add fixtures for the specific edge cases the workload encounters.

17.12.3 Durability and storage

The on-disk representation of `tquadbin` is byte-identical to `tbigint` (each instant carries one 64-bit QUADBIN cell ID plus a timestamp). Consequences for storage planning:

- *WKT* via `tquadbin_in` and the temporal output function. Cells render as canonical hex strings (e.g. `'480fffffffffffffff'`); both decimal and hex are accepted on input. Round-trip is bit-stable.
- *WKB / EWKB / HexWKB* via the standard PostGIS endian-flag-then-payload pattern. Stable across PostgreSQL major versions.
- *pg_dump* uses WKT in plain mode and WKB under `--binary-upgrade`; both round-trip cleanly.
- *Cross-cast with tbigint:* the bidirectional `tquadbin :: tbigint` casts are zero-cost bitwise reinterpretations. Use them when working with both type-safe (`tquadbin`) and arithmetic-friendly (`tbigint`) views of the same trajectory in a single query.

Chapter 18

Temporal Point Clouds

The [pgPointCloud](#) extension stores LIDAR point cloud data in PostgreSQL using two static types: `pcpoint`, a single multi-dimensional point whose dimensions are described by a *schema* stored in the `pointcloud_formats` catalog table; and `pcpatch`, a compressed batch of `pcpoint` values that share the same `pcid` (schema id). Each schema fixes the dimensions present (X, Y, Z, Intensity, ...), their numeric type, scale, and offset.

MobilityDB lifts these types into the temporal world via `tpcpoint` (a moving LIDAR/GPS sensor) and `tpcpatch` (a time series of compressed point clusters). It also adds set types `pcpointset` / `pcpatchset` and the spatiotemporal bounding-box type `tpcbox`. A complete reference for the static `pcpoint` and `pcpatch` types is given in the [pgPointCloud documentation](#); this chapter covers MobilityDB's additions on top of `pgPointCloud`.

To use the types described here, MobilityDB must be built with the `POINTCLOUD=ON` CMake option. On such a build the generated `mobilitydb.control` declares `requires = 'postgis, pointcloud'`, so a single `CASCADE` creates the full stack:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE: installing required extension "postgis"
-- NOTICE: installing required extension "pointcloud"
```

18.1 Quickstart

This section walks through a minimal end-to-end example: registering a schema, building values, lifting to a temporal type, querying with the `bbox` surface, and indexing. The goal is to make every later section in this chapter pin to a concrete usage. Follow the steps in order against a fresh database.

18.1.1 Register a schema

Every `pcpoint` and `pcpatch` value carries a `pcid` that resolves through `pointcloud_formats` to an XML schema. The schema declares the dimensions (X, Y, Z, ...), their on-disk numeric type, and per-dimension scale. A minimal 3D schema with all dimensions stored as `int32` at unit scale:

```
INSERT INTO pointcloud_formats (pcid, srid, schema) VALUES (1, 4326, '
<?xml version="1.0" encoding="UTF-8"?>
<pc:PointCloudSchema xmlns:pc="http://pointcloud.org/schemas/PC/1.1">
  <pc:dimension><pc:position>1</pc:position><pc:size>4</pc:size>
    <pc:name>X</pc:name><pc:interpretation>int32_t</pc:interpretation>
    <pc:scale>1</pc:scale></pc:dimension>
  <pc:dimension><pc:position>2</pc:position><pc:size>4</pc:size>
    <pc:name>Y</pc:name><pc:interpretation>int32_t</pc:interpretation>
    <pc:scale>1</pc:scale></pc:dimension>
  <pc:dimension><pc:position>3</pc:position><pc:size>4</pc:size>
```

```
<pc:name>Z</pc:name><pc:interpretation>int32_t</pc:interpretation>
<pc:scale>1</pc:scale></pc:dimension>
</pc:PointCloudSchema>');

```

Schemas are global to the database; you usually register them once at fixture-load time. SRID is per-schema and inherited by every value with that `pcid`.

18.1.2 Build base and temporal values

MobilityDB ships ergonomic constructors over `pgPointCloud`'s `PC_MakePoint` / `PC_Patch`:

```
-- a single point at (1, 2, 3) under pcid=1
SELECT pcpoint(1, 1.0, 2.0, 3.0);

-- a patch holding two points
SELECT pcpatch(1, pcpoint(1, 1.0, 2.0, 3.0),
               pcpoint(1, 4.0, 5.0, 6.0));

-- a single-instant tpcpoint at (10, 20, 30) at 2024-01-01
SELECT tpcpoint(pcpoint(1, 10, 20, 30), '2024-01-01'::timestampz);

-- a 3-instant moving sensor track (step interpolation, the default for
-- pointcloud temporals because dimensions like Intensity don't interpolate
-- linearly the way coordinates do)
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 0, 0, 0), '2024-01-01'::timestampz),
  tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-02'::timestampz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-03'::timestampz)]);

```

For long sessions, declare the column with a *typmod* so every row's `pcid` is enforced at INSERT time:

```
CREATE TABLE scans (id int, traj tpcpoint(1), full_scan tpcpatch(1));
INSERT INTO scans VALUES (1,
  tpcpointSeq(ARRAY[
    tpcpoint(pcpoint(1, 0, 0, 0), '2024-01-01'::timestampz),
    tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-02'::timestampz)]),
  tpcpatch(pcpatch(1, pcpoint(1,1,1,1), pcpoint(1,2,2,2)),
    '2024-01-01'::timestampz));

```

Inserting a value whose `pcid` disagrees with the column's *typmod* raises `ERROR: Pcid of tpcpoint value (N) does not match column typmod pcid (M); mixing pcids in an unconstrained column is also rejected at extent aggregation time.`

18.1.3 Query with the bbox surface

Every `tpcpoint` and `tpcpatch` carries a `tpcbox` (mirror of `stbox` + `pcid`) computed at construction time. The `bbox`-level operator surface — topological (`&&`, `@>`, `<@`, `~=`, `-|-`), directional on `X/Y/Z/time`, and `KNN` distance `|=|` — works between any pair of (`tpcbox`, `tpcpoint`, `tpcpatch`):

```
-- rows whose track ever entered a 3D-temporal box
SELECT id FROM scans
WHERE traj && tpcbox_zt(0, 0, 0, 1, 1, 1,
  tstzspan '[2024-01-01, 2024-01-02]', 1, 4326);

-- 5 nearest tracks to a query box (KNN, GiST-orderable)
SELECT id, traj |=| my_box AS dist FROM scans
ORDER BY traj |=| my_box LIMIT 5;

```

Patch-level filtering (drop entire instants whose patch does not overlap a box) is provided by `atTpcbox` / `minusTpcbox`. See [Per-point operations](#) for the granularity scope.

18.1.4 Index for bbox-driven scans

R-tree GiST and quadtree / kd-tree SP-GiST opclasses accelerate every bbox predicate. Pick GiST for general-purpose workloads (and for KNN ordering); SP-GiST kd-tree is competitive on read-heavy point-only data:

```
CREATE INDEX scans_traj_gist ON scans USING gist(traj);
CREATE INDEX scans_full_spgist ON scans USING spgist(full_scan);
```

The SP-GiST opclasses store an stbox internally and recover the poid filter at recheck time, so a column with mixed pcids returns a slightly larger candidate set for recheck — a non-issue when each table holds values of a single schema, which the typmod recipe in **Build base and temporal values** enforces.

18.1.5 Aggregate

```
-- spatiotemporal extent of every track (one tpcbox)
SELECT extent(traj) FROM scans;

-- count of active tracks at each timestamp (tint)
SELECT tcount(traj) FROM scans;

-- merge per-source instants into a single tpcpoint
SELECT id, merge(traj) FROM scans GROUP BY id;
```

extent rejects mixed pcids (the bbox dimensions would be uninterpretable across schemas); merge and tcount follow the same rules as their non-pointcloud counterparts.

18.1.6 Per-point access

The **points** set-returning function explodes each instant's patch into individual pcpoints, useful for joins against per-point predicates that the bbox layer cannot express:

```
-- explode every patch in the column into one row per (instant timestamp, point)
SELECT id, t, point FROM scans, points(full_scan);

-- total points across every instant of every track
SELECT id, numPoints(full_scan) FROM scans;
```

Per-point *filtering* — building a new patch from a subset of the original's points — is provided by **atTpcboxFine** / **minusTpcboxFine** (against a tpcbox) and **atGeometry** / **minusGeometry** (against a 2D geometry); see **Per-point operations** for the granularity matrix.

18.1.7 What's next

The rest of the chapter expands on each of the surfaces touched here: **pcpoint** / **pcpatch** accessors, **pcpointset** / **pcpatchset**, the **tpcbox bounding box**, **tpcpoint**, **tpcpatch**, **indexes**, and **aggregations**.

18.2 Static Types pcpoint and pcpatch

The static types come from pgPointCloud. MobilityDB exposes the schema-aware coordinate accessors needed to project pg-PointCloud values into the rest of the temporal-types ecosystem.

18.2.1 Ergonomic constructors

pgPointCloud's `pcpoint_in` only accepts hex-WKB, so the canonical builder is `PC_MakePoint` (`pcid`, `ARRAY[...]::float`). MobilityDB ships two thin SQL overloads on top of it for shorter test literals and ad-hoc queries; the underlying schema-dimension validation still happens inside `PC_MakePoint`.

- Build a 2D or 3D `pcpoint` directly from x/y(/z) coordinates

`pcpoint(pcid integer, x float, y float) → pcpoint`

`pcpoint(pcid integer, x float, y float, z float) → pcpoint`

```
SELECT pcpoint(1, 10.0, 20.0, 30.0);
-- equivalent to PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]::float[])
```

- Build a `pcpatch` from a variadic list of `pcpoints` sharing the same `pcid`

`pcpatch(pcid integer, VARIADIC points pcpoint[]) → pcpatch`

```
SELECT pcpatch(1, pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0));
-- equivalent to PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[1.0, 1.0, 1.0]::float[]),
--                          PC_MakePoint(1, ARRAY[2.0, 2.0, 2.0]::float[])])
```

18.2.2 Accessors

- Return the schema id of a `pcpoint` or `pcpatch`

`pcid(pcpoint) → integer`

`pcid(pcpatch) → integer`

```
SELECT pcid(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]));
-- 1
```

- Return a coordinate of a `pcpoint`, NULL if the dimension is absent from the schema

`getX(pcpoint) → float`

`getY(pcpoint) → float`

`getZ(pcpoint) → float`

```
WITH pt AS (SELECT PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]) p)
SELECT getX(p), getY(p), getZ(p) FROM pt;
-- 10 | 20 | 30
```

- Return any named dimension of a `pcpoint`, NULL if the name is unknown for the schema

`getDim(pcpoint, text) → float`

```
SELECT getDim(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), 'X');
-- 10
```

- Return the spatial reference identifier (inherited from the schema)

`SRID(pcpoint) → integer`

`SRID(pcpatch) → integer`

```
SELECT SRID(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]));
-- 0
```

18.3 Set Types `pcpointset` and `pcpatchset`

A `pcpointset` is an ordered set of distinct `pcpoint` values; a `pcpatchset` is the analogous set type for `pcpatch`. Both follow MobilityDB's general set semantics (see Chapter 2): values are deduplicated on construction, the binary representation is canonical, and all generic set-level operators (`=`, `<>`, `<`, `<=`, `>`, `>=`, `@>`, `<@`, `&&`, `+`, `-`, `*`) and accessors (`numValues`, `memSize`, `asText`, `asBinary`, `asHexWKB`, ...) are available.

All values in a set must share the same `pcid`. Mixing schemas in a single set raises an error at construction time.

18.3.1 Constructors

- Construct a set from an array of values, all of which must share the same `pcid`

```
set(pcpoint[]) → pcpointset
```

```
set(pcpatch[]) → pcpatchset
```

```
SELECT numValues(set(ARRAY[PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]),
  PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), -- duplicate, deduped
  PC_MakePoint(1, ARRAY[11.0, 21.0, 31.0])]));
-- 2
```

18.4 Bounding Box `tpcbox`

The `tpcbox` type is the spatiotemporal bounding box for the temporal point-cloud types. It mirrors the structure of `stbox` (see Chapter 3) — same X/Y/Z/T axes, same flag bits for which dimensions are present — and adds a single field: the `pcid`. Two `tpcbox` values with different `pcid` values are never considered to overlap, contain, equal, or be adjacent to each other, regardless of their geometric or temporal extent.

Carrying the `pcid` through bounding-box arithmetic guarantees that bounding-box pruning never drops a row whose schema differs from the query's schema, and never returns a hit across mismatched schemas.

18.4.1 Constructors

- Construct a `tpcbox` from explicit X/Y/Z/T bounds and a `pcid`

```
tpcbox(xmin,ymin,xmax,ymax,pcid,srid=0) → tpcbox
```

```
tpcbox_z(xmin,ymin,zmin,xmax,ymax,zmax,pcid,srid=0) → tpcbox
```

```
tpcbox_t(period,pcid) → tpcbox
```

```
tpcbox_xt(xmin,ymin,xmax,ymax,period,pcid,srid=0) → tpcbox
```

```
tpcbox_zt(xmin,ymin,zmin,xmax,ymax,zmax,period,pcid,srid=0) → tpcbox
```

```
SELECT tpcbox(0, 0, 10, 10, 1, 0);
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcbox_zt(0, 0, 0, 10, 10, 10, tstzspan '[2024-01-01, 2024-01-02]', 1, 0);
-- TPCBOX(ZT(((0,0,0),(10,10,10)), [2024-01-01, 2024-01-02]), 1)
```

18.4.2 Conversions

- Convert a `pcpoint` to a degenerate (zero-extent) `tpcbox`; SRID and `pcid` come from the schema

```
tpcbox(pcpoint) → tpcbox
```

```
SELECT tpcbox(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]));
-- TPCBOX(Z((10,20,30),(10,20,30)), 1)
```


- Convert a pcpatch to a tpcbox using the patch's embedded 2D PCBOUNDS; the second form takes an explicit SRID instead of looking it up from the schema

tpcbox(pcpatch) → tpcbox

tpcbox(pcpatch, integer) → tpcbox

```
SELECT tpcbox(PC_Patch(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0])));
```

18.4.3 Accessors

- Return whether a tpcbox has the X/Y, Z, or T dimensions set

hasX(tpcbox) → boolean

hasZ(tpcbox) → boolean

hasT(tpcbox) → boolean

```
WITH b AS (SELECT tpcbox(0, 0, 10, 10, 1, 0) AS b)
SELECT hasX(b), hasZ(b) FROM b;
-- t | f
```

- Return a coordinate bound, NULL if the corresponding dimension is absent

xmin(tpcbox) → float

xmax(tpcbox) → float

ymin(tpcbox) → float

ymax(tpcbox) → float

zmin(tpcbox) → float

zmax(tpcbox) → float

```
WITH b AS (SELECT tpcbox(0, 0, 10, 10, 1, 0) AS b)
SELECT xmin(b), xmax(b) FROM b;
-- 0 | 10
```

- Return a temporal bound, NULL if the box has no T dimension

tmin(tpcbox) → timestampz

tmax(tpcbox) → timestampz

```
SELECT tmin(tpcbox_t(tstzspan '[2024-01-01, 2024-01-02]', 1));
-- 2024-01-01 00:00:00+00
```

- Return the schema id and SRID

pcid(tpcbox) → integer

SRID(tpcbox) → integer

```
WITH b AS (SELECT tpcbox(0, 0, 10, 10, 7, 4326) AS b)
SELECT pcid(b), SRID(b) FROM b;
-- 7 | 4326
```

18.4.4 Transformations

- Return a `tpcbox` with coordinates rounded to a given number of decimal digits

`round(tpcbox, integer=0) → tpcbox`

```
SELECT round(tpcbox(0.123456, 1.234567, 2.345678, 3.456789, 1, 0), 2);
-- TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)
```

- Return a `tpcbox` with the SRID overwritten. Does not reproject coordinates — for that, project the underlying `tpcpoint` first and take the `bbox` of the result

`setSRID(tpcbox, integer) → tpcbox`

```
SELECT SRID(setSRID(tpcbox(0, 0, 10, 10, 1, 0), 4326));
-- 4326
```

18.4.5 Set Operations

- Return the union of two `tpcboxes`; both must share the same `pcid`

`tpcbox_union(tpcbox, tpcbox) → tpcbox`

`tpcbox + tpcbox → tpcbox`

```
SELECT tpcbox(0, 0, 5, 5, 1, 0) + tpcbox(3, 3, 10, 10, 1, 0);
-- TPCBOX(X((0,0),(10,10)), 1)
```

- Return the intersection of two `tpcboxes`, NULL if disjoint or `pcid` mismatch

`tpcbox_intersection(tpcbox, tpcbox) → tpcbox`

`tpcbox * tpcbox → tpcbox`

```
SELECT tpcbox(0, 0, 5, 5, 1, 0) * tpcbox(3, 3, 10, 10, 1, 0);
-- TPCBOX(X((3,3),(5,5)), 1)
```

Note: a generic box-vs-box minus is not provided, mirroring the convention for `stbox` and `tbox`: the difference of two boxes is not generally a box.

18.4.6 Topological Operators

All topological operators take two `tpcbox` values with matching `pcid`. A `pcid` mismatch always yields false (not an error) so that index scans behave intuitively.

- Return true if the first `tpcbox` contains the second

`tpcbox @> tpcbox → boolean`

```
SELECT tpcbox(0, 0, 10, 10, 1, 0) @> tpcbox(2, 2, 8, 8, 1, 0);
-- t
```

- Return true if the first `tpcbox` is contained in the second

`tpcbox <@ tpcbox → boolean`

```
SELECT tpcbox(2, 2, 8, 8, 1, 0) <@ tpcbox(0, 0, 10, 10, 1, 0);
-- t
```

- Return true if two `tpcboxes` overlap

`tpcbox && tpcbox → boolean`

```
SELECT tpcbox(0, 0, 5, 5, 1, 0) && tpcbox(3, 3, 10, 10, 1, 0); -- t
SELECT tpcbox(0, 0, 5, 5, 1, 0) && tpcbox(0, 0, 5, 5, 2, 0); -- f (pcid mismatch)
```

- Return true if two `tpcboxes` are exactly equal
`tpcbox ~= tpcbox → boolean`
- Return true if two `tpcboxes` are adjacent (share a boundary)
`tpcbox -|- tpcbox → boolean`

18.4.7 Position Operators

Sixteen axis-aligned position predicates, parallel to those on `stbox`. Each tests strict-or-overlap relationship along one axis. All require matching `pcid`; mismatch returns false. Z-axis predicates require both boxes to have a Z dimension; T-axis predicates require both to have a T dimension.

- Test the X-axis ordering of two `tpcboxes`
`tpcbox << tpcbox → boolean (strictly left)`
`tpcbox <<| tpcbox → boolean (does not extend right)`
`tpcbox >> tpcbox → boolean (strictly right)`
`tpcbox >>| tpcbox → boolean (does not extend left)`
- Test the Y-axis ordering of two `tpcboxes`
`tpcbox <<| tpcbox → boolean (strictly below)`
`tpcbox <<| tpcbox → boolean (does not extend above)`
`tpcbox |>> tpcbox → boolean (strictly above)`
`tpcbox |>>| tpcbox → boolean (does not extend below)`
- Test the Z-axis ordering of two `tpcboxes` (returns false if either lacks Z)
`tpcbox <</ tpcbox → boolean (strictly in front)`
`tpcbox <</ tpcbox → boolean (does not extend behind)`
`tpcbox />> tpcbox → boolean (strictly behind)`
`tpcbox />>| tpcbox → boolean (does not extend in front)`
- Test the time-axis ordering of two `tpcboxes` (returns false if either lacks T)
`tpcbox <<# tpcbox → boolean (strictly before)`
`tpcbox <<# tpcbox → boolean (does not extend after)`
`tpcbox #>> tpcbox → boolean (strictly after)`
`tpcbox #>>| tpcbox → boolean (does not extend before)`

18.4.8 Comparisons

- Compare two `tpcboxes`; total order over the canonical encoding (`pcid`, `srid`, `flags`, `period`, then spatial bounds in XYZ-min/max order)
`tpcbox = tpcbox → boolean`
`tpcbox <> tpcbox → boolean`
`tpcbox < tpcbox → boolean`
`tpcbox <= tpcbox → boolean`
`tpcbox > tpcbox → boolean`
`tpcbox >= tpcbox → boolean`

18.5 Temporal Type `tpcpoint`

A `tpcpoint` is the lifting of `pcpoint` through MobilityDB's temporal machinery. It supports the same three subtypes as the other temporal types — instant, sequence (with discrete or step interpolation), and sequence set. All instants in a single `tpcpoint` value must share the same `pcid`; the constructor enforces this.

The bounding box of a `tpcpoint` is a `tpcbox`, computed at construction time from the X/Y/Z dimensions of the underlying `pcpoint` values together with the timestamps of the instants.

A column or domain may pin the schema by adding the `pcid` as a single-integer `typmod`, exactly like PostGIS' `geometry(Point, SRID)`. Mixing schemas in such a column is rejected at INSERT or cast time. The same construct applies to `tpcpatch`.

```
CREATE TABLE scans (id int, traj tpcpoint(1), full_scan tpcpatch(1));
-- accepts pcid 1
INSERT INTO scans VALUES (1,
    tpcpoint(pcpoint(1, 1.0, 2.0, 3.0), '2024-01-01'::timestampz),
    tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
        '2024-01-01'::timestampz));
-- rejects any other pcid:
-- ERROR: Pcid of tpcpoint value (2) does not match column typmod pcid (1)
```

Omitting the `typmod` (`tpcpoint` with no parenthesised `pcid`) leaves the column schema-agnostic. The runtime check in the extent aggregate still rejects mid-query mixed-`pcid` input even on unconstrained columns.

18.5.1 Constructors

- Construct a temporal `pcpoint` of instant subtype

`tpcpoint(pcpoint,timestampz) → tpcpoint`

```
SELECT tpcpoint(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), '2024-01-01 12:00+00');
```

- Construct a temporal `pcpoint` of sequence subtype from an array of instants. The interpolation is one of 'discrete', 'step'; 'step' is the default for `tpcpoint` since `pcpoints` carry heterogeneous dimensions that do not interpolate linearly

`tpcpointSeq(tpcpoint[]) → tpcpoint`

`tpcpointSeq(tpcpoint[],text) → tpcpoint`

```
SELECT tpcpointSeq(ARRAY[tpcpoint(PC_MakePoint(1, ARRAY[1.0, 2.0, 3.0]), '2024-01-01':: ←
    timestampz),
    tpcpoint(PC_MakePoint(1, ARRAY[4.0, 5.0, 6.0]), '2024-01-02'::timestampz)], 'step');
```

- Construct a temporal `pcpoint` of sequence-set subtype from an array of sequences

`tpcpointSeqSet(tpcpoint[]) → tpcpoint`

18.5.2 Accessors

All generic temporal accessors apply (see Chapter 4): `numInstants`, `startInstant`, `endInstant`, `instantN`, `instants`, `startValue`, `endValue`, `getValues`, `numTimestamps`, `startTimestamp`, `endTimestamp`, `getTime`, `duration`, `memSize`, `interp`. In addition:

- Return the `pgPointCloud` schema id shared by all instants

`pcid(tpcpoint) → integer`

- Return the SRID inherited from the schema

`SRID(tpcpoint) → integer`

18.5.3 Per-Dimension Projections

- Project a tpcpoint onto a spatial dimension, yielding a tfloat

getX(tpcpoint) → tfloat

getY(tpcpoint) → tfloat

getZ(tpcpoint) → tfloat

```
SELECT startValue(getX(tpcpoint(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]),
    '2024-01-01'::timestampz)));
-- 10
```

- Project a tpcpoint onto any named schema dimension, yielding a tfloat

getDim(tpcpoint, text) → tfloat

18.5.4 Conversions

- Cast a tpcpoint to a temporal geometry point, projecting away every non-spatial dimension while preserving timestamps. The result has the same SRID as the schema; its dimensionality (2D vs. 3D) follows the presence of Z in the schema. A reverse cast is not provided: reconstructing a pcpoint requires a target schema that the temporal value alone cannot supply

tgeompoint::tpcpoint is not defined

tpcpoint::tgeompoint

```
SELECT ST_AsText(startValue(tpcpoint(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]),
    '2024-01-01'::timestampz)::tgeompoint));
-- POINT Z (10 20 30)
```

18.5.5 Comparisons

- Compare two tpcpoints lexicographically over their canonical encodings

tpcpoint = tpcpoint → boolean

tpcpoint <> tpcpoint → boolean

tpcpoint < tpcpoint → boolean

tpcpoint <= tpcpoint → boolean

tpcpoint > tpcpoint → boolean

tpcpoint >= tpcpoint → boolean

- Test whether the value at any (ever) or every (always) instant equals a given pcpoint

tpcpoint ?= pcpoint → boolean

tpcpoint %= pcpoint → boolean

18.5.6 Restrictions

- Restrict a tpcpoint to (the complement of) a pcpoint value or a set of pcpoint values

atValue(tpcpoint, pcpoint) → tpcpoint

minusValue(tpcpoint, pcpoint) → tpcpoint

atValues(tpcpoint, pcpointset) → tpcpoint

minusValues(tpcpoint, pcpointset) → tpcpoint

- Restrict a `tpcpoint` to a time selector (timestamp, period, set, or spanset), or remove it


```
atTime(tpcpoint,timestamptz) → tpcpoint
atTime(tpcpoint,tstzspan) → tpcpoint
atTime(tpcpoint,tstzset) → tpcpoint
atTime(tpcpoint,tstzspanset) → tpcpoint
minusTime(tpcpoint,timestamptz) → tpcpoint
minusTime(tpcpoint,tstzspan) → tpcpoint
minusTime(tpcpoint,tstzset) → tpcpoint
minusTime(tpcpoint,tstzspanset) → tpcpoint
```
- Restrict a `tpcpoint` to the spatial / temporal extent of a `tpcbox`, or remove it. Returns NULL when the `tpcbox`'s `pcid` does not match the `tpcpoint`'s. Internally projects to a `tgeompoint` via the schema cache, restricts the projection by the equivalent `stbox`, and lifts the result's time span back onto the original `tpcpoint` to preserve the full `pcpoint` values


```
atTpcbox(tpcpoint,tpcbox,border_inc bool=TRUE) → tpcpoint
minusTpcbox(tpcpoint,tpcbox,border_inc bool=TRUE) → tpcpoint
```

18.5.7 Bounding-box operators

The `bbox` operators evaluate against the value's `tpcbox`; see Section 18.4.6 and Section 18.4.7 for the per-operator geometry. Each operator below is wired against `tpcbox`, `tstzspan`, and `tpcpoint` itself.

- Topological predicates: `&&` overlaps, `@>` contains, `<@` contained by, `~=` same, `-|-` adjacent.
- Strict directional predicates on the X axis (`<<`, `>>`), Y axis (`<<|`, `|>>`), Z axis (`<</`, `/>>`), and time axis (`<<#`, `#>>`), plus their "overlaps-or-X" variants `&<`, `&>`, `&<|`, `|&>`, `&</`, `/&>`, `&<#`, `#&>`.
- Nearest-approach distance, KNN-orderable through the GiST opclass: `SELECT ... ORDER BY temp |=| query LIMIT N` is index-accelerated. `Pcid` mismatch yields infinity.


```
nearestApproachDistance(tpcpoint,tpcbox) → float
nearestApproachDistance(tpcpoint,tpcpoint) → float
```

18.5.8 B-tree ordering — what ORDER BY tpcpoint means

The `tpcpoint_btree_ops` opclass produces a total order over `tpcpoint` values, but the order is byte-wise on the underlying `pcpoint` serialisation, not geometric. The same caveats that apply to [tpcpatch's B-tree](#) apply here:

- **pcid is the primary discriminator.** Points with smaller `pcid` sort before points with larger `pcid`, regardless of coordinates. Mixing `pcids` in a single column groups rows by schema first when ordering.
- **Within a single pcid, ordering follows the schema's on-disk dimension layout** — typically X, Y, Z encoded as `int32` (after applying the schema's per-dimension scale). Two points whose Euclidean distance is small can sort far apart if their X dimensions differ, since X is compared first.
- **Equality is exact-bytes equality.** Two `pcpoints` with the same dimension values in the same schema are equal; the same coordinate set encoded in a schema with different scale or dimension order is not.

For spatial-meaningful ordering, project to a `tgeompoint` via the schema-aware cast and use PostGIS' KNN `<->` on the resulting geometry, or use the GiST KNN operator `!=|` directly on the `tpcpoint`. For time-meaningful ordering, project to `startTimestamp` or `timespan` and order on that.

18.5.9 Spatial relationships

Evaluated by projecting the `tpcpoint` to a `tgeompoint` via the schema cache and delegating to the corresponding `tgeompoint` relation. Each function is wired in three argument orders: `(geometry, tpcpoint)`, `(tpcpoint, geometry)`, and `(tpcpoint, tpcpoint)`. `tpcpatch` is intentionally out of scope — patch-level intersection would require per-point decomposition and a different design.

- Ever / always intersects.

```
eIntersects(tpcpoint, geometry) → bool, aIntersects(tpcpoint, geometry) → bool
eIntersects(tpcpoint, tpcpoint) → bool, aIntersects(tpcpoint, tpcpoint) → bool
```

- Ever / always disjoint.

```
eDisjoint(tpcpoint, geometry) → bool, aDisjoint(tpcpoint, geometry) → bool
eDisjoint(tpcpoint, tpcpoint) → bool, aDisjoint(tpcpoint, tpcpoint) → bool
```

- Ever / always within distance.

```
eDwithin(tpcpoint, geometry, float) → bool, aDwithin(tpcpoint, geometry, float) → bool
eDwithin(tpcpoint, tpcpoint, float) → bool, aDwithin(tpcpoint, tpcpoint, float) → bool
```

18.6 Temporal Type `tpcpatch`

A `tpcpatch` is the lifting of `pcpatch`. It mirrors `tpcpoint` in every regard — same subtypes, same accessors, same casts and operators — except that each instant carries an entire compressed batch of points instead of a single point. The bounding box is again a `tpcbox`, computed in $O(1)$ per instant from the patch's embedded `PCBOUNDS`.

18.6.1 Constructors

- Construct a temporal `pcpatch` of instant subtype

```
tpcpatch(pcpatch, timestampz) → tpcpatch
```

- Construct a temporal `pcpatch` of sequence subtype

```
tpcpatchSeq(tpcpatch[]) → tpcpatch
tpcpatchSeq(tpcpatch[], text) → tpcpatch
```

- Construct a temporal `pcpatch` of sequence-set subtype

```
tpcpatchSeqSet(tpcpatch[]) → tpcpatch
```

18.6.2 Accessors

All the generic temporal accessors apply, plus the `tpcpoint`-style `pcid` and `SRID`. In addition:

- Return the number of points in the patch carried by the first or last instant

```
startNumPoints(tpcpatch) → integer
endNumPoints(tpcpatch) → integer
```

```
SELECT startNumPoints(tpcpatch(PC_Patch(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0])),
    '2024-02-01'::timestampz));
-- 1
```

- Return the total number of points across every instant's patch. Read directly from each patch's header — no decompression.

```
numPoints(tpcpatch) → bigint
```

```
SELECT numPoints(tpcpatchSeq(ARRAY[
  tpcpatch(PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[1.0, 1.0, 1.0]),
    PC_MakePoint(1, ARRAY[2.0, 2.0, 2.0])])),
    '2024-01-01'::timestamp),
  tpcpatch(PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[3.0, 3.0, 3.0])]),
    '2024-01-02'::timestamp)]));
-- 3
```

- Set-returning function: emits one row per (instant timestamp, point) by walking each instant's patch and decomposing it through pgPointCloud's in-memory C API. Useful for joining a tpcpatch column against per-point predicates that the bbox-level operators can't express. Cost is O(total points) — one per-instant decompression plus one row emission per point.

points(tpcpatch) → setof (t timestamp, point pcpoint)

```
SELECT t, point FROM points(tpcpatch(
  PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[1.0, 1.0, 1.0]),
    PC_MakePoint(1, ARRAY[2.0, 2.0, 2.0])])),
  '2024-01-01'::timestamp);
-- ('2024-01-01', '01:000000000000000000000000F03F0000000000000000F03F0000000000000000F03F':: ↵
  pcpoint)
-- ('2024-01-01', '01:0000000000000000000000004000000000000000400000000000000040'::pcpoint ↵
  )
```

See Section 18.6.7 for the limits of these accessors and what is and is not available at per-point granularity.

18.6.3 Restrictions

- Restrict a tpcpatch to (the complement of) a pcpatch value or a set of pcpatch values

atValue(tpcpatch, pcpatch) → tpcpatch

minusValue(tpcpatch, pcpatch) → tpcpatch

atValues(tpcpatch, pcpatchset) → tpcpatch

minusValues(tpcpatch, pcpatchset) → tpcpatch

- Restrict a tpcpatch to the instants whose patch passes a coarse PCBOUNDS overlap test against the tpcbox, or remove them. Granularity is patch-level: each surviving instant keeps its pcpatch payload verbatim, with no per-point decompression. Because pgPointCloud's PCBOUNDS is 2D, the Z dimension of the box is ignored at this granularity. Returns NULL when the tpcbox's pcid does not match.

atTpcbox(tpcpatch, tpcbox, border_inc bool=TRUE) → tpcpatch

minusTpcbox(tpcpatch, tpcbox, border_inc bool=TRUE) → tpcpatch

- Per-point variant of the previous: each surviving instant carries a freshly-built pcpatch holding only the points inside (or outside, for minus) the tpcbox in 2D — and 3D when the box has a Z dimension. Instants whose patch filters to zero points are dropped. Slower than the coarse variant because every patch is decompressed and rebuilt; use when you need point-level fidelity.

atTpcboxFine(tpcpatch, tpcbox, border_inc bool=TRUE) → tpcpatch

minusTpcboxFine(tpcpatch, tpcbox, border_inc bool=TRUE) → tpcpatch

- Restrict a tpcpatch to the points whose XY projection intersects (or does not intersect, for minus) a 2D geometry. Z is ignored. SRID compatibility between the patch schema and the geometry must be ensured by the caller.

atGeometry(tpcpatch, geometry) → tpcpatch

minusGeometry(tpcpatch, geometry) → tpcpatch

18.6.4 Spatial relationships

- Return true iff at least one point in any of the `tpcpatch`'s instants intersects the geometry (XY only).

`eIntersects(tpcpatch, geometry) → boolean`

18.6.5 Bounding-box operators

The same `bbox` operator surface as Section 18.5.7 applies to `tpcpatch`. The predicate evaluates against the value's `tpcbox`, computed in $O(1)$ per instant from the patch's embedded `PCBOUNDS`.

- Topological predicates: `&&`, `@>`, `<@`, `~>`, `-|`. See Section 18.4.6.
- Strict directional predicates on the X / Y / Z / time axes (`<<`, `>>`, `<<|`, `|>>`, `<</`, `/>>`, `<<#`, `#>>`) and their "overlaps-or-X" variants (`&<`, `&>`, `&<|`, `|&>`, `&</`, `/&>`, `&<#`, `#&>`). See Section 18.4.7.
- Nearest-approach distance (`|=|`) is KNN-orderable through the GiST opclass.

`nearestApproachDistance(tpcpatch, tpcbox) → float`

`nearestApproachDistance(tpcpatch, tpcpatch) → float`

18.6.6 B-tree ordering — what `ORDER BY tpcpatch` means

The `tpcpatch_btree_ops` opclass produces a total order over `tpcpatch` values, but the order is not spatial. The comparator on the underlying `pcpatch` is byte-wise (`memcmp` over the meaningful varlena bytes); the temporal-level comparator is the lexicographic combination of per-instant timestamps and per-instant `pcpatch` byte order. The observable consequences:

- **pcid is the primary discriminator.** A patch with a smaller `pcid` sorts before one with a larger `pcid`, regardless of point coordinates, count, or compression. This makes `ORDER BY` on a mixed-`pcid` column group rows by schema first, which is sometimes useful as a coarse-grained "chunk by source schema" pass.
- **Within a single `pcid`, ordering follows the on-disk byte layout:** `compression` scheme, then `npoints`, then the 2D `PCBOUNDS` (`xmin`, `xmax`, `ymin`, `ymax`), then the compressed data payload. This is well-defined and stable for a given `pgPointCloud` release, but it is *not* a geometric or spatial ordering — two patches whose bounding boxes overlap in 3D space can sort arbitrarily relative to each other if their compression scheme or point count differs.
- **Equality is exact-bytes equality** over the meaningful bytes (the trailing zero-padding `pgPointCloud`'s varlena reserves is excluded). Two patches built from the same point set in the same order, with the same compression, are equal; reordering the points or changing compression makes them unequal.

For spatial-meaningful ordering, use the GiST KNN operator `|=|` (see [nearest-approach distance on tpcpatch](#)); for time-meaningful ordering, project to `startTimeStamp` or `timespan` and order on that.

18.6.7 Per-point operations: what is and is not available

Operations on `tpcpatch` values come at two granularities: *patch-level* and *per-point*. The split matters because `pgPointCloud` patches store their points in a compressed payload that the `bbox` layer cannot inspect.

- **Patch-level — supported.** Each instant's patch is treated as opaque and is kept or dropped as a whole, using only the 4-double `PCBOUNDS` header (`xmin` / `xmax` / `ymin` / `ymax`) and the instant's timestamp. This is the granularity of `atTpcbox` / `minusTpcbox` and the bounding-box operators in Section 18.6.5, and of the `bbox`-driven `tpcbox` aggregate. No payload decompression happens; queries are $O(\text{number of instants})$.

Because `PCBOUNDS` is 2D, the Z dimension of any `tpcbox` argument is ignored at this granularity even when the underlying schema has a Z dimension.

- **Per-point inspection — supported.** The `points` set-returning function emits one row per (instant timestamp, point) by decomposing each instant's patch through pgPointCloud's in-memory C API. Cost is $O(\text{total points})$ — one per-instant decompression plus one row emission per point.
- **Per-point filtering / construction — supported.** `atTpcboxFine / minusTpcboxFine`, `atGeometry / minusGeometry`, and `eIntersects(tpcpatch, geometry)` walk every point of every instant in C, applying their predicate against the actual point coordinates rather than just the patch bounding box. Surviving instants carry a freshly-built patch holding only the points that passed; instants whose patches filter to zero points are dropped.

The recommended workflow when both granularities are available is: (a) prune at the patch level with `atTpcbox` or an SP-GiST/GiST index scan to drop entire instants whose PCBOUNDS doesn't overlap the area of interest, and (b) refine on the survivors with `atTpcboxFine / atGeometry` when point-level fidelity is needed.

18.7 Indexes

B-tree (`tpcpoint_btree_ops`, `tpcpatch_btree_ops`, `tpcbox_btree_ops`) and hash (`tpcpoint_hash_ops`, `tpcpatch_hash_ops`) operator classes support equality and ordering predicates. The R-tree GiST and quadtree / kd-tree SP-GiST operator classes accelerate the bbox-based predicates (`&&`, `@>`, `<@`, `~`, `=`, `-|-`) on all three of `tpcbox`, `tpcpoint`, and `tpcpatch`:

```
-- GiST (R-tree)
CREATE INDEX ON my_table USING gist(my_tpcpoint_column);
CREATE INDEX ON my_table USING gist(my_tpcpatch_column);

-- SP-GiST quadtree (default) or kd-tree
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column);
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column tpcpoint_kdtree_ops);
```

SP-GiST opclasses store an `stbox` internally (the `pcid` filter is recovered by the operator's recheck), so a query against an index on a column whose values mix multiple `pcids` will return a slightly larger set of candidate rows for recheck — a non-issue when each table holds values of a single schema, which is the common case.

18.8 Aggregations

- Bounding-box aggregate over a column of `tpcpoint`, `tpcpatch`, or `tpcbox` values. Returns the smallest `tpcbox` containing every input. Aggregating across distinct `pcids` raises an error — the `bbox` would be uninterpretable since dimensions are `pcid`-relative.


```
extent(tpcpoint) → tpcbox
extent(tpcpatch) → tpcbox
extent(tpcbox) → tpcbox
```
- Temporal count and window count of overlapping rows at each timestamp. Result is a `tint`. All input rows must share the same temporal subtype (instant / sequence / sequence set).


```
tcount(tpcpoint) → tint, tcount(tpcpatch) → tint
wcount(tpcpoint, interval) → tint, wcount(tpcpatch, interval) → tint
```
- Per-instant `pcpatch` point count, summed across rows. Distinct from `tcount(tpcpatch)`, which counts the number of patches (always 1 per instant). `tnpoints` reads as "how many points were in the cloud at time `t`", the natural primitive for ingest-QC and density-over-time dashboards.


```
tnpoints(tpcpatch) → tint
```
- Per-instant point density (`numPoints / xy-bbox-area`) summed across rows. Each `pcpatch` carries the inline PCBOUNDS (`xmin / xmax / ymin / ymax`) so the `bbox-area` term is read directly — no recomputation. A 1-point or co-linear patch yields `+Infinity` for that instant; filter with `isfinite()` in downstream queries if needed.


```
tdensity(tpcpatch) → tfloat
```

- Combine multiple temporal values into a single one with all input instants merged in temporal order. All inputs must share the same `pcid` and subtype.

```
merge(tpcpoint) → tpcpoint, merge(tpcpatch) → tpcpatch
```

```
mergeAgg(tpcpoint) → tpcpoint, mergeAgg(tpcpatch) → tpcpatch
```

- Streaming construction aggregates: `appendInstant` assembles instants into a sequence; `appendSequence` assembles sequences into a sequence set. Useful for trajectory loaders that consume rows in time order.

```
appendInstant(tpcpoint[,interp,maxt]) → tpcpoint, appendInstant(tpcpatch) → tpcpatch
```

```
appendInstantAgg(tpcpoint[,interp,maxt]) → tpcpoint, appendInstantAgg(tpcpatch) → tpcpatch
```

```
appendSequence(tpcpoint) → tpcpoint, appendSequence(tpcpatch) → tpcpatch
```

```
appendSequenceAgg(tpcpoint) → tpcpoint, appendSequenceAgg(tpcpatch) → tpcpatch
```

18.9 Binary representation and portability

Both `tpcpoint` and `tpcpatch` support standard MobilityDB binary I/O through `asBinary / tpcpointFromBinary / tpcpatchFromBinary`, and through PostgreSQL's `COPY ... (FORMAT BINARY)`. The encoding embeds the `pcid` of each value alongside the dimension payload, so the binary blob is unambiguous about which schema the values were written under.

Whenever the encoding backend has the schema XML for a given `pcid` registered in its MEOS-level schema cache (which the PG extension populates lazily on first use, scanning `pointcloud_formats`), the encoder additionally embeds the schema XML in the WKB blob. The `MEOS_WKB_PCSCHEMAFLAG` bit (0x80) in the temporal-flags byte signals this. A receiver that does not yet know the `pcid` — for example, a different cluster after a `pg_dump --format=binary` — automatically parses and registers the embedded schema, so the import succeeds without an out-of-band schema preload step.

If the encoding backend has only a parsed `PCSCHEMA` (no source XML) for a `pcid`, the schema bit is left clear and the blob is portable only between backends that share the same `pointcloud_formats` catalog. This matches the behaviour of the prior, schema-less encoding and is the fallback path when the cache was populated by hand from a standalone-MEOS program.

The receiver also needs `mobilitydb_init` to have run — automatic in any backend that has loaded the `mobilitydb` extension, since the function-info module installs the XML-parse hook there.

18.10 MF-JSON output

- For `tpcpoint`, the temporal type is emitted as `{"type": "MovingPCPoint", ... "coordinates": [[X,Y,Z], ...], "datetimes": [...]}`. `X/Y/[Z]` are extracted from each instant's `pcpoint` via the schema cache; if the schema lacks `X/Y` the coordinate array is empty for that instant.

```
asMFJSON(tpcpoint, options int=0, flags int=0, maxdecimaldigits int=15) → text
```

- For `tpcpatch`, the type tag is `"MovingPCPatch"` and each instant emits a small object `{"pcid":N, "npoints":N, "bounds": [xmin,xmax,ymin,ymax]}`. The compressed point payload is intentionally not in the JSON — use `asBinary / asHexWKB` for round-trip.

```
asMFJSON(tpcpatch, options int=0, flags int=0, maxdecimaldigits int=15) → text
```

When `options` is set to 1 the output also includes a `"bbox"` key — for `tpcpoint / tpcpatch` this is a `TPCBox` JSON object that adds a `"pcid"` field next to the standard `stbox`-shaped `"bbox"` array.

MF-JSON is output-only for `tpcpoint / tpcpatch`. The JSON form does not carry the schema (`pcid` + dim layout) for `tpcpoint`, and is a per-instant summary that drops the per-point payload for `tpcpatch` — so a reverse parser cannot reconstruct the original value. For lossless round-trip use `asBinary / tpcpointFromBinary / tpcpatchFromBinary`, or `asHexWKB / tpcpointFromHexWKB / tpcpatchFromHexWKB`.

18.11 PDAL integration: `readers.tpcpatch` and `writers.tpcpatch`

MobilityDB ships native PDAL plugins under `contrib/pdal/` that read and write `tpcpatch` values directly without staging through `readers.pgpointcloud+tpcpatchSeq()`. Build them with:

```
cd contrib/pdal
cmake -B build && cmake --build build -j
sudo cmake --install build
```

PDAL discovers the plugins via `PDAL_DRIVER_PATH` or, after install, on its default search path. Verify with `pdal --drivers | grep tpcpatch`.

18.11.1 `readers.tpcpatch`

Reads any SQL query returning rows of `(timestamp, pcpatch, pcid)` and emits a PDAL `PointView` with all schema dimensions plus a `time_t` dimension carrying microseconds since epoch. Decode is delegated to `pgPointCloud`'s `pc_patch_from_wkb` so all three compression types (`PC_NONE`, `PC_DIMENSIONAL`, `PC_LAZPERF`) are handled.

To unpack a stored `tpcpatch` into per-instant rows the reader can consume:

```
{
  "type": "readers.tpcpatch",
  "connection": "host=/tmp dbname=mydb",
  "query": "SELECT (EXTRACT(EPOCH FROM timestampN(traj, n))*1000000)::bigint AS t,
               valueN(traj, n)          AS pcp,
               PC_PCId(valueN(traj, n)) AS pcid
  FROM trajectories,
       generate_series(1, numInstants(traj)) n
 WHERE id = 42
 ORDER BY n"
}
```

18.11.2 `writers.tpcpatch`

Receives a `PointView` with a configurable time dimension, groups points by timestamp into per-instant `pcpatch` values, then dispatches an `INSERT`, `UPDATE`, `append-merge`, or `upsert` against a `tpcpatch` column using MobilityDB's `tpcpatch(pcpatch, timestamptz) + tpcpatchSeq(pcpatch[])` constructors. Encode is delegated to `pgPointCloud`'s `pc_patch_to_wkb` with optional `pc_patch_compress`. Schema scale / offset is applied symmetrically — values flow as their semantic units (metres, degrees, ...) through PDAL even when the schema stores them as scaled `int32`.

Options:

- `connection` (positional) — libpq connection string.
- `table` (positional) — destination table name.
- `column` — `tpcpatch` column name (default `traj`).
- `pcid` (required) — entry in `pointcloud_formats` whose schema the input dimensions must match.
- `id_column, id_value` — identify the target row for any non-insert mode.
- `mode` — `insert` (default — new row per pipeline run), `update` (overwrite the row identified by `id_column = id_value`), `append` (concatenate via `merge()` onto the existing row), or `upsert` (insert if absent, append if present). `append / update` raise a clear "matched no row" error if `id_value` is not found, so silent zero-row writes cannot happen.
- `time_dim` — name of the source time dimension (default `time_t`; set to `GpsTime` to consume LAS timestamps directly without a `filters.assign` stage).

- `time_format` — interpretation of `time_dim`: `unix_microseconds` (default), `unix_seconds`, or `gps_adjusted` (LAS-1.4 Adjusted GPS Standard Time — the writer applies the GPS-Unix epoch shift and 2024 leap-second offset).
- `compression` — `none` (default), `dimensional`, or `laz`. MobilityDB normalises `tpcpatch` on-disk storage to `PC_NONE`, so the option only affects `libpq` transport size — typical reduction is $\sim 3\times$ for `dimensional` on `dim-redundant` data.
- `flush_threshold` (default 1024) — number of completed per-instant patches buffered in memory before issuing an `INSERT/append`. Honoured for `mode=append/upsert`; `mode=insert/update` defer to a single tail-flush since each pipeline run produces a single row.

The writer emits periodic `Info`-level progress lines via PDAL's logger and a final summary in `done()`: total points / patches written, destination column, `pcid`, `compression`, and `mode`. Visible at `pdal pipeline -v 5`.

18.11.3 Supported compression schemes

The reader and writer support all three `pgPointCloud` compression types: `PC_NONE`, `PC_DIMENSIONAL`, and `PC_LAZPERF`. The `PC_LAZPERF` regression at `contrib/pdal/examples/test_lazperf.sql` requires the running `pgPointCloud` install to be built with `--with-lazperf=DIR`; the SQL fixture aborts with a clear message otherwise. The plugins are kept under `contrib/pdal/` rather than in the main MobilityDB build tree so they can be built and updated independently.

18.11.4 Limitations

- *WKB endianness.* `readers.tpcpatch` accepts only little-endian (NDR) `pcpatch` WKB. A big-endian patch (first byte `0x00`) is rejected with a clear error rather than silently mis-decoded. `pgPointCloud` emits NDR on every common platform; XDR support would be additive.
- *Compression of stored `tpcpatch`.* The `writers.tpcpatch` `compression` option controls only the `libpq` transport form. MobilityDB's `tpcpatch` normalises its on-disk `pcpatch` storage to `PC_NONE`.
- *Streaming write.* `writers.tpcpatch` implements PDAL's `Streamable::processOne` interface, so a `readers.las` → `writers.tpcpatch` pipeline runs in PDAL streaming mode automatically and never holds the full point cloud in memory. The completed-patch queue is bounded by the `flush_threshold` writer option (default 1024); on `mode=append` / `mode=upsert` each threshold-triggered flush issues an incremental SQL batch via `merge()`, while `mode=insert` / `mode=update` defer to a single tail-flush in `done()` (single-row semantics). This is the path that makes multi-GB LAS ingest tractable.

18.12 PCL bridge: SQL-callable filters and registration

An optional companion extension `mobilitydb_pcl` lives under `contrib/pcl/`. It links the running PostgreSQL backend against the [Point Cloud Library](#) and exposes a small set of per-`pcpatch` operations as SQL functions. Schema-aware point-type dispatch picks `pcl::PointXYZ`, `pcl::PointXYZI`, or `pcl::PointXYZRGB` at call time based on the `dim` layout of the input `pcid`.

- `pcpatch_to_pcd(pcpatch) → bytea` and `pcpatch_from_pcd(bytea, pcid) → pcpatch` — round-trip through PCL's PCD wire format. Round-trip preserves all `dim` values bit-exactly.
- `pcpatch_voxel_grid(pcpatch, leaf double precision) → pcpatch` — `VoxelGrid` down-sampling, `leaf` in the schema's spatial units. Preserves `Intensity` / `RGB`.
- `pcpatch_sor(pcpatch, k integer DEFAULT 50, stddev_mul double precision DEFAULT 1.0) → pcpatch` — `Statistical Outlier Removal`. Drops points whose mean distance to their `k` nearest neighbours falls outside ($\text{mean} \pm \text{stddev_mul} \times \text{stddev}$). Preserves `Intensity` / `RGB`.

- `pcpatch_icp(source pcpatch, target pcpatch, max_iter int DEFAULT 50, max_corr double DEFAULT 1.0) → double[]` and `pcpatch_gicp(...)` with the same signature — IterativeClosestPoint and GeneralizedICP rigid registration. Both return an 8-element double array `[tx, ty, tz, qw, qx, qy, qz, fitness]` composable into a MobilityDB pose via `pose_make_3d`. GICP's Mahalanobis cost over per-point local covariances is markedly more robust than ICP on non-planar / vegetated surfaces (road-corridor scans, photogrammetric heritage facades).
- `pcpatch_normals(pcpatch, k int DEFAULT 10) → double[]` — per-point surface normal + curvature via PCL's `NormalEstimation<PointXYZ, Normal>` over the `k` nearest neighbours. Returns a flat array of length `4 × npoints` laid out as `[nx_0, ny_0, nz_0, curv_0, nx_1, ny_1, nz_1, curv_1, ...]`. Use cases: oriented-surface change detection across repeat surveys, Potree `--normal` export, photogrammetric mesh seeding. For very large patches (>50 k points) consider down-sampling with `pcpatch_voxel_grid` first.

The bridge is gated behind `CREATE EXTENSION mobilitydb_pcl` in databases that have already loaded `pointcloud` + `mobilitydb`. Build instructions and a deterministic regression suite are in `contrib/pcl/README.md`.

18.13 Caveats

- The `pointcloud` extension must exist in the database before `mobilitydb`'s SQL definitions reference `pcpoint` / `pcpatch`. On a `POINTCLOUD=ON` build the generated `mobilitydb.control` declares this dependency in its `requires =` clause, so `CREATE EXTENSION mobilitydb CASCADE` resolves `pointcloud` automatically. Manual creation in the wrong order without `CASCADE` still raises a missing-type error.
- Schema mixing is rejected at construction time: a `tpcpoint` sequence whose instants have different `pcid` values cannot be built, and the same constraint applies to `pcpointset` and `pcpatchset`. The schema is what gives the dimensions their meaning, and silently mixing schemas would produce values whose coordinates are uninterpretable.
- The `POINTCLOUD=ON` CMake flag is required. Builds without it produce a `libMobilityDB` that does not declare any of the types in this chapter; the SQL files for them are not generated.

Chapter 19

Raster Sampling

The **PostGIS raster** extension stores gridded coverage data (elevation, temperature, satellite imagery, ...) in the `raster` type. Each raster has one or more bands; a band holds a 2D array of pixel values, a spatial extent, a pixel size, and an optional nodata sentinel value.

MobilityDB's raster sampling operator evaluates a raster band at the instants of a moving-point trajectory and returns a `tfloat`. Instants whose position falls outside the raster extent or on a nodata pixel are silently dropped. This enables queries such as *what elevation profile did each vehicle follow?* or *which trips entered a temperature threshold band?*

To use the operators described here, MobilityDB must be built with `-DRASTER=ON`. On such a build the generated `mobilitydb` control declares `requires = 'postgis, postgis_raster'`, so a single `CASCADE` creates the full stack:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE: installing required extension "postgis"
-- NOTICE: installing required extension "postgis_raster"
```

19.1 Sampling Operator

- Sample a raster band at each instant of a trajectory

`raster_value(raster, tgeompoint, band integer DEFAULT 1) → tfloat`

Evaluates pixel band at every instant of `traj` using bilinear-nearest sampling (`ST_Value` with `resample=false`). Returns an instant-set `tfloat` whose values are rounded to four decimal places. Returns `NULL` when no instant intersects the raster.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(
    ST_AddBand(
      ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
      '32BF'::text, 0.0::float8, NULL::float8),
    1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
          [40.0::float4, 50.0::float4, 60.0::float4],
          [70.0::float4, 80.0::float4, 90.0::float4]]) AS r
)
SELECT raster_value(r,
  tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2000-01-01 00:00:00+00,
                POINT(2.5 2.5)@2000-01-02 00:00:00+00,
                POINT(0.5 0.5)@2000-01-03 00:00:00+00]')::text
FROM rast;
-- {10@2000-01-01 00:00:00+00, 30@2000-01-02 00:00:00+00, 70@2000-01-03 00:00:00+00}
```

Applied to the BerlinMOD trajectory dataset with a terrain elevation raster:

```
-- Elevation profile per trip (requires a loaded elevation raster)
SELECT tripid, raster_value(elev, trip4326) AS elevation
FROM trips, elevation_raster;

-- Average elevation per trip
SELECT tripid, avgValue(raster_value(elev, trip4326)) AS mean_elev
FROM trips, elevation_raster;

-- Trips that crossed terrain above 200 m
SELECT tripid
FROM trips, elevation_raster
WHERE atSpan(raster_value(elev, trip4326),
             floatspan '[200, 9999]') IS NOT NULL;
```

19.2 Raquet / QUADBIN Sampling

Raquet is a cloud-native raster format that stores raster tiles as rows in an Apache Parquet file. Each row holds the pixel bytes for one Web-Mercator tile together with its **QUADBIN** cell identifier — a 64-bit integer that encodes the tile's zoom level and Morton-interleaved x/y coordinates. No separate spatial metadata is required: the bounding box and pixel-to-coordinate mapping are fully determined by the QUADBIN value.

The typical query pattern joins a trajectory against a Raquet table in two steps:

```
-- Step 1: identify which tiles the trajectory overlaps
SELECT DISTINCT q
FROM unnest(trajectory_quadbins(traj, 8)) AS q;

-- Step 2: sample each matching tile
SELECT raster_tile_value_quadbin(
    band_data, width, height, quadbin, 'FLOAT32', nodata, true, traj)
FROM elevation_raquet
WHERE quadbin = ANY(trajectory_quadbins(traj, 8));
```

- Return the distinct QUADBIN cells at a zoom level covered by a trajectory

```
trajectory_quadbins(tgeompoint, integer) → bigint[]
```

Returns the set of unique QUADBIN cells (deduplicated) whose Web-Mercator tile envelopes contain at least one instant of `traj`. The result is suitable as a WHERE-clause join key against a Raquet table. `zoom` must be in `[0, 15]`.

```
-- Three instants spanning three tiles at zoom 3
SELECT trajectory_quadbins(
    tgeompointFromText('SRID=4326;{Point(-60 45)@2024-01-01,
                          Point(0 45)@2024-01-02,
                          Point(60 45)@2024-01-03}'), 3);
-- {5202501994543054848,5203346419473186816,5203416788217364480}

-- Count of Raquet tiles a ship trajectory overlaps at zoom 8
SELECT array_length(trajectory_quadbins(trip4326, 8), 1) AS tile_count
FROM trips
ORDER BY tile_count DESC;
```

- Sample a Raquet raster chip along a trajectory using a QUADBIN cell for georeferencing

```
raster_tile_value_quadbin(bytea, integer, integer, bigint, text, float8, boolean, tgeompoint)
→ tfloat
```

Evaluates a row-major single-band pixel array at each instant of `traj` (SRID 4326). The tile georeferencing (bounding box, pixel-to-coordinate mapping) is derived from `quadbin` alone via the Web-Mercator slippy-tile transform. The `pixtype`

argument must be one of `UINT8`, `INT16`, `INT32`, `FLOAT32`, or `FLOAT64`. When `has_nodata` is `true`, instants whose pixel equals `nodata` are silently dropped. Returns `NULL` when no instant survives.

```
-- Sample elevation tiles from a Raquet Parquet table
-- for all ship trajectories
SELECT t.tripid,
       raster_tile_value_quadbin(
         r.band_data, r.width, r.height, r.quadbin,
         'FLOAT32', r.nodata, true, t.trip4326)
FROM trips t
JOIN elevation_raquet r
  ON r.quadbin = ANY(trajectory_quadbins(t.trip4326, 8));

-- Average sea-surface temperature per trajectory
SELECT t.tripid,
       avgValue(raster_tile_value_quadbin(
         r.band_data, 256, 256, r.quadbin,
         'INT16', -9999, true, t.trip4326)) AS mean_sst
FROM trips t
JOIN sst_raquet r
  ON r.quadbin = ANY(trajectory_quadbins(t.trip4326, 6));
```

- Construct a Raquet raster tile from its pixel bytes, dimensions, QUADBIN cell and pixel type

`raquet(bytea, integer, integer, bigint, text, float8) → raquet`

Packages a row-major single-band pixels array of width × height pixels of type `pixtype` (one of `UINT8`, `INT16`, `INT32`, `FLOAT32`, or `FLOAT64`), georeferenced by the `quadbin` cell, into a single self-describing `raquet` value. The `nodata` argument is optional; when omitted (`NULL`) the tile carries no `nodata` sentinel. An error is raised when the pixels array is smaller than width × height × the pixel size. A `raquet` is a GDAL-free, variable-length value with a portable Hex-WKB text and binary representation, distinct from and coexisting with the PostGIS raster type used by `raster_value`.

```
-- Package the loose tile columns of a Raquet table into raquet values
SELECT raquet(band_data, width, height, quadbin, 'FLOAT32', nodata) AS tile
FROM elevation_raquet;

-- A 2 x 2 UINT8 tile with no nodata
SELECT raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'UINT8');
```

- Decode an in-memory raster file into a Raquet raster tile via GDAL

`raquet_read(bytea, bigint) → raquet`

Reads the bytes of `rasterfile` — a raster in any format GDAL supports (GeoTIFF, PNG, ...) — through GDAL's `/vsimem/` virtual filesystem, and packs its first band, row-major, into a single self-describing `raquet` value georeferenced by the `quadbin` cell. The band data type must be one of `Byte`, `Int16`, `Int32`, `Float32`, or `Float64`; the band `nodata` value, when set, is carried into the tile. Because the file is supplied as bytes, no server-side file access is required. GDAL performs the decode, so the raster's format driver must be permitted by PostGIS's `postgis.gdal_enabled_drivers` setting (which disables all drivers by default); enable it with, for example, `SET postgis.gdal_enabled_drivers = 'ENABLE_ALL'`. This is the ingest counterpart of the `raquet` constructor, which builds a tile from already-decoded pixel bytes.

When `quadbin` is omitted or `NULL`, the tile identifier is derived from the raster's georeferencing: the raster must carry an EPSG:3857 (Web-Mercator) spatial reference and an axis-aligned geotransform describing a single QUADBIN tile. A raster with no spatial reference, one that is not in EPSG:3857, or a geotransform that is rotated or not aligned to the tile grid raises an error rather than being coerced.

```
-- Decode a GeoTIFF stored in a bytea column into a raquet tile
SELECT raquet_read(file_bytes, quadbin) AS tile
FROM elevation_files;

-- Derive the QUADBIN cell from an EPSG:3857 GeoTIFF's georeferencing
SELECT raquet_read(file_bytes) AS tile
FROM elevation_files;
```

- Sample a raquet raster tile along a trajectory

```
raster_tile_value(raquet,tgeompoint) → tfloat
```

Evaluates the tile at each instant of `traj` (SRID 4326), returning the sampled pixel values as a `tfloat`. This is the typed equivalent of `raster_tile_value_quadbin`: the dimensions, QUADBIN cell, pixel type and nodata handling are taken from the `raquet` value instead of from separate arguments. Returns `NULL` when no instant falls inside the tile or survives nodata filtering.

```
-- Sample pre-packaged raquet tiles along ship trajectories
SELECT t.tripid, raster_tile_value(r.tile, t.trip4326)
FROM trips t
JOIN elevation_tiles r
  ON r.quadbin = ANY(trajectory_quadbins(t.trip4326, 8));
```

- Sample an array of raquet raster tiles along a trajectory

```
raster_tile_value(raquet[],tgeompoint) → tfloat
```

Evaluates every tile of `rast` at each instant of `traj` (SRID 4326) and returns the sampled pixel values as a single `tfloat`. A trajectory that leaves one tile is covered by several of them, each contributing the instants that fall inside it. Tiles of one zoom level partition the plane, so they contribute disjoint instants. Tiles of different zoom levels overlap, and where two of them sample the same instant the value of the tile of higher zoom is kept, that being the one carrying the finer resolution. Returns `NULL` when no instant falls inside a tile or survives nodata filtering.

```
-- Sample a trajectory across every tile it crosses, in one value
SELECT t.tripid, raster_tile_value(array_agg(r.tile), t.trip4326)
FROM trips t
JOIN elevation_tiles r
  ON r.quadbin = ANY(trajectory_quadbins(t.trip4326, 8))
GROUP BY t.tripid, t.trip4326;
```

19.3 Raquet Accessors and Comparison

A `raquet` value is self-describing: the accessors below read back the georeferencing and layout it carries, so a tile packaged with the `raquet` constructor or decoded with `raquet_read` can be inspected without keeping the loose columns it was built from.

Tiles also support the full set of comparison operators. The ordering is total and is defined on the QUADBIN cell first, then the pixel type, the width, the height, and finally the pixel bytes; it is meant for indexing and deduplication rather than for a spatial interpretation. A default btree operator class makes `ORDER BY`, `DISTINCT` and merge joins work on `raquet` columns, and a default hash operator class enables hash joins and hash aggregation.

- Return the QUADBIN cell of a Raquet tile

```
quadbin(raquet) → bigint
```

- Return the width in pixels of a Raquet tile

```
width(raquet) → integer
```

- Return the height in pixels of a Raquet tile

```
height(raquet) → integer
```

- Return the nodata sentinel value of a Raquet tile

```
nodata(raquet) → float8
```

- Return the name of the pixel data type of a Raquet tile

```
pixtype(raquet) → text
```

The returned name is the one the constructors accept, that is, one of `UINT8`, `INT16`, `INT32`, `FLOAT32`, or `FLOAT64`.

```
-- Read back the layout of a packaged tile
SELECT quadbin(tile), width(tile), height(tile), pixtype(tile), nodata(tile)
FROM (SELECT raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'UINT8') AS tile) t;
-- 5193776270265024512 | 2 | 2 | UINT8 | 0
```

- Convert a Raquet tile into a spatiotemporal box

```
stbox(raquet) → stbox
```

Returns the tile footprint: the longitude and latitude envelope of its QUADBIN cell, in SRID 4326 and without a time dimension. The footprint follows from the cell alone, so tiles differing in pixels, dimensions or pixel type share it. The conversion is also available as a cast.

```
-- The footprint of a tile covering the north-eastern quadrant at zoom 1
SELECT round(stbox(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512,
'UINT8')), 6);
-- SRID=4326;STBOX X((0,0),(180,85.051129))

-- Keep the tiles whose footprint overlaps a region of interest
SELECT * FROM elevation_tiles
WHERE tile::stbox && stbox 'SRID=4326;STBOX X((0,0),(30,30))';
```

The footprint carries the stbox operators, aggregates and operator classes, so a tile table is searched by spatial overlap at any zoom level rather than by an equality test on the QUADBIN cell at a single fixed one. Indexing the conversion gives the tile table a spatial index, and the extent of a set of tiles is the extent of their footprints.

```
-- A spatial index over the tile footprints
CREATE INDEX elevation_tiles_gist ON elevation_tiles USING gist (stbox(tile));
CREATE INDEX elevation_tiles_spgist ON elevation_tiles USING spgist (stbox(tile));

-- The extent covered by a set of tiles
SELECT extent(stbox(tile)) FROM elevation_tiles;

-- Tiles a trajectory passes through, at whatever zoom levels the table holds
SELECT t.tripid, r.tile
FROM trips t, elevation_tiles r
WHERE stbox(r.tile) && stbox(t.trip4326);
```

- Compare two Raquet tiles

```
raquet = raquet → boolean
raquet <> raquet → boolean
raquet < raquet → boolean
raquet <= raquet → boolean
raquet >= raquet → boolean
raquet > raquet → boolean
```

The functions backing these operators are eq, ne, lt, le, ge and gt, together with cmp(raquet, raquet) → integer, which returns -1, 0, or 1.

```
-- Two tiles with the same cell, layout and pixels are equal
SELECT raquet('\x0102'::bytea, 2, 1, 5193776270265024512, 'UINT8') =
       raquet('\x0102'::bytea, 2, 1, 5193776270265024512, 'UINT8');
-- true

-- Deduplicate tiles, which the btree and hash operator classes support
SELECT DISTINCT tile FROM elevation_tiles;
```

19.4 Restriction and Predicate Operators

The following SQL-defined operators compose `raster_value` with the standard MobilityDB temporal restriction and predicate functions. They accept an optional `band` argument (default 1) and call `raster_value` exactly once per invocation.

- Return the instants of a trajectory where the sampled raster value falls inside a float range

`atRasterValue(tgeompoint, raster, floatspan, band integer DEFAULT 1) → tgeompoint`

Evaluates `raster_value` at every instant of `traj` and returns the sub-trajectory restricted to the times when the pixel value lies within `vspan`. Returns NULL when no instant satisfies the condition.

```
WITH rast AS (
  SELECT ST_SetValues(
    ST_AddBand(
      ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
      '32BF'::text, 0.0::float8, NULL::float8),
    1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
          [40.0::float4, 50.0::float4, 60.0::float4],
          [70.0::float4, 80.0::float4, 90.0::float4]]) AS r
)
SELECT asText(atRasterValue(
  tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2000-01-01,
                POINT(1.5 1.5)@2000-01-02,
                POINT(0.5 0.5)@2000-01-03]',
  r, floatspan '[40, 90]'))
FROM rast;
-- {[POINT(1.5 1.5)@2000-01-02 00:00:00+00], [POINT(0.5 0.5)@2000-01-03 00:00:00+00]}
```

- Return the instants of a trajectory where the sampled raster value falls outside a float range

`minusRasterValue(tgeompoint, raster, floatspan, band integer DEFAULT 1) → tgeompoint`

The complement of `atRasterValue`: returns the sub-trajectory restricted to the times when the pixel value lies outside `vspan`. Returns NULL when every instant satisfies the condition.

```
-- Keep only the instant whose raster value (10) is below 40
SELECT asText(minusRasterValue(traj, elev_raster, floatspan '[40, 9999]'))
FROM trips, elevation_raster;
```

- Return true if the trajectory ever samples a raster pixel value inside a float range

`eRasterValue(raster, tgeompoint, floatspan, band integer DEFAULT 1) → boolean`

Returns true when at least one in-extent instant of `traj` samples a pixel value within `vspan`, false otherwise. Returns NULL only when the input is NULL.

```
-- Trips that ever crossed terrain above 200 m
SELECT tripid
FROM trips, elevation_raster
WHERE eRasterValue(elev_raster, trip4326, floatspan '[200, 9999]');
```

- Return true if every in-raster-extent instant of the trajectory samples a pixel value inside a float range

`aRasterValue(raster, tgeompoint, floatspan, band integer DEFAULT 1) → boolean`

Returns true when every in-extent instant of `traj` samples a pixel value within `vspan`, false when at least one does not. Returns NULL only when the input is NULL.

```
-- Trips that stayed entirely below 100 m elevation
SELECT tripid
FROM trips, elevation_raster
WHERE aRasterValue(elev_raster, trip4326, floatspan '[0, 100]');
```

19.5 Build and Install

PostGIS raster is part of the standard `postgresql-N-postgis-3` package on Debian/Ubuntu; no extra package is required. Pass `-DRASTER=ON` to CMake:

```
cmake -DRASTER=ON ..
make -j$(nproc)
sudo make install
```

To include raster in the CI coverage build alongside other optional families, add `-DRASTER=ON` to the `cmake` invocation in `.github/workflows/pgversion.yml`.

19.6 Production Roadmap

The two paths described in this chapter serve different purposes and are both maintained. The `raquet` path is a self-contained MEOS value type: it carries its pixels and its QUADBIN georeferencing in a single value and needs no PostgreSQL to evaluate, so it is available to every language binding derived from MEOS. The PostGIS `raster` path offers the far richer raster processing of the `postgis_raster` extension — map algebra, statistics, resampling, clipping, conversion to polygons — inside PostgreSQL only.

Neither path replaces the other. Work on the `raquet` path extends what a cloud-native tile can do along a trajectory; it is not an attempt to reimplement raster processing.

The extensions below require a general raster data model, that is, arbitrary spatial reference systems, arbitrary affine georeferencing and several bands per tile. A `raquet` value is a single-band tile whose extent and pixel grid follow from its QUADBIN cell, so these lie outside that model and remain available through the PostGIS `raster` path:

- *Multi-band tiles* — one `tfloat` per band for multi-spectral imagery such as RGB or multi-channel satellite scenes.
- *Arbitrary spatial reference systems* — tiles outside the Web-Mercator grid, and reprojection between reference systems.
- *Raster processing* — map algebra, summary statistics, resampling, and conversion to polygons, together with clipping a tile to a region, such as the extent a set of trajectories visits over a time interval.

Chapter 20

Portable Named-Function Dialect

Every MobilityDB operator is also callable as a bare named function. A query written with named functions only, using no operator symbols, is portable: the same SQL runs unchanged across the MobilityDB ecosystem — the database engines MobilityDB (PostgreSQL), MobilityDuck (DuckDB) and MobilitySpark (Apache Spark), and the streaming engines — several of which cannot register PostgreSQL operator symbols. Each named alias reuses the operator's own implementation, so it returns exactly the same result as the operator.

This chapter is the canonical, ecosystem-wide reference for that mapping. An engine that does not accept a given operator can look it up here and use the function form instead. For example, DuckDB does not accept any operator containing # (it reserves # for positional column references), so on MobilityDuck the temporal comparisons (#=, ...) are written `tEq, ...` and the time-position tests (<<#, #>>, &<#, #&>) are written `before, after, overbefore, overafter`, while the operators it does accept may be used directly.

The spatial-relationship functions (`eIntersects`, `aIntersects`, `eTouches`, `eDwithin`, ...) and the restriction functions (`atTime`, `atGeometry`, `atValue`, ...) are only ever named functions (they have no operator). The B-tree ordering comparator is likewise the named function `cmp(a, b)`, which returns -1, 0 or 1 and has no operator. Every operator maps to a bare name as follows; for the arithmetic and set-theoretic operators the function name carries the operand's type-family prefix (`set`, `span` or `spanset`, and `t` for temporal numbers).

Category	Operator	Portable function
Topology	<code>&&</code>	<code>overlaps(a, b)</code>
	<code>@></code>	<code>contains(a, b)</code>
	<code><@</code>	<code>contained(a, b)</code>
	<code>- -</code>	<code>adjacent(a, b)</code>
Time position	<code><<#</code>	<code>before(a, b)</code>
	<code>#>></code>	<code>after(a, b)</code>
	<code>&<#</code>	<code>overbefore(a, b)</code>
	<code>#&></code>	<code>overafter(a, b)</code>
Space, X axis	<code><<</code>	<code>left(a, b)</code>
	<code>>></code>	<code>right(a, b)</code>
	<code>&<</code>	<code>overleft(a, b)</code>
	<code>&></code>	<code>overright(a, b)</code>
Space, Y axis	<code><< </code>	<code>below(a, b)</code>
	<code> >></code>	<code>above(a, b)</code>
	<code>&< </code>	<code>overbelow(a, b)</code>
	<code> &></code>	<code>overabove(a, b)</code>
Space, Z axis	<code><</</code>	<code>front(a, b)</code>
	<code>/>></code>	<code>back(a, b)</code>
	<code>&</</code>	<code>overfront(a, b)</code>
	<code>/&></code>	<code>overback(a, b)</code>
Temporal comparison	<code>#=</code>	<code>tEq(a, b)</code>
	<code>#<></code>	<code>tNe(a, b)</code>

Category	Operator	Portable function
	#<	tLt(a, b)
	#<=	tLe(a, b)
	#>	tGt(a, b)
	#>=	tGe(a, b)
Distance	<->	tDistance(a, b)
	=	nearestApproachDistance(a, b)
Same	~=	same(a, b)
Traditional comparison	=	eq(a, b)
	<>	ne(a, b)
	<	lt(a, b)
	<=	le(a, b)
	>	gt(a, b)
	>=	ge(a, b)
Ever comparison	?=	eEq(a, b)
	?<>	eNe(a, b)
	?<	eLt(a, b)
	?<=	eLe(a, b)
	?>	eGt(a, b)
	?>=	eGe(a, b)
Always comparison	%=	aEq(a, b)
	%<>	aNe(a, b)
	%<	aLt(a, b)
	%<=	aLe(a, b)
	%>	aGt(a, b)
	%>=	aGe(a, b)
Boolean (temporal boolean)	&	tAnd(a, b)
		tOr(a, b)
	~	tNot(a)
Arithmetic (temporal numbers)	+	tAdd(a, b)
	-	tSub(a, b)
	*	tMul(a, b)
	/	tDiv(a, b)
Union (set / span / span set)	+	setUnion / spanUnion / spansetUnion (a, b)
Intersection (set / span / span set)	*	setIntersection / spanIntersection / spansetIntersection (a, b)
Difference (set / span / span set)	-	setMinus / spanMinus / spansetMinus (a, b)

For example, the bounding-box overlap `t1.trip && t2.trip` is written portably as `overlaps(t1.trip, t2.trip)`, and the temporal-before test `p1.period <<# p2.period` as `before(p1.period, p2.period)`.

Aggregate functions follow the same principle, and are listed in their own table below so that an engine can look up the machinery it needs without hunting through the prose. A SQL aggregate is assembled from up to three PostgreSQL machinery functions, each exposed under a canonical name built from the aggregate name and its role: a transition function `<aggregate>Transition` (always present), a combine function `<aggregate>Combine` used for parallel aggregation, and a final function `<aggregate>Final`. The combine and final roles are each optional: an aggregate whose transition state is already its result has no final function (for example `extent` and `minDistance`), and an aggregate that does not support partial aggregation has no combine function (for example the append aggregates). The roles stay distinct and uniformly named so that every binding can reconstruct the aggregate from the catalog. The following table lists every aggregate and its machinery functions; a dash (—) marks a role the aggregate does not define.

Category	Transition	Combine	Final
Count	tCountTransition	tCountCombine	tCountFinal
	wCountTransition	wCountCombine	wCountFinal
Sum	tSumTransition	tSumCombine	tSumFinal
	wSumTransition	wSumCombine	wSumFinal
Average	tAvgTransition	tAvgCombine	tAvgFinal

Category	Transition	Combine	Final
	wAvgTransition	wAvgCombine	wAvgFinal
Minimum / maximum	tMinTransition	tMinCombine	tMinFinal
	tMaxTransition	tMaxCombine	tMaxFinal
	wMinTransition	wMinCombine	wMinFinal
	wMaxTransition	wMaxCombine	wMaxFinal
Boolean	tAndTransition	tAndCombine	tAndFinal
	tOrTransition	tOrCombine	tOrFinal
Centroid	tCentroidTransition	tCentroidCombine	tCentroidFinal
Point cloud	tnpointsTransition	tnpointsCombine	tnpointsFinal
	tdensityTransition	tdensityCombine	tdensityFinal
Bounding box	extentTransition	extentCombine	—
Distance	minDistanceTransition	minDistanceCombine	—
Union	setUnionTransition	setUnionCombine	setUnionFinal
	spanUnionTransition	spanUnionCombine	spanUnionFinal
	spansetUnionTransition	spansetUnionCombine	spansetUnionFinal
Merge	mergeTransition	mergeCombine	mergeFinal
Append	appendInstantTransition	—	appendInstantFinal
	appendSequenceTransition	—	appendSequenceFinal

Appendix A

MobilityDB Reference

A.1 MobilityDB Types

MobilityDB defines four *template types* that act as type constructors over *base types*. These template types are `set`, `span`, `spanset`, and `temporal`. Table A.1 shows the types defined over these template types. Furthermore, MobilityDB defines two bounding box types, namely, `tbox` and `stbox`.

Base Types	Template Types			
	<code>set</code>	<code>span</code>	<code>spanset</code>	<code>temporal</code>
<code>bool</code>				<code>tbool</code>
<code>text</code>	<code>textset</code>			<code>ttext</code>
<code>integer</code>	<code>intset</code>	<code>intspan</code>	<code>intspanset</code>	<code>tint</code>
<code>bigint</code>	<code>bigintset</code>	<code>bigintspan</code>	<code>bigintspanset</code>	<code>tbigint</code>
<code>float</code>	<code>floatset</code>	<code>floatspan</code>	<code>floatspanset</code>	<code>tfloat</code>
<code>date</code>	<code>dateset</code>	<code>datespan</code>	<code>datespanset</code>	
<code>timestampz</code>	<code>tstzset</code>	<code>tstzspan</code>	<code>tstzspanset</code>	
<code>geometry</code>	<code>geomset</code>			<code>tgeompoint</code> , <code>tgeometry</code>
<code>geography</code>	<code>geogset</code>			<code>tgeogpoint</code> , <code>tgeography</code>
<code>pose</code>	<code>poseset</code>			<code>tpose</code>
<code>npoint</code>	<code>npointset</code>			<code>tnpoint</code>
<code>cbuffer</code>	<code>cbufferset</code>			<code>tcbuffer</code>
<code>h3index</code>	<code>h3indexset</code>			<code>th3index</code>
<code>quadbin</code>	<code>quadbinset</code>			<code>tquadbin</code>
<code>pcpoint</code>	<code>pcpointset</code>			<code>tpcpoint</code>
<code>pcpatch</code>	<code>pcpatchset</code>			<code>tpcpatch</code>

Table A.1: MobilityDB current instantiations of template types

A.2 Set and Span Types

A.2.1 Input and Output

- **asText**: Return the Well-Known Text (WKT) representation
- **asBinary**: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- **settypeFromBinary**, **spantypeFromBinary**, **spansettypeFromBinary**: Input from the Well-Known Binary (WKB) representation
- **settypeFromHexWKB**, **spantypeFromHexWKB**, **spansettypeFromHexWKB**: Input from the Hexadecimal Well-Known Binary (HexWKB) representation

A.2.2 Constructors

- **set**: Constructor for set values
- **span**: Constructor for span values
- **spanset**: Constructor for span set values

A.2.3 Conversions

- **::, set(base), span(base), spanset(base)**: Convert a base value to a set, span, or span set value
- **::, spanset(set)**: Convert a set value to a span set value
- **::, spanset(span)**: Convert a span value to a span set value
- **::, range(span), span(range)**: Convert a span value to and from a PostgreSQL range value
- **::, multirange(spanset), spanset(multirange)**: Convert a span set value to and from a PostgreSQL multirange value

A.2.4 Accessors

- **memSize**: Return the memory size in bytes
- **lower, upper**: Return the lower or upper bound
- **lowerInc, upperInc**: Is the lower or upper bound inclusive?
- **width**: Return the width of the span as a float
- **duration**: Return the duration
- **span**: Return the bounding span of the set or span set ignoring the potential time gaps
- **numValues, getValues**: Return the (number of) values
- **startValue, endValue, valueN**: Return the start, end, or n-th value
- **numSpans, spans**: Return (the number of) composing spans
- **startSpan, endSpan, spanN**: Return the start, end, or n-th span
- **numDates, dates**: Return the (number of) different dates
- **startDate, endDate, dateN**: Return the start, end, or n-th date
- **numTimestamps, timestamps**: Return the (number of) different timestamps
- **startTimestamp, endTimestamp, timestampN**: Return the start, end, or n-th timestamp

A.2.5 Transformations

- **expand**: Expand or shrink the bounds by a value or an interval
 - **shift, scale, shiftScale**: Shift and/or scale by a value or an interval
 - **floor, ceil**: Round down or up to the lowest integer
 - **round**: Round to a number of decimal places
 - **degrees, radians**: Transform to degrees or radians
 - **lower, upper, initcap**: Transform to lowercase, uppercase, or initcap
 - **||**: Text concatenation
 - **tprecision**: Set the temporal precision to the interval with respect to the origin
-

A.2.6 Spatial Reference System

- **SRID, setSRID**: Return or set the spatial reference identifier
- **transform, transformPipeline**: Transform to a spatial reference identifier

A.2.7 Set Operations

- **+, -, ***: Union, difference, and intersection of sets or spans

A.2.8 Bounding Box Operations

A.2.8.1 Topological Operations

- **&&**: Do the sets or spans overlap (have values in common)?
- **@>**: Does the first set or span contain the second one?
- **<@**: Is the first set or span contained by the second one?
- **-|**: Is the first set or span adjacent to the second one?

A.2.8.2 Position Operations

- **<<, <<#**: Is the first set or span strictly to the left of the second one?
- **>>, #>>**: Is the first set or span strictly to the right of the second one?
- **&<, &<#**: Is the first set or span not to the right of the second one?
- **&>, #&>**: Is the first set or span not to the left of the second one?

A.2.8.3 Splitting Operations

- **splitNSpans**: Return an array of N spans obtained by merging the elements of a set or the spans of a spanset
- **splitEachNSpans**: Return an array of spans obtained by merging N consecutive elements of a set or N consecutive spans of a spanset

A.2.9 Distance Operations

- **<->**: Return the smallest distance ever

A.2.10 Comparisons

- **=, <>, <, >, <=, >=, cmp, cmp, and cmp**: Traditional comparisons

A.2.11 Aggregations

- **tCount**: Temporal count
- **extent**: Bounding span
- **setUnion, spanUnion, spansetUnion**: Aggregate union

A.3 Box Types

A.3.1 Input and Output

- **asText**: Return the Well-Known Text (WKT) representation
- **asBinary**, **asHexWKB**: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation as text
- **boxFromBinary**, **boxFromHexWKB**: Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

A.3.2 Constructors

- **tbox**: Constructor for `tbox`
- **stboxX**, **stboxZ**, **stboxT**, **stboxXT**, **stboxZT**, **geodstboxZ**, **geodstboxT**, **geodstboxZT**: Constructor for `stbox`

A.3.3 Conversions

- **tbox::type**: Convert a `tbox` to another type
- **type::tbox**: Convert another type to a `tbox`
- **stbox::type**: Convert an `stbox` to another type
- **type::stbox**: Convert another type to an `stbox`

A.3.4 Accessors

- **hasX**, **hasZ**, **hasT**: Has X/Z/T dimension?
- **isGeodetic**: Is geodetic?
- **xMin**, **yMin**, **zMin**, **tMin**: Return the minimum X/Y/Z/T value
- **xMax**, **yMax**, **zMax**, **tMax**: Return the maximum X/Y/Z/T value
- **xMinInc**, **tMinInc**: Is the minimum X/T value inclusive?
- **xMaxInc**, **tMaxInc**: Is the maximum X/T value inclusive?
- **area**, **volume**, **perimeter**: Area, volume, perimeter

A.3.5 Transformations

- **shiftValue**, **scaleValue**, **shiftScaleValue**: Shift and/or scale the span of a bounding box by a value
 - **shiftTime**, **scaleTime**, **shiftScaleTime**: Shift and/or scale the period of a bounding box by an interval
 - **getSpace**: Return the spatial dimension of a bounding box, removing the temporal dimension if any
 - **expandValue**, **expandSpace**, **expandTime**: Expand the numeric, spatial, or temporal dimension of a bounding box by a value or an interval
 - **round**: Round the values or the coordinates of a bounding box to a number of decimal places
-

A.3.6 Spatial Reference System

- **SRID, setSRID**: Return or set the spatial reference identifier
- **transform, transformPipeline**: Transform to a spatial reference identifier

A.3.7 Splitting Operations

- **quadSplit**: Split a bounding box in quadrants or octants

A.3.8 Set Operations

- **+, ***: Union and intersection of bounding boxes

A.3.9 Bounding Box Operations

A.3.9.1 Topological Operations

- **&&**: Do the bounding boxes overlap?
- **@>**: Does the first bounding box contain the second one?
- **<@**: Is the first bounding box contained in the second one?
- **~=**: Are the bounding boxes equal in their common dimensions?
- **-|**: Are the bounding boxes adjacent?

A.3.9.2 Position Operations

- **<<, <<|, <<|, <<#**: Are the X/Y/Z/T values of the first bounding box strictly less than those of the second one?
- **>>, |>>, />>, #>>**: Are the X/Y/Z/T values of the first bounding box strictly greater than those of the second one?
- **&<, &<|, &<|, &<#**: Are the X/Y/Z/T values of the first bounding box not greater than those of the second one?
- **&>, |&>, /&>, #&>**: Are the X/Y/Z/T values of the first bounding box not less than those of the second one?

A.3.10 Comparisons

- **=, <>, <, >, <=, >=cmp**: Traditional comparisons

A.3.11 Aggregations

- **extent**: Bounding box extent

A.4 Temporal Types

A.4.1 Input and Output

- **asText**: Return the Well-Known Text (WKT) representation
- **asMFJSON**: Return the Moving Features JSON (MF-JSON) representation
- **asBinary**, **asHexWKB**: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- **ttypeFromMFJSON**: Input from a Moving Features JSON (MF-JSON) representation
- **ttypeFromBinary**, **ttypeFromHexWKB**: Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

A.4.2 Constructors

- **ttype**: Constructor for temporal types from a base value and a time value
- **ttypeSeq**: Constructor for temporal types of sequence subtype
- **ttypeSeqSet**, **ttypeSeqSetGaps**: Constructor for temporal types of sequence set subtype

A.4.3 Conversions

- **ttype::tstzspan**: Convert a temporal value to a timestamptz span

A.4.4 Accessors

- **memSize**: Return the memory size in bytes
 - **tempSubtype**: Return the temporal subtype
 - **tempBasetype**: Return the base type
 - **interp**: Return the interpolation
 - **getValue**, **getTimestamp**: Return the value or the timestamp of an instant
 - **getValues**, **getTime**: Return the values or the time on which the temporal value is defined
 - **timeSpan**: Return the bounding period on which the temporal value is defined
 - **startValue**, **endValue**, **valueN**: Return the start, end, or n-th value
 - **valueAtTimestamp**: Return the value at a timestamp
 - **duration**: Return the duration
 - **lowerInc**, **upperInc**: Is the start/end instant inclusive?
 - **numInstants**, **instants**: Return the (number of) different instants
 - **startInstant**, **endInstant**, **instantN**: Return the start, end, or n-th instant
 - **numTimestamps**, **timestamps**: Return the (number of different) timestamps
 - **startTimestamp**, **endTimestamp**, **timestampN**: Return the start, end, or n-th timestamp
 - **numSequences**, **sequences**: Return the (number of) sequences
 - **startSequence**, **endSequence**, **sequenceN**: Return the start, end, or n-th sequence
 - **segments**: Return the segments
-

A.4.5 Transformations

- **ttypeInst, ttypeSeq, ttypeSeqSet**: Transform a temporal value to another subtype
- **setInterp**: Transform a temporal value to another interpolation
- **shiftTime, scaleTime, shiftScaleTime**: Shift and/or scale the time span of a temporal value by one or two interval
- **unnest**: Transform a nonlinear temporal value into a set of rows, each one containing a base value and a period set during which the temporal value has the base value

A.4.6 Modifications

- **insert**: Insert a temporal value into another one
- **update**: Update a temporal value with another one
- **deleteTime**: Delete the instants of a temporal value that intersect a time value
- **beforeTimestamp, afterTimestamp**: Keep the instants of a temporal value that are before/after or equal a timestamp
- **appendInstant, appendInstantAgg**: Append a temporal instant to a temporal value
- **appendSequence, appendSequenceAgg**: Append a temporal sequence to a temporal value
- **merge, mergeAgg**: Merge temporal values

A.4.7 Restrictions

- **atValue, minusValue, atValues, minusValues**: Restrict to (the complement of) a value or a set of values
- **atTime, minusTime**: Restrict to (the complement of) a time value

A.4.8 Comparisons

- **=, <>, <, >, <=, >=, ttype_cmp**: Traditional comparisons
- **?=, %=, ?<>, %<>, ?<, %<, ?>, %>, ?<=, %<=, ?>=, %>=**: Ever and always comparisons
- **#=, #<>, #<, #>, #<=, #>=**: Temporal comparisons

A.4.9 Miscellaneous

- **mobilitydb_version**: Return the version of the MobilityDB extension
- **mobilitydb_full_version**: Return the versions of the MobilityDB extension and its dependencies

A.5 Temporal Alphanumeric Types

A.5.1 Conversions

- **tnumber::{span,tbox}, valueSpan(tnumber), timeSpan(tnumber), tbox(tnumber)**: Convert a temporal number to a span or to a temporal box
- **tbool::tint, tint(tbool)**: Convert between a temporal boolean and a temporal integer
- **tint::tfloat, tfloat::tint, tfloat(tint), tint(tfloat)**: Convert between a temporal integer and a temporal float

A.5.2 Accessors

- **valueSet**: Return the set of values of a temporal number
- **minValue**, **maxValue**, **avgValue**: Return the minimum, maximum or average value
- **minInstant**, **maxInstant**: Return the instant with the minimum or maximum value

A.5.3 Transformations

- **shiftValue**, **scaleValue**, **shiftScaleValue**: Shift and/or scale the value span of a temporal number by one or two values
- **stops**: Extract from a temporal float with linear interpolation the subsequences where the values stay within an span of a given width for at least a given duration

A.5.4 Restrictions

- **atMin**, **atMax**, **minusMin**, **minusMax**: Restrict to (the complement of) the minimum or maximum value
- **atTbox**, **minusTbox**: Restrict to (the complement of) a `tbox`

A.5.5 Boolean Operations

- **&**, **|**: Temporal and, temporal or
- **~**: Temporal not
- **whenTrue**: Return the time when a temporal Boolean takes the value true

A.5.6 Mathematical Operations

- **+**, **-**, *****, **/**: Temporal addition, subtraction, multiplication, and division
- **abs**: Return the absolute value of a temporal number
- **floor**, **ceil**: Round down or up to the nearest integer
- **round**: Round to a number of decimal places
- **degrees**, **radians**: Convert to degrees or radians
- **deltaValue**: Return the value difference between consecutive instants of a temporal number
- **trend**: Return the trend of a temporal float with linear interpolation, which states whether its value is increasing, constant, or decreasing, represented, respectively, by 1, 0, and -1.
- **derivative**: Return the derivative over time of a temporal float in units per second
- **integral**: Return the area under the curve
- **twAvg**: Return the time-weighted average
- **ln**, **log10**: Return the natural and base 10 logarithm of a temporal float
- **exp**: Return the exponential (e raised to the given power) of a temporal float

A.5.7 Text Operations

- **||**: Text concatenation
 - **lower**, **upper**, **initcap**: Transform in lowercase, uppercase, or initcap
-

A.6 Temporal Geometry Types

A.6.1 Input and Output

- **asText, asEWKT**: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- **asMFJSON**: Return the Moving Features JSON (MF-JSON) representation
- **asBinary, asEWKB, asHexEWKB**: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (EWKB) representation
- **tspatialFromText, tspatialFromEWKT**: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- **tspatialFromMFJSON**: Input from the Moving Features JSON (MF-JSON) representation
- **tspatialFromBinary, tspatialFromEWKB, tspatialFromHexEWKB**: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), or from the Hexadecimal Extended Well-Known Binary (EWKB) representation

A.6.2 Conversions

- **::, stbox(tspatial)**: Convert a spatiotemporal value to a spatiotemporal box
- **tgeometry::tgeography, tgeography::tgeometry, tgeopoint::tgeogpoint, tgeogpoint::tgeopoint**: Convert between a temporal geometry and a temporal geography
- **tgeopoint::geometry, tgeogpoint::geography, geometry::tgeopoint, geography::tgeogpoint**: Convert between a temporal point and a PostGIS trajectory

A.6.3 Spatial Reference System

- **SRID, setSRID**: Return or set the spatial reference identifier
- **transform, transformPipeline**: Transform to a spatial reference identifier

A.6.4 Accessors

- **trajectory, traversedArea**: Return the trajectory or the traversed area
 - **twCentroid**: Return the time-weighted centroid
 - **getX, getY, getZ**: Return the X/Y/Z/T coordinate values as a temporal float
 - **isSimple**: Return true if the temporal point does not spatially self-intersect
 - **length**: Return the length traversed by the temporal point
 - **cumulativeLength**: Return the cumulative length traversed by the temporal point
 - **speed**: Return the speed of the temporal point in units per second
 - **direction**: Return the direction
 - **azimuth**: Return the temporal azimuth
 - **angularDifference**: Return the temporal angular difference
 - **bearing**: Return the temporal bearing
-

A.6.5 Transformations

- **round**: Round the coordinate values to a number of decimal places
- **makeSimple**: Return an array of fragments of the temporal point which are simple
- **geoMeasure**: Construct a geometry/geography with M measure from a temporal point and a temporal float
- **affine**: Return the 3D affine transform of a temporal point to do things like translate, rotate, scale in one step
- **rotate**: Return a temporal point rotated counter-clockwise about the origin point
- **scale**: Return a temporal point scaled by given factors
- **asMVTGeom**: Transform a temporal geometric point into the coordinate space of a Mapbox Vector Tile
- **stops**: Extract from a temporal point with linear interpolation the subsequences where the point stays within an area with a specified maximum size for at least the given duration

A.6.6 Restrictions

- **atGeometry, minusGeometry**: Restrict to (the complement of) a geometry, a span in Z and/or a period
- **atStbox, minusStbox**: Restrict to (the complement of) an `stbox`
- **atElevation, minusElevation**: Restrict to (the complement of) an elevation range

A.6.7 Distance Operations

- **l=**: Return the smallest distance ever
- **minDistance**: Return the minimum spatial distance between two temporal geometries over the union of their temporal extents
- **nearestApproachInstant**: Return the instant of the first temporal point at which the two arguments are at the nearest distance
- **shortestLine**: Return the line connecting the nearest approach point
- **<->**: Return the temporal distance

A.6.8 Spatial Relationships

A.6.8.1 Ever or Always Spatial Relationships

- **eContains, aContains**: Ever or always contains
- **eDisjoint, aDisjoint**: Is ever or always disjoint
- **eDwithin, aDwithin**: Is ever or always at distance within
- **eIntersects, aIntersects**: Ever or always intersects
- **eTouches, aTouches**: Ever or always touches

A.6.8.2 Spatiotemporal Relationships

- **tContains**: Temporal contains
 - **tDisjoint**: Temporal disjoint
 - **tDwithin**: Temporal distance within
 - **tIntersects**: Temporal intersects
 - **tTouches**: Temporal touches
-

A.7 Operations for Temporal Types: Analytics Operations

A.7.1 Simplification

- **minDistSimplify, minTimeDeltaSimplify**: Return a temporal float or a temporal point simplified ensuring that consecutive values are at least a certain distance or time interval apart
- **maxDistSimplify, douglasPeuckerSimplify**: Return a temporal float or a temporal point simplified using the Douglas-Peucker algorithm

A.7.2 Reduction

- **tsample**: Sample a temporal value with respect to an interval
- **tprecision**: Reduce the temporal precision of a temporal value with respect to an interval computing the time-weighted average/centroid in each time bin

A.7.3 Similarity

- **hausdorffDistance, frechetDistance, dynTimeWarpDistance, averageHausdorffDistance, lcssDistance**: Return the discrete Hausdorff distance, the discrete Fréchet distance, the Dynamic Time Warp distance, the average Hausdorff distance, or the Longest Common SubSequence (LCSS) distance between two temporal values
- **frechetDistancePath, dynTimeWarpPath**: Return the correspondence pairs between two temporal values with respect to the discrete Fréchet distance or the Dynamic Time Warp distance

A.7.4 Splitting Operations

- **splitNSpans**: Return an array of N time spans obtained by merging the instants or segments of a temporal value
- **splitEachNSpans**: Return an array of time spans obtained by merging N consecutive instants or segments of a temporal value
- **splitNTboxes**: Return an array of N temporal boxes obtained by merging the instants or segments of a temporal number
- **splitEachNTboxes**: Return an array of temporal boxes obtained by merging N consecutive instants or segments of a temporal number
- **splitNStboxes**: Return either an array of N spatial boxes obtained by merging the segments of a (multi)line or an array of N spatiotemporal boxes obtained by merging the instants or segments of a temporal point
- **splitEachNStboxes**: Return either an array of spatial boxes obtained by merging N consecutive segments of a (multi)line or an array of spatiotemporal boxes obtained by merging N consecutive instants or segments of a temporal point

A.7.5 Multidimensional Tiling

A.7.5.1 Bin Operations

- **bins**: Return a set of bins that cover a value or time span with bins of the same width or duration
 - **getBin**: Return the bin that contains a number or a timestamp
-

A.7.5.2 Tile Operations

- **valueTiles**, **timeTiles**, **valueTimeTiles**: Return the set of tiles that cover a temporal box with tiles of the same size and/or duration
- **spaceTiles**, **timeTiles**, **spaceTimeTiles**: Return the set of tiles that cover a spatiotemporal box with tiles of the same size and/or duration
- **getValueTile**, **getTboxTimeTile**, **getValueTimeTile**: Return the temporal tile that covers a value and/or a timestamp
- **getSpaceTile**, **getStboxTimeTile**, **getSpaceTimeTile**: Return the spatiotemporal tile that covers a point and/or a timestamp

A.7.5.3 Bounding Box Operations

- **valueBins**, **timeBins**: Return an array of value or times spans obtained from the instants or segments of a temporal number with respect to a value or time grid
- **valueBoxes**, **timeBoxes**, **valueTimeBoxes**: Return an array of temporal boxes obtained from the instants or segments of a temporal number with respect to a value and/or time grid
- **spaceBoxes**, **timeBoxes**, **spaceTimeBoxes**: Return an array of spatiotemporal boxes obtained from the instants or segments of a temporal point with respect to a space and/or time grid

A.7.5.4 Splitting Operations

- **valueSplit**, **timeSplit**, **valueTimeSplit**: Split a temporal number with respect to value and/or time bins
- **spaceSplit**, **spaceTimeSplit**: Split a temporal point with respect to a spatial or spatiotemporal grid

A.7.6 Aggregations

- **tCount**: Temporal count
- **extent**: Bounding box extent
- **tAnd**, **tOr**: Temporal and, temporal or
- **tMin**, **tMinAgg**, **tMax**, **tMaxAgg**, **tSum**, **tAvg**: Temporal minimum, maximum, sum, and average
- **wCount**: Window count
- **wMin**, **wMax**, **wSum**, **wAvg**: Window minimum, maximum, sum, and average
- **tCentroid**: Temporal centroid

A.8 Temporal Poses

A.8.1 Static Poses

A.8.1.1 Input and Output

- **asText**, **asEWKT**: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- **asBinary**, **asEWKB**, **asHexEWKB**: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- **poseFromText**, **poseFromEWKT**: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- **poseFromBinary**, **poseFromEWKB**, **poseFromHexEWKB**: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.8.1.2 Constructors

- **pose**: Constructors for poses

A.8.1.3 Conversions

- **pose::stbox**, **stbox(pose)**: Convert a pose and, optionally, a timestamp or a period, into a spatiotemporal box
- **pose::geometry**: Convert a pose into a geometry point

A.8.1.4 Accessors

- **point**: Return the point
- **rotation**: Return the rotation
- **orientation**: Return the orientation

A.8.1.5 Transformations

- **round**: Round the point and the orientation of the pose to the number of decimal places

A.8.1.6 Spatial Reference System

- **SRID**, **setSRID**: Return or set the spatial reference identifier
- **transform**, **transformPipeline**: Transform to a spatial reference identifier
- **same**: Spatial similarity

A.8.1.7 Comparisons

- **=**, **<>**, **<**, **>**, **<=**, **>=**: Traditional comparisons

A.8.2 Temporal Poses

A.8.2.1 Input and Output

- **asText**, **asEWKT**: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
 - **asMFJSON**: Return the Moving Features JSON (MF-JSON) representation
 - **asBinary**, **asEWKB**, **asHexEWKB**: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
 - **tposeFromText**, **tposeFromEWKT**: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
 - **tposeFromMFJSON**: Input from the Moving Features JSON (MF-JSON) representation
 - **tposeFromBinary**, **tposeFromEWKB**, **tposeFromHexEWKB**: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
-

A.8.2.2 Constructors

- **tpose**: Constructor for temporal poses from a base value and a time value
- **tposeSeq**: Constructors for temporal poses of sequence subtype
- **tposeSeqSet, tposeSeqSetGaps**: Constructor for temporal poses of sequence set subtype
- **tpose**: Construct a 2D temporal pose from a temporal geometry point and a temporal float

A.8.2.3 Conversions

- **tpose::tgeompoint**: Convert a temporal pose into a temporal geometry point

A.8.2.4 Accessors

- **getValues**: Return the values
- **points**: Return the points
- **rotation**: Return the rotation
- **valueAtTimestamp**: Return the value at a timestamp

A.8.2.5 Transformations

- **tposeInst, tposeSeq, tposeSeqSet**: Transform to another subtype
- **setInterp**: Transform to another interpolation
- **round**: Round the points and the orientations of the temporal pose to the number of decimal places

A.8.2.6 Modifications

- **appendInstant, appendInstantAgg, appendSequence, appendSequenceAgg**: Append a temporal instant or a temporal sequence
- **merge, mergeAgg**: Merge the temporal poses

A.8.2.7 Restrictions

- **atValue, minusValue, atValues, minusValues**: Restrict to (the complement of) a value or a set of values
- **atGeometry, minusGeometry**: Restrict to (the complement of) a geometry

A.8.2.8 Spatial Reference System

- **SRID, setSRID**: Return or set the spatial reference identifier
- **transform, transformPipeline**: Transform to a spatial reference identifier

A.8.2.9 Bounding Box Operations

- **&&, <@, @>, ~=, -|**: Topological operators
- **<<, &<, >>, &>, <<|, &<|, |>>, |&>, <<#, &<#, #>>, |&>**: Position operators

A.8.2.10 Distance Operations

- **|=**: Return the smallest distance ever
- **nearestApproachInstant**: Return the instant of the first temporal pose at which the two arguments are at the nearest distance
- **shortestLine**: Return the line connecting the nearest approach point
- **<->**: Return the temporal distance

A.8.2.11 Comparisons

- **=, <, <=, >, >=**: Traditional comparisons
- **?=, &=**: Ever and always comparisons
- **#=, #<>**: Temporal comparisons

A.8.2.12 Aggregations

- **tCount**: Temporal count
- **wCount**: Window count

A.9 Temporal Network Points

A.9.1 Static Network Types

A.9.1.1 Constructors

- **npoint, nsegment**: Constructors for network points and network segments

A.9.1.2 Conversions

- **npoint::geometry, nsegment::geometry, geometry::npoint, geometry::nsegment**: Convert between a network point or segment and a geometry

A.9.1.3 Accessors

- **route**: Return the route identifier
- **getPosition**: Return the position fraction
- **startPosition, endPosition**: Return the start or end position

A.9.1.4 Transformations

- **round**: Round the position(s) of the network point or the network segment to the number of decimal places

A.9.1.5 Spatial Operations

- **SRID**: Return the spatial reference identifier
 - **equals**: Spatial similarity
-

A.9.1.6 Comparisons

- `=`, `<>`, `<`, `>`, `<=`, `>=`: Traditional comparisons

A.9.2 Temporal Network Points

A.9.2.1 Constructors

- `tnpoint`: Constructor for temporal network points from a base value and a time value
- `tnpointSeq`: Constructors for temporal network points of sequence subtype
- `tnpointSeqSet`, `tnpointSeqSetGaps`: Constructor for temporal network points of sequence set subtype

A.9.2.2 Conversions

- `tnpoint::tgeompoint`, `tgeompoint::tnpoint`: Convert between a temporal network point and a temporal geometry point

A.9.2.3 Accessors

- `getValues`: Return the values
- `routes`: Return the road identifiers
- `valueAtTimestamp`: Return the value at a timestamp
- `length`: Return the length traversed by the temporal network point
- `cumulativeLength`: Return the cumulative length traversed by the temporal network point
- `speed`: Return the speed of the temporal network point in units per second

A.9.2.4 Transformations

- `tnpointInst`, `tnpointSeq`, `tnpointSeqSet`: Transform a temporal network point to another subtype
- `setInterp`: Transform a temporal network point to another interpolation
- `round`: Round the fraction of the temporal network point to the number of decimal places

A.9.2.5 Modifications

- `appendInstant`, `appendSequence`: Append a temporal instant or a temporal sequence
- `merge`, `mergeAgg`: Merge the temporal network points

A.9.2.6 Restrictions

- `atValue`, `minusValue`, `atValues`, `minusValues`: Restrict to (the complement of) a value or a set of values
- `atGeometry`, `minusGeometry`: Restrict to (the complement of) a geometry

A.9.2.7 Bounding Box Operations

- `&&`, `<@`, `@>`, `~=`, `-|`: Topological operators
- `<<`, `<<`, `>>`, `>>`, `<<|`, `&<|`, `|>>`, `|&>`, `<<#`, `&<#`, `#>>`, `|&>`: Position operators

A.9.2.8 Distance Operations

- **|=**: Return the smallest distance ever
- **nearestApproachInstant**: Return the instant of the first temporal network point at which the two arguments are at the nearest distance
- **shortestLine**: Return the line connecting the nearest approach point between the two arguments
- **<->**: Return the temporal distance

A.9.2.9 Spatial Operations

- **twCentroid**: Return the time-weighted centroid

A.9.2.10 Comparisons

- **=, <, >, <=, >=**: Traditional comparisons
- **?=, &=**: Ever and always comparisons
- **#=, #<>**: Temporal comparisons

A.9.2.11 Aggregations

- **tCount**: Temporal count
- **wCount**: Window count
- **tCentroid**: Temporal centroid

A.10 Temporal Circular Buffers

A.10.1 Static Circular Buffers

A.10.1.1 Constructors

- **cbuffer**: Constructor for circular buffers

A.10.1.2 Conversions

- **cbuffer::geometry, geometry::cbuffer**: Convert between a circular buffer and a geometry point
- **stbox**: Convert a circular buffer and, optionally, a timestamp or a period, into a spatiotemporal box

A.10.1.3 Accessors

- **point**: Return the point
- **radius**: Return the radius

A.10.1.4 Transformations

- **round**: Round the point and the radius of the circular buffer to the number of decimal places

A.10.1.5 Spatial Operations

- **SRID, setSRID**: Return or set the spatial reference identifier
- **transform, transformPipeline**: Transform to a spatial reference identifier

A.10.1.6 Comparisons

- **=, <>, <, >, <=, >=**: Traditional comparisons

A.10.2 Temporal Circular Buffers

A.10.2.1 Input and Output

- **asText, asEWKT**: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- **asMFJSON**: Return the Moving Features JSON (MF-JSON) representation
- **asBinary, asEWKB**: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation as text
- **tcbufferFromText, tcbufferFromEWKT**: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- **tcbufferFromMFJSON**: Input from the Moving Features JSON (MF-JSON) representation
- **tcbufferFromBinary, tcbufferFromEWKB, tcbufferFromHexEWKB**: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.10.2.2 Constructors

- **tcbuffer**: Constructor for temporal circular buffers having a constant value
- **tcbufferSeq**: Constructors for temporal circular buffers of sequence subtype
- **tcbufferSeqSet, tcbufferSeqSetGaps**: Constructor for temporal circular buffers of sequence set subtype
- **tcbuffer**: Construct a temporal circular buffer from a temporal geometry point and a temporal float

A.10.2.3 Conversions

- **tcbuffer::tgeompoint**: Convert a temporal circular buffer to a temporal geometry point
- **tcbuffer::tfloat**: Convert a temporal circular buffer to a temporal float

A.10.2.4 Accessors

- **getValues**: Return the values
- **points, radius**: Return the points or the radii
- **valueAtTimestamp**: Return the value at a timestamp

A.10.2.5 Transformations

- **tcbufferInst, tcbufferSeq, tcbufferSeqSet**: Transform a temporal circular buffer to another subtype
 - **setInterp**: Transform a temporal circular buffer to another interpolation
 - **round**: Round the points and the radii of the temporal circular buffer to the number of decimal places
-

A.10.2.6 Modifications

- **appendInstant**, **appendInstantAgg**, **appendSequence**, **appendSequenceAgg**: Append a temporal instant or a temporal sequence
- **merge**, **mergeAgg**: Merge the temporal circular buffers

A.10.2.7 Restrictions

- **atValue**, **minusValue**, **atValues**, **minusValues**: Restrict to (the complement of) a value or a set of values
- **atGeometry**, **minusGeometry**: Restrict to (the complement of) a geometry

A.10.2.8 Distance Operations

- **|=|**: Return the smallest distance ever between the two arguments
- **nearestApproachInstant**: Return the instant of the first temporal circular buffer at which the two arguments are at the nearest distance
- **shortestLine**: Return the line connecting the nearest approach point between the two arguments
- **<->**: Return the temporal distance

A.10.2.9 Spatial Operations

- **SRID**, **setSRID**: Return or set the spatial reference identifier
- **transform**, **transformPipeline**: Transform to a spatial reference identifier
- **traversedArea**: Return the area traversed by the temporal circular buffer

A.10.2.10 Bounding Box Operations

- **&&**, **<@**, **@>**, **~=**, **-|**: Topological operators
- **<<**, **&<**, **>>**, **&>**, **<<l**, **&<l**, **|>**, **|&>**, **<<#**, **&<#**, **#>>**, **|&>**: Position operators

A.10.2.11 Spatial Relationships

- **eContains**, **eDisjoint**, **eIntersects**, **eTouches**, **eDwithin**: Possible spatial relationships
- **tContains**, **tDisjoint**, **tIntersects**, **tTouches**, **tDwithin**: Spatiotemporal relationships

A.10.2.12 Comparisons

- **=**, **<>**, **<**, **>**, **<=**, **>=**: Traditional comparisons
- **?=**, **&=**: Ever and always comparisons
- **#=**, **#<>**: Temporal comparisons

A.10.2.13 Aggregations

- **tCount**: Temporal count
- **wCount**: Window count
- **tCentroid**: Temporal centroid

A.11 Temporal Rigid Geometries

A.11.1 Input and Output

- **asText**: Return the Well-Known Text (WKT) representation
- **asEWKT**: Return the Extended Well-Known Text (EWKT) representation, prefixed with the SRID
- **asMFJSON**: Return the Moving Features JSON representation
- **asEWKB**, **asHexEWKB**: Return the Extended Well-Known Binary (EWKB) representation, or its hex-encoded text
- **trgeometryFromText**, **trgeometryFromEWKT**, **trgeometryFromMFJSON**, **trgeometryFromEWKB**, **trgeometryFromHexEWKB**: Parse a trgeometry from its WKT, EWKT, MFJSON, EWKB, or hex-EWKB form

A.11.2 Constructors

- **trgeometry**: Construct a temporal rigid geometry from a reference geometry and a temporal pose
- **appendInstant**, **appendInstantAgg**: Append a single instant to an existing trgeometry
- **appendSequence**, **appendSequenceAgg**: Append an entire sequence to an existing trgeometry

A.11.3 Conversions

- **tpose**: Return the underlying temporal pose
- **tgeompoint**: Return the centroid (antenna) trajectory as a temporal point

A.11.4 Accessors

- **startValue**, **endValue**, **valueAtTimestamp**: Return the materialised geometry at the start, end, or a chosen timestamp
- **numInstants**, **numSequences**, **numTimestamps**: Return the number of distinct instants, sequences, or timestamps
- **instants**, **sequences**, **segments**: Return the array of constituent instants, sequences, or inter-instant segments
- **points**: Return the set of distinct centroid (antenna) points
- **rotation**: Return the rotation angle as a temporal float
- **geom**: Return the static reference geometry

A.11.5 Traversed Area

- **traversedArea**: Return the union of the materialised polygons across the time domain

A.11.6 Spatial Functions

- **centroid**: Return the centroid of the moving body as a temporal point
 - **convexHull**: Return the convex hull of the area traversed by the moving body
 - **bodyPointTrajectory**: Return the world-frame trajectory of a body-frame point carried by the geometry
-

A.11.7 Motion Metrics

- **length, cumulativeLength**: Return the total or cumulative length traversed by the centroid
- **speed**: Return the speed of the centroid as a temporal float
- **twCentroid**: Return the time-weighted centroid of the centroid trajectory as a geometry

A.11.8 Modifications

- **round**: Round all numeric components to a given number of decimal places
- **setInterp**: Convert between linear and step interpolations
- **shiftTime, scaleTime, shiftScaleTime**: Shift, scale, or both shift and scale the time domain

A.11.9 Restrictions

- **atGeometry, minusGeometry**: Restrict a trgeometry to (the complement of) a geometry
- **atStbox, minusStbox**: Restrict a trgeometry to (the complement of) a spatiotemporal box

A.11.10 Distance Operations

- **<->, tDistance**: Return the temporal distance between two trgeometries
- **l=, nearestApproachDistance**: Return the smallest distance between two trgeometries over their shared time domain
- **nearestApproachInstant**: Return the instant at which the arguments are at their smallest mutual distance
- **shortestLine**: Return the line connecting the two arguments at their nearest approach

A.11.11 Similarity

- **hausdorffDistance, frechetDistance, dynTimeWarpDistance**: Return the Hausdorff, Frechet, or Dynamic Time Warp distance
- **frechetDistancePath, dynTimeWarpPath**: Return the Frechet or Dynamic Time Warp correspondence pairs

A.11.12 Comparisons

- **=, <>**: Test exact (in)equality of two trgeometries
- **?=, ?<>**: Test whether the materialised geometry is ever (not) equal to the operand
- **%=, %<>**: Test whether the materialised geometry is always (not) equal to the operand

A.11.13 Bounding Box Operations

- **&&, @>, <@, ~=, -|**: Standard topological operators on bounding boxes
- **<<, >>, &<, &>, <<|, &<|, |>>, |&>, <<#, &<#, #>>, #&>**: Spatial and temporal position operators on bounding boxes

A.11.14 Tiles

- **spaceBoxes, timeBoxes, spaceTimeBoxes**: Return the boxes of a spatial, temporal, or spatiotemporal grid intersected

A.11.15 Bounding Boxes

- **stboxes, spans**: Return the spatiotemporal boxes or time spans of the segments
- **splitNStboxes, splitEachNStboxes, splitNSpans, splitEachNSpans**: Return the boxes or spans split into a given number of bins

A.11.16 Aggregations

- **tCount**: Number of overlapping trgeometries at each instant
- **extent**: Aggregate bounding box of a column of trgeometries
- **merge, mergeAgg**: Combine two trgeometries with the same reference geometry into a sequence-set

A.12 Temporal JSON

A.12.1 Input and Output

- **asText**: Return the Well-Known Text (WKT) representation
- **asBinary, asWKB**: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexEWKB) representation as text
- **asMFJSON**: Return the Moving Features JSON (MF-JSON) representation as text
- **tjsonbFromText, tjsonbFromWKT**: Input from the Well-Known Text (WKT) representation
- **tjsonbFromBinary, tjsonbFromHexWKB**: Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexEWKB) representation
- **tjsonbFromMFJSON**: Input from the Moving Features JSON (MF-JSON) representation

A.12.2 Constructors

- **tjsonb**: Constructor for temporal JSONB having a constant value
- **tjsonbSeq**: Constructors for temporal JSONB of sequence subtype
- **tjsonbSeqSet, tjsonbSeqSetGaps**: Constructor for temporal JSONB of sequence set subtype

A.12.3 Conversions

- **tjsonb::tstzspan**: Convert a temporal JSONB to a temporal geometry point
- **tjsonb::ttext**: Convert a temporal JSONB to a temporal text

A.12.4 Accessors

- **getValues**: Return the values
- **valueAtTimestamp**: Return the value at a timestamp

A.12.5 Transformations

- **tjsonbInst, tjsonbSeq, tjsonbSeqSet**: Transform a temporal JSONB to another subtype
 - **setInterp**: Transform a temporal JSONB to another interpolation
-

A.12.6 Temporal JSON operations

- `->`, `->>`, `tjson_object_field`, `tjsonb_object_field`, `tjsonb_object_field_text`: Extract a field from a temporal JSON value specified by a key
- `#>`, `#>>`: Extract a field from a temporal JSON value specified by a path
- `tjson_array_element`, `tjsonb_array_element`, `tjsonb_array_element_text`: Extract an element from temporal JSON array
- `||`: Temporal JSONB concatenation
- `-`, `#-`: Temporal JSONB deletion
- `tjsonb_set`, `tjsonb_set_lax`: Temporal JSONB set
- `?`, `?|`, `?&`: Temporal JSONB exists
- `@>`, `<@`: Temporal JSONB contains/contained
- `tbool`, `tint`, `tfloat`, `ttext`: Extract a temporal alphanumeric value from a temporal JSONB value given by a key
- `tjson_strip_nulls`, `tjsonb_strip_nulls`: Return a temporal JSON value without nulls
- `@?`, `tjsonb_path_exists`, `tjsonb_path_exists_tz`: Does the JSON path expression return any item from the JSONB value?
- `@@`, `tjsonb_path_match`, `tjsonb_path_match_tz`: Return the result of a JSON path expression check for a temporal JSONB value.
- `tjsonb_path_query`, `tjsonb_path_query_tz`: Return all items returned by a JSON path expression from a temporal JSONB value, as a JSON array
- `tjsonb_path_query_first`, `tjsonb_path_query_first_tz`: Return the first item returned by a JSON path expression from a temporal JSONB value

A.12.7 Restrictions

- `atValue`, `minusValue`, `atValues`, `minusValues`: Restrict to (the complement of) a value or a set of values

A.12.8 Bounding Box Operations

- `&&`, `<@`, `@>`, `~=`, `-|`: Topological operators
- `<<#`, `&<#`, `#>>`, `|&>`: Position operators

A.12.9 Comparisons

- `=`, `<>`, `<`, `>`, `<=`, `>=`: Traditional comparisons
- `?=`, `&=`: Ever and always comparisons
- `#=`, `#<>`: Temporal comparisons

A.12.10 Aggregations

- `tCount`: Temporal count
- `wCount`: Window count

A.13 Temporal H3 Cell Indices

A.13.1 Conversions

- **th3index::tbigint, tbigint::th3index**: Convert between a temporal H3 cell and a temporal bigint (explicit, zero-cost)
- **th3index::tgeogpoint, th3index::tgeompoint**: Cast a temporal H3 cell to a temporal point of cell centroids

A.13.2 Static Geometry Coverage

- **geoToH3Cell**: Return the H3 cell containing a point geometry at the given resolution
- **geoToH3IndexSet**: Return the set of H3 cells covering a geometry at the given resolution

A.13.3 Accessors

- **startValue, endValue**: Return the first or last H3 cell identifier of a temporal value
- **valueN**: Return the nth distinct H3 cell identifier
- **getValues**: Return the set of distinct H3 cell identifiers the trajectory takes
- **valueAtTimestamp**: Return the H3 cell identifier at a given timestamp

A.13.4 Index Inspection

- **th3GetResolution**: Return the temporal resolution level (0-15) of each cell
- **th3GetBaseCellNumber**: Return the temporal base-cell number (0-121) of each cell
- **isValidCell**: Return a temporal boolean of whether each value is a valid cell
- **th3IsResClassIii**: Return a temporal boolean of whether each cell has Class-III orientation
- **th3IsPentagon**: Return a temporal boolean of whether each cell is a pentagon

A.13.5 Hierarchy

- **th3CellToParent**: Return the temporal parent cell at the given resolution
- **th3CellToCenterChild**: Return the temporal center-child cell at the given resolution
- **th3CellToChildPos**: Return each cell's ordinal position among its parent's children
- **th3ChildPosToCell**: Return the temporal child cell at a given ordinal position

A.13.6 Lat/Lng Conversions

- **tgeompoint_to_th3, tgeogpoint_to_th3**: Return the temporal H3 cell containing each point at the given resolution
 - **th3CellToLatlng**: Return the temporal geodetic centroid of each cell
 - **th3CellToBoundary**: Return the temporal polygon boundary of each cell as a geography
-

A.13.7 Directed Edges

- **th3AreNeighborCells**: Return a temporal boolean of whether two cells are grid neighbours
- **th3CellsToDirectedEdge**: Return a temporal directed-edge identifier from an origin/destination cell pair
- **isValidDirectedEdge**: Return a temporal boolean of whether each value is a valid directed edge
- **th3GetDirectedEdgeOrigin**, **th3GetDirectedEdgeDestination**: Return the temporal origin or destination cell of a directed edge
- **th3DirectedEdgeToBoundary**: Return the temporal polygon boundary of each directed edge as a geography

A.13.8 Vertices

- **th3CellToVertex**: Return the temporal H3 vertex at the given vertex number
- **th3VertexToLatlng**: Return the temporal geodetic coordinates of each vertex
- **isValidVertex**: Return a temporal boolean of whether each value is a valid vertex

A.13.9 Grid Traversal

- **th3GridDistance**, **<->**: Return the temporal grid-hop distance between two temporal cells
- **th3CellToLocalIj**, **th3LocalIjToCell**: Convert between a temporal cell and a temporal local (I, J) coordinate

A.13.10 Metrics

- **th3CellArea**: Return the temporal area of each cell in the requested unit
- **th3EdgeLength**: Return the temporal length of each directed edge in the requested unit
- **greatCircleDistance**: Return the temporal great-circle distance between two temporal geodetic points

A.13.11 Ever and Always Comparisons

- **ever_eq**, **?=**: Return whether a temporal H3 cell is ever equal to the probe
- **always_eq**, **%=**: Return whether a temporal H3 cell is always equal to the probe
- **ever_ne**, **?<>**: Return whether a temporal H3 cell is ever different from the probe
- **always_ne**, **%<>**: Return whether a temporal H3 cell is always different from the probe

A.13.12 Temporal Comparisons

- **#=**, **#<>**: Return the per-instant equality or inequality between a temporal H3 cell and a probe

A.13.13 Bounding Box Operations

- **&&**, **@>**, **<@**, **~=**, **-|**: Bounding-box overlap, contains, contained-by, same, and adjacent
- **<<**, **&<**, **&>**, **>>**, **<<|**, **&<|**, **|&>**, **|>>**: Relative position along the spatial axes
- **<<#**, **&<#**, **#>>**, **#&>**: Relative position along the time axis

A.13.14 Set-Returning Helpers

- **h3GridDisk**: All cells within k grid steps of the origin
- **h3GridRing**: The cells at exactly k grid steps from the origin
- **h3GridPathCells**: The inclusive path of cells between two cells
- **h3CellToChildren**: All children of a cell at the given target resolution
- **h3CompactCells**: Compacted mixed-resolution representation of a cell set
- **h3UncompactCells**: Uncompacted representation at the given resolution
- **h3OriginToDirectedEdges**: All outgoing directed edges of a cell
- **h3CellToVertexes**: All vertexes of a cell
- **h3GetIcosahedronFaces**: The icosahedron face indexes intersected by a cell

A.13.15 Spatial Relationships

- **eContains, aContains, tContains**: Whether one footprint contains the other (interiors only)
- **eCovers, aCovers, tCovers**: Whether one footprint covers the other (boundary included)
- **eDisjoint, aDisjoint, tDisjoint**: Whether the footprints share no point
- **eIntersects, aIntersects, tIntersects**: Whether the footprints share at least one point
- **eTouches, aTouches, tTouches**: Whether the footprints share a boundary point but no interior point
- **eDwithin, aDwithin, tDwithin**: Whether the footprints lie within a given distance

A.13.16 Indexing

- **th3index_rtree_ops**: GiST operator class indexing the spatiotemporal bounding box
- **th3index_quadtree_ops, th3index_kdtree_ops**: SP-GiST operator classes keyed on the spatiotemporal bounding box

A.14 Temporal Quadbin Cell Indices

A.14.1 Conversions

- **tquadbin::tbigint, tbigint::tquadbin**: Convert between a temporal quadbin cell and a temporal bigint
- **tquadbin::tgeompoint**: Convert a temporal quadbin cell to a temporal geometry point of cell centroids

A.14.2 Static Cells

- **isValidIndex**: Return whether a value is a structurally valid quadbin identifier
 - **isValidCell**: Return whether a value is a valid quadbin cell
-

A.14.3 Accessors

- **startValue, endValue**: Return the first or the last cell
- **valueN**: Return the nth distinct cell
- **getValues**: Return the set of distinct cells
- **valueAtTimestamp**: Return the cell at a timestamp

A.14.4 Index Inspection

- **quadbinGetResolution, tquadbinGetResolution**: Return the resolution (zoom) level of a cell
- **isValidCell**: Return a temporal boolean of cell validity

A.14.5 Hierarchy

- **quadbinCellToParent, tquadbinCellToParent**: Return the parent cell at a coarser resolution
- **quadbinCellToChildren**: Return the child cells at a finer resolution
- **quadbinCellSibling**: Return the neighbouring cell in a direction
- **quadbinGridDisk**: Return the cells within k grid steps of an origin

A.14.6 Geometry Conversions

- **geoToQuadbinCell**: Return the cell containing a point at a resolution
- **quadbinCellToPoint, tquadbinCellToPoint**: Return the centroid of a cell
- **quadbinCellToBoundary, tquadbinCellToBoundary**: Return the boundary polygon of a cell

A.14.7 Tiles and Quadkeys

- **quadbinTileToCell**: Return the cell from slippy-map tile coordinates
- **quadbinCellToTile**: Return the tile coordinates of a cell
- **quadbinCellToQuadkey, tquadbinCellToQuadkey**: Return the quadkey string of a cell

A.14.8 Metrics

- **quadbinCellArea, tquadbinCellArea**: Return the area of a cell

A.14.9 Comparisons

- **=, <>, <, >, <=, >=**: Traditional comparisons

A.14.10 Indexing

- **tquadbin_btree_ops**: Default B-tree operator class
 - **tquadbin_hash_ops**: Default hash operator class
-

A.15 Temporal Point Clouds

A.15.1 Static Types `pcpoint` and `pcpatch`

A.15.1.1 Constructors

- `pcpoint`: Build a 2D or 3D `pcpoint` from x/y(/z) coordinates
- `pcpatch`: Build a `pcpatch` from a variadic list of `pcpoints`

A.15.1.2 Accessors

- `pcid`: Return the schema id of a `pcpoint` or `pcpatch`
- `getX`, `getY`, `getZ`: Return a coordinate of a `pcpoint`
- `getDim`: Return any named dimension of a `pcpoint`
- `SRID`: Return the spatial reference identifier of a `pcpoint` or `pcpatch`

A.15.2 Set Types `pcpointset` and `pcpatchset`

A.15.2.1 Constructors

- `set`: Construct a set from an array of `pcpoint` or `pcpatch` values sharing the same `pcid`

A.15.3 Bounding Box `tpcbox`

A.15.3.1 Constructors

- `tpcbox`, `tpcbox_z`, `tpcbox_t`, `tpcbox_xt`, `tpcbox_zt`: Construct a `tpcbox` from explicit X/Y/Z/T bounds and a `pcid`

A.15.3.2 Conversions

- `tpcbox(pcpoint)`: Convert a `pcpoint` to a degenerate (zero-extent) `tpcbox`
- `tpcbox(pcpatch)`: Convert a `pcpatch` to a `tpcbox` using the patch's embedded PCBOUNDS

A.15.3.3 Accessors

- `hasX`, `hasZ`, `hasT`: Return whether a `tpcbox` has the X/Y, Z, or T dimensions set
- `xmin`, `xmax`, `ymin`, `ymax`, `zmin`, `zmax`: Return a coordinate bound of a `tpcbox`
- `tmin`, `tmax`: Return a temporal bound of a `tpcbox`
- `pcid`, `SRID`: Return the schema id and SRID of a `tpcbox`

A.15.3.4 Transformations

- `round`: Return a `tpcbox` with coordinates rounded to a given number of decimal digits
 - `setSRID`: Return a `tpcbox` with the SRID overwritten
-

A.15.3.5 Set Operations

- **tpcbox_union, +**: Return the union of two tpcboxes
- **tpcbox_intersection, ***: Return the intersection of two tpcboxes

A.15.3.6 Topological Operators

- **&&**: Return true if two tpcboxes overlap
- **@>**: Return true if the first tpcbox contains the second
- **<@**: Return true if the first tpcbox is contained in the second
- **~=**: Return true if two tpcboxes are exactly equal
- **-|-**: Return true if two tpcboxes are adjacent

A.15.3.7 Position Operators

- **<<, &<, >>, &>**: X-axis position operators
- **<<|, &<|, |>>, |&>**: Y-axis position operators
- **<</, &</, />>, /&>**: Z-axis position operators
- **<<#, &<#, #>>, #&>**: Time-axis position operators

A.15.3.8 Comparisons

- **=, <>, <, >, <=, >=**: Traditional comparisons

A.15.4 Temporal Type **tpcpoint**

A.15.4.1 Constructors

- **tpcpoint**: Construct a temporal pcpoint of instant subtype
- **tpcpointSeq**: Construct a temporal pcpoint of sequence subtype
- **tpcpointSeqSet**: Construct a temporal pcpoint of sequence-set subtype

A.15.4.2 Accessors

- **pcid**: Return the pgPointCloud schema id shared by all instants
- **SRID**: Return the SRID inherited from the schema
- **getX, getY, getZ**: Project a tpcpoint onto a spatial dimension, yielding a tfloat
- **getDim**: Project a tpcpoint onto any named schema dimension, yielding a tfloat

A.15.4.3 Conversions

- **tpcpoint::tgeompoint**: Cast a tpcpoint to a temporal geometry point

A.15.4.4 Comparisons

- `=`, `<>`, `<`, `>`, `<=`, `>=`: Traditional comparisons
- `?=`, `%=`: Ever and always equality comparisons

A.15.4.5 Restrictions

- `atValue`, `minusValue`, `atValues`, `minusValues`: Restrict to (the complement of) a value or a set of values
- `atTime`, `minusTime`: Restrict to (the complement of) a time selector
- `atTpcbox`, `minusTpcbox`: Restrict to (the complement of) the extent of a tpcbox

A.15.4.6 Bounding-box Operators

- `&&`, `@>`, `<@`, `~=`, `-|`: Topological operators
- `<<`, `>>`, `<<|`, `|>>`, `<</`, `/>`, `<<#`, `#>`: Position operators
- `|=`, `nearestApproachDistance`: Nearest-approach distance, KNN-orderable

A.15.4.7 Spatial Relationships

- `eIntersects`, `aIntersects`: Ever / always intersects
- `eDisjoint`, `aDisjoint`: Ever / always disjoint
- `eDwithin`, `aDwithin`: Ever / always within distance

A.15.5 Temporal Type `tpcpatch`

A.15.5.1 Constructors

- `tpcpatch`: Construct a temporal pcpatch of instant subtype
- `tpcpatchSeq`: Construct a temporal pcpatch of sequence subtype
- `tpcpatchSeqSet`: Construct a temporal pcpatch of sequence-set subtype

A.15.5.2 Accessors

- `startNumPoints`, `endNumPoints`: Return the number of points in the first or last instant's patch
- `numPoints`: Return the total number of points across all instants
- `points`: Set-returning function emitting one row per (timestamp, pcpoint)

A.15.5.3 Restrictions

- `atValue`, `minusValue`, `atValues`, `minusValues`: Restrict to (the complement of) a value or a set of values
 - `atTpcbox`, `minusTpcbox`: Restrict to the instants whose patch overlaps a tpcbox (patch-level)
 - `atTpcboxFine`, `minusTpcboxFine`: Restrict to (the complement of) the points inside a tpcbox (per-point)
 - `atGeometry`, `minusGeometry`: Restrict to (the complement of) the points intersecting a 2D geometry
-

A.15.5.4 Spatial Relationships

- **eIntersects**: Return true if at least one point in any instant intersects a geometry

A.15.5.5 Bounding-box Operators

- **&&, @>, <@, ~=, -|**: Topological operators
- **<<, >>, <<|, |>>, <</, />>, <<#, #>>**: Position operators
- **|=|, nearestApproachDistance**: Nearest-approach distance, KNN-orderable

A.15.6 Aggregations

- **extent**: Bounding-box aggregate over tpcpoint, tpcpatch, or tpcbox values
- **tcoun**t, **wcount**: Temporal count and window count
- **tnpoints**: Per-instant pcpatch point count summed across rows
- **tdensity**: Per-instant point density summed across rows
- **merge, mergeAgg**: Combine multiple temporal values into a single one
- **appendInstant, appendInstantAgg, appendSequence, appendSequenceAgg**: Streaming construction aggregates

A.15.7 Indexes

- **GiST, SP-GiST, B-tree, and hash opclasses** for tpcpoint, tpcpatch, and tpcbox

A.15.8 Binary Representation

- **asBinary, tpcpointFromBinary, tpcpatchFromBinary**: Binary I/O with embedded schema for portability

A.15.9 MF-JSON Output

- **asMFJSON**: Return the Moving Features JSON representation of a tpcpoint
- **asMFJSON**: Return the Moving Features JSON representation of a tpcpatch

A.15.10 PDAL Integration

- **readers.tpcpatch**: PDAL reader that reads tpcpatch values from a SQL query
- **writers.tpcpatch**: PDAL writer that writes tpcpatch values to a PostgreSQL table

A.15.11 PCL Bridge

- **pcpatch_to_pcd, pcpatch_from_pcd, pcpatch_voxel_grid, pcpatch_sor, pcpatch_icp, pcpatch_normals**: PCL-backed SQL functions for point cloud processing

A.16 Raster Sampling

A.16.1 Sampling Operator

- **raster_value**: Sample a raster band at each instant of a trajectory

A.16.2 Raquet / QUADBIN Sampling

- **trajectory_quadbins**: Return the distinct QUADBIN cells at a zoom level covered by a trajectory
- **raster_tile_value_quadbin**: Sample a Raquet raster chip along a trajectory using a QUADBIN cell for georeferencing
- **raquet**: Construct a Raquet raster tile from its pixel bytes, dimensions, QUADBIN cell and pixel type
- **raquet_read**: Decode an in-memory raster file into a Raquet raster tile via GDAL
- **raster_tile_value**: Sample a Raquet raster tile along a trajectory
- **raster_tile_value**: Sample an array of Raquet raster tiles along a trajectory, keeping the finer resolution where tiles overlap

A.16.3 Raquet Accessors and Comparison

- **quadbin**: Return the QUADBIN cell of a Raquet tile
- **width**: Return the width in pixels of a Raquet tile
- **height**: Return the height in pixels of a Raquet tile
- **nodata**: Return the nodata sentinel value of a Raquet tile
- **pixtype**: Return the name of the pixel data type of a Raquet tile
- **stbox**: Convert a Raquet tile into a spatiotemporal box
- **raquet comparison**: Compare two Raquet tiles

A.16.4 Restriction and Predicate Operators

- **atRasterValue**: Return the instants of a trajectory where the sampled raster value falls inside a float range
 - **minusRasterValue**: Return the instants of a trajectory where the sampled raster value falls outside a float range
 - **eRasterValue**: Return true if the trajectory ever samples a raster pixel value inside a float range
 - **aRasterValue**: Return true if every in-raster-extent instant of the trajectory samples a pixel value inside a float range
-

Appendix B

Synthetic Data Generator

In many circumstances, it is necessary to have a test dataset to evaluate alternative implementation approaches or to perform benchmarks. It is often required that such a data set have particular requirements in size or in the intrinsic characteristics of its data. Even if a real-world dataset could be available, it may be not ideal for such experiments for multiple reasons. Therefore, a synthetic data generator that could be customized to produce data according to the given requirements is often the best solution. Obviously, experiments with synthetic data should be complemented with experiments with real-world data to have a thorough understanding of the problem at hand.

MobilityDB provides a simple synthetic data generator that can be used for such purposes. In particular, such a data generator was used for generating the database used for the regression tests in MobilityDB. The data generator is programmed in PL/pgSQL so it can be easily customized. It is located in the directory `datagen` in the repository. In this appendix, we briefly introduce the basic functionality of the generator. We first list the functions generating random values for the various PostgreSQL, PostGIS, and MobilityDB data types, and then give examples how to use these functions for generating tables of such values. The parameters of the functions are not specified, refer to the source files where detailed explanations about the various parameters can be found.

B.1 Generator for PostgreSQL Types

- `random_bool`: Generate a random boolean
 - `random_int`: Generate a random integer
 - `random_int_array`: Generate a random array of integers
 - `random_int4range`: Generate a random integer range
 - `random_float`: Generate a random float
 - `random_float_array`: Generate a random array of floats
 - `random_text`: Generate a random text
 - `random_timestamptz`: Generate a random timestamp with time zone
 - `random_timestamptz_array`: Generate a random array of timestamps with time zone
 - `random_minutes`: Generate a random interval of minutes
 - `random_tstzrange`: Generate a random timestamp with time zone range
 - `random_tstzrange_array`: Generate a random array of timestamp with time zone ranges
-

B.2 Generator for PostGIS Types

- `random_geom_point`: Generate a random 2D geometric point
 - `random_geom_point3D`: Generate a random 3D geometric point
 - `random_geog_point`: Generate a random 2D geographic point
 - `random_geog_point3D`: Generate a random 3D geographic point
 - `random_geom_point_array`: Generate a random array of 2D geometric points
 - `random_geom_point3D_array`: Generate a random array of 3D geometric points
 - `random_geog_point_array`: Generate a random array of 2D geographic points
 - `random_geog_point3D_array`: Generate a random array of 3D geographic points
 - `random_geom_linestring`: Generate a random geometric 2D linestring
 - `random_geom_linestring3D`: Generate a random geometric 3D linestring
 - `random_geog_linestring`: Generate a random geographic 2D linestring
 - `random_geog_linestring3D`: Generate a random geographic 3D linestring
 - `random_geom_polygon`: Generate a random geometric 2D polygon without holes
 - `random_geom_polygon3D`: Generate a random geometric 3D polygon without holes
 - `random_geog_polygon`: Generate a random geographic 2D polygon without holes
 - `random_geog_polygon3D`: Generate a random geographic 3D polygon without holes
 - `random_geom_multipoint`: Generate a random geometric 2D multipoint
 - `random_geom_multipoint3D`: Generate a random geometric 3D multipoint
 - `random_geog_multipoint`: Generate a random geographic 2D multipoint
 - `random_geog_multipoint3D`: Generate a random geographic 3D multipoint
 - `random_geom_multilinestring`: Generate a random geometric 2D multilinestring
 - `random_geom_multilinestring3D`: Generate a random geometric 3D multilinestring
 - `random_geog_multilinestring`: Generate a random geographic 2D multilinestring
 - `random_geog_multilinestring3D`: Generate a random geographic 3D multilinestring
 - `random_geom_multipolygon`: Generate a random geometric 2D multipolygon without holes
 - `random_geom_multipolygon3D`: Generate a random geometric 3D multipolygon without holes
 - `random_geog_multipolygon`: Generate a random geographic 2D multipolygon without holes
 - `random_geog_multipolygon3D`: Generate a random geographic 3D multipolygon without holes
-

B.3 Generator for MobilityDB Span, Time, and Box Types

- `random_intspan`: Generate a random integer span
- `random_floatspan`: Generate a random float span
- `random_tstzspan`: Generate a random `tstzspan`
- `random_tstzspan_array`: Generate a random array of `tstzspan` values
- `random_tstzset`: Generate a random `tstzset`
- `random_tstzspanset`: Generate a random `tstzspanset`
- `random_tbox`: Generate a random `tbox`
- `random_stbox`: Generate a random 2D `stbox`
- `random_stbox3D`: Generate a random 3D `stbox`
- `random_geodstbox`: Generate a random 2D geodetic `stbox`
- `random_geodstbox3D`: Generate a random 3D geodetic `stbox`

B.4 Generator for MobilityDB Temporal Types

- `random_tbool_inst`: Generate a random temporal Boolean of instant subtype
 - `random_tint_inst`: Generate a random temporal integer of instant subtype
 - `random_tfloat_inst`: Generate a random temporal float of instant subtype
 - `random_ttext_inst`: Generate a random temporal text of instant subtype
 - `random_tgeompoint_inst`: Generate a random temporal geometric 2D point of instant subtype
 - `random_tgeompoint3D_inst`: Generate a random temporal geometric 3D point of instant subtype
 - `random_tgeogpoint_inst`: Generate a random temporal geographic 2D point of instant subtype
 - `random_tgeogpoint3D_inst`: Generate a random temporal geographic 3D point of instant subtype
 - `random_tbool_discseq`: Generate a random temporal Boolean of sequence subtype and discrete interpolation
 - `random_tint_discseq`: Generate a random temporal integer of sequence subtype and discrete interpolation
 - `random_tfloat_discseq`: Generate a random temporal float of sequence subtype and discrete interpolation
 - `random_ttext_discseq`: Generate a random temporal text of sequence subtype and discrete interpolation
 - `random_tgeompoint_discseq`: Generate a random temporal geometric 2D point of sequence subtype and discrete interpolation
 - `random_tgeompoint3D_discseq`: Generate a random temporal geometric 3D point of sequence subtype and discrete interpolation
 - `random_tgeogpoint_discseq`: Generate a random temporal geographic 2D point of sequence subtype and discrete interpolation
 - `random_tgeogpoint3D_discseq`: Generate a random temporal geographic 3D point of sequence subtype and discrete interpolation
 - `random_tbool_seq`: Generate a random temporal Boolean of sequence subtype
-

- `random_tint_seq`: Generate a random temporal integer of sequence subtype
- `random_tfloat_seq`: Generate a random temporal float of sequence subtype
- `random_ttext_seq`: Generate a random temporal text of sequence subtype
- `random_tgeompoint_seq`: Generate a random temporal geometric 2D point of sequence subtype
- `random_tgeompoint3D_seq`: Generate a random temporal geometric 3D point of sequence subtype
- `random_tgeogpoint_seq`: Generate a random temporal geographic 2D point of sequence subtype
- `random_tgeogpoint3D_seq`: Generate a random temporal geographic 3D point of sequence subtype
- `random_tbool_seqset`: Generate a random temporal Boolean of sequence set subtype
- `random_tint_seqset`: Generate a random temporal integer of sequence set subtype
- `random_tfloat_seqset`: Generate a random temporal float of sequence set subtype
- `random_ttext_seqset`: Generate a random temporal text of sequence set subtype
- `random_tgeompoint_seqset`: Generate a random temporal geometric 2D point of sequence set subtype
- `random_tgeompoint3D_seqset`: Generate a random temporal geometric 3D point of sequence set subtype
- `random_tgeogpoint_seqset`: Generate a random temporal geographic 2D point of sequence set subtype
- `random_tgeogpoint3D_seqset`: Generate a random temporal geographic 3D point of sequence set subtype

B.5 Generation of Tables with Random Values

The files `create_test_tables_temporal.sql` and `create_test_tables_tpoint.sql` provide usage examples for the functions generating random values listed above. For example, the first file defines the following function.

```
CREATE OR REPLACE FUNCTION create_test_tables_temporal(size integer DEFAULT 100)
RETURNS text AS $$
DECLARE
    perc integer;
BEGIN
    perc := size * 0.01;
    IF perc < 1 THEN perc := 1; END IF;

    -- ... Table generation ...

RETURN 'The End';
END;
$$ LANGUAGE 'plpgsql';
```

The function has a `size` parameter that defines the number of rows in the tables. If not provided, it creates by default tables of 100 rows. The function defines a variable `perc` that computes the 1% of the size of the tables. This parameter is used, for example, for generating tables having 1% of null values. We illustrate next some of the commands generating tables.

The creation of a table `tbl_float` containing random `float` values in the range `[0,100]` with 1% of null values is given next.

```
CREATE TABLE tbl_float AS
/* Add perc NULL values */
SELECT k, NULL AS f
FROM generate_series(1, perc) AS k UNION
SELECT k, random_float(0, 100)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_tbox` containing random `tbox` values where the bounds for values are in the range [0,100] and the bounds for timestamps are in the range [2001-01-01, 2001-12-31] is given next.

```
CREATE TABLE tbl_tbox AS
/* Add perc NULL values */
SELECT k, NULL AS b
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tbox(0, 100, '2001-01-01', '2001-12-31', 10, 10)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_floatspan` containing random `floatspan` values where the bounds for values are in the range [0,100] and the maximum difference between the lower and the upper bounds is 10 is given next.

```
CREATE TABLE tbl_floatspan AS
/* Add perc NULL values */
SELECT k, NULL AS f
FROM generate_series(1, perc) AS k UNION
SELECT k, random_floatspan(0, 100, 10)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_tstzset` containing random `tstzset` values having between 5 and 10 timestamps where the timestamps are in the range [2001-01-01, 2001-12-31] and the maximum interval between consecutive timestamps is 10 minutes is given next.

```
CREATE TABLE tbl_tstzset AS
/* Add perc NULL values */
SELECT k, NULL AS ts
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tstzset('2001-01-01', '2001-12-31', 10, 5, 10)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_tstzspan` containing random `tstzspan` values where the timestamps are in the range [2001-01-01, 2001-12-31] and the maximum difference between the lower and the upper bounds is 10 minutes is given next.

```
CREATE TABLE tbl_tstzspan AS
/* Add perc NULL values */
SELECT k, NULL AS p
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tstzspan('2001-01-01', '2001-12-31', 10)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_geom_point` containing random geometry 2D point values, where the x and y coordinates are in the range [0, 100] and in SRID 3812 is given next.

```
CREATE TABLE tbl_geom_point AS
SELECT 1 AS k, geometry 'SRID=3812;point empty' AS g UNION
SELECT k, random_geom_point(0, 100, 0, 100, 3812)
FROM generate_series(2, size) k;
```

Notice that the table contains an empty point value. If the SRID is not given it is set by default to 0.

The creation of a table `tbl_geog_point3D` containing random geography 3D point values, where the x, y, and z coordinates are, respectively, in the ranges [-10, 32], [35, 72], and [0, 1000] and in SRID 7844 is given next.

```
CREATE TABLE tbl_geog_point3D AS
SELECT 1 AS k, geography 'SRID=7844;pointZ empty' AS g UNION
SELECT k, random_geog_point3D(-10, 32, 35, 72, 0, 1000, 7844)
FROM generate_series(2, size) k;
```

Notice that latitude and longitude values are chosen to approximately cover continental Europe. If the SRID is not given it is set by default to 4326.

The creation of a table `tbl_geom_linestring` containing random geometry 2D linestring values having between 5 and 10 vertices, where the x and y coordinates are in the range [0, 100] and in SRID 3812 and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k, random_geom_linestring(0, 100, 0, 100, 10, 5, 10, 3812)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geom_linestring` containing random geometry 2D linestring values having between 5 and 10 vertices, where the x and y coordinates are in the range [0, 100] and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k, random_geom_linestring(0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geom_polygon3D` containing random geometry 3D polygon values without holes, having between 5 and 10 vertices, where the x, y, and z coordinates are in the range [0, 100] and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_polygon3D AS
SELECT 1 AS k, geometry 'polygon Z empty' AS g UNION
SELECT k, random_geom_polygon3D(0, 100, 0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geom_multipoint` containing random geometry 2D multipoint values having between 5 and 10 points, where the x and y coordinates are in the range [0, 100] and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_multipoint AS
SELECT 1 AS k, geometry 'multipoint empty' AS g UNION
SELECT k, random_geom_multipoint(0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geog_multilinestring` containing random geography 2D multilinestring values having between 5 and 10 linestrings, each one having between 5 and 10 vertices, where the x and y coordinates are, respectively, in the ranges [-10, 32] and [35, 72], and the maximum difference between consecutive coordinate values is 10 is given next.

```
CREATE TABLE tbl_geog_multilinestring AS
SELECT 1 AS k, geography 'multilinestring empty' AS g UNION
SELECT k, random_geog_multilinestring(-10, 32, 35, 72, 10, 5, 10, 5, 10)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geometry3D` containing random geometry 3D values of various types is given next. This function requires that the tables for the various geometry types have been created previously.

```
CREATE TABLE tbl_geometry3D (
  k serial PRIMARY KEY,
  g geometry);
INSERT INTO tbl_geometry3D(g)
(SELECT g FROM tbl_geom_point3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_linestring3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_polygon3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipoint3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geog_multilinestring3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipolygon3D ORDER BY k LIMIT (size * 0.2));
```

The creation of a table `tbl_tbool_inst` containing random `tbool` values of instant subtype where the timestamps are in the range [2001-01-01, 2001-12-31] is given next.

```

CREATE TABLE tbl_tbool_inst AS
/* Add perc NULL values */
SELECT k, NULL AS inst
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tbool_inst('2001-01-01', '2001-12-31')
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT inst FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows with the same timestamp */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT tboolInst(random_bool(), getTimestamp(inst))
            FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);

```

As can be seen above, the table has a percentage of null values, of duplicates, and of rows with the same timestamp.

The creation of a table `tbl_tint_discseq` containing random `tint` values of sequence subtype and discrete interpolation having between 5 and 10 timestamps where the integer values are in the range [0, 100], the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between two consecutive values is 10, and the maximum interval between two consecutive instants is 10 minutes is given next.

```

CREATE TABLE tbl_tint_discseq AS
/* Add perc NULL values */
SELECT k, NULL AS ti
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tint_discseq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS ti
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows with the same timestamp */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti + random_int(1, 2) FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc rows that meet */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, endTimestamp(ti)-startTimestamp(ti))
            FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc rows that overlap */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute', (endTimestamp(ti)-startTimestamp(ti))/2))
            FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);

```

As can be seen above, the table has a percentage of null values, of duplicates, of rows with the same timestamp, of rows that meet, and of rows that overlap.

The creation of a table `tbl_tfloat_seq` containing random `tfloat` values of sequence subtype having between 5 and 10 timestamps where the float values are in the range [0, 100], the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between two consecutive values is 10, and the maximum interval between two consecutive instants is 10 minutes is given next.

```

CREATE TABLE tbl_tfloat_seq AS
/* Add perc NULL values */
SELECT k, NULL AS seq
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tfloat_seq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS seq

```

```

FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq + random_int(1, 2) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, timeSpan(seq)) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, date_trunc('minute', timeSpan(seq)/2))
FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);

```

The creation of a table `tbl_ttext_seqset` containing random `ttext` values of sequence set subtype having between 5 and 10 sequences, each one having between 5 and 10 timestamps, where the text values have at most 10 characters, the timestamps are in the range [2001-01-01, 2001-12-31], and the maximum interval between two consecutive instants is 10 minutes is given next.

```

CREATE TABLE tbl_ttext_seqset AS
/* Add perc NULL values */
SELECT k, NULL AS ts
FROM generate_series(1, perc) AS k UNION
SELECT k, random_ttext_seqset('2001-01-01', '2001-12-31', 10, 10, 5, 10, 5, 10) AS ts
FROM generate_series(perc+1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts || text 'A' FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, timeSpan(ts)) FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);

```

The creation of a table `tbl_tgeompoint_discseq` containing random `tgeompoint` 2D values of sequence subtype and discrete interpolation having between 5 and 10 instants, where the x and y coordinates are in the range [0, 100] and in SRID 3812, the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between successive coordinates is at most 10 units in the underlying SRID, and the maximum interval between two consecutive instants is 10 minutes is given next.

```

CREATE TABLE tbl_tgeompoint_discseq AS
SELECT k, random_tgeompoint_discseq(0, 100, 0, 100, '2001-01-01', '2001-12-31',
10, 10, 5, 10, 3812) AS ti
FROM generate_series(1, size) k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT ti FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);

```



```

/* Add perc tuples with the same timestamp */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT round(ti,6) FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, endTimeStamp(ti)-startTimeStamp(ti))
FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute', (endTimeStamp(ti)-startTimeStamp(ti))/2))
FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);

```

Finally, the creation of a table `tbl_tgeompoint3D_seqset` containing random `tgeompoint` 3D values of sequence set subtype having between 5 and 10 sequences, each one having between 5 and 10 timestamps, where the x, y, and z coordinates are in the range [0, 100] and in SRID 3812, the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between successive coordinates is at most 10 units in the underlying SRID, and the maximum interval between two consecutive instants is 10 minutes is given next.

```

DROP TABLE IF EXISTS tbl_tgeompoint3D_seqset;
CREATE TABLE tbl_tgeompoint3D_seqset AS
SELECT k, random_tgeompoint3D_seqset(0, 100, 0, 100, 0, 100, '2001-01-01', '2001-12-31',
10, 10, 5, 10, 5, 10, 3812) AS ts
FROM generate_series(1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT ts FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT round(ts,3) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts, timeSpan(ts)) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+ ←
perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);

```

B.6 Generator for Temporal Network Point Types

- `random_fraction`: Generate a random fraction in the range [0,1]
- `random_npoint`: Generate a random network point
- `random_nsegment`: Generate a random network segment
- `random_tnpoint_inst`: Generate a random temporal network point of instant subtype
- `random_tnpoint_discseq`: Generate a random temporal network point of sequence subtype and discrete interpolation
- `random_tnpoint_seq`: Generate a random temporal network point of sequence subtype and linear or step interpolation
- `random_tnpoint_seqset`: Generate a random temporal network point of sequence set subtype

The file `/datagen/npoint/create_test_tables_tnpoint.sql` provide usage examples for the functions generating random values listed above.